

dslar

July 10, 2026

basics.lib/name	Faust Basic Element Library
basics.lib/sAndH:author	Romain Michon
basics.lib/version	1.22.0
compile_options	-lang cpp -fpga-mem-th 4 -ct 1 -es 1 -mcd 16 -mdd 1024 -mdy 33 -singl
delays.lib/name	Faust Delay Library
delays.lib/version	1.2.0
filename	dslar.dsp
filters.lib/lowpass0_highpass1	MIT-style STK-4.3 license, Copyright (C) 2003-2019 by Julius O. Smit
filters.lib/name	Faust Filters Library
filters.lib/pole:author	Julius O. Smith III
filters.lib/pole:copyright	Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.e
filters.lib/pole:license	MIT-style STK-4.3 license
filters.lib/version	1.7.1
filters.lib/zero:author	Julius O. Smith III
filters.lib/zero:copyright	Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.e
filters.lib/zero:license	MIT-style STK-4.3 license
maths.lib/author	GRAME
maths.lib/copyright	GRAME
maths.lib/license	LGPL with exception
maths.lib/name	Faust Math Library
maths.lib/version	2.9.0
name	dslar
platform.lib/name	Generic Platform Library
platform.lib/version	1.3.0
seam.discipio.lib/author	Luca Spanedda, Dario Sanfilippo
seam.discipio.lib/description	Realised on composer's instructions of the year 2017 edited in L'Aquila
seam.discipio.lib/name	Agostino Di Scipio - AUDIBLE ECOSYSTEMICS n.2 Library, granula
seam.discipio.lib/version	xxx
seam.lib/author	Giuseppe Silvi, Davide Tedesco
seam.lib/license	CC3
seam.lib/name	Faust SEAM main lib
seam.lib/version	0.2
seam.pdclone.lib/author	Giuseppe Silvi
seam.pdclone.lib/license	GPL-3.0
seam.pdclone.lib/name	PdClone Faust Library
seam.pdclone.lib/version	0.1

This document provides a mathematical description of the Faust program text stored in the `dslar.dsp` file. See the notice in Section 3 (page 5) for details.

1 Mathematical definition of process

The *dslar* program evaluates the signal transformer denoted by **process**, which is mathematically defined as follows:

1. Output signal y such that

$$y(t) = u_{s1}(t) \cdot r_1(t) \cdot r_{19}(t - p_5(t))$$

2. Input signal x

3. User-interface input signals u_{si} for $i \in [1, 7]$ and u_{c1} such that

"Output"	$u_{s1}(t) \in [0, 1]$	(default value = 1)
"Power"	$u_{c1}(t) \in \{0, 1\}$	(default value = 0)
"Decorrelation" (ms)	$u_{s2}(t) \in [1, 200]$	(default value = 20)
"Target"	$u_{s3}(t) \in [0, 1]$	(default value = 1)
"Steepness"	$u_{s4}(t) \in [1, 80]$	(default value = 40)
"Control smoothing" (ms)	$u_{s5}(t) \in [1, 1000]$	(default value = 200)
"Drive"	$u_{s6}(t) \in [1, 4]$	(default value = 1)
"Loop delay" (ms)	$u_{s7}(t) \in [1, 200]$	(default value = 50)

4. Intermediate signals p_i for $i \in [1, 5]$, s_i for $i \in [1, 12]$, r_i for $i \in [1, 16]$ and q_i for $i \in [1, 11]$ such that

$$\begin{aligned}
p_1(t) &= u_{c1}(t) \\
p_2(t) &= \min(38400, \max(0, \text{int}(k_4 \cdot u_{s2}(t) + 0.5))) \\
p_3(t) &= \left(\frac{k_5}{u_{s5}(t)} \right) \\
p_4(t) &= k_7 \cdot u_{s6}(t) \\
p_5(t) &= \min(38400, \max(0, \text{int}(k_9 \cdot u_{s7}(t) + 0.5)))
\end{aligned}$$

$$\begin{aligned}
s_1(t) &= r_2(t) \leq r_2(t-1) \\
s_2(t) &= ((1 \ominus r_8(t)) \bmod 2048 \oplus 2048) \bmod 2048 \\
s_3(t) &= (s_2(t) \equiv 0) \\
s_4(t) &= ((1025 \ominus r_8(t)) \bmod 2048 \oplus 2048) \bmod 2048 \\
s_5(t) &= (s_4(t) \equiv 0) \\
s_6(t) &= p_1(t) \neq p_1(t-1) \\
s_7(t) &= x(t) \cdot r_{10}(t) \\
s_8(t) &= s_7(t - p_2(t))^2
\end{aligned}$$

$$\begin{aligned}
s_9(t) &= 0.00048828125 \cdot (1 - \cos(0.003067958984375 \cdot s_2(t))) \cdot s_8(t) \\
s_{10}(t) &= 0.00048828125 \cdot s_8(t) \cdot (1 - \cos(0.003067958984375 \cdot s_4(t))) \\
s_{11}(t) &= |q_3(t) - u_{s3}(t)|^{u_{s4}(t)} \\
s_{12}(t) &= s_{11}(t) \neq s_{11}(t-1)
\end{aligned}$$

$$\begin{aligned}
r_3(t) &= 1 \\
r_2(t) &= (r_3(t-1) \oplus r_2(t-1)) \bmod k_2 \\
r_8(t) &= (r_8(t-1) \oplus 1) \bmod 2048 \\
r_{13}(t) &= q_8(t) \\
r_{11}(t) &= r_{13}(t) + (p_1(t) - r_{13}(t)) \cdot \min(k_3 \cdot q_9(t), 1) \\
r_{12}(t) &= q_9(t) \\
r_{10}(t) &= q_7(t) \\
r_9(t) &= q_6(t) \\
r_{16}(t) &= q_{10}(t) \\
r_7(t) &= q_5(t) \\
r_6(t) &= q_2(t) \\
r_4(t) &= r_6(t) + \min(p_3(t) \cdot q_{11}(t), 1) \cdot (s_{11}(t) - r_6(t)) \\
r_5(t) &= q_{11}(t) \\
r_1(t) &= q_1(t) \\
r_{18}(t) &= s_7(t) + k_8 \cdot r_{18}(t-1) \\
r_{19}(t) &= p_4(t) \cdot (r_{18}(t) - r_{18}(t-1))
\end{aligned}$$

$$q_8(t) = \begin{cases} r_{13}(t-1) & \text{if } s_6(t) = 0 \\ r_{11}(t-1) & \text{if } s_6(t) = 1 \end{cases} \quad q_9(t) = \begin{cases} r_{12}(t-1) \oplus 1 & \text{if } s_6(t) = 0 \\ 0 & \text{if } s_6(t) = 1 \end{cases} \quad q_7(t) = \begin{cases} r_{10}(t-1) & \text{if } s_1(t) = \\ r_{11}(t) & \text{if } s_1(t) = \end{cases}$$

5. Constants k_i for $i \in [1, 9]$ such that

$$\begin{aligned} k_1 &= \min(192000, \max(1, f_S)) \\ k_2 &= \text{int}(0.02 \cdot k_1) \\ k_3 &= \left(\frac{0.5}{k_1}\right) \\ k_4 &= 0.001 \cdot k_1 \\ k_5 &= \frac{1000}{k_1} \\ k_6 &= \min(192000, \max(1, f_S)) \\ k_7 &= 0.5 \cdot \left(\min\left(1, \max\left(0, 1 - \frac{628.318}{k_6}\right)\right) + 1\right) \\ k_8 &= \min\left(1, \max\left(0, 1 - \frac{628.318}{k_1}\right)\right) \\ k_9 &= 0.001 \cdot k_6 \end{aligned}$$

2 Block diagram of process

The block diagram of `process` is shown on Figure 1 (page 5).

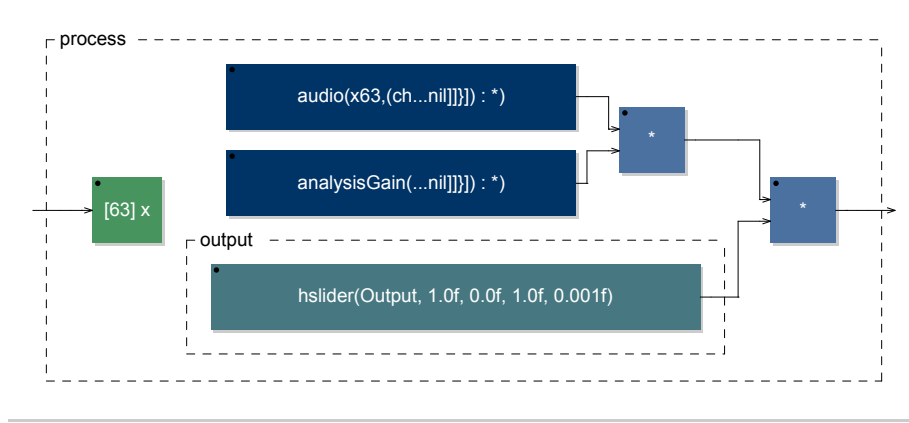


Figure 1: Block diagram of `process`

3 Notice

- This document was generated using Faust version 2.85.5 on July 10, 2026.

- The value of a Faust program is the result of applying the signal transformer denoted by the expression to which the **process** identifier is bound to input signals, running at the f_S sampling frequency.
- Faust (*Functional Audio Stream*) is a functional programming language designed for synchronous real-time signal processing and synthesis applications. A Faust program is a set of bindings of identifiers to expressions that denote signal transformers. A signal s in S is a function mapping¹ times $t \in \mathbb{Z}$ to values $s(t) \in \mathbb{R}$, while a signal transformer is a function from S^n to S^m , where $n, m \in \mathbb{N}$. See the Faust manual for additional information (<http://faust.grame.fr>).
- Every mathematical formula derived from a Faust expression is assumed, in this document, to having been normalized (in an implementation-dependent manner) by the Faust compiler.
- A block diagram is a graphical representation of the Faust binding of an identifier I to an expression E ; each graph is put in a box labeled by I . Subexpressions of E are recursively displayed as long as the whole picture fits in one page.
- $\forall x \in \mathbb{R}$,

$$\text{int}(x) = \begin{cases} \lfloor x \rfloor & \text{if } x > 0 \\ \lceil x \rceil & \text{if } x < 0 \\ 0 & \text{if } x = 0 \end{cases}.$$

- This document uses the following integer operations:

<i>operation</i>	<i>name</i>	<i>semantics</i>
$i \oplus j$	integer addition	$\text{normalize}(i + j)$, in $\mathbb{Z}$
$i \ominus j$	integer substraction	$\text{normalize}(i - j)$, in $\mathbb{Z}$

Integer operations in Faust are inspired by the semantics of operations on the n -bit two's complement representation of integer numbers; they are internal composition laws on the subset $[-2^{n-1}, 2^{n-1}-1]$ of $\mathbb{Z}$, with $n = 32$. For any integer binary operation $\times$ on $\mathbb{Z}$, the $\otimes$ operation is defined as: $i \otimes j = \text{normalize}(i \times j)$, with

$$\text{normalize}(i) = i - N \cdot \text{sign}(i) \cdot \left\lfloor \frac{|i| + N/2 + (\text{sign}(i) - 1)/2}{N} \right\rfloor,$$

where $N = 2^n$ and $\text{sign}(i) = 0$ if $i = 0$ and $i/|i|$ otherwise. Unary integer operations are defined likewise.

- The **dslar-mdoc/** directory may also include the following subdirectories:
 - **cpp/** for Faust compiled code;

¹Faust assumes that $\forall s \in S, \forall t \in \mathbb{Z}, s(t) = 0$ when $t < 0$.

- pdf/ which contains this document;
- src/ for all Faust sources used (even libraries);
- svg/ for block diagrams, encoded using the Scalable Vector Graphics format (<http://www.w3.org/Graphics/SVG/>);
- tex/ for the L^AT_EX source of this document.

4 Faust code listings

This section provides the listings of the Faust code used to generate this document, including dependencies.

Listing 1: ds1ar.dsp

```

1 import("seam.lib");
2 // Canonical DSP: sds.lar in seam.discipio.lib -the feedforward mono LAR
3 // processor of Di Scipio's LAR.pd (the Larsen loop is acoustic, external).
4 gate = checkbox("Power"); // system on/off, 2000 ms anti-click fade
5 drive = hslider("Drive", 1, 1, 4, 0.01); // audio pre-gain (Pd presets 1/2/4)
6 target = hslider("Target", 1, 0, 1, 0.001); // homeostat reference (Pd: - 1)
7 steep = hslider("Steepness", 40, 1, 80, 0.1); // homeostat exponent (Pd: pow 40)
8 tsmooth = hslider("Control smoothing [unit:ms]", 200, 1, 1000, 1);
9 tab1 = hslider("Loop delay [unit:ms]", 50, 1, 200, 0.1);
10 tab2 = hslider("Decorrelation [unit:ms]", 20, 1, 200, 0.1);
11 output = hslider("Output", 1, 0, 1, 0.001); // final VCA to host
12 process = sds.lar(gate, drive, target, steep, tsmooth, tab1, tab2, output);

```

Listing 2: seam.lib

```

1 declare name "Faust SEAM main lib";
2 declare version "0.2";
3 declare author "Giuseppe Silvi";
4 declare author "Davide Tedesco";
5 declare license "CC3";
6 // calling standard faust libraries
7 import("stdfaust.lib");
8 // ### standard faust lib extensions
9 san = library("seam.analyzers.lib");
10 sba = library("seam.basic.lib");
11 sma = library("seam.math.lib");
12 sfi = library("seam.filters.lib");
13 sre = library("seam.reverbs.lib");
14 sno = library("seam.noises.lib");
15 // ### general Ambisonics theory
16 sam = library("seam.ambisonics.lib");
17 sdw = library("seam.dwt.lib");
18 // ### stereophony and perception
19 sst = library("seam.stereophony.lib");
20 // shr = library("seam.hrtf.lib");
21 // ### CSound and MaxMsp object cloning
22 scs = library("seam.csound.lib");
23 scy = library("seam.cyclone.lib");
24 spd = library("seam.pdclone.lib");
25 // ### author specific literature
26 sds = library("seam.discipio.lib");

```

```

27 smg = library("seam.gerzon.lib");
28 slw = library("seam.linkwitz.lib");
29 // sln = library("seam.nono.lib");
30 // sgn = library("seam.nottoli.lib");
31 sjm = library("seam.moorer.lib");
32 scr = library("seam.roads.lib");
33 sms = library("seam.schroeder.lib");
34 sfv = library("seam.freeverb.lib");
35
36 sff = library("seam.ffunctions.lib");
37
38 // ### live electronics stuff
39 // sgu = library("seam.gui.lib");
40 // shw = library("seam.hardware.lib");
41 // smx = library("seam.mixer.lib");
42
43 // ### instrument specific literature
44 // svc = library("seam.vcs3.lib");

```

Listing 3: stdfaust.lib

```

1  /##### stdfaust.lib #####
2  // The purpose of this library is to give access to all the Faust standard libraries
3  // through a series of environments.
4  /#####
5
6  aa = library("aanl.lib");
7  sf = library("all.lib");
8  an = library("analyzers.lib");
9  ba = library("basics.lib");
10 co = library("compressors.lib");
11 db = library("debug.lib");
12 de = library("delays.lib");
13 dm = library("demos.lib");
14 dx = library("dx7.lib");
15 en = library("envelopes.lib");
16 fd = library("fds.lib");
17 fi = library("filters.lib");
18 ho = library("hoa.lib");
19 hy = library("hysteresis.lib");
20 it = library("interpolators.lib");
21 la = library("linearalgebra.lib");
22 ma = library("maths.lib");
23 mi = library("mi.lib");
24 mo = library("motion.lib");
25 ef = library("misceffects.lib");
26 no = library("noises.lib");
27 os = library("oscillators.lib");
28 pf = library("phaflangers.lib");
29 pl = library("platform.lib");
30 pm = library("physmodels.lib");
31 qu = library("quantizers.lib");
32 rm = library("reducemaps.lib");
33 re = library("reverbs.lib");
34 ro = library("routes.lib");
35 sp = library("spats.lib");
36 si = library("signals.lib");
37 so = library("soundfiles.lib");
38 sy = library("synths.lib");
39 ve = library("vaeffects.lib");
40 vl = library("version.lib");
41 wa = library("webaudio.lib");
42 wd = library("wdmodels.lib");

```

Listing 4: seam.discipio.lib

```

1 declare name "Agostino Di Scipio - AUDIBLE ECOSYSTEMICS n.2 Library";
2 import("seam.lib");
3 //----- DELAY ---
4 delMax = 2^16;
5 //Specific delay line with a maximum delay of 1 second
6 //delayfb(del,fb) = (+ : de.delay(ma.SR,d))-*(fb)
7 //with{
8 // d = min(0, del) : ba.sec2samp;
9 //};
10
11 //delayfb(del,fb) = (+ : de.delay(ma.SR,ba.sec2samp(del)))-*(fb);
12 delayfb(del,fb) = sre.dflcc(ma.SR,ba.sec2samp(del),fb);
13 //----- MAPPERS ---
14 mapsum(x,s,m) = s+(x*(m));
15 mapsub(x,s,m) = s-(x*(m));
16 mapmul(x,s,m) = m*(x+s);
17 mapcond(x) = select2(x>0.5, 1, (1-(max(0,(min(1,x)))))*2);
18 //----- INTEGRATOR ---
19 // returns the average absolute value over a specific time frame
20 // (one may use RMS measures, instead, or other amplitude-following methods);
21 // output range is [0, 1]
22 integrator(s) = an.abs_envelope_rect(s);
23 //----- LOCALMAX ---
24 // returns the maximum signal amplitude (absolute value) in a given time frame;
25 // frame duration is dynamically adjusted: the next frame duration is set at the
26 // end of the previous frame
27 localmax(x, frame) = x : ba.peakholder(frame*ma.SR); //CORRETTO
28 //----- SAMPLEREAD ---
29 /*
30 samplerread(var1,ratio, memchunk, sig) =
31 // delay frazionario * crossfade
32 de.fdelay(delMax,d,sig) * ma.fmin(d/xfade,1) +
33 de.fdelay(delMax,d+bufsize,sig) * (1-ma.fmin(d/xfade,1))
34 with {
35 bufsize = int(min(delMax, var1) * (memchunk) : si.smoo);
36 xfade = 128;
37 d = (1 - ratio : si.smoo) : (+ : +(bufsize) : fmod(_,bufsize)) ~ _;
38 };
39 */
40
41
42 sampleRead(memBuffer, ratio, maxChunk,x) =
43 it.fwtable(1,192000 * (memBuffer), .0, ba.period(memBuffer * ma.SR), x, rIdx)
44 with {
45 //clip the smallest chunk
46 memChunk(maxChunk) = (maxChunk : max(0.001, min(1))) * memBuffer * ma.SR;
47 rIdx = os.phasor(memChunk(maxChunk), (ma.SR / memChunk (maxChunk)) * ratio);
48 };
49
50 //process = os.osc(4) <: _, sampleRead(1,0.1,1);
51 //----- GRANULAR SAMPLING ---
52 // read sample sequences off subsequent buffer memory chunks, and envelopes the
53 // signal chunk with a pseudo-Gaussian envelope curve; the particular
54 // implementation should allow for time-stretching (slower memory pointer
55 // increments at grain level), as well as for "grain density" controls and
56 // slight random deviations ("jitter") on grain parameters; no frequency shift
57 // necessary
58
59 /*
60 Granular Sampling -
61 version with - Overlap Add To One - Granulator
62 for:
63 Agostino Di Scipio - AUDIBLE ECOSYSTEMICS
64 n.2a / Feedback Study (2003)
65 n.2b / Feedback Study, sound installation (2004).
66 */

```

```

67  /*
68  - mem.pointer is the pointer to the next location in the memory
69    buffer; in the present notation, it varies between -1 (beginning
70    of buffer) and 1 (end of buffer)
71  - mem.pointer : timeIndex1 - a signal between -1 and -0.5
72
73  - mem.pointer.jitter is a random deviation of the current
74    mem.pointer value; any viable method can be used to
75    calculate the current actual value of mem.pointer
76  - mem.pointer.jitter: (1 - memWriteDel1) / 100
77
78  - memWriteDel1 = a signal between 0 and 1
79
80  - grain.duration: 0.023 + ((1 - memWriteDel1) / 21) s
81
82  - grain.dur.jitter is a random deviation of the current
83    grain.duration value: the current actual grain duration =
84    grain.duration + (rnd grain.dur.jitter(0.1) grain.duration)
85  - with rnd = random value in the interval [-1, 1]
86  - grain.dur.jitter: 0.1 - constant value
87
88  - density: cntrlLev: a signal between 0 and 1 (1 max, 0 no grains)
89  */
90
91  declare name "granular_sampling for AUDIBLE ECOSYSTEMICS n.2";
92  declare version "xxx";
93  declare author "Luca Spanedda";
94  declare author "Dario Sanfilippo";
95  declare description "Realised on composer's instructions of the year 2017 edited in 'LAquila
96    , Italy";
97
98  primeNumbers(index) = ba.take(index , list)
99    with{
100      list = sma.primes;
101    };
102
103  noise(seed) = (+ (primeNumbers(seed + 1)) ~ *(1103515245)) / 2147483647;
104
105
106  grain(seed,var1,timeIndex,memWriteDel,cntrlLev,divDur,x) =
107  (hann(readingSegment) * buffer(bufferSize, readPtr, x)) : vdelay
108    with{
109
110      // density
111      _grainRate = (cntrlLev*(100-1))+1;
112      // target grain duration in seconds
113      _grainDuration = 0.023 + ((1 - memWriteDel) / divDur);
114      // target grain position in the buffer
115      _grainPosition = ((timeIndex)+1)/2;
116      // make sure to have decorrelated noises
117      // grain.dur.jitter: 0.1 - constant value
118      durationJitter = noise(2 * seed) * .1 + .1;
119      positionJitter = noise(2 * seed + 1) * (1 - memWriteDel) / 100;
120
121      // buffer size
122      bufferSize = var1 * ma.SR;
123      // hann window
124      hann(x) = sin(ma.PI * x) ^ 2.0;
125
126      // a phasor that read new params only when: y_1 < y_2
127      phasorLocking = loop ~ _
128        with {
129          loop(y_1) = ph , unlock
130          with{
131            y_2 = y_1';
132            ph = os.phasor(1, ba.sAndH(unlock, _grainRate));
133            unlock = (y_1 < y_2) + 1 - 1';

```

```

134     };
135 };
136
137 // two outputs of the phasor: phasor, trigger(y_1<y_2)
138 phasor = phasorLocking : _ , !;
139 unlocking = phasorLocking : ! , _;
140
141 // new param with lock function based on the phasor
142 lock(param) = ba.sAndH(unlocking, param);
143
144 // TO DO: wrap & receive param from AE2
145 grainPosition = lock(_grainPosition * positionJitter);
146 // TO DO: wrap & receive param from AE2
147 grainRate = lock(_grainRate);
148 // TO DO: wrap & receive param from AE2
149 grainDuration = lock(_grainDuration * durationJitter);
150
151 // maximum allowed grain duration in seconds
152 maxGrainDuration = 1.0 / grainRate;
153 // phase segment multiplication factor to complete a Hann cycle in the target duration
154 phasorSlopeFactor = maxGrainDuration / min(maxGrainDuration, grainDuration);
155
156
157 readingSegment = min(1.0, phasor * phasorSlopeFactor);
158
159 // decorrelation delay. Instead of 1 control w: hslider("decorrelation", 1, 0, 1, .001)
160 noisePadding = 1 * lock(noise(seed+3)) : abs;
161 vdelay(x) = x : de.sdelay(ma.SR, 1024, noisePadding * ma.SR);
162
163 // read pointer
164 readPtr = grainPosition * bufferSize + readingSegment
165         * (ma.SR / (grainRate * phasorSlopeFactor));
166
167 buffer(length, readPtr, x) = it.frwtable(5, 1920000, .0, writePtr, x, readPtr)
168     with{
169         writePtr = ba.period(length);
170     };
171 };
172
173 // par (how much grains/instances do you want?)
174 grainN(voices,var1,timeIndex,memWriteDel,cntrlLev,divDur,x) =
175     par(i, voices, grain(i,var1,timeIndex,memWriteDel,cntrlLev,divDur,(x/voices)));
176
177 granular_sampling(nVoices,var1,timeIndex,memWriteDel,cntrlLev,divDur,x) = x*0;
178 /*grainN(nVoices,var1,
179     timeIndexN(timeIndex),
180     memWriteDelN(memWriteDel),
181     cntrlLevN(cntrlLev),
182     divDur,x) :-> _
183     with{
184         timeIndexN(timeIndex) = timeIndex : min(1) : max(-1);
185         memWriteDelN(memWriteDel) = memWriteDel : min(1) : max(0);
186         cntrlLevN(cntrlLev) = cntrlLev : min(1) : max(0) ;
187     };*/
188
189
190 // DEMO GUI //////////////////////////////////////
191 // timeIndex1 - a signal between -1 and -0.5
192 GtimeIndex = hslider("timeIndex1", -1, -1, -0.5, .001);
193 // memWriteDel1 - a signal between 0 and 1
194 GmemWriteDel = hslider("memWriteDel1", 0, 0, 1, .001);
195 volume = hslider("volume", 0, 0, 10, .001);
196 // cntrlLev: a signal between 0 and 1 (1 max, 0 no grains)
197 GcntrlLev = hslider("cntrlLev", .5, 0, 1, .001);
198 // var1 distance (in meters) between the two farthest removed loudspeakers
199 Gvar1 = 3;
200 GnVoices = 2;
201 GdivDur = 21;

```

```

202 ///////////////////////////////////////////////////
203
204 //process = os.osc(1000) <: granular_sampling(GnVoices,Gvar1,GtimeIndex,GmemWriteDel,
      GcntrlLev,GdivDur);
205
206 //Triangular waveform wavetable
207
208 triwaveform(tablesize) =
209     1-abs(float(ba.period(tablesize) / tablesize) - 0.5)*(2);
210
211 osctri(freq) = rdtable(tablesize, triwaveform(tablesize), int(os.phasor(tablesize,freq)))
212 with {
213     tablesize = pl.tablesize;
214 };
215
216 //freq = 1000;
217 //process = osctri(freq), os.oscsin(freq);
218
219 //===== AGOSTINO DI SCIPIO - MODES OF INTERFERENCE (LAR) ===
220 // Self-regulating acoustic-feedback (Larsen) control law reconstructed from
221 // Di Scipio's LAR.pd (see the plugins/dslar study). The analysis branch measures
222 // the loop's Hann-weighted RMS level and maps it to a loop gain that holds the
223 // system at homeostatic equilibrium ("high enough to result into Larsen tones ...
224 // but not too high to cause clipping", Di Scipio 2006). Composes the Pd-object
225 // clones spd.env / spd.dbtorms; it does NOT re-implement them
226 // (memory project_pdcclone_library_architecture).
227
228 // Hann-weighted RMS at control rate = Pd's LAR analysis chain (env~ -> dbtorms).
229 // Algebraically dbtorms(powtodb(ms)) = sqrt(ms) (the dB round-trip cancels), so
230 // this IS the Hann-weighted RMS; we compose the real clones so env's control-rate
231 // step-hold stays faithful. Pd LAR uses env~ 2048, hence period = npoints/2 = 1024.
232 hannrms(npoints, period) = spd.env(npoints, period) : spd.dbtorms;
233
234 // Homeostatic loop-gain law (LAR analysis branch: -reference : abs : pow k):
235 // g = | r - reference | ^ k
236 // LAR.pd values: reference = 1, k = 40. Equilibrium sits at LOW r (r in [0, ~0.12]):
237 // as the room gets louder r rises toward the reference, |r-reference| -> 0 and the
238 // gain collapses (negative feedback); below equilibrium the gain approaches 1.
239 // r > reference re-opens |r-reference| (documented edge, LAR study $controllo).
240 homeostat(reference, k, r) = abs(r - reference) ^ k;
241
242 // Full LAR control brick: loop audio in -> homeostatic loop gain out.
243 // (The 200 ms control ramp and the audio loop itself belong to the dslar assembly.)
244 larsengain(npoints, period, reference, k) =
245     hannrms(npoints, period) : homeostat(reference, k);
246 // process = larsengain(64, 32, 1.0, 40.0); // compilable demo
247 // process = larsengain(2048, 1024, 1.0, 40.0); // LAR nominal (spec-only window)
248
249 //----- LAR brick -----
250 // Feedforward mono LAR processor (LAR.pd). The Larsen loop is ACOUSTIC and
251 // external (dac~ -> room -> mic -> adc~); the patch has NO internal feedback, so
252 // tab1 is a feedforward delay, not a recursion. See dslar Phase 3 spec + study.
253 // Delay buffers are sized once for the worst case (DELMAXMS at SRMAX); the read
254 // delay adapts to ma.SR at runtime (de.delay needs a compile-time buffer size).
255 SRMAX = 192000.0;
256 DELMAXMS = 200.0;
257 delms(ms) = de.delay(int(DELMAXMS*SRMAX/1000.0), int(ms*ma.SR/1000.0 + 0.5));
258 lar(gate, drive, ref, k, tsmooth, tab1, tab2, output, x) =
259     audio(fx) * analysisGain(fx) * output // loop multiplier, then output VCA
260 with {
261     fx = x * (gate : spd.line(2000.0)); // adc~ x fade, BEFORE the audio/analysis split
262     audio(s) = s : spd.hip(100.0) : *(drive) : delms(tab1);
263     analysisGain(s) = s : delms(tab2)
264         : larsengain(2048, 1024, ref, k)
265         : spd.line(tsmooth);
266 };
267 // process = lar(1.0, 1.0, 1.0, 40.0, 200.0, 50.0, 20.0, 1.0); // LAR nominal

```

Listing 5: seam.pdclone.lib

```

1 declare name "PdClone Faust Library";
2 declare version "0.1";
3 declare author "Giuseppe Silvi";
4 declare license "GPL-3.0";
5 //
6 // Faithful Faust clones of Pure Data signal objects, verified line-by-line
7 // against the Pd source (Miller Puckette). Sibling of seam.cyclone.lib (scy),
8 // which clones Max/gen~. Prefix: spd.
9 //
10 // Di Scipio's idioms (seam.discipio.lib, sds) COMPOSE these primitives; they
11 // do not re-implement the Pd math. See memory project_pdclone_library_architecture.
12 //
13 import("seam.lib");
14 //===== PURE DATA OBJECT CLONES =====
15 //=====
16 spd = library("seam.pdclone.lib");
17
18 //--- x_acoustics.c : amplitude <-> decibel converters -----
19 // Pd's dB is a 100 dB-offset scale: a power of 1.0 maps to 100 dB and silence
20 // clips to 0 (Pd never outputs -inf). LOGTEN = ln(10), so Pd's
21 // "100 + 10./LOGTEN*log(f)" is exactly "100 + 10*log10(f)".
22
23 //----- powtodb(p) ---
24 // powtodb(p) = 100 + 10*log10(p), clipped at >= 0 ; p <= 0 -> 0.
25 // Pd source x_acoustics.c (powtodb). Help patch: acoustics-help.pd (vendored under dslar/doc
26 // references/pd-help). Used by spd.env.
27 powtodb(p) = select2(p <= 0.0, max(0.0, 100.0 + 10.0*log10(p)), 0.0);
28 // process = powtodb; // powtodb(1) = 100 ; powtodb(0.01) = 80 ; powtodb(0) = 0
29
30 //----- rmstodb(r) ---
31 // rmstodb(r) = 100 + 20*log10(r), clipped at >= 0 ; r <= 0 -> 0.
32 rmstodb(r) = select2(r <= 0.0, max(0.0, 100.0 + 20.0*log10(r)), 0.0);
33 // process = rmstodb; // rmstodb(1) = 100 ; rmstodb(0) = 0
34
35 //----- dbtopow(d) ---
36 // dbtopow(d) = 10^((d-100)/10), d clamped to <= 870 ; d <= 0 -> 0.
37 dbtopow(d) = select2(d <= 0.0, pow(10.0, (min(870.0, d) - 100.0)/10.0), 0.0);
38 // process = dbtopow; // dbtopow(100) = 1 ; dbtopow(0) = 0
39
40 //----- dbtorms(d) ---
41 // dbtorms(d) = 10^((d-100)/20), d clamped to <= 485 ; d <= 0 -> 0.
42 // Inverse of the (env~ -> powtodb) chain: in the LAR loop, env~ -> dbtorms
43 // collapses to r = sqrt(Hann-weighted mean-square) = RMS. Pd source x_acoustics.c.
44 dbtorms(d) = select2(d <= 0.0, pow(10.0, (min(485.0, d) - 100.0)/20.0), 0.0);
45 // process = dbtorms; // dbtorms(100) = 1 ; dbtorms(80) = 0.1 ; dbtorms(0) = 0
46
47 //--- d_filter.c : hip~ -----
48 //----- hip(f) ---
49 // Pd source d_filter.c (hip~). Help patch: hip~-help.pd.
50 // One-pole/one-zero high-pass. Pd sighip_perform (compatibilitylevel > 43):
51 // coef = clip(1 - f*2pi/SR, 0, 1) (sighip_ft1)
52 // w[n] = x[n] + coef*w[n-1] (fi.pole(coef))
53 // y[n] = normal*(w[n] - w[n-1]) (fi.zero(1) then *normal)
54 // normal = (1 + coef)/2
55 // DIFFERS from fi.highpass (Butterworth biquad): this is Pd's exact topology.
56 // NOTE: Pd uses the truncated literal 3.14159 for pi (not M_PI); reproduced
57 // here so the coefficient is bit-identical to Pd's. f <= 0 gives coef = 1: Pd
58 // then copies the input verbatim (its coef==1 branch), and the pole-at-1 form
59 // below is algebraically identical (integrator then differencer) though not
60 // numerically robust at exact DC -- LAR uses hip~ 100, so coef < 1 always.
61 hip(f) = fi.pole(coef) : fi.zero(1) : *(normal)
62 with {
63   coef = 1.0 - f*(2.0*3.14159)/ma.SR : max(0.0) : min(1.0);
64   normal = (1.0 + coef)/2.0;
65 };
66 // process = hip(100); // 1-in/1-out DC-blocking loop high-pass (LAR: hip~ 100)

```

```

66 //--- d_ctl.c : env~ -----
67 //----- env(npoints, period) ---
68 // Pd source d_ctl.c (env~). Help patch: env~-help.pd.
69 // Hann-weighted RMS-power envelope follower, output in Pd dB (via powtodb).
70 //
71 // Pd splits env~ across two rates:
72 // * FULL RATE (env_tilde_perform) accumulates, over an npoints window,
73 //   result = sum_i buf[i] * x[n-i]^2 ,
74 //   buf[i] = (1 - cos(2pi*i/npoints)) / npoints (normalized Hann, sum = 1);
75 // * CONTROL RATE (env_tilde_tick, a clock callback every `period` samples)
76 //   outputs powtodb(result) and HOLDS it until the next tick.
77 //   (Pd default period = npoints/2.)
78 //
79 // The one-line spec is a sliding FIR, held at control rate:
80 // env = ( _ <: sum(i, npoints, ( _@i <: *) * (1-cos(2*3.14159*i/npoints))/npoints))
81 //       : powtodb : ba.sAndH(ba.pulse(period));
82 // clear, but sum() unrolls to `npoints` taps -- env(2048,*) takes minutes to compile.
83 // Pd does NOT do that: env_tilde_perform is OVERLAP-ADD (~L = npoints/period active
84 // Hann windows, O(L)/sample). We implement the overlap-add below; it is float-exact
85 // vs the FIR above AND vs the Pd env~ oracle (~4e-6 dB, summation-order rounding only)
86 // and compiles at env(2048,1024) in ~0.16 s. Derivation + A/B: the plugins/dslar study
87 // sketch doc/sketch/env_oa.dsp.
88 //
89 // Each lane runs a DOWN phase d that reaches 0 at its capture instant (n = off mod
90 // npoints); the L lanes are staggered by `period`, so their captures union to the
91 // n = 0 mod period grid -- Pd's control-rate clock. At d==0 the weight hann(k) sits on
92 // x(n-k), identical to the FIR sum. The step-hold is intrinsic (result changes only at a
93 // capture) -- the block-kernel behaviour of memory project_pd_vs_faust_block_validation,
94 // expressed directly. No `ondemand` needed: the win is the overlap-add STRUCTURE, not the
95 // rate primitive (ondemand stays an optional idiom for the per-hop powtodb).
96 //
97 // CONSTRAINT: npoints % period == 0 (integer overlap L). Pd rounds realperiod to a block
98 // multiple; here require exact divisibility (LAR env~ 2048 -> period 1024, L = 2).
99 // SR-independence (memory project_discipio_sr_independence) stays a porting concern: the
100 // C++ deliverable sizes npoints = round(fs*2048/44100) at prepare(fs). Pd uses the
101 // truncated literal 3.14159 for pi; reproduced. an.* followers do NOT match (rectangular /
102 // one-pole, 0 dB ref): new code, upstream candidate for GRAME (project_env_upstream_grame).
103 //
104 // `period` is explicit (Faust has no arity overloading); Pd's one-arg env~ npoints
105 // defaults period = npoints/2, so mirror that as env(npoints, npoints/2) at the call site.
106 env(npoints, period) = sqr : overlapAdd
107 with {
108   sqr(x) = x * x;
109   overlapAdd(x2) = ba.sAndH(anyCap, grabbed) : powtodb
110   with {
111     L = int(npoints / period); // overlap factor (LAR: 2048/1024 = 2)
112     wrapmod(a) = (a % npoints + npoints) % npoints;
113     up = step ~ _ with { step(c) = (c + 1) % npoints; }; // up(n) = (n+1)%npoints
114     d(off) = wrapmod(off + 1 - up); // d(off)==0 at n = off (mod npoints)
115     hann(p) = (1.0 - cos((2.0*3.14159*p)/npoints)) / npoints;
116     lane(off) = acc
117     with {
118       dd = d(off);
119       w = hann(dd);
120       newwin = dd == (npoints - 1); // first sample of a fresh window
121       acc = step ~ _ with { step(a) = select2(newwin, a + w*x2, w*x2); };
122     };
123     cap(off) = d(off) == 0; // capture instant for this lane
124     anyCap = par(1, L, cap(1*period)) :> _ > 0;
125     grabbed = par(1, L, lane(1*period) * cap(1*period)) :> _;
126   };
127 };
128 // process = env(2048, 1024); // LAR nominal: env~ 2048 (period 1024, L=2) -> level in dB
129 // process = env(64, 32); // small window used in the doc/sketch A/B
130
131 //----- line(ms, x) ---
132 // Pd source x_time.c (line, control-rate). Help patch: line-help.pd.

```

```

134 // Pd `line` ramps linearly toward its target and EMITS at a grain interval
135 // (DEFAULTLINEGRAIN = 20 ms), a control-rate staircase; it restarts from the
136 // current value on each new target (setval). Here ba.line is NOT used: the
137 // linear interp is rebuilt as Pd's v = setval + min(elapsed/R,1)*(target-setval)
138 // so the tick values match Pd's tau/R exactly, then sAndH on a 20 ms grain
139 // pulse imposes the staircase. Same idiom as spd.env (real math + control-rate
140 // hold). FAUST vs Pd: reproduces the grain staircase to float precision; the
141 // tick PHASE (Pd aligns ticks to message arrival, this to sample 0) is the
142 // documented residual, deferred to the block-kernel validation sub-project.
143 line(ms, x) = cont : ba.sAndH(ba.pulse(20.0 * ma.SR/1000.0))
144 with {
145     R = ms * ma.SR/1000.0; // ramp length in samples
146     chg = x != x'; // new target
147     cont = (loop ~ (_,_)) : (_, !) // continuous interp; keep v, drop counter
148     with {
149         loop(v, e) = nv, ne
150         with {
151             ne = ba.if(chg, 0, e + 1); // samples elapsed since change (0 at change)
152             sv = ba.sAndH(chg, v); // setval = output frozen at the change instant
153             nv = sv + min(ne/R, 1.0) * (x - sv);
154         };
155     };
156 };
157 // process = line(100.0); // 100 ms ramp, 20 ms grain staircase; vs Pd line oracle < 1e-6
158
159 //----- delread~ / delwrite~ (de.delay) ---
160 // Pd source d.delay.c. Help patch: delay-tilde-objects-help.pd.
161 // LAR's named lines are single-writer/single-reader (tab1 50 ms loop delay,
162 // tab2 20 ms decorrelation tap), so de.delay(n,d,x) = x@min(n,max(0,d)) is an
163 // exact match; no spd re-implementation is written. The Pd one-block minimum
164 // does not bind (50/20 ms >> 64 samples). Documented here for completeness so
165 // every Pd object LAR touches carries its dual source (C source + help patch).

```

Listing 6: basics.lib

```

1 //##### basics.lib #####
2 // Basics library. Its official prefix is `ba`.
3 //
4 // This library provides reusable building blocks for core DSP and Faust
5 // programming. It typically includes low-level utilities for math, routing,
6 // signal conditioning, timing, control, and helper components used across
7 // higher-level libraries.
8 //
9 // The Basics library is organized into 8 sections:
10 //
11 // * [Conversion Tools](#conversion-tools)
12 // * [Counters and Time/Tempo Tools](#counters-and-timetempo-tools)
13 // * [Array Processing/Pattern Matching](#array-processingpattern-matching)
14 // * [Function tabulation](#function-tabulation)
15 // * [Selectors (Conditions)](#selectors-conditions)
16 // * [Other](#other)
17 // * [Sliding Reduce](#sliding-reduce)
18 // * [Parallel Operators](#parallel-operators)
19 //
20 // ### References
21 //
22 // * <https://github.com/grame-cncm/faustlibraries/blob/master/basics.lib>
23 //#####
24
25 /*****
26 *****/
27 FAUST library file, GRAME section
28
29 Except where noted otherwise, Copyright (C) 2003-2017 by GRAME,

```

```

30 Centre National de Creation Musicale.
31 -----
32 GRAME LICENSE
33
34 This program is free software; you can redistribute it and/or modify
35 it under the terms of the GNU Lesser General Public License as
36 published by the Free Software Foundation; either version 2.1 of the
37 License, or (at your option) any later version.
38
39 This program is distributed in the hope that it will be useful,
40 but WITHOUT ANY WARRANTY; without even the implied warranty of
41 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
42 GNU Lesser General Public License for more details.
43
44 You should have received a copy of the GNU Lesser General Public
45 License along with the GNU C Library; if not, write to the Free
46 Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
47 02111-1307 USA.
48
49 EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
50 larger FAUST program which directly or indirectly imports this library
51 file and still distribute the compiled code generated by the FAUST
52 compiler, or a modified version of this compiled code, under your own
53 copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
54 grants you the right to freely choose the license for the resulting
55 compiled code. In particular the resulting compiled code has no obligation
56 to be LGPL or GPL. For example you are free to choose a commercial or
57 closed source license or any other license if you decide so.
58 *****
59 *****/
60
61 ma = library("maths.lib");
62 ro = library("routes.lib");
63 ba = library("basics.lib"); // for compatible copy/paste out of this file
64 fi = library("filters.lib");
65 it = library("interpolators.lib");
66 si = library("signals.lib");
67
68 declare name "Faust Basic Element Library";
69 declare version "1.22.0";
70
71 //=====Conversion Tools=====
72 //=====
73
74 //-----`ba.samp2sec`-----
75 // Converts a number of samples to a duration in seconds at the current sampling rate (see `
76   ma.SR`).
77 // `samp2sec` is a standard Faust function.
78 //
79 // ### Usage
80 //
81 // `samp2sec(n) : _
82 //   `
83 //
84 // Where:
85 //
86 // * `n`: number of samples
87 //
88 // ### Test
89 //
90 // ba = library("basics.lib");
91 // samp2sec_test = ba.samp2sec(512);
92 //
93 //-----
94 samp2sec(n) = n/ma.SR;
95
96

```

```

97 //-----`(ba.)sec2samp`-----
98 // Converts a duration in seconds to a number of samples at the current sampling rate (see `
    ma.SR`).
99 // `samp2sec` is a standard Faust function.
100 //
101 // #### Usage
102 //
103 // ```
104 // sec2samp(d) : _
105 // ```
106 //
107 // Where:
108 //
109 // * `d`: duration in seconds
110 //
111 // #### Test
112 //
113 // ba = library("basics.lib");
114 // sec2samp_test = ba.sec2samp(0.01);
115 //
116 //-----
117 sec2samp(d) = d*ma.SR;
118
119
120 //-----`(ba.)db2linear`-----
121 // dB-to-linear value converter. It can be used to convert an amplitude in dB to a linear
    gain [0-N].
122 // `db2linear` is a standard Faust function.
123 //
124 // #### Usage
125 //
126 // ```
127 // db2linear(l) : _
128 // ```
129 //
130 // Where:
131 //
132 // * `l`: amplitude in dB
133 //
134 // #### Test
135 //
136 // ba = library("basics.lib");
137 // db2linear_test = ba.db2linear(-6);
138 //
139 //-----
140 db2linear(l) = pow(10.0, l/20.0);
141
142
143 //-----`(ba.)linear2db`-----
144 // line-to-dB value converter. It can be used to convert a linear gain [0-N] to an amplitude
    in dB.
145 // `linear2db` is a standard Faust function.
146 //
147 // #### Usage
148 //
149 // ```
150 // linear2db(g) : _
151 // ```
152 //
153 // Where:
154 //
155 // * `g`: a linear gain
156 //
157 // #### Test
158 //
159 // ba = library("basics.lib");
160 // linear2db_test = ba.linear2db(0.5);
161 //

```

```

162 //-----
163 linear2db(g) = 20.0*log10(max(ma.MIN, g));
164
165
166 //-----`(ba.)lin2LogGain`-----
167 // Converts a linear gain (0-1) to a log gain (0-1).
168 //
169 // #### Usage
170 //
171 // ```
172 // lin2LogGain(n) : _
173 // ```
174 //
175 // Where:
176 //
177 // * `n`: the linear gain
178 //
179 // #### Test
180 // ```
181 // ba = library("basics.lib");
182 // lin2LogGain_test = ba.lin2LogGain(0.5);
183 // ```
184 //-----
185 lin2LogGain(n) = n*n;
186
187
188 //-----`(ba.)log2LinGain`-----
189 // Converts a log gain (0-1) to a linear gain (0-1).
190 //
191 // #### Usage
192 //
193 // ```
194 // log2LinGain(n) : _
195 // ```
196 //
197 // Where:
198 //
199 // * `n`: the log gain
200 //
201 // #### Test
202 // ```
203 // ba = library("basics.lib");
204 // log2LinGain_test = ba.log2LinGain(0.25);
205 // ```
206 //-----
207 log2LinGain(n) = sqrt(n);
208
209
210 // end GRAME section
211 //#####
212 //*****
213 FAUST library file, jos section
214
215 Except where noted otherwise, The Faust functions below in this
216 section are Copyright (C) 2003-2017 by Julius O. Smith III <jos@ccrma.stanford.edu>
217 ([jos](http://ccrma.stanford.edu/~jos/)), and released under the
218 (MIT-style) [STK-4.3](#stk-4.3-license) license.
219
220 The MarkDown comments in this section are Copyright 2016-2017 by Romain
221 Michon and Julius O. Smith III, and are released under the
222 [CCA4I](https://creativecommons.org/licenses/by/4.0/) license (TODO: if/when Romain agrees)
223
224 *****/
225
226 //-----`(ba.)tau2pole`-----
227 // Returns a real pole giving exponential decay.
228 // Note that t60 (time to decay 60 dB) is ~6.91 time constants.
229 // `tau2pole` is a standard Faust function.

```

```

230 //
231 // #### Usage
232 //
233 // ```
234 // _ : smooth(tau2pole(tau)) : _
235 // ```
236 //
237 // Where:
238 //
239 // * `tau`: time-constant in seconds
240 //
241 // tau2pole(tau) = exp(-1.0/(tau*ma.SR));
242 //
243 // #### Test
244 // ```
245 // ba = library("basics.lib");
246 // tau2pole_test = ba.tau2pole(0.01);
247 // ```
248 //-----
249 tau2pole(tau) = ba.if(clipCond, 0.0, exp(-1.0/(tauCenterClipped*float(ma.SR))))
250 with {
251     clipCond = abs(tau)<ma.EPSILON;
252     tauCenterClipped = ba.if(clipCond, 1.0, tau); // 1.0 can be any nonzero value (not used)
253 };
254
255
256 //-----`ba.)pole2tau`-----
257 // Returns the time-constant, in seconds, corresponding to the given real,
258 // positive pole in (0-1).
259 // `pole2tau` is a standard Faust function.
260 //
261 // #### Usage
262 //
263 // ```
264 // pole2tau(pole) : _
265 // ```
266 //
267 // Where:
268 //
269 // * `pole`: the pole
270 //
271 // #### Test
272 // ```
273 // ba = library("basics.lib");
274 // pole2tau_test = ba.pole2tau(0.9);
275 // ```
276 //-----
277 pole2tau(pole) = -1.0/(log(max(ma.MIN, pole))*ma.SR);
278
279
280 //-----`ba.)midikey2hz`-----
281 // Converts a MIDI key number to a frequency in Hz (MIDI key 69 = A440).
282 // `midikey2hz` is a standard Faust function.
283 //
284 // #### Usage
285 //
286 // ```
287 // midikey2hz(mk) : _
288 // ```
289 //
290 // Where:
291 //
292 // * `mk`: the MIDI key number
293 //
294 // #### Test
295 // ```
296 // ba = library("basics.lib");
297 // midikey2hz_test = ba.midikey2hz(60);

```

```

298 // ``
299 //-----
300 midikey2hz(mk) = 440.0*pow(2.0, (mk-69.0)/12.0);
301
302
303 //-----`(ba.)hz2midikey`-----
304 // Converts a frequency in Hz to a MIDI key number (MIDI key 69 = A440).
305 // `hz2midikey` is a standard Faust function.
306 //
307 // #### Usage
308 //
309 // ``
310 // hz2midikey(freq) : _
311 // ``
312 //
313 // Where:
314 //
315 // * `freq`: frequency in Hz
316 //
317 // #### Test
318 //
319 // ba = library("basics.lib");
320 // hz2midikey_test = ba.hz2midikey(440);
321 // ``
322 //-----
323 hz2midikey(freq) = 12.0*ma.log2(freq/440.0) + 69.0;
324
325
326 //-----`(ba.)semi2ratio`-----
327 // Converts semitones in a frequency multiplicative ratio.
328 // `semi2ratio` is a standard Faust function.
329 //
330 // #### Usage
331 //
332 // ``
333 // semi2ratio(semi) : _
334 // ``
335 //
336 // Where:
337 //
338 // * `semi`: number of semitone
339 //
340 // #### Test
341 //
342 // ba = library("basics.lib");
343 // semi2ratio_test = ba.semi2ratio(7);
344 // ``
345 //-----
346 semi2ratio(semi) = pow(2.0, semi/12.0);
347
348
349 //-----`(ba.)ratio2semi`-----
350 // Converts a frequency multiplicative ratio in semitones.
351 // `ratio2semi` is a standard Faust function.
352 //
353 // #### Usage
354 //
355 // ``
356 // ratio2semi(ratio) : _
357 // ``
358 //
359 // Where:
360 //
361 // * `ratio`: frequency multiplicative ratio
362 //
363 // #### Test
364 //
365 // ba = library("basics.lib");

```

```

366 // ratio2semi_test = ba.ratio2semi(2.0);
367 // ``
368 //-----
369 ratio2semi(ratio) = 12.0*log(ratio)/log(2.0);
370
371
372 //-----`(ba.)cent2ratio`-----
373 // Converts cents in a frequency multiplicative ratio.
374 //
375 // #### Usage
376 //
377 // ``
378 // cent2ratio(cent) : _
379 // ``
380 //
381 // Where:
382 //
383 // * `cent`: number of cents
384 //
385 // #### Test
386 // ``
387 // ba = library("basics.lib");
388 // cent2ratio_test = ba.cent2ratio(100);
389 // ``
390 //-----
391 cent2ratio(cent) = pow(2.0, cent/1200.0);
392
393
394 //-----`(ba.)ratio2cent`-----
395 // Converts a frequency multiplicative ratio in cents.
396 //
397 // #### Usage
398 //
399 // ``
400 // ratio2cent(ratio) : _
401 // ``
402 //
403 // Where:
404 //
405 // * `ratio`: frequency multiplicative ratio
406 //
407 // #### Test
408 // ``
409 // ba = library("basics.lib");
410 // ratio2cent_test = ba.ratio2cent(1.5);
411 // ``
412 //-----
413 ratio2cent(ratio) = 1200.0*log(ratio)/log(2.0);
414
415
416 //-----`(ba.)pianokey2hz`-----
417 // Converts a piano key number to a frequency in Hz (piano key 49 = A440).
418 //
419 // #### Usage
420 //
421 // ``
422 // pianokey2hz(pk) : _
423 // ``
424 //
425 // Where:
426 //
427 // * `pk`: the piano key number
428 //
429 // #### Test
430 // ``
431 // ba = library("basics.lib");
432 // pianokey2hz_test = ba.pianokey2hz(49);
433 // ``

```

```

434 //-----
435 pianokey2hz(pk) = 440.0*pow(2.0, (pk-49.0)/12.0);
436
437
438 //-----`ba.)hz2pianokey`-----
439 // Converts a frequency in Hz to a piano key number (piano key 49 = A440).
440 //
441 // #### Usage
442 //
443 // ```
444 // hz2pianokey(freq) : _
445 // ```
446 //
447 // Where:
448 //
449 // * `freq`: frequency in Hz
450 //
451 // #### Test
452 // ```
453 // ba = library("basics.lib");
454 // hz2pianokey_test = ba.hz2pianokey(440);
455 // ```
456 //-----
457 hz2pianokey(freq) = 12.0*ma.log2(freq/440.0) + 49.0;
458
459
460 // end jos section
461 //#####
462 //*****
463 FAUST library file, GRAME section 2
464 //*****/
465
466 //=====Counters and Time/Tempo Tools=====
467 //=====
468
469 //-----`ba.)counter`-----
470 // Starts counting 0, 1, 2, 3..., and raise the current integer value
471 // at each upfront of the trigger.
472 //
473 // #### Usage
474 //
475 // ```
476 // counter(trig) : _
477 // ```
478 //
479 // Where:
480 //
481 // * `trig`: the trigger signal, each upfront will move the counter to the next integer
482 //
483 // #### Test
484 // ```
485 // ba = library("basics.lib");
486 // counter_test = ba.counter(button("trig"));
487 // ```
488 //-----
489 declare counter author "Stephane Letz";
490
491 counter(trig) = upfront(trig) : + ~ _ with { upfront(x) = x > x'; };
492
493
494 //-----`ba.)countdown`-----
495 // Starts counting down from n included to 0. While trig is 1 the output is n.
496 // The countdown starts with the transition of trig from 1 to 0. At the end
497 // of the countdown the output value will remain at 0 until the next trig.
498 // `countdown` is a standard Faust function.
499 //
500 // #### Usage
501 //

```

```

502 // ---
503 // countdown(n,trig) : _
504 // ---
505 //
506 // Where:
507 //
508 // * `n`: the starting point of the countdown
509 // * `trig`: the trigger signal (1: start at `n`; 0: decrease until 0)
510 //
511 // #### Test
512 // ---
513 // ba = library("basics.lib");
514 // countdown_test = ba.countdown(8, button("trig"));
515 // ---
516 //-----
517 countdown(n, trig) = \c.(if(trig>0, n, max(0, c-1))) ~ _;
518
519
520 //-----`(ba.)countup`-----
521 // Starts counting up from 0 to n included. While trig is 1 the output is 0.
522 // The countup starts with the transition of trig from 1 to 0. At the end
523 // of the countup the output value will remain at n until the next trig.
524 // `countup` is a standard Faust function.
525 //
526 // #### Usage
527 //
528 // ---
529 // countup(n,trig) : _
530 // ---
531 //
532 // Where:
533 //
534 // * `n`: the maximum count value
535 // * `trig`: the trigger signal (1: start at 0; 0: increase until `n`)
536 //
537 // #### Test
538 // ---
539 // ba = library("basics.lib");
540 // countup_test = ba.countup(8, ba.pulse(16));
541 // ---
542 //-----
543 countup(n, trig) = \c.(if(trig>0, 0, min(n, c+1))) ~ _;
544
545
546 //-----`(ba.)sweep`-----
547 // Counts from 0 to `period-1` repeatedly, generating a
548 // sawtooth waveform, like `os.lf_rawsaw`,
549 // starting at 1 when `run` transitions from 0 to 1.
550 // Outputs zero while `run` is 0.
551 //
552 // #### Usage
553 //
554 // ---
555 // sweep(period,run) : _
556 // ---
557 //
558 // Where:
559 //
560 // * `period`: sweep length in samples
561 // * `run`: enable signal, non-zero while the sweep runs
562 //
563 // #### Test
564 // ---
565 // ba = library("basics.lib");
566 // sweep_test = ba.sweep(64, checkbox("run"));
567 // ---
568 //-----
569 declare sweep author "Jonatan Liljedahl";

```

```

570 sweep = %(int(*:max(1)))~+(1);
571
572
573
574 //-----`(ba.)time`-----
575 // A simple counter that produces the sequence of 0,1,2...N integer values.
576 // `time` is a standard Faust function.
577 //
578 // #### Usage
579 //
580 // ```
581 // time : _
582 // ```
583 //
584 // #### Test
585 // ```
586 // ba = library("basics.lib");
587 // time_test = ba.time;
588 // ```
589 //-----
590 time = +(1)~_ : mem;
591
592
593 //-----`(ba.)ramp`-----
594 // A linear ramp with a slope of '(+/-)1/n' samples to reach the next target value.
595 //
596 // #### Usage
597 //
598 // ```
599 // _ : ramp(n) : _
600 // ```
601 // Where:
602 //
603 // * `n`: number of samples to increment/decrement the value by one
604 //
605 // #### Test
606 // ```
607 // ba = library("basics.lib");
608 // os = library("oscillators.lib");
609 // ramp_test = os.osc(1) : ba.ramp(256);
610 // ```
611 //-----
612 ramp = case {
613   (0) => _;
614   (n) => \(y,x).(if(y+1.0/n < x, y+1.0/n, if(y-1.0/n > x, y-1.0/n, x))) ~ _;
615 };
616
617
618 //-----`(ba.)line`-----
619 // A ramp interpolator that generates a linear transition to reach a target value:
620 //
621 // - the interpolation process restarts each time a new and distinct input value is received
622 // - it utilizes 'n' samples to achieve the transition to the target value
623 // - after reaching the target value, the output value is maintained.
624 //
625 // #### Usage
626 //
627 // ```
628 // _ : line(n) : _
629 // ```
630 // Where:
631 //
632 // * `n`: number of samples to reach the new target received at its input
633 //
634 // #### Test
635 // ```
636 // ba = library("basics.lib");
637 // os = library("oscillators.lib");

```

```

638 // line_test = os.osc(1) : ba.line(256);
639 // ```
640 //-----
641 line(n, x) = state ~ (_,_) : !, _
642 with {
643     state(t, c) = nt, nc
644     with {
645         nt = ba.if(x != x', n, t-1);
646         nc = ba.if(nt > 0, c + (x - c)/nt, x);
647     };
648 };
649
650
651 //-----`(ba.)tempo`-----
652 // Converts a tempo in BPM into a number of samples.
653 //
654 // #### Usage
655 //
656 // ```
657 // tempo(t) : _
658 // ```
659 //
660 // Where:
661 //
662 // * `t`: tempo in BPM
663 //
664 // #### Test
665 // ```
666 // ba = library("basics.lib");
667 // tempo_test = ba.temp(120);
668 // ```
669 //-----
670 tempo(t) = (60*ma.SR)/t;
671
672
673 //-----`(ba.)period`-----
674 // Basic sawtooth wave of period `p`.
675 //
676 // #### Usage
677 //
678 // ```
679 // period(p) : _
680 // ```
681 //
682 // Where:
683 //
684 // * `p`: period as a number of samples
685 //
686 // NOTE: may be this should go in oscillators.lib
687 // #### Test
688 // ```
689 // ba = library("basics.lib");
690 // period_test = ba.period(64);
691 // ```
692 //-----
693 period(p) = %(int(p))~+(1');
694
695
696 //-----`(ba.)spulse`-----
697 // Produces a single pulse of n samples when trig goes from 0 to 1.
698 //
699 // #### Usage
700 //
701 // ```
702 // spulse(n,trig) : _
703 // ```
704 //
705 // Where:

```

```

706 //
707 // * `n`: pulse length as a number of samples
708 // * `trig`: the trigger signal (1: start the pulse)
709 //
710 // #### Test
711 // ```
712 // ba = library("basics.lib");
713 // spulse_test = ba.spulse(32, button("trig"));
714 // ```
715 //-----
716 spulse(n, trig) = trig : trigger(n)
717 with {
718   upfront(x) = x > x'; // detect rising edge
719   reset = upfront(trig); // reset signal
720   decay(n, x) = ba.if(reset, 0, x - (x>0.0)/n); // decay the pulse, possibly using reset to
721   restart the pulse
722   release(n) = + ~ decay(n);
723   trigger(n) = upfront : release(n) : >(0.0);
724 }
725 //-----`(ba.)pulse`-----
726 // Pulses (like 10000) generated at period `p`.
727 //
728 // #### Usage
729 //
730 // ```
731 // pulse(p) : _
732 // ```
733 //
734 // Where:
735 //
736 // * `p`: period as a number of samples
737 //
738 // NOTE: may be this should go in oscillators.lib
739 // #### Test
740 // ```
741 // ba = library("basics.lib");
742 // pulse_test = ba.pulse(64);
743 // ```
744 //-----
745 pulse(p) = period(p) : \ (x).(x <= x');
746
747 //-----`(ba.)pulsen`-----
748 // Pulses (like 11110000) of length `n` generated at period `p`.
749 //
750 // #### Usage
751 //
752 // ```
753 // pulsen(n,p) : _
754 // ```
755 //
756 // Where:
757 //
758 // * `n`: pulse length as a number of samples
759 // * `p`: period as a number of samples
760 //
761 // NOTE: may be this should go in oscillators.lib
762 // #### Test
763 // ```
764 // ba = library("basics.lib");
765 // pulsen_test = ba.pulsen(8, 64);
766 // ```
767 //-----
768 pulsen(n,p) = period(p)<n;
769
770
771 //-----`(ba.)cycle`-----
772

```

```

773 // Split nonzero input values into `n` cycles.
774 //
775 // #### Usage
776 //
777 // ```
778 // _ : cycle(n) : si.bus(n)
779 // ```
780 //
781 // Where:
782 //
783 // * `n`: the number of cycles/output signals
784 //
785 // #### Test
786 // ```
787 // ba = library("basics.lib");
788 // cycle_test = button("gate") : ba.cycle(3);
789 // ```
790 //-----
791 declare cycle author "Mike Olsen";
792
793 cycle(n) = _ <: par(i,n,resetCtr(n,(i+1)));
794
795
796 //-----`(ba.)beat`-----
797 // Pulses at tempo `t` in BPM.
798 // `beat` is a standard Faust function.
799 //
800 // #### Usage
801 //
802 // ```
803 // beat(t) : _
804 // ```
805 //
806 // Where:
807 //
808 // * `t`: tempo in BPM
809 //
810 // #### Test
811 // ```
812 // ba = library("basics.lib");
813 // beat_test = ba.beat(120);
814 // ```
815 //-----
816 beat(t) = pulse(tempo(t));
817
818
819 //-----`(ba.)pulse_countup`-----
820 // Starts counting up pulses. While trig is 1 the output is
821 // counting up, while trig is 0 the counter is reset to 0.
822 //
823 // #### Usage
824 //
825 // ```
826 // _ : pulse_countup(trig) : _
827 // ```
828 //
829 // Where:
830 //
831 // * `trig`: the trigger signal (1: start at next pulse; 0: reset to 0)
832 //
833 // #### Test
834 // ```
835 // ba = library("basics.lib");
836 // pulse_countup_test = ba.pulse_countup(1) + 0.001;
837 // ```
838 //-----
839 declare pulse_countup author "Vince";
840

```

```

841 pulse_countup(trig) = + ~ _ * trig;
842
843
844 //-----`(ba.)pulse_countdown`-----
845 // Starts counting down pulses. While trig is 1 the output is
846 // counting down, while trig is 0 the counter is reset to 0.
847 //
848 // #### Usage
849 //
850 // ```
851 // _ : pulse_countdown(trig) : _
852 // ```
853 //
854 // Where:
855 //
856 // * `trig`: the trigger signal (1: start at next pulse; 0: reset to 0)
857 //
858 // #### Test
859 // ```
860 // ba = library("basics.lib");
861 // pulse_countdown_test = ba.pulse_countdown(1) + 0.001;
862 // ```
863 //-----
864 declare pulse_countdown author "Vince";
865
866 pulse_countdown(trig) = - ~ _ * trig;
867
868
869 //-----`(ba.)pulse_countup_loop`-----
870 // Starts counting up pulses from 0 to n included. While trig is 1 the output is
871 // counting up, while trig is 0 the counter is reset to 0. At the end
872 // of the countup (n) the output value will be reset to 0.
873 //
874 // #### Usage
875 //
876 // ```
877 // _ : pulse_countup_loop(n,trig) : _
878 // ```
879 //
880 // Where:
881 //
882 // * `n`: the highest number of the countup (included) before reset to 0
883 // * `trig`: the trigger signal (1: start at next pulse; 0: reset to 0)
884 //
885 // #### Test
886 // ```
887 // ba = library("basics.lib");
888 // pulse_countup_loop_test = ba.pulse_countup_loop(4, 1) + 0.001;
889 // ```
890 //-----
891 declare pulse_countup_loop author "Vince";
892
893 pulse_countup_loop(n, trig) = + ~ cond(n)*trig
894 with {
895   cond(n, x) = x * (x <= n);
896 };
897
898
899 //-----`(ba.)pulse_countdown_loop`-----
900 // Starts counting down pulses from 0 to n included. While trig is 1 the output
901 // is counting down, while trig is 0 the counter is reset to 0. At the end
902 // of the countdown (n) the output value will be reset to 0.
903 //
904 // #### Usage
905 //
906 // ```
907 // _ : pulse_countdown_loop(n,trig) : _
908 // ```

```

```

909 //
910 // Where:
911 //
912 // * `n`: the highest number of the countup (included) before reset to 0
913 // * `trig`: the trigger signal (1: start at next pulse; 0: reset to 0)
914 //
915 // #### Test
916 // ```
917 // ba = library("basics.lib");
918 // pulse_countdown_loop_test = ba.pulse_countdown_loop(4, 1) + 0.001;
919 // ```
920 //-----
921 declare pulse_countdown_loop author "Vince";
922
923 pulse_countdown_loop(n, trig) = - ~ cond(n)*trig
924 with {
925   cond(n, x) = x * (x >= n);
926 };
927
928
929 //-----`(ba.)resetCtr`-----
930 // Function that lets through the mth impulse out of
931 // each consecutive group of `n` impulses.
932 //
933 // #### Usage
934 //
935 // ```
936 // _ : resetCtr(n,m) : _
937 // ```
938 //
939 // Where:
940 //
941 // * `n`: the total number of impulses being split
942 // * `m`: index of impulse to allow to be output
943 //
944 // #### Test
945 // ```
946 // ba = library("basics.lib");
947 // resetCtr_test = ba.pulse(16) : ba.resetCtr(4, 2);
948 // ```
949 //-----
950 declare resetCtr author "Mike Olsen";
951
952 resetCtr(n,m) = _ <: (_,pulse_countup_loop(n-1,1)) : (_,(_==m)) : *;
953
954
955 //=====Array Processing/Pattern Matching=====
956 //=====
957
958 //-----`(ba.)count`-----
959 // Count the number of elements of list l.
960 // `count` is a standard Faust function.
961 //
962 // #### Usage
963 //
964 // ```
965 // count(l)
966 // count((10,20,30,40)) -> 4
967 // ```
968 //
969 // Where:
970 //
971 // * `l`: list of elements
972 //
973 // #### Test
974 // ```
975 // ba = library("basics.lib");
976 // count_test = ba.count((10,20,30,40));

```

```

977 // ```
978 //-----
979 count((xs, xxs)) = 1 + count(xxs);
980 count(xx) = 1;
981
982
983 //-----`(ba.)take`-----
984 // Take an element from a list.
985 // `take` is a standard Faust function.
986 //
987 // #### Usage
988 //
989 // ```
990 // take(P,l)
991 // take(3,(10,20,30,40)) -> 30
992 // ```
993 //
994 // Where:
995 //
996 // * `P`: position (int, known at compile time, P > 0)
997 // * `l`: list of elements
998 //
999 // #### Test
1000 //
1001 // ba = library("basics.lib");
1002 // take_test = ba.take(3, (10,20,30,40));
1003 //
1004 //-----
1005 take(1, (xs, xxs)) = xs;
1006 take(1, xs) = xs;
1007 take(N, (xs, xxs)) = take(N-1, xxs);
1008
1009 //-----`(ba.)pick`-----
1010 // Pick the nth element from a list.
1011 // Similar to `ba.take(n+1,l)` but faster and more powerful.
1012 //
1013 // #### Usage
1014 //
1015 // ```
1016 // pick(l,n) : _
1017 // ```
1018 //
1019 // Where:
1020 //
1021 // * `l`: list of elements
1022 // * `n`: index of element to pick, compile time constant.
1023 //       if n < 0 or n >= length of `l`, `pick()` outputs 0.
1024 //
1025 // #### Example test program
1026 //
1027 // ```
1028 // pick((10,20,30,40), 2) -> 30
1029 // ```
1030 //
1031 // ```
1032 // pick(si.bus(3), 1) // same as !,_,!
1033 // ```
1034 //
1035 // while `ba.take(2, si.bus(3))` acts as `_,_`.
1036 //
1037 // Unlike `take()`, `pick()` always flattens the list, so
1038 // `pick((10,(20,30),40), 1)` outputs `20`, not `20,30`.
1039 //
1040 // #### Test
1041 //
1042 // ba = library("basics.lib");
1043 // pick_test = ba.pick((10,20,30,40), 2);
1044 //

```

```

1045 //-----
1046 pick(1,n) = 1 : route(outputs(1),1, n+1, 1);
1047
1048 //-----`ba.`pickN`-----
1049 // Select the inputs listed in `0` among `N` at compile time.
1050 //
1051 // #### Usage
1052 //
1053 // ```
1054 // si.bus(N) : pickN(N,0) : si.bus(outputs(0))
1055 // ```
1056 //
1057 // Where:
1058 //
1059 // * `N`: number of inputs, compile time constant
1060 // * `0`: list of the inputs to select, compile time constants
1061 //
1062 // #### Example test program
1063 //
1064 // ```
1065 // pickN(4,2) : _ // same as selector(2,4) but faster
1066 // ```
1067 //
1068 // ```
1069 // pick(4,(1,3)) : _,_ // same as !,!,_
1070 // ```
1071 //
1072 // ```
1073 // pickN(4,(1,3), (10,20,30,40)) -> (20,40)
1074 // ```
1075 //
1076 // ```
1077 // process = pickN(2, (1,0,0,1)) // same as `process(x,y) = y,x,x,y`
1078 // ```
1079 //
1080 // #### Test
1081 //
1082 // ```
1083 // ba = library("basics.lib");
1084 // pickN_test = (1,2,3,4) : ba.pickN(4, (0,2));
1085 // ```
1086 //-----
1087 pickN(N,0) = route(N,outputs(0), par(o,outputs(0), pick(0,o)+1, o+1));
1088
1089 //-----`ba.`subseq`-----
1090 // Extract a part of a list.
1091 //
1092 // #### Usage
1093 //
1094 // ```
1095 // subseq(1, P, N)
1096 // subseq((10,20,30,40,50,60), 1, 3) -> (20,30,40)
1097 // subseq((10,20,30,40,50,60), 4, 1) -> 50
1098 // ```
1099 //
1100 // Where:
1101 //
1102 // * `l`: list
1103 // * `P`: start point (int, known at compile time, 0: begin of list)
1104 // * `N`: number of elements (int, known at compile time)
1105 //
1106 // #### Note:
1107 //
1108 // Faust doesn't have proper lists. Lists are simulated with parallel
1109 // compositions and there is no empty list.
1110 //
1111 // #### Test
1112 //
1113 // ```
1114 // ba = library("basics.lib");

```

```

1113 // subseq_test = ba.subseq((10,20,30,40,50), 1, 3);
1114 // ...
1115 //-----
1116 subseq((head, tail), 0, 1) = head;
1117 subseq((head, tail), 0, N) = head, subseq(tail, 0, N-1);
1118 subseq((head, tail), P, N) = subseq(tail, P-1, N);
1119 subseq(head, 0, N) = head;
1120
1121
1122 //=====Function tabulation=====
1123 // The purpose of function tabulation is to speed up the computation of heavy functions over
// an interval,
1124 // so that the computation at runtime can be faster than directly using the function.
1125 // Two techniques are implemented:
1126 //
1127 // * `tabulate` computes the function in a table and read the points using interpolation. `
// tabulateNd` is the N dimensions version of `tabulate`
1128 //
1129 // * `tabulate_chebychev` uses Chebyshev polynomial approximation
1130 //
1131 // #### Comparison program example
1132 // ...
1133 ///* Both tabulate() and tabulate_chebychev() create rdtbl of size = 200, both use */
1134 ///* cubic polynomials, so this comparison is more or less fair. */
1135 // process = line(50000, r0, r1) <: FX-tb,FX-ch : par(i, 2, maxerr)
1136 // with {
1137 //   C = 0;
1138 //   FX = sin;
1139 //   NX = 50;
1140 //   CD = 3;
1141 //   r0 = 0;
1142 //   r1 = ma.PI;
1143 //   tb(x) = ba.tabulate(C, FX, NX*(CD+1), r0, r1, x).cub;
1144 //   ch(x) = ba.tabulate_chebychev(C, FX, NX, CD, r0, r1, x);
1145 //   maxerr = abs : max ~ _;
1146 //   line(n, x0, x1) = x0 + (ba.time%n)/n * (x1-x0);
1147 // };
1148 // ...
1149
1150 //-----`ba.`tabulate`-----
1151 // Tabulate a 1D function over the range [r0, r1] for access via nearest-value, linear, cubic
// interpolation.
1152 // In other words, the uniformly tabulated function can be evaluated using interpolation of
// order 0 (none), 1 (linear), or 3 (cubic).
1153 //
1154 // #### Usage
1155 //
1156 // ...
1157 // tabulate(C, FX, S, r0, r1, x).(val|lin|cub) : _
1158 // ...
1159 //
1160 // Where:
1161 //
1162 // * `C`: whether to dynamically force the `x` value to the range [r0, r1]: 1 forces the
// check, 0 deactivates it (constant numerical expression)
1163 // * `FX`: unary function Y=F(X) with one output (scalar function of one variable)
1164 // * `S`: size of the table in samples (constant numerical expression)
1165 // * `r0`: minimum value of argument x
1166 // * `r1`: maximum value of argument x
1167 // * `x`: input value used to read the tabulated function
1168 //
1169 // ...
1170 // tabulate(C, FX, S, r0, r1, x).val uses the value in the table closest to x
1171 // ...
1172 //
1173 // ...
1174 // tabulate(C, FX, S, r0, r1, x).lin evaluates at x using linear interpolation between the
// closest stored values

```

```

1175 // ...
1176 //
1177 // ...
1178 // tabulate(C, FX, S, r0, r1, x).cub evaluates at x using cubic interpolation between the
        closest stored values
1179 // ...
1180 //
1181 // #### Example test program
1182 //
1183 // ...
1184 // midikey2hz(mk) = ba.tabulate(1, ba.midikey2hz, 512, 0, 127, mk).lin;
1185 // process = midikey2hz(ba.time), ba.midikey2hz(ba.time);
1186 // ...
1187 //
1188 // #### Test
1189 // ...
1190 // ba = library("basics.lib");
1191 // tabulate_test = ba.tabulate(1, ba.midikey2hz, 128, 0, 127, 60).lin;
1192 // ...
1193 //-----
1194 tabulate(C, FX, S, r0, r1, x) = environment {
1195
1196     // Maximum index to access
1197     mid = S-1;
1198
1199     // Create the table
1200     wf = r0 + float(rid(ba.time, 1))*(r1-r0)/float(mid) : FX;
1201
1202     // Prepare the 'float' table read index
1203     id = (x-r0)/(r1-r0)*mid;
1204
1205     // Limit the table read index in [0, mid] if C = 1
1206     rid(x, 0) = x;
1207     rid(x, 1) = max(0, min(x, mid));
1208
1209     // Tabulate an unary 'FX' function on a range [r0, r1]
1210     val = y0 with { y0 = rdtbl(S, wf, rid(int(id+0.5), C)); };
1211
1212     // Tabulate an unary 'FX' function over the range [r0, r1] with linear interpolation
1213     lin = it.interpolate_linear(d,y0,y1)
1214     with {
1215         x0 = int(id);
1216         x1 = x0+1;
1217         d = id-x0;
1218         y0 = rdtbl(S, wf, rid(x0, C));
1219         y1 = rdtbl(S, wf, rid(x1, C));
1220     };
1221
1222     // Tabulate an unary 'FX' function over the range [r0, r1] with cubic interpolation
1223     cub = it.interpolate_cubic(d,y0,y1,y2,y3)
1224     with {
1225         x0 = x1-1;
1226         x1 = int(id);
1227         x2 = x1+1;
1228         x3 = x2+1;
1229         d = id-x1;
1230         y0 = rdtbl(S, wf, rid(x0, C));
1231         y1 = rdtbl(S, wf, rid(x1, C));
1232         y2 = rdtbl(S, wf, rid(x2, C));
1233         y3 = rdtbl(S, wf, rid(x3, C));
1234     };
1235 };
1236
1237 declare tabulate author "Stephane Letz";
1238
1239
1240 //-----`ba.)tabulate_chebychev`-----
1241 // Tabulate a 1D function over the range [r0, r1] for access via Chebyshev polynomial

```

```

1242 // approximation.
1243 // In contrast to `(ba.)tabulate`, which interpolates only between tabulated samples, `(ba.)
1244 // tabulate_chebychev`
1245 // stores coefficients of Chebyshev polynomials that are evaluated to provide better
1246 // approximations in many cases.
1247 // Two new arguments controlling this are NX, the number of segments into which [r0, r1] is
1248 // divided, and CD,
1249 // the maximum Chebyshev polynomial degree to use for each segment. A `rdtable` of size NX*(
1250 // CD+1) is internally used.
1251 //
1252 // Note that processing `r1` the last point in the interval is not safe. So either be sure
1253 // the input stays in [r0, r1[
1254 // or use `C = 1`.
1255 //
1256 // #### Usage
1257 //
1258 // _ : tabulate_chebychev(C, FX, NX, CD, r0, r1) : _
1259 //
1260 // Where:
1261 //
1262 // * `C`: whether to dynamically force the value to the range [r0, r1]: 1 forces the check, 0
1263 //   deactivates it (constant numerical expression)
1264 // * `FX`: unary function Y=F(X) with one output (scalar function of one variable)
1265 // * `NX`: number of segments for uniformly partitioning [r0, r1] (constant numerical
1266 //   expression)
1267 // * `CD`: maximum polynomial degree for each Chebyshev polynomial (constant numerical
1268 //   expression)
1269 // * `r0`: minimum value of argument x
1270 // * `r1`: maximum value of argument x
1271 // * `x`: input value used to evaluate the approximation
1272 //
1273 // #### Example test program
1274 //
1275 //
1276 // midikey2hz_chebychev(mk) = ba.tabulate_chebychev(1, ba.midikey2hz, 100, 4, 0, 127, mk);
1277 // process = midikey2hz_chebychev(ba.time), ba.midikey2hz(ba.time);
1278 //
1279 // #### Test
1280 //
1281 // ba = library("basics.lib");
1282 // tabulate_chebychev_test = ba.tabulate_chebychev(1, ba.midikey2hz, 32, 4, 0, 127, 60);
1283 //
1284 //-----
1285 tabulate_chebychev(C, FX, NX, CD, r0, r1, x) = y with {
1286   ck(0) = _;
1287   ck(1) = max(0) : min(NX-1);
1288
1289   // number of chebyshev coefficients
1290   NC = CD + 1;
1291   // length of the segments
1292   DX = (r1 - r0) / NX;
1293   // number of segment 'x' falls in
1294   nx = (x - r0) / DX : int : ck(C);
1295   // center of n's segment
1296   xc(n) = r0 + DX * (n + 1/2);
1297   // so ch(0) .. ch(NC) are the coeffs we use for approximation
1298   // on nx's segment
1299   ch(i) = chtab(NC * nx + i);
1300
1301   // map the input in segment [nx*DX, (nx+1)*DX] to [-1,1]
1302   y = (x - xc(nx)) * 2/DX <: sum(i, NC, ch(i) * ma.chebychev(i));
1303
1304   // map [-1,1] to the segment [nx*DX, (nx+1)*DX] so mapfx(nx)
1305   // is simply the "renormalized" FX defined on [-1,1]
1306   mapfx(nx, x) = FX(xc(nx) + DX/2 * x);

```

```

1301 // calculate the nc's chebyshev coefficient we use on nx's segment
1302 gench(nx, nc) = (1+(nc!=0))/NC * sum(k,NC,
1303     mapfx(nx, cos(ma.PI*(k+1/2)/NC)) * cos(ma.PI*nc*(k+1/2)/NC));
1304
1305 // record gench(nx, nc) in rdtable() to avoid the run-time calculations
1306 chtab = rdtable(NX*NC, (ba.time <: int(/(NC)), %(NC) : gench));
1307 };
1308
1309 declare tabulate_chebychev author "Oleg Nesterov";
1310 declare tabulate_chebychev copyright "Copyright (C) 2022 Oleg Nesterov <oleg@redhat.com>";
1311 declare tabulate_chebychev license "MIT-style STK-4.3 license";
1312
1313 //-----`ba.)tabulateNd`-----
1314 // Tabulate an nD function for access via nearest-value or linear or cubic interpolation. In
1315 // other words, the tabulated function can be evaluated using interpolation of order 0 (
1316 // none), 1 (linear), or 3 (cubic).
1317 //
1318 // The table size and parameter range of each dimension can and must be separately specified.
1319 // You can use it anywhere you have an expensive function with multiple parameters with
1320 // known ranges. You could use it to build a wavetable synth, for example.
1321 //
1322 // The number of dimensions is deduced from the number of parameters you give, see below.
1323 //
1324 // Note that processing the last point in each interval is not safe. So either be sure the
1325 // inputs stay in their respective ranges, or use `C = 1`. Similarly for the first point
1326 // when doing cubic interpolation.
1327 //
1328 // #### Usage
1329 //
1330 // tabulateNd(C, function, (parameters) ).(val/lin/cub) : _
1331 //
1332 // Where:
1333 //
1334 // * `C`: whether to dynamically force the parameter values for each dimension to the ranges
1335 //   specified in parameters: 1 forces the check, 0 deactivates it (constant numerical
1336 //   expression)
1337 // * `function`: the function we want to tabulate. Can have any number of inputs, but needs
1338 //   to have just one output.
1339 // * `(parameters)`: sizes, ranges and read values. Note: these need to be in brackets, to
1340 //   make them one entity.
1341 //
1342 // If N is the number of dimensions, we need:
1343 //
1344 // * N times `S`: number of values to store for this dimension (constant numerical
1345 //   expression)
1346 // * N times `r0`: minimum value of this dimension
1347 // * N times `r1`: maximum value of this dimension
1348 // * N times `x`: read value of this dimension
1349 //
1350 // By providing these parameters, you indirectly specify the number of dimensions; it's the
1351 // number of parameters divided by 4.
1352 //
1353 // The user facing functions are:
1354 //
1355 // tabulateNd(C, function, S, parameters).val
1356 //
1357 // - Uses the value in the table closest to x.
1358 //
1359 // tabulateNd(C, function, S, parameters).lin
1360 //
1361 // - Evaluates at x using linear interpolation between the closest stored values.
1362 //
1363 // tabulateNd(C, function, S, parameters).cub
1364 //
1365 //

```

```

1357 // - Evaluates at x using cubic interpolation between the closest stored values.
1358 //
1359 //
1360 // ##### Example test program
1361 //
1362 // ```
1363 // powSin(x,y) = sin(pow(x,y)); // The function we want to tabulate
1364 // powSinTable(x,y) = ba.tabulateNd(1, powSin, (sizeX,sizeY, rx0,ry0, rx1,ry1, x,y) ).lin;
1365 // sizeX = 512; // table size of the first parameter
1366 // sizeY = 512; // table size of the second parameter
1367 // rx0 = 2; // start of the range of the first parameter
1368 // ry0 = 2; // start of the range of the second parameter
1369 // rx1 = 10; // end of the range of the first parameter
1370 // ry1 = 10; // end of the range of the second parameter
1371 // x = hslider("x", rx0, rx0, rx1, 0.001):si.smoo;
1372 // y = hslider("y", ry0, ry0, ry1, 0.001):si.smoo;
1373 // process = powSinTable(x,y), powSin(x,y);
1374 // ```
1375 //
1376 // ##### Working principle
1377 //
1378 // The ``.val`` function just outputs the closest stored value.
1379 // The ``.lin`` and ``.cub`` functions interpolate in N dimensions.
1380 //
1381 // ##### Multi dimensional interpolation
1382 //
1383 // To understand what it means to interpolate in N dimensions, here's a quick reminder on the
1384 // general principle of 2D linear interpolation:
1385 //
1386 // * We have a grid of values, and we want to find the value at a point (x, y) within this
1387 // grid.
1388 // * We first find the four closest points (A, B, C, D) in the grid surrounding the point (x,
1389 // y).
1390 //
1391 // Then, we perform linear interpolation in the x-direction between points A and B, and
1392 // between points C and D. This gives us two new points E and F. Finally, we perform linear
1393 // interpolation in the y-direction between points E and F to get our value.
1394 //
1395 // To implement this in Faust, we need N sequential groups of interpolators, where N is the
1396 // number of dimensions.
1397 //
1398 // Each group feeds into the next, with the last "group" being a single interpolator, and the
1399 // group before it containing one interpolator for each input of the group it's feeding.
1400 //
1401 // Some examples:
1402 //
1403 // * Our 2D linear example has two interpolators feeding into one.
1404 // * A 3D linear interpolator has four interpolators feeding into two, feeding into one.
1405 // * A 2D cubic interpolator has four interpolators feeding into one.
1406 // * A 3D cubic interpolator has sixteen interpolators feeding into four, feeding into one.
1407 //
1408 // To understand which values we need to look up, let's consider the 2D linear example again.
1409 // The four values going into the first group represent the four closest points (A, B, C, D)
1410 // mentioned above.
1411 //
1412 // 1) The first interpolator gets:
1413 //
1414 // * The closest value that is stored (A)
1415 // * The next value in the x dimension, keeping y fixed (B)
1416 //
1417 // 2) The second interpolator gets:
1418 //
1419 // * One step over in the y dimension, keeping x fixed (C)
1420 // * One step over in both the x dimension and the y dimension (D)
1421 //
1422 // The outputs of these two interpolators are points E and F.
1423 // In other words: the interpolated x values and, respectively, the following y values:
1424 //
1425 // * The closest stored value of the y dimension

```

```

1417 // * One step forward in the y dimension
1418 //
1419 // The last interpolator takes these two values and interpolates them in the y dimension.
1420 //
1421 // To generalize for N dimensions and linear interpolation:
1422 //
1423 // * The first group has  $2^{(n-1)}$  parallel interpolators interpolating in the first dimension.
1424 // * The second group has  $2^{(n-2)}$  parallel interpolators interpolating in the second
    dimension.
1425 // * The process continues until the n-th group, which has a single interpolator
    interpolating in the n-th dimension.
1426 //
1427 // The same principle applies to the cubic interpolation in nD. The only difference is that
    there would be  $4^{(n-1)}$  parallel interpolators in the first group, compared to  $2^{(n-1)}$ 
    for linear interpolation.
1428 //
1429 // This is what the ``mixers`` function does.
1430 //
1431 // Besides the values, each interpolator also needs to know the weight of each value in it's
    output.
1432 // Let's call this `d`, like in ``ba.interpolate``. It is the same for each group of
    interpolators, since it correlates to a dimension.
1433 // It's value is calculated the similarly to ``ba.interpolate``:
1434 //
1435 // * First we prepare a "float table read-index" for that dimension (``id`` in ``ba.tabulate
    ``)
1436 // * If the table only had that dimension and it could read a float index, what would it be.
1437 // * Then we ``int`` the float index to get the value we have stored that is closest to, but
    lower than the input value; the actual index for that dimension.
1438 // Our ``d`` is the difference between the float index and the actual index.
1439 //
1440 // The ``ids`` function calculates the ``id`` for each dimension and inside the ``mixer``
    function they get turned into ``d``'s.
1441 //
1442 // ##### Storage method
1443 //
1444 // The elephant in the room is: how do we get these indexes? For that we need to know how the
    values are stored.
1445 // We use one big table to store everything.
1446 //
1447 // To understand the concept, let's look at the 2D example again, and then we'll extend it to
    3d and the general nD case.
1448 //
1449 // Let's say we have a 2D table with dimensions A and B where:
1450 // A has 3 values between 0 and 5 and B has 4 values between 0 and 1.
1451 // The 1D array representation of this 2D table will have a size of  $3 * 4 = 12$ .
1452 //
1453 // The values are stored in the following way:
1454 //
1455 // * First 3 values: A is 0, then 3, then 5 while B is at 0.
1456 // * Next 3 values: A changes from 0 to 5 while B is at 1/3.
1457 // * Next 3 values: A changes from 0 to 5 while B is at 2/3.
1458 // * Last 3 values: A changes from 0 to 5 while B is at 1.
1459 //
1460 // For the 3D example, let's extend the 2D example with an additional dimension C having 2
    values between 0 and 2.
1461 // The total size will be  $3 * 4 * 2 = 24$ .
1462 //
1463 // The values are stored like so:
1464 //
1465 // * First 3 values: A changes from 0 to 5, B is at 0, and C is at 0.
1466 // * Next 3 values: A changes from 0 to 5, B is at 1/3, and C is at 0.
1467 // * Next 3 values: A changes from 0 to 5, B is at 2/3, and C is at 0.
1468 // * Next 3 values: A changes from 0 to 5, B is at 1, and C is at 0.
1469 //
1470 // The last 12 values are the same as the first 12, but with C at 2.
1471 //
1472 // For the general n-dimensional case, we iterate through all dimensions, changing the values

```

```

of the innermost dimension first, then moving towards the outer dimensions.
1473 //
1474 // ##### Read indexes
1475 //
1476 // To get the float read index (`id`) corresponding to a particular dimension, we scale the
        function input value to be between 0 and 1, and multiply it by the size of that
        dimension minus one.
1477 //
1478 // To understand how we get the `readIndex` for `val`, let's work through how we'd do it
        in our 2D linear example.
1479 // For simplicity's sake, the ranges of the inputs to our `function` are both 0 to 1.
1480 // Say we wanted to read the value closest to `x=0.5` and `y=0`, so the `id` of `x`
        is `1` (the second value) and the `id` of `y` is 0 (first value). In this case,
        the read index is just the `id` of `x`, rounded to the nearest integer, just like in
        `ba.tabulate`.
1481 //
1482 // If we want to read the value belonging to `x=0.5` and `y=2/3`, things get more
        complicated. The `id` for `y` is now `2`, the third value. For each step in the `y`
        direction, we need to increase the index by `3`, the number of values that are
        stored for `x`. So the influence of the `y` is: the size of `x` times the rounded
        `id` of `y`. The final read index is the rounded `id` of `x` plus the influence
        of `y`.
1483 //
1484 // For the general nD case, we need to do the same operation N times, each feeding into the
        next. This operation is the `riN` function. We take four parameters: the size of the
        dimension before it `prevSize`, the index of the previous dimension `prevIX`, the
        current size `sizeX` and the current id `idX`. `riN` has 2 outputs, the size, for
        feeding into the next dimension's `prevSize`, and the read index feeding into the next
        dimension's `prevIX`.
1485 // The size is the `sizeX` times `prevSize`. The read index is the rounded `idX` times
        `prevSize` added to the `prevIX`. Our final `readIndex` is the read index output
        of the last dimension.
1486 //
1487 // To get the read values for the interpolators need a pattern of offsets in each dimension,
        since we are looking for the read indexes surrounding the point of interest. These
        offsets are best explained by looking at the code of `tabulate2d`, the hardcoded 2D
        version:
1488 //
1489 // ```
1490 // tabulate2d(C,function, sizeX,sizeY, rx0,ry0, rx1,ry1, x,y) =
1491 //   environment {
1492 //     size = sizeX*sizeY;
1493 //     // Maximum X index to access
1494 //     midX = sizeX-1;
1495 //     // Maximum Y index to access
1496 //     midY = sizeY-1;
1497 //     // Maximum total index to access
1498 //     mid = size-1;
1499 //     // Create the table
1500 //     wf = function(wfX,wfY);
1501 //     // Prepare the 'float' table read index for X
1502 //     idX = (x-rx0)/(rx1-rx0)*midX;
1503 //     // Prepare the 'float' table read index for Y
1504 //     idY = ((y-ry0)/(ry1-ry0))*midY;
1505 //     // table creation X:
1506 //     wfX =
1507 //       rx0+float(ba.time%sizeX)*(rx1-rx0)
1508 //       /float(midX);
1509 //     // table creation Y:
1510 //     wfY =
1511 //       ry0+
1512 //       ((float(ba.time-(ba.time%sizeX))
1513 //       /float(sizeX))
1514 //       *(ry1-ry0))
1515 //       /float(midY);
1516 //
1517 //     // Limit the table read index in [0, mid] if C = 1
1518 //     rid(x,mid, 0) = x;

```

```

1519 // rid(x,mid, 1) = max(0, min(x, mid));
1520 //
1521 // // Tabulate a binary 'FX' function on a range [rx0, rx1] [ry0, ry1]
1522 // val(x,y) =
1523 //     rdtable(size, wf, readIndex);
1524 // readIndex =
1525 //     rid(
1526 //         rid(int(idX+0.5),midX, C)
1527 //         +yOffset
1528 //         , mid, C);
1529 // yOffset = sizeX*rid(int(idY),midY,C);
1530 //
1531 // // Tabulate a binary 'FX' function over the range [rx0, rx1] [ry0, ry1] with linear
interpolation
1532 // lin =
1533 //     it.interpolate_linear(
1534 //         dy
1535 //         , it.interpolate_linear(dx,v0,v1)
1536 //         , it.interpolate_linear(dx,v2,v3))
1537 // with {
1538 //     i0 = rid(int(idX), midX, C)+yOffset;
1539 //     i1 = i0+1;
1540 //     i2 = i0+sizeX;
1541 //     i3 = i1+sizeX;
1542 //     dx = idX-int(idX);
1543 //     dy = idY-int(idY);
1544 //     v0 = rdtable(size, wf, rid(i0, mid, C));
1545 //     v1 = rdtable(size, wf, rid(i1, mid, C));
1546 //     v2 = rdtable(size, wf, rid(i2, mid, C));
1547 //     v3 = rdtable(size, wf, rid(i3, mid, C));
1548 // };
1549 //
1550 // // Tabulate a binary 'FX' function over the range [rx0, rx1] [ry0, ry1] with cubic
interpolation
1551 // cub =
1552 //     it.interpolate_cubic(
1553 //         dy
1554 //         , it.interpolate_cubic(dx,v0,v1,v2,v3)
1555 //         , it.interpolate_cubic(dx,v4,v5,v6,v7)
1556 //         , it.interpolate_cubic(dx,v8,v9,v10,v11)
1557 //         , it.interpolate_cubic(dx,v12,v13,v14,v15)
1558 //         )
1559 // with {
1560 //     i0 = i4-sizeX;
1561 //     i1 = i5-sizeX;
1562 //     i2 = i6-sizeX;
1563 //     i3 = i7-sizeX;
1564 //
1565 //     i4 = i5-1;
1566 //     i5 = rid(int(idX), midX, C)+yOffset;
1567 //     i6 = i5+1;
1568 //     i7 = i6+1;
1569 //
1570 //     i8 = i4+sizeX;
1571 //     i9 = i5+sizeX;
1572 //     i10 = i6+sizeX;
1573 //     i11 = i7+sizeX;
1574 //
1575 //     i12 = i4+(2*sizeX);
1576 //     i13 = i5+(2*sizeX);
1577 //     i14 = i6+(2*sizeX);
1578 //     i15 = i7+(2*sizeX);
1579 //
1580 //     dx = idX-int(idX);
1581 //     dy = idY-int(idY);
1582 //     v0 = rdtable(size, wf, rid(i0 , mid, C));
1583 //     v1 = rdtable(size, wf, rid(i1 , mid, C));
1584 //     v2 = rdtable(size, wf, rid(i2 , mid, C));

```

```

1585 //      v3 = rdtable(size, wf, rid(i3 , mid, C));
1586 //      v4 = rdtable(size, wf, rid(i4 , mid, C));
1587 //      v5 = rdtable(size, wf, rid(i5 , mid, C));
1588 //      v6 = rdtable(size, wf, rid(i6 , mid, C));
1589 //      v7 = rdtable(size, wf, rid(i7 , mid, C));
1590 //      v8 = rdtable(size, wf, rid(i8 , mid, C));
1591 //      v9 = rdtable(size, wf, rid(i9 , mid, C));
1592 //      v10 = rdtable(size, wf, rid(i10, mid, C));
1593 //      v11 = rdtable(size, wf, rid(i11, mid, C));
1594 //      v12 = rdtable(size, wf, rid(i12, mid, C));
1595 //      v13 = rdtable(size, wf, rid(i13, mid, C));
1596 //      v14 = rdtable(size, wf, rid(i14, mid, C));
1597 //      v15 = rdtable(size, wf, rid(i15, mid, C));
1598 //    };
1599 //  };
1600 //  ```
1601 //
1602 // In the interest of brevity, we'll stop explaining here. If you have any more questions,
// feel free to open an issue on [faustlibraries](https://github.com/grame-cncm/
// faustlibraries) and tag @magnetophon.
1603 //
1604 // #### Test
1605 // ```
1606 // ba = library("basics.lib");
1607 // powSin(x,y) = sin(pow(x,y));
1608 // tabulateNd_test = ba.tabulateNd(1, powSin, (8,8, 2.0,2.0, 8.0,8.0, 3.0,4.0)).lin;
1609 // ```
1610 //-----
1611 tabulateNd(C,function,parameters) =
1612   environment {
1613     val =
1614       parameters
1615       // our sizes can be int, should be faster to calculate
1616       : (par(i, N, int),si.bus(N*3))
1617       // table size, waveform, read index
1618       <: (tableSize,wf,readIndex(idsGetRoundedNotFloored))
1619       : rdtable
1620     with {
1621       // for val we want the rounded version of id, the interpolators need the rounded down
// version
1622       // see: https://github.com/grame-cncm/faustlibraries/pull/152
1623       idsGetRoundedNotFloored =
1624         // table sizes and ids for each dimension
1625         sizesIds:
1626         (bs,par(i, N, _+0.5));
1627     };
1628
1629     lin =
1630       parameters
1631       // our sizes can be int, should be faster to calculate
1632       : (par(i, N, int),si.bus(N*3))
1633       // the mixers need the float indexes and the values read from the tables
1634       <: (ids,tables(nrReadIndexes,readIndexes))
1635       // the actual interpolation
1636       :mixers(0,nrReadIndexes)
1637     with {
1638       // the read indexes to form the closest points in the grid surrounding the point of
// interest.
1639       readIndexes =
1640         // for cleaner block diagrams
1641         si.bus(nParams) <:
1642         // the closest stored value at or below the point of interest
1643         ((readIndex(sizesIds) <: si.bus(nrReadIndexes))
1644         // the offsets to get to the points around it
1645         , offsets)
1646         // add them up
1647         : ro.interleave(nrReadIndexes,2) : par(i, nrReadIndexes, +) ;
1648       offsets =

```

```

1649         // the size of a step for each dimension
1650         stepSizes
1651         // for each read index
1652         <: par(i, nrReadIndexes,
1653             // sum up the N components that determine the final offset
1654             par(j, N, switch(i,j))>_)
1655     with {
1656         // the truth table of which step size to use is a binary counting table
1657         // so we use a variant on a function I wrote for slidingReduce
1658         switch(i,j) = _*int2bin(j,i,nrReadIndexes);
1659     };
1660     // since each interpolator has 2 inputs
1661     nrReadIndexes = pow(2,N);
1662 };
1663
1664 cub =
1665     // same as .lin so far
1666     parameters
1667     : (par(i, N, int),si.bus(N*3))
1668     <: (ids,tables(nrReadIndexes,readIndexes))
1669     :mixers(1,nrReadIndexes)
1670 with {
1671     readIndexes =
1672         si.bus(nParams) <:
1673         ((stepSizes:ro.cross(N))
1674         , readIndex(sizesIds))
1675         // calculate the offsets for each interpolation read index and add them to the "
1676         // base" readindex
1677         :cubifiers;
1678     // since each interpolator has 4 inputs
1679     nrReadIndexes = pow(4,N);
1680     // calculate the offsets and add them to the "base" readindex
1681     // we do this by creating a tree, where each branch has 4 sub-branches
1682     // there are N splits, so we end up with nrReadIndexes "leaves"
1683     cubifiers =
1684         // iterate through the dimensions, each feeding into the next
1685         seq(i, N,
1686             si.bus(N-i-1),cubifier(pow(4,i)));
1687     // take a step size and ``len`` input indexes and create 4*len output indexes
1688     // offset the input index(es) with the amount needed for that dimension
1689     cubifier(len) =
1690         ((_<:si.bus(len*4))
1691         , (si.bus(len)<:si.bus(len*4)))
1692         : ro.interleave(len*4,2)
1693         :par(i, 4,
1694             par(j, len, off(i)+_))
1695     with {
1696         // the hardcoded numbers are the same offsets as in ``ba.tabulate``
1697         // but they get multiplied by the step size of the current dimension
1698         off(0,stepSize) = -1*stepSize;
1699         off(1,stepSize) = 0;
1700         off(2,stepSize) = 1*stepSize;
1701         off(3,stepSize) = 2*stepSize;
1702     };
1703 };
1704
1705 // how many parameters have we been given?
1706 nParams = outputs(parameters);
1707 // the number of dimensions
1708 N = int(nParams/4);
1709 // the values we read from the tables
1710 tables(nrReadIndexes,readIndexes) =
1711     si.bus(nParams)<:
1712     // table size, waveform, read index for each table
1713     ((tableSize<:si.bus(nrReadIndexes))
1714     , (wf<:si.bus(nrReadIndexes))
1715     , readIndexes)
1716     :ro.interleave(nrReadIndexes,3)

```

```

1716 // the actual tables
1717 :par(i, nrReadIndexes, rdtable);
1718
1719 // the interpolators
1720 mixers(linCub,nrReadIndexes)=
1721 (ro.cross(N),si.bus(nrReadIndexes))
1722 : seq(i, N, mixer(linCub,i));
1723
1724 // the interpolator for a single dimension
1725 // linear version
1726 mixer(0,i) =
1727 mixerUniversal(i,2,(_,!,_),it.interpolate_linear) ;
1728 // cubic version
1729 mixer(1,i) =
1730 mixerUniversal(i,4,(_,!,_,!,_),it.interpolate_cubic) ;
1731
1732 // i is the current dimension
1733 // mult is the number of inputs of each interpolator
1734 // sieve tells us which outputs to let trough and which to block, more on this soon
1735 mixerUniversal(i,mult,sieve,it) =
1736 // bypass the weights needed for the next dimension
1737 si.bus(N-i-1),
1738 // split our own weight
1739 // in the end we need one weight per interpolator
1740 // but since we need to interleave these with nrInterpolators*mult read values, we
1741 // split it into as many busses as we have read values
1742 (((_<:si.bus(nrInterpolators(i)*mult))
1743 // the read values bypass this step
1744 , (si.bus(nrInterpolators(i)*mult)))
1745 // interleave the weights with the read values
1746 : ro.interleave(nrInterpolators(i)*mult,2)
1747 // the actual interpolators
1748 : par(i, nrInterpolators(i),
1749 // take the id and turn it into the weight for this dimension
1750 ((_<:(_-int(_)))
1751 // throw away the extra weights we created for the interleave
1752 ,sieve)
1753 // a single interpolator
1754 :it))
1755
1756 with {
1757 // the number of interpolators for this dimension
1758 nrInterpolators(i) = pow(mult,N-i-1);
1759 };
1760
1761 // total size of the table: s(0) * s(1) ... * s(N-2) * s(N-1)
1762 // N in, 1 out
1763 size(1) = _;
1764 size(N) = _*size(N-1);
1765 tableSize = size(N),par(i, N*3, !);
1766 // the size of a step for each dimension.
1767 // the first is 1, the second is the size of the first dimension times one
1768 // the third is the second number, times the size of the second dimension
1769 stepSizes =
1770 (int(1),si.bus(N-1),par(i, 3*N+1, !))
1771 : seq(i, N-1,
1772 (si.bus(i),(_<:(_,_)), si.bus(N-i-1))
1773 : (si.bus(i+1),*,si.bus(N-i-2)));
1774 // Prepare the 'float' table read index for one parameter
1775 id(sizeX,r0,r1,x) = (x-r0)/(r1-r0)*(sizeX-1);
1776 // Prepare the 'float' table read index for all parameters
1777 ids =
1778 ro.interleave(N,4)
1779 : par(i, N, id) ;
1780
1781 // one waveform parameter write value:
1782 wfp(prevSize,sizeX,r0,r1) =
1783 r0+
1784 ((float(int(ba.time%(prevSize*sizeX)/prevSize))*(r1-r0))

```

```

1783         /float(sizeX-1))
1784         ,(prevSize*sizeX);
1785
1786         // all waveform parameters write values:
1787         wfps =
1788             ro.interleave(N,3)
1789             : (1,si.bus(3*N))
1790             : seq(i, N, si.bus(i),wfp, si.bus(3*N-(3*(i+1))))
1791             : (si.bus(N),!);
1792
1793         // Create the table
1794         wf = (wfps,par(i, N, !)):function;
1795
1796         // Limit the table read index in [0, mid] if C = 1
1797         // we do this both for the total table and per dimension
1798         rid(x,mid, 0) = int(x);
1799         rid(x,mid, 1) = max(int(0), min(int(x), mid));
1800
1801         // for ``val`` this is the stored value closest to the point of interest
1802         // for ``lin`` and ``cub`` it's the closest stored value at or below the point of
1803         // interest
1804         readIndex(sizesIds) =
1805             // table sizes and ids for each dimension
1806             sizesIds
1807             // get the raw read index
1808             : ri
1809             // limit it and make it an int
1810             : riPost ;
1811         // helper function to route the arguments of rid
1812         riPost(size,ri) =
1813             rid(ri,size-int(1),C);
1814         // the raw read index
1815         ri =
1816             // interleave the sizes and the ids
1817             ro.interleave(N,2)
1818             // the first iteration gets:
1819             // 1 as the size of the previous dimension and 0 as the read index of the previous
1820             // dimension
1821             : (1,0
1822             // pass through the sizes and ids
1823             ,si.bus(2*N))
1824             // for each dimension
1825             : seq(i, N,
1826             // get the step size and the partial index for the next dimension
1827             riN
1828             // pass through the sizes and ids for the next dimensions
1829             , si.bus(2*(N-i-1))) ;
1830
1831         // get the step size and the partial index for the next dimension
1832         riN(prevSize,prevIX,sizeX,idX) =
1833             // each step size is the previous step size times the current
1834             (prevSize*sizeX)
1835             // but the step we actually take in this dimension is the stepsize we calculated in
1836             // the previous dimension
1837             , ( (prevSize*
1838             // round down and limit the id for this dimension to get the number of steps
1839             rid(int(idX),(sizeX-int(1)),C))
1840             // add the result of that to the one we calculated in the previous dimension
1841             +prevIX) ;
1842
1843         // table sizes and ids for each dimension
1844         // just a helper function to simplify routing
1845         sizesIds =
1846             (
1847                 // from sizes
1848                 ( bs<:si.bus(N*2) )
1849                 // from r0s,rls,xs
1850                 , si.bus(N*3)

```

```

1848         ) :
1849             // from sizes
1850             (si.bus(N)
1851             // takes (midX,r0,r1,x)
1852             ,ids);
1853
1854             // the value of a single bit when binary counting
1855             int2bin(i,n,maxN) = int(int((n)/(1<<i))%int(2));
1856             // shortcut
1857             bs = si.bus(N);
1858     };
1859
1860     declare tabulateNd author "Bart Brouns";
1861     declare tabulateNd copyright "Copyright (C) 2023 Bart Brouns <bart@magnetophon.nl>";
1862     declare tabulateNd license "AGPL-3.0";
1863
1864     //=====Selectors (Conditions)=====
1865     //=====
1866
1867     //-----`(ba.)if`-----
1868     // if-then-else implemented with a select2. WARNING: since `select2` is strict (always
1869     // evaluating both branches),
1870     // the resulting if does not have the usual "lazy" semantic of the C if form, and thus cannot
1871     // be used to
1872     // protect against forbidden computations like division-by-zero for instance.
1873     //
1874     // #### Usage
1875     //
1876     // * `if(cond, then, else) : _`
1877     // Where:
1878     // * `cond`: condition
1879     // * `then`: signal selected while cond is true
1880     // * `else`: signal selected while cond is false
1881     //
1882     // #### Test
1883     // ```
1884     // ba = library("basics.lib");
1885     // if_test = ba.if(1, 0.5, -0.5);
1886     // ```
1887     //-----
1888     if(cond,then,else) = select2(cond,else,then);
1889
1890     // TODO: perhaps it would make more sense to have an if(a,b) and an ifelse(a,b,c)?
1891
1892     //-----`(ba.)ifNc`-----
1893     // if-then-elseif-then-...elseif-then-else implemented on top of `ba.if`.
1894     //
1895     // #### Usage
1896     //
1897     // ```
1898     // ifNc((cond1,then1, cond2,then2, ... condN,thenN, else)) : _
1899     // or
1900     // ifNc(Nc, cond1,then1, cond2,then2, ... condN,thenN, else) : _
1901     // or
1902     // cond1,then1, cond2,then2, ... condN,thenN, else : ifNc(Nc) : _
1903     // ```
1904     //
1905     // Where:
1906     //
1907     // * `Nc` : number of branches/conditions (constant numerical expression)
1908     // * `condX`: condition
1909     // * `thenX`: signal selected if condX is the 1st true condition
1910     // * `else`: signal selected if all the cond1-condN conditions are false
1911     //
1912     // #### Example test program
1913     //

```

```

1914 // ``
1915 //   process(x,y) = ifNc((x<y,-1, x>y,+1, 0));
1916 // or
1917 //   process(x,y) = ifNc(2, x<y,-1, x>y,+1, 0);
1918 // or
1919 //   process(x,y) = x<y,-1, x>y,+1, 0 : ifNc(2);
1920 // ``
1921 //
1922 // outputs `-1` if `x<y`, `+1` if `x>y`, `0` otherwise.
1923 //
1924 // #### Test
1925 // ``
1926 // ba = library("basics.lib");
1927 // ifNc_test = ba.ifNc((1, 10, 0, 20, 30));
1928 // ``
1929 //-----
1930 ifNc((c,r)) = c,r : ifNc(outputs(r)/2);
1931 ifNc(1) = if;
1932 ifNc(n) = _,_,ifNc(n-1) : ifNc(1);
1933
1934 declare ifNc author "Oleg Nesterov";
1935 declare ifNc copyright "Copyright (C) 2023 Oleg Nesterov <oleg@redhat.com>";
1936 declare ifNc license "MIT-style STK-4.3 license";
1937
1938 //-----`ba.)ifNcNo`-----
1939 // `ifNcNo(Nc,No)` is similar to `ifNc(Nc)` above but then/else branches have `No` outputs.
1940 //
1941 // #### Usage
1942 //
1943 // ``
1944 //   ifNcNo(Nc,No, cond1,then1, cond2,then2, ... condN,thenN, else) : sig.bus(No)
1945 // ``
1946 //
1947 // Where:
1948 //
1949 // * `Nc` : number of branches/conditions (constant numerical expression)
1950 // * `No` : number of outputs (constant numerical expression)
1951 // * `condX` : condition
1952 // * `thenX` : list of No signals selected if condX is the 1st true condition
1953 // * `else` : list of No signals selected if all the cond1-condN conditions are false
1954 //
1955 // #### Example test program
1956 //
1957 // ``
1958 //   process(x) = ifNcNo(2,3, x<0, -1,-1,-1, x>0, 1,1,1, 0,0,0);
1959 // ``
1960 //
1961 // outputs `-1,-1,-1` if `x<0`, `1,1,1` if `x>0`, `0,0,0` otherwise.
1962 //
1963 // #### Test
1964 // ``
1965 // ba = library("basics.lib");
1966 // ifNcNo_test = (1, 10, 0, 20, 30) : ba.ifNcNo(2, 1);
1967 // ``
1968 //-----
1969 ifNcNo(Nc,No) = si.bus(Ni) <: par(no,No, slice(no) : ifNc(Nc))
1970 with {
1971   Ni = Nc*No + Nc+No;
1972   slice(no) = pickN(Ni, (par(nc,Nc, ct(nc,no)), else(no)));
1973   ct(nc,no) = nc*(No+1)+0, // cond[nc]
1974             nc*(No+1)+1+no; // then[nc][no]
1975   else(no) = Nc*(No+1)+0+no; // else[no]
1976 };
1977
1978 declare ifNcNo author "Oleg Nesterov";
1979 declare ifNcNo copyright "Copyright (C) 2023 Oleg Nesterov <oleg@redhat.com>";
1980 declare ifNcNo license "MIT-style STK-4.3 license";
1981

```

```

1982 //-----`(ba.)selector`-----
1983 // Selects the ith input among N at compile time.
1984 //
1985 // #### Usage
1986 //
1987 // ```
1988 // selector(I,N)
1989 // _,_ : selector(2,4) : _ // selects the 3rd input among 4
1990 // ```
1991 //
1992 // Where:
1993 //
1994 // * `I`: input to select (int, numbered from 0, known at compile time)
1995 // * `N`: number of inputs (int, known at compile time, N > I)
1996 //
1997 // There is also `cselector` for selecting among complex input signals of the form (real,imag
1998 // ).
1999 //
2000 // #### Test
2001 // ```
2002 // ba = library("basics.lib");
2003 // selector_test = (0.1, 0.2, 0.3, 0.4) : ba.selector(2, 4);
2004 // ```
2005 //-----
2006 selector(i,n) = par(j, n, S(i, j)) with { S(i,i) = _; S(i,j) = !; };
2007 cselector(i,n) = par(j, n, S(i, j)) with { S(i,i) = (_,_); S(i,j) = (!,!); }; // for complex
2008 // numbers
2009
2010 //-----`(ba.)select2stereo`-----
2011 // Select between 2 stereo signals.
2012 //
2013 // #### Usage
2014 //
2015 // ```
2016 // _,_ : select2stereo(bpc) : _,_
2017 // ```
2018 //
2019 // Where:
2020 //
2021 // * `bpc`: the selector switch (0/1)
2022 //
2023 // #### Test
2024 // ```
2025 // ba = library("basics.lib");
2026 // select2stereo_test = ba.select2stereo(1, (0.1,0.2, 0.3,0.4));
2027 // ```
2028 //-----
2029 select2stereo(bpc) = ro.cross2 : select2(bpc), select2(bpc) : _,_;
2030
2031
2032 //-----`(ba.)selectn`-----
2033 // Selects the ith input among N at run time.
2034 //
2035 // #### Usage
2036 //
2037 // ```
2038 // selectn(N,i)
2039 // _,_ : selectn(4,2) : _ // selects the 3rd input among 4
2040 // ```
2041 //
2042 // Where:
2043 //
2044 // * `N`: number of inputs (int, known at compile time, N > 0)
2045 // * `i`: input to select (int, numbered from 0)
2046 //
2047 // #### Example test program

```

```

2048 //
2049 // `--
2050 // N = 64;
2051 // process = par(n, N, (par(i,N,i) : selectn(N,n)));
2052 // `--
2053 //
2054 // #### Test
2055 // `--
2056 // ba = library("basics.lib");
2057 // selectn_test = (1,2,3,4) : ba.selectn(4, 2);
2058 // `--
2059 //-----
2060 selectn(N,i) = selectnX(N,i,selector)
2061 with {
2062     selector(i,j,x,y) = select2((i >= j), x, y);
2063 };
2064
2065
2066 // The generic version with a 'sel' function to be applied on:
2067 // - the channel index as a (possibly) fractional value
2068 // - the next channel index as an integer value
2069 // - the 2 signals to be selected between
2070
2071 selectnX(N,i,sel) = S(N,0)
2072 with {
2073     S(1,offset) = _;
2074     S(n,offset) = S(left, offset), S(right, offset+left) : sel(i, offset+left)
2075     with {
2076         right = int(n/2);
2077         left = n-right;
2078     };
2079 };
2080
2081
2082 //-----`ba).selectbus`-----
2083 // Select a bus among `NUM_BUSES` buses, where each bus has `BUS_SIZE` outputs.
2084 // The order of the signal inputs should be the signals of the first bus, the
2085 // signals of the second bus, and so on.
2086 //
2087 // #### Usage
2088 // `--
2089 // `--
2090 // process = si.bus(BUS_SIZE*NUM_BUSES) : selectbus(BUS_SIZE, NUM_BUSES, id) : si.bus(
2091 //     BUS_SIZE);
2092 // `--
2093 // Where:
2094 //
2095 // * `BUS_SIZE`: number of outputs from each bus (int, known at compile time).
2096 // * `NUM_BUSES`: number of buses (int, known at compile time).
2097 // * `id`: index of the bus to select (int, `0<=id<NUM_BUSES`)
2098 //
2099 // #### Test
2100 // `--
2101 // ba = library("basics.lib");
2102 // selectbus_test = (1,2,3,4) : ba.selectbus(2, 2, 1);
2103 // `--
2104 //-----
2105 selectbus(BUS_SIZE, NUM_BUSES, id) = ro.interleave(BUS_SIZE, NUM_BUSES) : par(j, BUS_SIZE,
2106     selectn(NUM_BUSES, id));
2107 declare selectbus author "David Braun";
2108 declare selectbus license "MIT";
2109
2110 //-----`ba).selectxbus`-----
2111 // Like `ba.selectbus`, but with a cross-fade when selecting the bus using the same
2112 // technique than `ba.selectmulti`.
2113 //

```

```

2114 // #### Usage
2115 //
2116 // ...
2117 // process = si.bus(BUS_SIZE*NUM_BUSES) : selectbus(BUS_SIZE, NUM_BUSES, FADE, id) : si.bus(
    BUS_SIZE);
2118 // ...
2119 //
2120 // Where:
2121 //
2122 // * `BUS_SIZE`: number of outputs from each bus (int, known at compile time).
2123 // * `NUM_BUSES`: number of buses (int, known at compile time).
2124 // * `fade`: number of samples for the crossfade.
2125 // * `id`: index of the bus to select (int, `0<=id<NUM_BUSES`)
2126 //
2127 // #### Test
2128 // ...
2129 // ba = library("basics.lib");
2130 // selectxbus_test = (1,2,3,4) : ba.selectxbus(2, 2, 16, checkbox("bus"));
2131 // ...
2132 //-----
2133 selectxbus(BUS_SIZE, NUM_BUSES, fade, id) =
2134   ro.interleave(BUS_SIZE, NUM_BUSES):
2135   par(i, BUS_SIZE, selectnX(NUM_BUSES, id, xfade))
2136   with {
2137     xfade(i, j, x, y) = x*(1-xb) + y*xb with {
2138       xb = ramp(fade, (i >= j));
2139     };
2140   };
2141 declare selectxbus author "Marc Lavallée";
2142 declare selectxbus license "MIT";
2143
2144
2145 //-----`(ba.)selectmulti`-----
2146 // Selects the ith circuit among N at run time (all should have the same number of inputs and
    outputs)
2147 // with a crossfade.
2148 //
2149 // #### Usage
2150 //
2151 // ...
2152 // selectmulti(n,lgen,id)
2153 // ...
2154 //
2155 // Where:
2156 //
2157 // * `n`: crossfade in samples
2158 // * `lgen`: list of circuits
2159 // * `id`: circuit to select (int, numbered from 0)
2160 //
2161 // #### Example test program
2162 //
2163 // ...
2164 // process = selectmulti(ma.SR/10, ((3,9),(2,8),(5,7)), nentry("choice", 0, 0, 2, 1));
2165 // process = selectmulti(ma.SR/10, ((_*3,_*9),(_*2,_*8),(_*5,_*7)), nentry("choice", 0, 0, 2,
    1));
2166 // ...
2167 //
2168 // #### Test
2169 // ...
2170 // ba = library("basics.lib");
2171 // os = library("oscillators.lib");
2172 // ma = library("maths.lib");
2173 // effects = ((_*0.5,_*0.5),(_*0.25,_*0.25));
2174 // choice = int(checkbox("choice"));
2175 // selectmulti_test = (os.osc(440), os.osc(660)) : ba.selectmulti(ma.SR/100, effects, choice)
    ;
2176 // ...
2177 //-----

```

```

2178 selectmulti(n, lgen, id) = selectmultiX(ins, lgen, id)
2179 with {
2180   selectmultiX(0, lgen, id) = selector;           // No inputs
2181   selectmultiX(N, lgen, id) = par(i, ins, _) <: selector; // General case
2182
2183   selector = lgen : ro.interleave(outs, N) : par(i, outs, selectnX(N, id, xfade))
2184   with {
2185     // crossfade of 'n' samples between 'x' and 'y' channels when the channel index changes
2186     xfade(i, j, x, y) = x*(1-xb) + y*xb with { xb = ramp(n, (i >= j)); };
2187   };
2188
2189   outs = outputs(take(1, lgen)); // Number of outputs of the first circuit (all should have
                                   // the same value)
2190   ins = inputs(take(1, lgen)); // Number of inputs of the first circuit (all should have the
                                   // same value)
2191   N = outputs(lgen)/outs;      // Number of items in the list
2192 };
2193
2194 //-----`(ba.)selectoutn`-----
2195 // Route input to the output among N at run time.
2196 //
2197 // #### Usage
2198 //
2199 // ```
2200 // _ : selectoutn(N, i) : si.bus(N)
2201 // ```
2202 //
2203 // Where:
2204 //
2205 // * `N`: number of outputs (int, known at compile time, N > 0)
2206 // * `i`: output number to route to (int, numbered from 0) (i.e. slider)
2207 //
2208 // #### Example test program
2209 //
2210 // ```
2211 // process = 1 : selectoutn(3, sel) : par(i, 3, vbargraph("v.bargraph %i", 0, 1));
2212 // sel = hslider("volume", 0, 0, 2, 1) : int;
2213 // ```
2214 //
2215 // #### Test
2216 //
2217 // ```
2218 // ba = library("basics.lib");
2219 // selectoutn_test = 1 : ba.selectoutn(3, 1);
2220 // ```
2221 //-----
2222 declare selectoutn author "Vince";
2223
2224 selectoutn(N, s) = _ <: par(i, N, *(s==i));
2225
2226
2227 //=====Other=====
2228 //=====
2229
2230 //-----`(ba.)latch`-----
2231 // Latch input on the rising edge of trig.
2232 // Captures ("records") the input x whenever trig crosses from 0 to >0,
2233 // and holds the last captured value at all other times.
2234 //
2235 // #### Usage
2236 //
2237 // ```
2238 // _ : latch(trig) : _
2239 // ```
2240 // Where:
2241 //
2242 // * `trig`: trigger signal. A rising edge (0 → >0) samples the input.
2243 //

```

```

2244 // #### Test
2245 // ```
2246 // ba = library("basics.lib");
2247 // os = library("oscillators.lib");
2248 // latch_test = os.osc(2) : ba.latch(ba.pulse(32));
2249 // ```
2250 //-----
2251 latch(trig, x) = x * s : + ~ *(1-s) with { s = (trig' <= 0) & (trig > 0); };
2252
2253
2254 //-----`ba.sAndH`-----
2255 // Sample And Hold: "records" the input when trig is 1, outputs a frozen value when trig is
2256 // 0.
2257 // `sAndH` is a standard Faust function.
2258 //
2259 // #### Usage
2260 // ```
2261 // _ : sAndH(trig) : _
2262 // ```
2263 //
2264 // Where:
2265 //
2266 // * `trig`: hold trigger (0 for hold, 1 for bypass)
2267 //
2268 // #### Test
2269 // ```
2270 // ba = library("basics.lib");
2271 // os = library("oscillators.lib");
2272 // sAndH_test = os.osc(2) : ba.sAndH(ba.pulse(32));
2273 // ```
2274 //-----
2275 declare sAndH author "Romain Michon";
2276
2277 sAndH(trig) = select2(trig) ~ _;
2278
2279 //-----`ba.tAndH`-----
2280 // Test And Hold: "records" the input when pred(input) is true, outputs a frozen value
2281 // otherwise.
2282 //
2283 // #### Usage
2284 // ```
2285 // _ : tAndH(pred) : _
2286 // ```
2287 //
2288 // Where:
2289 //
2290 // * `pred`: predicate to test the input
2291 //
2292 // #### Test
2293 // ```
2294 // ba = library("basics.lib");
2295 // os = library("oscillators.lib");
2296 // isPositive(x) = x > 0.0;
2297 // tAndH_test = os.osc(2) : ba.tAndH(isPositive);
2298 // ```
2299 //-----
2300 tAndH(pred) = _ <: pred, _ : sAndH;
2301
2302 //-----`ba.downSample`-----
2303 // Down sample a signal. WARNING: this function doesn't change the
2304 // rate of a signal, it just holds samples...
2305 // `downSample` is a standard Faust function.
2306 //
2307 // #### Usage
2308 // ```
2309 // ```

```

```

2310 // _ : downSample(freq) : _
2311 // ...
2312 //
2313 // Where:
2314 //
2315 // * `freq`: new rate in Hz
2316 //
2317 // #### Test
2318 // ...
2319 // ba = library("basics.lib");
2320 // os = library("oscillators.lib");
2321 // downSample_test = os.osc(440) : ba.downSample(11025);
2322 // ...
2323 //-----
2324 declare downSample author "Romain Michon";
2325
2326 downSample(freq) = sAndH(hold)
2327 with {
2328   hold = time%int(ma.SR/freq) == 0;
2329 };
2330
2331
2332 //-----`(ba.)downSampleCV`-----
2333 //
2334 // A version of `ba.downSample` where the frequency parameter has
2335 // been replaced by an `amount` parameter that is in the range zero
2336 // to one. WARNING: this function doesn't change the rate of a
2337 // signal, it just holds samples...
2338 //
2339 // #### Usage
2340 // ...
2341 // _ : downSampleCV(amount) : _
2342 // ...
2343 // Where:
2344 //
2345 // * `amount`: The amount of down-sampling to perform [0..1]
2346 //
2347 // #### Test
2348 // ...
2349 // ba = library("basics.lib");
2350 // os = library("oscillators.lib");
2351 // downSampleCV_test = os.osc(440) : ba.downSampleCV(0.5);
2352 // ...
2353 //-----
2354 downSampleCV(amt) = sAndH(hold)
2355 with {
2356   // If amt is 0, then freq should be ma.SR/2.
2357   // If amt is 1, then freq should be 491.
2358   // 491 is derived from listening to the Serum synthesizer.
2359   freq = amt : it.remap(0, 1, hz2midikey(ma.SR/2), hz2midikey(491)) : midikey2hz;
2360   hold = time%int(ma.SR/freq) == 0;
2361 };
2362
2363 declare downSampleCV author "Romain Michon/David Braun";
2364
2365
2366 //-----`(ba.)peakhold`-----
2367 // Outputs current max value above zero.
2368 //
2369 // #### Usage
2370 //
2371 // ...
2372 // _ : peakhold(mode) : _
2373 // ...
2374 //
2375 // Where:
2376 //
2377 // * `mode`: operation mode

```

```

2378 // `0` passes the signal through and a single-sample `0` trigger resets the held value
2379 // `1` tracks and holds the maximum value
2380 //
2381 // #### Test
2382 // ...
2383 // ba = library("basics.lib");
2384 // os = library("oscillators.lib");
2385 // peakhold_test = os.osc(440) : ba.peakhold(1);
2386 // ...
2387 //-----
2388 declare peakhold author "Jonatan Liljedahl, revised by Romain Michon";
2389
2390 peakhold = (*, _ : max) ~ _;
2391
2392 //-----`(ba.)peakholder`-----
2393 //
2394 // While peak-holder functions are scarcely discussed in the literature
2395 // (please do send me an email if you know otherwise), common sense
2396 // tells that the expected behaviour should be as follows: the absolute
2397 // value of the input signal is compared with the output of the peak-holder;
2398 // if the input is greater or equal to the output, a new peak is detected
2399 // and sent to the output; otherwise, a timer starts and the current peak
2400 // is held for N samples; once the timer is out and no new peaks have been
2401 // detected, the absolute value of the current input becomes the new peak.
2402 //
2403 // #### Usage
2404 //
2405 // ...
2406 // _ : peakholder(holdTime) : _
2407 // ...
2408 //
2409 // Where:
2410 //
2411 // * `holdTime`: hold time in samples
2412 //
2413 // #### Test
2414 // ...
2415 // ba = library("basics.lib");
2416 // os = library("oscillators.lib");
2417 // peakholder_test = os.osc(440) : ba.peakholder(ba.sec2samp(0.1));
2418 // ...
2419 //-----
2420 declare peakholder author "Dario Sanfilippo";
2421 declare peakholder copyright
2422 "Copyright (C) 2022 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
2423 declare peakholder license "MIT-style STK-4.3 license";
2424 peakholder(holdTime, x) = loop ~ si.bus(2) : !, _
2425   with {
2426     loop(timerState, outState) = timer , output
2427     with {
2428       isNewPeak = abs(x) >= outState;
2429       isTimeOut = timerState >= holdTime;
2430       bypass = isNewPeak | isTimeOut;
2431       timer = ba.if(bypass, 0, timerState + 1);
2432       output = ba.if(bypass, abs(x), outState);
2433     };
2434   };
2435
2436 /*
2437 // Alternate version with branchless code
2438 //-----
2439 peakholder(holdTime, x) = loop ~ si.bus(2) : !, _
2440   with {
2441     loop(timerState, outState) = timer , output
2442     with {
2443       isNewPeak = abs(x) >= outState;
2444       isTimeOut = timerState >= holdTime;

```

```

2446         bypass = isNewPeak | isTimeout;
2447         timer = (1 - bypass) * (timerState + 1);
2448         output = bypass * (abs(x) - outState) + outState;
2449     };
2450 };
2451 */
2452
2453 /*
2454 // The function below is kept for back-compatibility in case any user relies
2455 // on it for their software. However, the function behaves differently than
2456 // expected: currently, the timer is not reset when a new peak is detected.
2457 //-----
2458 declare peakholder author "Jonatan Liljedahl";
2459 peakholder(n) = peakhold2 ~ reset : (!,_) with {
2460     reset = sweep(n) > 0;
2461     // first out is gate that is 1 while holding last peak
2462     peakhold2 = _,abs <: peakhold,!,_ <: >=,_,!;
2463 };
2464 */
2465
2466 //-----`(ba.)kr2ar`-----
2467 // Force a control rate signal to be used as an audio rate signal.
2468 //
2469 // #### Usage
2470 //
2471 // ```
2472 // kr2ar : _
2473 // ```
2474 //
2475 // Typical use:
2476 //
2477 // ```
2478 // hslider("freq", 200, 200, 2000, 0.1) : kr2ar;
2479 // ```
2480 //
2481 // #### Test
2482 //
2483 // ```
2484 // ba = library("basics.lib");
2485 // kr2ar_test = button("gate") : ba.kr2ar;
2486 // ```
2487 //-----
2488 kr2ar = + ~ * (0);
2489
2490 //-----`(ba.)impulsify`-----
2491 // Turns a signal into an impulse with the value of the current sample
2492 // (0.3,0.2,0.1 becomes 0.3,0.0,0.0). This function is typically used with a
2493 // `button` to turn its output into an impulse. `impulsify` is a standard Faust
2494 // function.
2495 //
2496 // #### Usage
2497 //
2498 // ```
2499 // button("gate") : impulsify;
2500 // ```
2501 //
2502 // #### Test
2503 //
2504 // ```
2505 // ba = library("basics.lib");
2506 // os = library("oscillators.lib");
2507 // impulsify_test = os.osc(440) : ba.impulsify;
2508 // ```
2509 //-----
2510 impulsify = _ <: _,mem : - <: >(0)*_;
2511
2512 //-----`(ba.)automat`-----
2513 // Record and replay in a loop the successive values of the input signal.

```

```

2514 //
2515 // ##### Usage
2516 //
2517 // ...
2518 // hslider(...) : automat(t, size, init) : _
2519 // ...
2520 //
2521 // Where:
2522 //
2523 // * `t`: tempo in BPM
2524 // * `size`: number of items in the loop
2525 // * `init`: init value in the loop
2526 //
2527 // ##### Test
2528 // ...
2529 // ba = library("basics.lib");
2530 // autoControl = hslider("autoControl", 0.2, 0, 1, 0.01);
2531 // automat_test = autoControl : ba.automat(120, 4, 0.0);
2532 // ...
2533 //-----
2534 automat(t, size, init, input) = rwttable(size+1, init, winindex, input, rindex)
2535 with {
2536   clock = beat(t);
2537   rindex = int(clock) : (+ : %(size)) ~ _; // each clock read the next entry of the table
2538   winindex = if(timeToRenew, rindex, size); // we ignore input unless it is time to renew
2539   timeToRenew = int(clock) & (inputHasMoved | (input <= init));
2540   inputHasMoved = abs(input-input') : countfrom(int(clock)') : >(0);
2541   countfrom(reset) = (+ : if(reset, 0, _)) ~ _;
2542 };
2543
2544
2545 //-----`ba.`bpf`-----
2546 // bpf is an environment (a group of related definitions) that can be used to
2547 // create break-point functions. It contains three functions:
2548 //
2549 // * `start(x,y)` to start a break-point function
2550 // * `end(x,y)` to end a break-point function
2551 // * `point(x,y)` to add intermediate points to a break-point function, using linear
2552   interpolation
2553 //
2554 // A minimal break-point function must contain at least a start and an end point:
2555 //
2556 // ...
2557 // f = bpf.start(x0,y0) : bpf.end(x1,y1);
2558 // ...
2559 //
2560 // A more involved break-point function can contains any number of intermediate
2561 // points:
2562 //
2563 // ...
2564 // f = bpf.start(x0,y0) : bpf.point(x1,y1) : bpf.point(x2,y2) : bpf.end(x3,y3);
2565 // ...
2566 //
2567 // In any case the `x_{i}` must be in increasing order (for all `i`, `x_{i} < x_{i+1}`).
2568 // For example the following definition:
2569 //
2570 // ...
2571 // f = bpf.start(x0,y0) : ... : bpf.point(xi,yi) : ... : bpf.end(xn,yn);
2572 // ...
2573 //
2574 // implements a break-point function f such that:
2575 //
2576 // * `f(x) = y_{0}` when `x < x_{0}`
2577 // * `f(x) = y_{n}` when `x > x_{n}`
2578 // * `f(x) = y_{i} + (y_{i+1}-y_{i})*(x-x_{i})/(x_{i+1}-x_{i})` when `x_{i} <= x`
2579 // and `x < x_{i+1}`
2580 //
2581 // In addition to `bpf.point`, there are also `step` and `curve` functions:

```

```

2581 //
2582 // * `step(x,y)` to add a flat section
2583 // * `step_end(x,y)` to end with a flat section
2584 // * `curve(B,x,y)` to add a curved section
2585 // * `curve_end(B,x,y)` to end with a curved section
2586 //
2587 // These functions can be combined with the other `bpf` functions.
2588 //
2589 // Here's an example using `bpf.step`:
2590 //
2591 // `f(x) = x : bpf.start(0,0) : bpf.step(.2,.3) : bpf.step(.4,.6) : bpf.step_end(1,1);`
2592 //
2593 // For `x < 0.0`, the output is 0.0.
2594 // For `0.0 <= x < 0.2`, the output is 0.0.
2595 // For `0.2 <= x < 0.4`, the output is 0.3.
2596 // For `0.4 <= x < 1.0`, the output is 0.6.
2597 // For `1.0 <= x`, the output is 1.0
2598 //
2599 // For the `curve` functions, `B` (compile-time constant)
2600 // is a "bias" value strictly greater than zero and less than or equal to 1. When `B` is 0.5,
2601 // the
2602 // output curve is exactly linear and equivalent to `bpf.point`. When `B` is less than 0.5,
2603 // the
2604 // output is biased towards the `y` value of the previous breakpoint. When `B` is greater
2605 // than 0.5,
2606 // the output is biased towards the `y` value of the curve breakpoint. Here's an example:
2607 //
2608 // `f = bpf.start(0,0) : bpf.curve(.15,.5,.5) : bpf.curve_end(.85,1,1);`
2609 //
2610 // In the following example, the output is biased towards zero (the latter y value) instead
2611 // of
2612 // being a linear ramp from 1 to 0.
2613 //
2614 // `f = bpf.start(0,1) : bpf.curve_end(.9,1,0);`
2615 //
2616 // `bpf` is a standard Faust function.
2617 //-----
2618 bpf = environment
2619 {
2620     // Start a break-point function
2621     start(x0,y0) = \x.(x0,y0,x,y0);
2622     // Add a break-point
2623     point(x1,y1) = \x0,y0,x,y.(x1, y1, x, if(x < x0, y, if(x < x1, y0 + (x-x0)*(y1-y0)/(x1-x0), y1)));
2624     // End a break-point function
2625     end(x1,y1) = \x0,y0,x,y.(if(x < x0, y, if(x < x1, y0 + (x-x0)*(y1-y0)/(x1-x0), y1)));
2626
2627     // Add a stepped "flat" section
2628     step(x1,y1) = \x0,y0,x,y.(x1, y1, x, if(x < x0, y, if(x < x1, y0, y1)));
2629     // End with a stepped "flat" section
2630     step_end(x1,y1) = \x0,y0,x,y.(if(x < x0, y, if(x < x1, y0, y1)));
2631
2632     // Add a curve-point. `B` (compile-time constant) must be in range (0,1]
2633     curve(B,x1,y1) = \x0,y0,x,y.(x1, y1, x, if(x < x0, y, if(x < x1, _curve_util(x0,x1,y0,
2634     y1,B,x), y1)));
2635     // End with a curve-point. `B` (compile-time constant) must be in range (0,1]
2636     curve_end(B,x1,y1) = \x0,y0,x,y.(if(x < x0, y, if(x < x1, _curve_util(x0,x1,y0,y1,B,x),
2637     y1)));
2638
2639     /*
2640     In the functions below, we use the equation `y=bias_curve(b,x)`.
2641     * `b`: bias in range (0,1]. Bias of 0.5 results in `y=x`. Bias above 0.5 pulls y upward.
2642     Bias below 0.5 pulls y downward.
2643     * `x`: input in range [0,1] that needs to be remapped/biased into `y`
2644     * `y`: `x` after it has been biased. Note that `(y==0 iff x==0) AND (y==1 iff x==1)`.
2645     */
2646     _bias_curve(b,x) = (x / (((1.0/b) - 2.0)*(1.0 - x))+1.0));

```

```

2641 _curve_util(x0,x1,y0,y1,b,x) = y
2642 with {
2643     x_norm = (x-x0)/(x1-x0); // x_norm is in [0-1]
2644     y_norm = _bias_curve(b, x_norm); // y_norm is in [0-1]
2645     y = (1-y_norm)*y0 + y_norm*y1; // linear interpolation between y0 and y1
2646 };
2647 };
2648
2649
2650 //-----`(ba.)listInterp`-----
2651 // Linearly interpolates between the elements of a list.
2652 //
2653 // #### Usage
2654 //
2655 // ```
2656 // index = 1.69; // range is 0-4
2657 // process = listInterp((800,400,350,450,325),index);
2658 // ```
2659 //
2660 // Where:
2661 //
2662 // * `index`: the index (float) to interpolate between the different values.
2663 // The range of `index` depends on the size of the list.
2664 //
2665 // #### Test
2666 //
2667 // ```
2668 // ba = library("basics.lib");
2669 // listInterp_test = ba.listInterp((800,400,350,450,325), 1.5);
2670 // ```
2671 //-----
2672 declare listInterp author "Romain Michon";
2673
2674 listInterp(v) =
2675     bpf.start(0,take(1,v)) :
2676     seq(i,count(v)-2,bpf.point(i+1,take(i+2,v))) :
2677     bpf.end(count(v)-1,take(count(v),v));
2678
2679 //-----`(ba.)bypass1`-----
2680 // Takes a mono input signal, route it to `e` and bypass it if `bpc = 1`.
2681 // When bypassed, `e` is feed with zeros so that its state is cleanup up.
2682 // `bypass1` is a standard Faust function.
2683 //
2684 // #### Usage
2685 //
2686 // ```
2687 // _ : bypass1(bpc,e) : _
2688 // ```
2689 //
2690 // Where:
2691 //
2692 // * `bpc`: bypass switch (0/1)
2693 // * `e`: a mono effect
2694 //
2695 // #### Test
2696 //
2697 // ```
2698 // ba = library("basics.lib");
2699 // os = library("oscillators.lib");
2700 // bypass1_test = os.osc(440) : ba.bypass1(button("bypass"), *(0.5));
2701 // ```
2702 //-----
2703 declare bypass1 author "Julius Smith";
2704 // License: STK-4.3
2705
2706 bypass1(bpc,e) = _ <: select2(bpc,(inswitch:e),_)
2707 with {
2708     inswitch = select2(bpc,_,0);

```

```

2709 };
2710
2711
2712 //-----`(ba.)bypass2`-----
2713 // Takes a stereo input signal, route it to `e` and bypass it if `bpc = 1`.
2714 // When bypassed, `e` is feed with zeros so that its state is cleanup up.
2715 // `bypass2` is a standard Faust function.
2716 //
2717 // #### Usage
2718 //
2719 // ```
2720 // _ : bypass2(bpc,e) : _,_
2721 // ```
2722 //
2723 // Where:
2724 //
2725 // * `bpc`: bypass switch (0/1)
2726 // * `e`: a stereo effect
2727 //
2728 // #### Test
2729 // ```
2730 // ba = library("basics.lib");
2731 // os = library("oscillators.lib");
2732 // bypass2_test = (os.osc(440), os.osc(660)) : ba.bypass2(button("bypass"), par(i,2, *(0.5)))
2733 // ```
2734 //-----
2735 declare bypass2 author "Julius Smith";
2736 // License: STK-4.3
2737
2738 bypass2(bpc,e) = _,_ <: ((inswitch:e),_,_) : select2stereo(bpc)
2739 with {
2740   inswitch = _,_ : (select2(bpc,_,0), select2(bpc,_,0)) : _,_;
2741 };
2742
2743
2744 //-----`(ba.)bypass1to2`-----
2745 // Bypass switch for effect `e` having mono input signal and stereo output.
2746 // Effect `e` is bypassed if `bpc = 1`.When bypassed, `e` is feed with zeros
2747 // so that its state is cleanup up.
2748 // `bypass1to2` is a standard Faust function.
2749 //
2750 // #### Usage
2751 //
2752 // ```
2753 // _ : bypass1to2(bpc,e) : _,_
2754 // ```
2755 //
2756 // Where:
2757 //
2758 // * `bpc`: bypass switch (0/1)
2759 // * `e`: a mono-to-stereo effect
2760 //
2761 // #### Test
2762 // ```
2763 // ba = library("basics.lib");
2764 // os = library("oscillators.lib");
2765 // monoToStereo(x) = (x*0.5, x*0.25);
2766 // bypass1to2_test = os.osc(440) : ba.bypass1to2(button("bypass"), monoToStereo);
2767 // ```
2768 //-----
2769 declare bypass1to2 author "Julius Smith";
2770 // License: STK-4.3
2771
2772 bypass1to2(bpc,e) = _ <: ((inswitch:e),_,_) : select2stereo(bpc)
2773 with {
2774   inswitch = select2(bpc,_,0);
2775 };

```

```

2776
2777
2778 //-----`(ba.)bypass_fade`-----
2779 // Bypass an arbitrary (N x N) circuit with 'n' samples crossfade.
2780 // Inputs and outputs signals are faded out when 'e' is bypassed,
2781 // so that 'e' state is cleanup up.
2782 // Once bypassed the effect is replaced by `par(i,N,_)`.
2783 // Bypassed circuits can be chained.
2784 //
2785 // #### Usage
2786 //
2787 // ```
2788 // _ : bypass_fade(n,b,e) : _
2789 // or
2790 // _ : bypass_fade(n,b,e) : _,_
2791 // ```
2792 //
2793 // Where:
2794 //
2795 // * `n`: number of samples for the crossfade
2796 // * `b`: bypass switch (0/1)
2797 // * `e`: N x N circuit
2798 //
2799 // #### Example test program
2800 //
2801 // ```
2802 // process = bypass_fade(ma.SR/10, checkbox("bypass echo"), echo);
2803 // process = bypass_fade(ma.SR/10, checkbox("bypass reverb"), freeverb);
2804 // ```
2805 //
2806 // #### Test
2807 // ```
2808 // ba = library("basics.lib");
2809 // os = library("oscillators.lib");
2810 // bypass_fade_test = (os.osc(440), os.osc(660)) : ba.bypass_fade(128, button("bypass"), par(
2811 //   i,2, *(0.5)));
2812 // ```
2813 //-----
2814 bypass_fade(n, b, e) = par(i, ins, _)
2815   <: (par(i, ins, *(1-xb)) : e : par(i, outs, *(1-xb))), par(i, ins, *(xb))
2816   >: par(i, outs, _)
2817
2818 with {
2819   ins = inputs(e);
2820   outs = outputs(e);
2821   xb = ramp(n, b);
2822 }
2823
2824 //-----`(ba.)toggle`-----
2825 // Triggered by the change of 0 to 1, it toggles the output value
2826 // between 0 and 1.
2827 //
2828 // #### Usage
2829 //
2830 // ```
2831 // _ : toggle : _
2832 // ```
2833 //
2834 // #### Example test program
2835 //
2836 // ```
2837 // button("toggle") : toggle : vbargraph("output", 0, 1)
2838 // (an.amp_follower(0.1) > 0.01) : toggle : vbargraph("output", 0, 1) // takes audio input
2839 // ```
2840 //
2841 // #### Test
2842 // ```
2843 // ba = library("basics.lib");
2844 // toggle_test = ba.toggle(button("trig"));

```

```

2843 // ---
2844 //-----
2845 declare toggle author "Vince";
2846
2847 toggle = trig : loop
2848 with {
2849     trig(x) = (x-x') == 1;
2850     loop = != ~ _;
2851 };
2852
2853
2854 //-----`(ba.)on_and_off`-----
2855 // The first channel set the output to 1, the second channel to 0.
2856 //
2857 // #### Usage
2858 //
2859 // ---
2860 // _ : on_and_off : _
2861 // ---
2862 //
2863 // #### Example test program
2864 //
2865 // ---
2866 // button("on"), button("off") : on_and_off : vbargraph("output", 0, 1)
2867 // ---
2868 //
2869 // #### Test
2870 // ---
2871 // ba = library("basics.lib");
2872 // on_and_off_test = button("on"), button("off") : ba.on_and_off;
2873 // ---
2874 //-----
2875 declare on_and_off author "Vince";
2876
2877 on_and_off(a, b) = (a : trig) : loop(b)
2878 with {
2879     trig(x) = (x-x') == 1;
2880     loop(b) = + ~ (_ >= 1) * ((b : trig) == 0);
2881 };
2882
2883
2884 //-----`(ba.)bitcrusher`-----
2885 // Produce distortion by reduction of the signal resolution.
2886 //
2887 // #### Usage
2888 //
2889 // ---
2890 // _ : bitcrusher(nbits) : _
2891 // ---
2892 //
2893 // Where:
2894 //
2895 // * `nbits`: the number of bits of the wanted resolution
2896 //
2897 // #### Test
2898 // ---
2899 // ba = library("basics.lib");
2900 // os = library("oscillators.lib");
2901 // bitcrusher_test = os.osc(440) : ba.bitcrusher(8);
2902 // ---
2903 //-----
2904 declare bitcrusher author "Julius O. Smith III, revised by Stephane Letz";
2905
2906 bitcrusher(nbits,x) = round(x * scaler) / scaler
2907 with {
2908     scaler = float(2^nbits - 1);
2909 };
2910

```

```

2911 //-----`(ba.)mulaw_bitcrusher
2912 // Produce distortion by reducing the signal resolution using -law compression.
2913 //
2914 // #### Usage
2915 //
2916 // ...
2917 // _ : mulaw_bitcrusher(mu,nbits) : _
2918 // ...
2919 //
2920 // Where:
2921 //
2922 // * `mu`: controls the degree of -law compression, larger values result in stronger
2923 // compression
2924 // * `nbits`: the number of bits of the wanted resolution
2925 //
2926 // #### Description
2927 //
2928 // The `mulaw_bitcrusher` applies a combination of -law compression, quantization, and
2929 // expansion
2930 // to create a non-linear bitcrushed effect. This method retains finer detail in lower-
2931 // amplitude signals
2932 // compared to linear bitcrushing, making it suitable for creative sound design.
2933 //
2934 // #### Theory
2935 //
2936 // 1. -law Compression:
2937 // emphasizes lower-amplitude signals by applying a logarithmic curve to the signal.
2938 // The formula used is:
2939 // ...
2940 // 
$$F(x) = \text{ma.signum}(x) * \log(1 + \mu * \text{abs}(x)) / \log(1 + \mu);$$

2941 // ...
2942 // 2. Quantization:
2943 // reduces the signal resolution to `nbits` by rounding values to the nearest step within
2944 // the specified bit depth.
2945 //
2946 // 3. -law Expansion:
2947 // reverses the compression applied earlier to restore the signal to its original dynamic
2948 // range:
2949 // ...
2950 // 
$$F'(y) = \text{ma.signum}(y) * (\text{pow}(1 + \mu, \text{abs}(y)) - 1) / \mu;$$

2951 // ...
2952 // #### Example test program
2953 //
2954 // ...
2955 // process = os.osc(440) : mulaw_bitcrusher(255, 8);
2956 // ...
2957 //
2958 // In this example, a sine wave at 440 Hz is passed through the -law bitcrusher, with a
2959 // compression
2960 // parameter `mu` of 255 and 8-bit quantization. This creates a distorted, "lo-fi" effect.
2961 //
2962 // #### Test
2963 //
2964 // ...
2965 // ba = library("basics.lib");
2966 // os = library("oscillators.lib");
2967 // mulaw_bitcrusher_test = os.osc(440) : ba.mulaw_bitcrusher(2.0, 8);
2968 // ...
2969 // #### References
2970 //
2971 // * <https://en.wikipedia.org/wiki/M/-law\_algorithm>
2972 //-----
2973
2974 declare mulaw_bitcrusher author "Stephane Letz";
2975
2976 mulaw_bitcrusher(mu,nbits,x) = x : muLawCompress(mu) : bitcrusher(nbits) : muLawExpand(mu)
2977 with {

```

```

2972 // -law compression
2973 muLawCompress(mu, x) = ma.signum(x) * log(1 + mu * abs(x)) / log(1 + mu);
2974
2975 // -law expansion (decompression)
2976 muLawExpand(mu, x) = ma.signum(x) * (pow(1 + mu, abs(x)) - 1) / mu;
2977 };
2978
2979
2980 //=====Sliding Reduce=====
2981 // Provides various operations on the last n samples using a high order
2982 // `slidingReduce(op,n,maxN,disabledVal,x)` fold-like function:
2983 //
2984 // * `slidingSum(n)`: the sliding sum of the last n input samples, CPU-light
2985 // * `slidingSump(n,maxN)`: the sliding sum of the last n input samples, numerically stable "
  forever"
2986 // * `slidingMax(n,maxN)`: the sliding max of the last n input samples
2987 // * `slidingMin(n,maxN)`: the sliding min of the last n input samples
2988 // * `slidingMean(n)`: the sliding mean of the last n input samples, CPU-light
2989 // * `slidingMeanp(n,maxN)`: the sliding mean of the last n input samples, numerically stable
  "forever"
2990 // * `slidingRMS(n)`: the sliding RMS of the last n input samples, CPU-light
2991 // * `slidingRMSp(n,maxN)`: the sliding RMS of the last n input samples, numerically stable "
  forever"
2992 //
2993 // #### Working Principle
2994 //
2995 // If we want the maximum of the last 8 values, we can do that as:
2996 //
2997 // ...
2998 // simpleMax(x) =
2999 // (
3000 //   (
3001 //     max(x@0,x@1),
3002 //     max(x@2,x@3)
3003 //   ):max
3004 // ),
3005 // (
3006 //   (
3007 //     max(x@4,x@5),
3008 //     max(x@6,x@7)
3009 //   ):max
3010 // )
3011 // :max;
3012 // ...
3013 //
3014 // `max(x@2,x@3)` is the same as `max(x@0,x@1)@2` but the latter re-uses a
3015 // value we already computed,so is more efficient. Using the same trick for
3016 // values 4 trough 7, we can write:
3017 //
3018 // ...
3019 // efficientMax(x)=
3020 // (
3021 //   (
3022 //     max(x@0,x@1),
3023 //     max(x@0,x@1)@2
3024 //   ):max
3025 // ),
3026 // (
3027 //   (
3028 //     max(x@0,x@1),
3029 //     max(x@0,x@1)@2
3030 //   ):max@4
3031 // )
3032 // :max;
3033 // ...
3034 //
3035 // We can rewrite it recursively, so it becomes possible to get the maximum at
3036 // have any number of values, as long as it's a power of 2.

```

```

3037 //
3038 // ...
3039 // recursiveMax =
3040 // case {
3041 //   (1,x) => x;
3042 //   (N,x) => max(recursiveMax(N/2,x), recursiveMax(N/2,x)@(N/2));
3043 // };
3044 // ...
3045 //
3046 // What if we want to look at a number of values that's not a power of 2?
3047 // For each value, we will have to decide whether to use it or not.
3048 // If n is bigger than the index of the value, we use it, otherwise we replace
3049 // it with (0-(ma.MAX)):
3050 // ...
3051 //
3052 // variableMax(n,x) =
3053 // max(
3054 //   max(
3055 //     (
3056 //       (x@0 : useVal(0)),
3057 //       (x@1 : useVal(1))
3058 //     ):max,
3059 //     (
3060 //       (x@2 : useVal(2)),
3061 //       (x@3 : useVal(3))
3062 //     ):max
3063 //   ),
3064 //   max(
3065 //     (
3066 //       (x@4 : useVal(4)),
3067 //       (x@5 : useVal(5))
3068 //     ):max,
3069 //     (
3070 //       (x@6 : useVal(6)),
3071 //       (x@7 : useVal(7))
3072 //     ):max
3073 //   )
3074 // )
3075 // with {
3076 //   useVal(i) = select2((n>=i) , (0-(ma.MAX)),_);
3077 // };
3078 // ...
3079 //
3080 // Now it becomes impossible to re-use any values. To fix that let's first look
3081 // at how we'd implement it using recursiveMax, but with a fixed n that is not
3082 // a power of 2. For example, this is how you'd do it with `n=3`:
3083 // ...
3084 //
3085 // binaryMaxThree(x) =
3086 // (
3087 //   recursiveMax(1,x)@0, // the first x
3088 //   recursiveMax(2,x)@1 // the second and third x
3089 // ):max;
3090 // ...
3091 //
3092 // `n=6`
3093 // ...
3094 //
3095 // binaryMaxSix(x) =
3096 // (
3097 //   recursiveMax(2,x)@0, // first two
3098 //   recursiveMax(4,x)@2 // third through sixth
3099 // ):max;
3100 // ...
3101 //
3102 // Note that `recursiveMax(2,x)` is used at a different delay than in
3103 // `binaryMaxThree`, since it represents 1 and 2, not 2 and 3. Each block is
3104 // delayed the combined size of the previous blocks.

```

```

3105 //
3106 // `n=7`
3107 //
3108 // ...
3109 // binaryMaxSeven(x) =
3110 // (
3111 // (
3112 //     recursiveMax(1,x)@0, // first x
3113 //     recursiveMax(2,x)@1 // second and third
3114 // ):max,
3115 // (
3116 //     recursiveMax(4,x)@3 // fourth through seventh
3117 // )
3118 // ):max;
3119 // ...
3120 //
3121 // To make a variable version, we need to know which powers of two are used,
3122 // and at which delay time.
3123 //
3124 // Then it becomes a matter of:
3125 //
3126 // * lining up all the different block sizes in parallel: `sequentialOperatorParOut()`
3127 // * delaying each the appropriate amount: `sumOfPrevBlockSizes()`
3128 // * turning it on or off: `useVal()`
3129 // * getting the maximum of all of them: `parallelOp()`
3130 //
3131 // In Faust, we can only do that for a fixed maximum number of values: `maxN`, known at
    compile time.
3132
3133 //=====
3134 // Section contributed by Bart Brouns (bart@magnetophon.nl).
3135 // SPDX-License-Identifier: GPL-3.0
3136 // Copyright (C) 2018 Bart Brouns
3137
3138
3139 //-----`(ba).slidingReduce`-----
3140 // Fold-like high order function. Apply a commutative binary operation `op` to
3141 // the last `n` consecutive samples of a signal `x`. For example :
3142 // `slidingReduce(max,128,128,0-(ma.MAX))` will compute the maximum of the last
3143 // 128 samples. The output is updated each sample, unlike reduce, where the
3144 // output is constant for the duration of a block.
3145 //
3146 // #### Usage
3147 //
3148 // ...
3149 // _ : slidingReduce(op,n,maxN,disabledVal) : _
3150 // ...
3151 //
3152 // Where:
3153 //
3154 // * `n`: the number of values to process
3155 // * `maxN`: the maximum number of values to process (int, known at compile time, maxN > 0)
3156 // * `op`: the operator. Needs to be a commutative one.
3157 // * `disabledVal`: the value to use when we want to ignore a value.
3158 //
3159 // In other words, `op(x,disabledVal)` should equal to `x`. For example,
3160 // `+(x,0)` equals `x` and `min(x,ma.MAX)` equals `x`. So if we want to
3161 // calculate the sum, we need to give 0 as `disabledVal`, and if we want the
3162 // minimum, we need to give `ma.MAX` as `disabledVal`.
3163 //
3164 // #### Test
3165 // ...
3166 // ba = library("basics.lib");
3167 // os = library("oscillators.lib");
3168 // ma = library("maths.lib");
3169 // slidingReduce_test = os.osc(440) : ba.slidingReduce(max, 64, 64, 0 - ma.MAX);
3170 // ...
3171 //-----

```

```

3172 slidingReduce(op,n,0,disabledVal) = 0!;
3173 slidingReduce(op,n,1,disabledVal) = _;
3174 slidingReduce(op,n,maxN,disabledVal) =
3175   sequentialOperatorParOut(maxNrBits(maxN)-1,op) : par(i, maxNrBits(maxN),
3176     _@sumOfPrevBlockSizes(i) : useVal(i)) : parallelOp(op, maxNrBits(maxN))
3177   with {
3178     sequentialOperatorParOut(N,op) = seq(i, N, operator(i));
3179     operator(i) = si.bus(i), (_<: _ , op(_,_@pow2(i))));
3180
3181     // The sum of all the sizes of the previous blocks
3182     sumOfPrevBlockSizes(0) = 0;
3183     sumOfPrevBlockSizes(i) = (ba.subseq((allBlockSizes),0,i):>_);
3184
3185     allBlockSizes = par(i, maxNrBits(maxN-1), (pow2(i)) * isUsed(i));
3186     maxNrBits(n) = int2nrOfBits(n);
3187
3188     // Decide whether or not to use a certain value, based on n
3189     useVal(i) = select2(isUsed(i), disabledVal, _);
3190
3191     isUsed(i) = ba.take(i+1, (int2bin(n,(maxN-1)*2+1)));
3192     pow2(i) = 1<<i;
3193     // same as:
3194     // pow2(i) = int(pow(2,i));
3195     // but in the block diagram, it will be displayed as a number, instead of a formula
3196
3197     // convert n into a list of ones and zeros
3198     int2bin(n,maxN) = par(j, maxNrBits(maxN-1), int(floor((n)/(pow2(j))))%2);
3199     // calculate how many ones and zeros are needed to represent maxN
3200     int2nrOfBits(n) = int(floor(log(n)/log(2))+1);
3201   };
3202
3203 //-----`ba.slidingSum`-----
3204 // The sliding sum of the last n input samples.
3205 //
3206 // It will eventually run into numerical trouble when there is a persistent dc component.
3207 // If that matters in your application, use the more CPU-intensive `ba.slidingSump`.
3208 //
3209 // #### Usage
3210 //
3211 // ```
3212 // _ : slidingSum(n) : _
3213 // ```
3214 //
3215 // Where:
3216 //
3217 // * `n`: the number of values to process
3218 //
3219 // #### Test
3220 // ```
3221 // ba = library("basics.lib");
3222 // os = library("oscillators.lib");
3223 // slidingSum_test = os.osc(440) : ba.slidingSum(64);
3224 // ```
3225 //-----
3226 slidingSum(n) = fi.integrator <: _ , _@int(max(0,n)) :> -;
3227
3228 //-----`ba.slidingSump`-----
3229 // The sliding sum of the last n input samples.
3230 //
3231 // It uses a lot more CPU than `ba.slidingSum`, but is numerically stable "forever" in return
3232 //
3233 //
3234 // #### Usage
3235 //
3236 // ```
3237 // _ : slidingSump(n,maxN) : _

```

```

3238 //
3239 //
3240 // Where:
3241 //
3242 // * `n`: the number of values to process
3243 // * `maxN`: the maximum number of values to process (int, known at compile time, maxN > 0)
3244 //
3245 // #### Test
3246 //
3247 // ba = library("basics.lib");
3248 // os = library("oscillators.lib");
3249 // slidingSump_test = os.osc(440) : ba.slidingSump(64, 128);
3250 //
3251 //-----
3252 slidingSump(n,maxN) = slidingReduce(+,n,maxN,0);
3253
3254
3255 //-----`(ba.)slidingMax`-----
3256 // The sliding maximum of the last n input samples.
3257 //
3258 // #### Usage
3259 //
3260 //
3261 // _ : slidingMax(n,maxN) : _
3262 //
3263 //
3264 // Where:
3265 //
3266 // * `n`: the number of values to process
3267 // * `maxN`: the maximum number of values to process (int, known at compile time, maxN > 0)
3268 //
3269 // #### Test
3270 //
3271 // ba = library("basics.lib");
3272 // os = library("oscillators.lib");
3273 // ma = library("maths.lib");
3274 // slidingMax_test = os.osc(440) : ba.slidingMax(64, 128);
3275 //
3276 //-----
3277 slidingMax(n,maxN) = slidingReduce(max,n,maxN,0-(ma.MAX));
3278
3279 //-----`(ba.)slidingMin`-----
3280 // The sliding minimum of the last n input samples.
3281 //
3282 // #### Usage
3283 //
3284 //
3285 // _ : slidingMin(n,maxN) : _
3286 //
3287 //
3288 // Where:
3289 //
3290 // * `n`: the number of values to process
3291 // * `maxN`: the maximum number of values to process (int, known at compile time, maxN > 0)
3292 //
3293 // #### Test
3294 //
3295 // ba = library("basics.lib");
3296 // os = library("oscillators.lib");
3297 // ma = library("maths.lib");
3298 // slidingMin_test = os.osc(440) : ba.slidingMin(64, 128);
3299 //
3300 //-----
3301 slidingMin(n,maxN) = slidingReduce(min,n,maxN,ma.MAX);
3302
3303
3304 //-----`(ba.)slidingMean`-----
3305 // The sliding mean of the last n input samples.

```

```

3306 //
3307 // It will eventually run into numerical trouble when there is a persistent dc component.
3308 // If that matters in your application, use the more CPU-intensive `ba.slidingMeanp`.
3309 //
3310 // #### Usage
3311 //
3312 // ```
3313 // _ : slidingMean(n) : _
3314 // ```
3315 //
3316 // Where:
3317 //
3318 // * `n`: the number of values to process
3319 //
3320 // #### Test
3321 // ```
3322 // ba = library("basics.lib");
3323 // os = library("oscillators.lib");
3324 // slidingMean_test = os.osc(440) : ba.slidingMean(64);
3325 // ```
3326 //-----
3327 slidingMean(n) = slidingSum(n)/n;
3328
3329
3330 //-----`(ba.)slidingMeanp`-----
3331 // The sliding mean of the last n input samples.
3332 //
3333 // It uses a lot more CPU than `ba.slidingMean`, but is numerically stable "forever" in
3334 // return.
3335 //
3336 // #### Usage
3337 //
3338 // ```
3339 // _ : slidingMeanp(n,maxN) : _
3340 // ```
3341 //
3342 // Where:
3343 //
3344 // * `n`: the number of values to process
3345 // * `maxN`: the maximum number of values to process (int, known at compile time, maxN > 0)
3346 //
3347 // #### Test
3348 // ```
3349 // ba = library("basics.lib");
3350 // os = library("oscillators.lib");
3351 // slidingMeanp_test = os.osc(440) : ba.slidingMeanp(64, 128);
3352 // ```
3353 //-----
3354 slidingMeanp(n,maxN) = slidingSump(n,maxN)/n;
3355
3356
3357 //-----`(ba.)slidingRMS`-----
3358 // The root mean square of the last n input samples.
3359 //
3360 // It will eventually run into numerical trouble when there is a persistent dc component.
3361 // If that matters in your application, use the more CPU-intensive `ba.slidingRMSp`.
3362 //
3363 // #### Usage
3364 //
3365 // ```
3366 // _ : slidingRMS(n) : _
3367 // ```
3368 //
3369 // Where:
3370 //
3371 // * `n`: the number of values to process
3372 //

```

```

3373 // #### Test
3374 // ```
3375 // ba = library("basics.lib");
3376 // os = library("oscillators.lib");
3377 // slidingRMS_test = os.osc(440) : ba.slidingRMS(64);
3378 // ```
3379 //-----
3380 slidingRMS(n) = pow(2) : slidingMean(n) : sqrt;
3381
3382 //-----`(ba.)slidingRMSp`-----
3383 // The root mean square of the last n input samples.
3384 //
3385 //
3386 // It uses a lot more CPU than `ba.slidingRMS`, but is numerically stable "forever" in return
3387 //
3388 // #### Usage
3389 //
3390 // ```
3391 // _ : slidingRMSp(n,maxN) : _
3392 // ```
3393 //
3394 // Where:
3395 //
3396 // * `n`: the number of values to process
3397 // * `maxN`: the maximum number of values to process (int, known at compile time, maxN > 0)
3398 //
3399 // #### Test
3400 // ```
3401 // ba = library("basics.lib");
3402 // os = library("oscillators.lib");
3403 // slidingRMSp_test = os.osc(440) : ba.slidingRMSp(64, 128);
3404 // ```
3405 //-----
3406 slidingRMSp(n,maxn) = pow(2) : slidingMeanp(n,maxn) : sqrt;
3407
3408 //=====
3409 // section contributed by Bart Brouns (bart@magnetophon.nl).
3410 // spdx-license-identifier: gpl-3.0
3411 // copyright (c) 2020 Bart Brouns
3412
3413 //=====Parallel Operators
3414 //=====
3415 // Provides various operations on N parallel inputs using a high order
3416 // `parallelOp(op,N,x)` function:
3417 //
3418 // * `parallelMax(N)`: the max of n parallel inputs
3419 // * `parallelMin(N)`: the min of n parallel inputs
3420 // * `parallelMean(N)`: the mean of n parallel inputs
3421 // * `parallelRMS(N)`: the RMS of n parallel inputs
3422
3423 //-----`(ba.)parallelOp`-----
3424 // Apply a commutative binary operation `op` to N parallel inputs.
3425 //
3426 // #### usage
3427 //
3428 // ```
3429 // si.bus(N) : parallelOp(op,N) : _
3430 // ```
3431 //
3432 // where:
3433 //
3434 // * `N`: the number of parallel inputs known at compile time
3435 // * `op`: the operator which needs to be commutative
3436 //
3437 // #### Test
3438 // ```

```

```

3439 // ba = library("basics.lib");
3440 // parallelOp_test = (0.2, 0.5, 0.1) : ba.parallelOp(max, 3);
3441 // ```
3442 //-----
3443 parallelOp(op,1) = _;
3444 parallelOp(op,2) = op;
3445 parallelOp(op,n) = op(parallelOp(op,n-1));
3446
3447 declare parallelOp author "Bart Brouns";
3448 declare parallelOp licence "GPL-3.0";
3449 declare parallelOp copyright "Copyright (c) 2020 Bart Brouns <bart@magnetophon.nl>";
3450
3451
3452 //-----`(ba.)parallelMax`-----
3453 // The maximum of N parallel inputs.
3454 //
3455 // #### Usage
3456 //
3457 // ```
3458 // si.bus(N) : parallelMax(N) : _
3459 // ```
3460 //
3461 // Where:
3462 //
3463 // * `N`: the number of parallel inputs known at compile time
3464 //
3465 // #### Test
3466 // ```
3467 // ba = library("basics.lib");
3468 // parallelMax_test = (0.2, 0.5, 0.1) : ba.parallelMax(3);
3469 // ```
3470 //-----
3471 parallelMax(n) = parallelOp(max,n);
3472
3473 declare parallelMax author "Bart Brouns";
3474 declare parallelMax licence "GPL-3.0";
3475 declare parallelMax copyright "Copyright (c) 2020 Bart Brouns <bart@magnetophon.nl>";
3476
3477
3478 //-----`(ba.)parallelMin`-----
3479 // The minimum of N parallel inputs.
3480 //
3481 // #### Usage
3482 //
3483 // ```
3484 // si.bus(N) : parallelMin(N) : _
3485 // ```
3486 //
3487 // Where:
3488 //
3489 // * `N`: the number of parallel inputs known at compile time
3490 //
3491 // #### Test
3492 // ```
3493 // ba = library("basics.lib");
3494 // parallelMin_test = (0.2, 0.5, 0.1) : ba.parallelMin(3);
3495 // ```
3496 //-----
3497 parallelMin(n) = parallelOp(min,n);
3498
3499 declare parallelMin author "Bart Brouns";
3500 declare parallelMin licence "GPL-3.0";
3501 declare parallelMin copyright "Copyright (c) 2020 Bart Brouns <bart@magnetophon.nl>";
3502
3503
3504 //-----`(ba.)parallelMean`-----
3505 // The mean of N parallel inputs.
3506 //

```

```

3507 // #### Usage
3508 //
3509 // ...
3510 // si.bus(N) : parallelMean(N) : _
3511 // ...
3512 //
3513 // Where:
3514 //
3515 // * `N`: the number of parallel inputs known at compile time
3516 //
3517 // #### Test
3518 // ...
3519 // ba = library("basics.lib");
3520 // parallelMean_test = (0.2, 0.5, 0.1) : ba.parallelMean(3);
3521 // ...
3522 //-----
3523 parallelMean(n) = si.bus(n):>_/n;
3524
3525 declare parallelMean author "Bart Brouns";
3526 declare parallelMean licence "GPL-3.0";
3527 declare parallelMean copyright "Copyright (c) 2020 Bart Brouns <bart@magnetophon.nl>";
3528
3529 //-----`(ba.)parallelRMS`-----
3530 // The RMS of N parallel inputs.
3531 //
3532 // #### Usage
3533 //
3534 // ...
3535 // si.bus(N) : parallelRMS(N) : _
3536 // ...
3537 //
3538 // Where:
3539 //
3540 // * `N`: the number of parallel inputs known at compile time
3541 //
3542 // #### Test
3543 // ...
3544 // ba = library("basics.lib");
3545 // parallelRMS_test = (0.2, 0.5, 0.1) : ba.parallelRMS(3);
3546 // ...
3547 //-----
3548 parallelRMS(n) = par(i, n, pow(2)) : parallelMean(n) : sqrt;
3549
3550 declare parallelRMS author "Bart Brouns";
3551 declare parallelRMS licence "GPL-3.0";
3552 declare parallelRMS copyright "Copyright (c) 2020 Bart Brouns <bart@magnetophon.nl>";
3553
3554 ///////////////////////////////////////////////////////////////////Deprecated Functions/////////////////////////////////////////////////////////////////
3555 // This section implements functions that used to be in music.lib but that are now
3556 // considered as "deprecated".
3557 ///////////////////////////////////////////////////////////////////
3558
3559 millisec = ma.SR/1000.0;
3560
3561 time1s = hslider("time", 0, 0, 1000, 0.1)*millisec;
3562 time2s = hslider("time", 0, 0, 2000, 0.1)*millisec;
3563 time5s = hslider("time", 0, 0, 5000, 0.1)*millisec;
3564 time10s = hslider("time", 0, 0, 10000, 0.1)*millisec;
3565 time21s = hslider("time", 0, 0, 21000, 0.1)*millisec;
3566 time43s = hslider("time", 0, 0, 43000, 0.1)*millisec;

```

Listing 7: maths.lib

```

1 //##### maths.lib #####

```

```

2 // Maths library. Its official prefix is `ma`.
3 //
4 // This library provides mathematical functions and utilities for numerical
5 // computations in Faust. It includes trigonometric, exponential, logarithmic, and
6 // statistical functions, constants, and complex-number
7 // operations used throughout Faust DSP and control code.
8 //
9 // The Maths library is organized into 1 section:
10 //
11 // * [Functions Reference](#functions-reference)
12 //
13 // Some functions are implemented as Faust foreign functions of `math.h` functions
14 // that are not part of Faust's primitives. Defines also various constants and several
15 // utilities.
16 //
17 // #### References
18 //
19 // * <https://github.com/grame-cncm/faustlibraries/blob/master/maths.lib>
20 // #####
21
22 // ## History
23 // * 06/13/2016 [RM] normalizing and integrating to new libraries
24 // * 07/08/2015 [YO] documentation comments
25 // * 20/06/2014 [SL] added FTZ function
26 // * 22/06/2013 [YO] added float/double/quad variants of some foreign functions
27 // * 28/06/2005 [YO] postfixed functions with 'f' to force float version instead of double
28 // * 28/06/2005 [YO] removed 'modf' because it requires a pointer as argument
29
30 /*****
31 *****/
32 FAUST library file
33 Copyright (C) 2003-2016 GRAME, Centre National de Creation Musicale
34 -----
35 This program is free software; you can redistribute it and/or modify
36 it under the terms of the GNU Lesser General Public License as
37 published by the Free Software Foundation; either version 2.1 of the
38 License, or (at your option) any later version.
39
40 This program is distributed in the hope that it will be useful,
41 but WITHOUT ANY WARRANTY; without even the implied warranty of
42 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
43 GNU Lesser General Public License for more details.
44
45 You should have received a copy of the GNU Lesser General Public
46 License along with the GNU C Library; if not, write to the Free
47 Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
48 02111-1307 USA.
49
50 EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
51 larger FAUST program which directly or indirectly imports this library
52 file and still distribute the compiled code generated by the FAUST
53 compiler, or a modified version of this compiled code, under your own
54 copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
55 grants you the right to freely choose the license for the resulting
56 compiled code. In particular the resulting compiled code has no obligation
57 to be LGPL or GPL. For example you are free to choose a commercial or
58 closed source license or any other license if you decide so.
59 *****/
60 *****/
61
62 // This library contains platform specific constants
63 ba = library("basics.lib");
64 pl = library("platform.lib");
65 ma = library("maths.lib"); // for compatible copy/paste out of this file
66
67 declare name "Faust Math Library";
68 declare version "2.9.0";
69 declare author "GRAME";

```

```

70 declare copyright "GRAME";
71 declare license "LGPL with exception";
72
73 //=====Functions Reference=====
74 //=====
75
76
77 //-----`(ma.)SR`-----
78 // Current sampling rate given at init time. Constant during program execution.
79 //
80 // ### Usage
81 //
82 // ```
83 // SR : _
84 // ```
85 //
86 // Where:
87 //
88 // * `SR`: initialization-time sampling rate constant
89 //
90 // ### Test
91 // ```
92 // ma = library("maths.lib");
93 // SR_test = ma.SR;
94 // ```
95 //-----
96 SR = pl.SR;
97
98 //-----`(ma.)T`-----
99 // Current sample duration in seconds computed from the sampling rate given at init time.
100 // Constant during program execution.
101 //
102 // ### Usage
103 //
104 // ```
105 // T : _
106 // ```
107 //
108 // Where:
109 //
110 // * `T`: sample duration ( $1/SR$ ) constant
111 //
112 // ### Test
113 // ```
114 // ma = library("maths.lib");
115 // T_test = ma.T;
116 // ```
117 //-----
118 T = 1.0 / SR;
119
120 //-----`(ma.)BS`-----
121 // Current block-size. Can change during the execution at each block.
122 //
123 // ### Usage
124 //
125 // ```
126 // BS : _
127 // ```
128 //
129 // Where:
130 //
131 // * `BS`: current processing block size
132 //
133 // ### Test
134 // ```
135 // ma = library("maths.lib");
136 // BS_test = ma.BS;
137 // ```

```

```

137 //-----`ma.PI`-----
138 BS = pl.BS;
139
140
141 //-----`ma.PI`-----
142 // Constant PI in double precision.
143 //
144 // #### Usage
145 //
146 // ```
147 // PI : _
148 // ```
149 //
150 // Where:
151 //
152 // * `PI`: double-precision constant
153 //
154 // #### Test
155 // ```
156 // ma = library("maths.lib");
157 // PI_test = ma.PI;
158 // ```
159 //-----
160 PI = 3.14159265358979323846;
161
162
163 //-----`ma.deg2rad`-----
164 // Convert degrees to radians.
165 //
166 // #### Usage
167 //
168 // ```
169 // 45. : deg2rad
170 // ```
171 //
172 // Where:
173 //
174 // * input: angle in degrees to convert
175 //
176 // #### Test
177 // ```
178 // ma = library("maths.lib");
179 // deg2rad_test = 45.0 : ma.deg2rad;
180 // ```
181 //-----
182 deg2rad = _ * PI / 180.;
183
184
185 //-----`ma.rad2deg`-----
186 // Convert radians to degrees.
187 //
188 // #### Usage
189 //
190 // ```
191 // ma.PI : rad2deg
192 // ```
193 //
194 // Where:
195 //
196 // * input: angle in radians to convert
197 //
198 // #### Test
199 // ```
200 // ma = library("maths.lib");
201 // rad2deg_test = ma.PI : ma.rad2deg;
202 // ```
203 //-----
204 rad2deg = _ * 180. / PI;

```

```

205
206
207 //-----`(ma.)E`-----
208 // Constant e in double precision.
209 //
210 // #### Usage
211 //
212 // ```
213 // E : _
214 // ```
215 //
216 // Where:
217 //
218 // * `E`: double-precision Euler's number constant
219 //
220 // #### Test
221 // ```
222 // ma = library("maths.lib");
223 // E_test = ma.E;
224 // ```
225 //-----
226 E = 2.71828182845904523536;
227
228
229 //-----`(ma.)EPSILON`-----
230 // Constant EPSILON available in simple/double/quad precision,
231 // as defined in the [floating-point standard](https://en.wikipedia.org/wiki/IEEE_754)
232 // and [machine epsilon](https://en.wikipedia.org/wiki/Machine_epsilon),
233 // that is smallest positive number such that `1.0 + EPSILON != 1.0`.
234 //
235 // #### Usage
236 //
237 // ```
238 // EPSILON : _
239 // ```
240 //
241 // Where:
242 //
243 // * `EPSILON`: machine epsilon constant for the current floating-point precision
244 //
245 // #### Test
246 // ```
247 // ma = library("maths.lib");
248 // EPSILON_test = ma.EPSILON;
249 // ```
250 //-----
251 singleprecision EPSILON = 1.192092896e-07;
252 doubleprecision EPSILON = 2.2204460492503131e-016;
253 quadprecision EPSILON = 1.084202172485504434007452e-019;
254 fixedpointprecision EPSILON = 2.2204460492503131e-016;
255
256
257 //-----`(ma.)MIN`-----
258 // Constant MIN available in simple/double/quad precision (minimal positive value).
259 //
260 // #### Usage
261 //
262 // ```
263 // MIN : _
264 // ```
265 //
266 // Where:
267 //
268 // * `MIN`: minimal positive normalized value for the current precision
269 //
270 // #### Test
271 // ```
272 // ma = library("maths.lib");

```

```

273 // MIN_test = ma.MIN * 1e307;
274 // ```
275 //-----
276 singleprecision MIN = 1.175494351e-38;
277 doubleprecision MIN = 2.2250738585072014e-308;
278 quadprecision MIN = 2.2250738585072014e-308;
279 fixedpointprecision MIN = 2.2250738585072014e-308;
280
281
282 //-----`(ma.)MAX`-----
283 // Constant MAX available in simple/double/quad precision (maximal positive value).
284 //
285 // #### Usage
286 //
287 // ```
288 // MAX : _
289 // ```
290 //
291 // Where:
292 //
293 // * `MAX`: maximal finite value for the current precision
294 //
295 // #### Test
296 //
297 // ma = library("maths.lib");
298 // MAX_test = ma.MAX;
299 // ```
300 //-----
301 singleprecision MAX = 3.402823466e+38;
302 doubleprecision MAX = 1.7976931348623158e+308;
303 quadprecision MAX = 1.7976931348623158e+308;
304 fixedpointprecision MAX = 1.7976931348623158e+308;
305
306 // Obsolete, kept for compatibility reasons
307 INFINITY = MAX;
308
309 //-----`(ma.)FTZ`-----
310 // Flush to zero: force samples under the "maximum subnormal number"
311 // to be zero. Usually not needed in C++ because the architecture
312 // file take care of this, but can be useful in JavaScript for instance.
313 //
314 // #### Usage
315 //
316 // ```
317 // _ : FTZ : _
318 // ```
319 //
320 // Where:
321 //
322 // * `x`: input signal to flush if its magnitude is subnormal
323 //
324 // #### Test
325 //
326 // ma = library("maths.lib");
327 // FTZ_test = ((ma.MIN * 0.5) : ma.FTZ), ((ma.MIN * 1e307) : ma.FTZ);
328 // ```
329 //
330 // #### References
331 //
332 // * <http://docs.oracle.com/cd/E19957-01/806-3568/ncg\_math.html>
333 //-----
334 FTZ(x) = x * (abs(x) > MIN);
335
336
337 //-----`(ma.)copysign`-----
338 // Changes the sign of x (first input) to that of y (second input).
339 //
340 // #### Usage

```

```

341 //
342 // ``
343 // _ : copysign : _
344 // ``
345 //
346 // Where:
347 //
348 // * `x`: value whose magnitude is preserved
349 // * `y`: value providing the sign
350 //
351 // #### Test
352 // ``
353 // ma = library("maths.lib");
354 // copysign_test = (-1.0, 2.0) : ma.copysign;
355 // ``
356 //-----
357 copysign = ffunction(float copysignf|copysign|copysignl (float, float), <math.h>,"");
358
359 //-----`(ma.)neg`-----
360 // Invert the sign (-x) of a signal.
361 //
362 //
363 // #### Usage
364 //
365 // ``
366 // _ : neg : _
367 // ``
368 //
369 // Where:
370 //
371 // * `x`: value to negate
372 //
373 // #### Test
374 // ``
375 // ma = library("maths.lib");
376 // neg_test = 3.5 : ma.neg;
377 // ``
378 //-----
379 neg(x) = -x;
380
381 //-----`(ma.)not`-----
382 // Bitwise `not` implemented with [xor](https://faustdoc.grame.fr/manual/syntax/#xor-
383 // primitive) as `not(x) = x xor -1;`.
384 // So working regardless of the size of the integer, assuming negative numbers in two's
385 // complement.
386 //
387 // #### Usage
388 //
389 // ``
390 // _ : not : _
391 // ``
392 //
393 // Where:
394 //
395 // * `x`: integer input value
396 //
397 // #### Test
398 // ``
399 // ma = library("maths.lib");
400 // not_test = 5 : ma.not;
401 // ``
402 //-----
403 not(x) = x xor -1;
404
405 //-----`(ma.)sub(x,y)`-----
406 // Subtract `x` and `y`.

```

```

407 //
408 // #### Usage
409 //
410 // ```
411 // _ : sub : _
412 // ```
413 //
414 // Where:
415 //
416 // * `x`: first operand
417 // * `y`: second operand
418 //
419 // #### Test
420 // ```
421 // ma = library("maths.lib");
422 // sub_test = (3, 10) : ma.sub;
423 // ```
424 //-----
425 sub(x,y) = y-x;
426
427
428 //-----`(ma.)inv`-----
429 // Compute the inverse (1/x) of the input signal.
430 //
431 // #### Usage
432 //
433 // ```
434 // _ : inv : _
435 // ```
436 //
437 // Where:
438 //
439 // * `x`: denominator input (non-zero)
440 //
441 // #### Test
442 // ```
443 // ma = library("maths.lib");
444 // inv_test = 4.0 : ma.inv;
445 // ```
446 //-----
447 inv(x) = 1/x;
448
449
450 //-----`(ma.)cbrt`-----
451 // Computes the cube root of of the input signal.
452 //
453 // #### Usage
454 //
455 // ```
456 // _ : cbrt : _
457 // ```
458 //
459 // Where:
460 //
461 // * `x`: value whose cube root is computed
462 //
463 // #### Test
464 // ```
465 // ma = library("maths.lib");
466 // cbrt_test = 8.0 : ma.cbrt;
467 // ```
468 //-----
469 cbrt = ffunction(float cbrtf|cbrt|cbrtl (float), <math.h>,"");
470
471
472 //-----`(ma.)hypot`-----
473 // Computes the euclidian distance of the two input signals
474 // sqrt(x*x+y*y) without undue overflow or underflow.

```

```

475 //
476 // ##### Usage
477 //
478 // ...
479 // _ : hypot : _
480 // ...
481 //
482 // Where:
483 //
484 // * `x`: first operand
485 // * `y`: second operand
486 //
487 // ##### Test
488 // ...
489 // ma = library("maths.lib");
490 // hypot_test = (3.0, 4.0) : ma.hypot;
491 // ...
492 //-----
493 hypot = ffunction(float hypotf|hypot|hypotl (float, float), <math.h>,"");
494
495
496 //-----`(ma.)ldexp`-----
497 // Takes two input signals: x and n, and multiplies x by 2 to the power n.
498 //
499 // ##### Usage
500 //
501 // ...
502 // _ : ldexp : _
503 // ...
504 //
505 // Where:
506 //
507 // * `x`: significand input
508 // * `n`: exponent (integer) input
509 //
510 // ##### Test
511 // ...
512 // ma = library("maths.lib");
513 // ldexp_test = (1.5, 3) : ma.ldexp;
514 // ...
515 //-----
516 ldexp = ffunction(float ldexpf|ldexp|ldexpl (float, int), <math.h>,"");
517
518
519 //-----`(ma.)scalb`-----
520 // Takes two input signals: x and n, and multiplies x by 2 to the power n.
521 //
522 // ##### Usage
523 //
524 // ...
525 // _ : scalb : _
526 // ...
527 //
528 // Where:
529 //
530 // * `x`: significand input
531 // * `n`: exponent (integer) input
532 //
533 // ##### Test
534 // ...
535 // ma = library("maths.lib");
536 // scalb_test = (2.0, -1) : ma.scalb;
537 // ...
538 //-----
539 scalb = ffunction(float scalbnf|scalbn|scalbnl (float, int), <math.h>,"");
540
541
542 //-----`(ma.)log1p`-----

```

```

543 // Computes log(1 + x) without undue loss of accuracy when x is nearly zero.
544 //
545 // #### Usage
546 //
547 // ```
548 // _ : log1p : _
549 // ```
550 //
551 // Where:
552 //
553 // * `x`: offset used in `log(1 + x)` (must be greater than -1)
554 //
555 // #### Test
556 //
557 // ma = library("maths.lib");
558 // log1p_test = 0.5 : ma.log1p;
559 // ```
560 //-----
561 log1p = ffunction(float log1pf|log1p|log1pl (float), <math.h>,"");
562
563
564 //-----`(ma.)logb`-----
565 // Return exponent of the input signal as a floating-point number.
566 //
567 // #### Usage
568 //
569 // ```
570 // _ : logb : _
571 // ```
572 //
573 // Where:
574 //
575 // * `x`: positive value whose exponent part is returned
576 //
577 // #### Test
578 //
579 // ma = library("maths.lib");
580 // logb_test = 8.0 : ma.logb;
581 // ```
582 //-----
583 logb = ffunction(float logbf|logb|logbl (float), <math.h>,"");
584
585
586 //-----`(ma.)ilogb`-----
587 // Return exponent of the input signal as an integer number.
588 //
589 // #### Usage
590 //
591 // ```
592 // _ : ilogb : _
593 // ```
594 //
595 // Where:
596 //
597 // * `x`: positive value whose exponent part is returned
598 //
599 // #### Test
600 //
601 // ma = library("maths.lib");
602 // ilogb_test = 8.0 : ma.ilogb;
603 // ```
604 //-----
605 ilogb = ffunction(int ilogbf|ilogb|ilogbl (float), <math.h>,"");
606
607
608 //-----`(ma.)log2`-----
609 // Returns the base 2 logarithm of x.
610 //

```

```

611 // #### Usage
612 //
613 // ...
614 // _ : log2 : _
615 // ...
616 //
617 // Where:
618 //
619 // * `x`: positive value whose base-2 logarithm is computed
620 //
621 // #### Test
622 // ...
623 // ma = library("maths.lib");
624 // log2_test = 8.0 : ma.log2;
625 // ...
626 //-----
627 log2(x) = log(x)/log(2.0);
628
629
630 //-----`(ma.)expm1`-----
631 // Return exponent of the input signal minus 1 with better precision.
632 //
633 // #### Usage
634 //
635 // ...
636 // _ : expm1 : _
637 // ...
638 //
639 // Where:
640 //
641 // * `x`: input value used for the `exp(x) - 1` computation
642 //
643 // #### Test
644 // ...
645 // ma = library("maths.lib");
646 // expm1_test = 0.5 : ma.expm1;
647 // ...
648 //-----
649 expm1 = ffunction(float expm1f|expm1|expm1l (float), <math.h>,"");
650
651
652 //-----`(ma.)acosh`-----
653 // Computes the principle value of the inverse hyperbolic cosine
654 // of the input signal.
655 //
656 // #### Usage
657 //
658 // ...
659 // _ : acosh : _
660 // ...
661 //
662 // Where:
663 //
664 // * `x`: input value (greater than or equal to 1)
665 //
666 // #### Test
667 // ...
668 // ma = library("maths.lib");
669 // acosh_test = 1.5 : ma.acosh;
670 // ...
671 //-----
672 acosh = ffunction(float acoshf|acosh|acoshl (float), <math.h>, "");
673
674
675 //-----`(ma.)asinh`-----
676 // Computes the inverse hyperbolic sine of the input signal.
677 //
678 // #### Usage

```

```

679 //
680 // \
681 // _ : asinh : _
682 // \
683 //
684 // Where:
685 //
686 // * `x`: input value
687 //
688 // #### Test
689 // \
690 // ma = library("maths.lib");
691 // asinh_test = 0.5 : ma.asinh;
692 // \
693 //-----
694 asinh = ffunction(float asinhf|asinh|asinh1 (float), <math.h>, "");
695
696
697 //-----`(ma.)atanh`-----
698 // Computes the inverse hyperbolic tangent of the input signal.
699 //
700 // #### Usage
701 // \
702 // \
703 // _ : atanh : _
704 // \
705 //
706 // Where:
707 //
708 // * `x`: input value in (-1, 1)
709 //
710 // #### Test
711 // \
712 // ma = library("maths.lib");
713 // atanh_test = 0.5 : ma.atanh;
714 // \
715 //-----
716 atanh = ffunction(float atanhf|atanh|atanh1 (float), <math.h>, "");
717
718
719 //-----`(ma.)sinh`-----
720 // Computes the hyperbolic sine of the input signal.
721 //
722 // #### Usage
723 // \
724 // \
725 // _ : sinh : _
726 // \
727 //
728 // Where:
729 //
730 // * `x`: input value
731 //
732 // #### Test
733 // \
734 // ma = library("maths.lib");
735 // sinh_test = 0.5 : ma.sinh;
736 // \
737 //-----
738 sinh = ffunction(float sinhf|sinh|sinh1 (float), <math.h>, "");
739
740
741 //-----`(ma.)cosh`-----
742 // Computes the hyperbolic cosine of the input signal.
743 //
744 // #### Usage
745 // \
746 // \

```

```

747 // _ : cosh : _
748 // ---
749 //
750 // Where:
751 //
752 // * `x`: input value
753 //
754 // #### Test
755 // ---
756 // ma = library("maths.lib");
757 // cosh_test = 0.5 : ma.cosh;
758 // ---
759 //-----
760 cosh = ffunction(float coshf|cosh|coshl (float), <math.h>, "");
761
762
763 //-----`(ma.)tanh`-----
764 // Computes the hyperbolic tangent of the input signal.
765 //
766 // #### Usage
767 //
768 // ---
769 // _ : tanh : _
770 // ---
771 //
772 // Where:
773 //
774 // * `x`: input value
775 //
776 // #### Test
777 // ---
778 // ma = library("maths.lib");
779 // tanh_test = 0.5 : ma.tanh;
780 // ---
781 //-----
782 tanh = ffunction(float tanhf|tanh|tanhl (float), <math.h>,"");
783
784
785 //-----`(ma.)erf`-----
786 // Computes the error function of the input signal.
787 //
788 // #### Usage
789 //
790 // ---
791 // _ : erf : _
792 // ---
793 //
794 // Where:
795 //
796 // * `x`: input value
797 //
798 // #### Test
799 // ---
800 // ma = library("maths.lib");
801 // erf_test = 0.5 : ma.erf;
802 // ---
803 //-----
804 erf = ffunction(float erff|erf|erfl(float), <math.h>,"");
805
806
807 //-----`(ma.)erfc`-----
808 // Computes the complementary error function of the input signal.
809 //
810 // #### Usage
811 //
812 // ---
813 // _ : erfc : _
814 // ---

```

```

815 //
816 // Where:
817 //
818 // * `x`: input value
819 //
820 // #### Test
821 // ```
822 // ma = library("maths.lib");
823 // erfc_test = 0.5 : ma.erfc;
824 // ```
825 //-----
826 erfc = ffunction(float erfcf|erfc|erfcl(float), <math.h>,"");
827
828
829 //-----`(ma.)gamma`-----
830 // Computes the gamma function of the input signal.
831 //
832 // #### Usage
833 //
834 // ```
835 // _ : gamma : _
836 // ```
837 //
838 // Where:
839 //
840 // * `x`: positive input value
841 //
842 // #### Test
843 // ```
844 // ma = library("maths.lib");
845 // gamma_test = 3.0 : ma.gamma;
846 // ```
847 //-----
848 gamma = ffunction(float tgammaf|tgamma|tgamma1(float), <math.h>,"");
849
850
851 //-----`(ma.)lgamma`-----
852 // Calculates the natural logarithm of the absolute value of
853 // the gamma function of the input signal.
854 //
855 // #### Usage
856 //
857 // ```
858 // _ : lgamma : _
859 // ```
860 //
861 // Where:
862 //
863 // * `x`: positive input value
864 //
865 // #### Test
866 // ```
867 // ma = library("maths.lib");
868 // lgamma_test = 3.0 : ma.lgamma;
869 // ```
870 //-----
871 lgamma = ffunction(float lgammaf|lgamma|lgamma1(float), <math.h>,"");
872
873
874 //-----`(ma.)J0`-----
875 // Computes the Bessel function of the first kind of order 0
876 // of the input signal.
877 //
878 // #### Usage
879 //
880 // ```
881 // _ : J0 : _
882 // ```

```

```

883 //
884 // Where:
885 //
886 // * `x`: input value
887 //
888 // #### Test
889 // ```
890 // ma = library("maths.lib");
891 // J0_test = 1.0 : ma.J0;
892 // ```
893 //-----
894 J0 = ffunction(float j0(float), <math.h>,"");
895
896 //-----`ma.`J1`-----
897 // Computes the Bessel function of the first kind of order 1
898 // of the input signal.
899 //
900 //
901 // #### Usage
902 //
903 // ```
904 // _ : J1 : _
905 // ```
906 //
907 // Where:
908 //
909 // * `x`: input value
910 //
911 // #### Test
912 // ```
913 // ma = library("maths.lib");
914 // J1_test = 1.0 : ma.J1;
915 // ```
916 //-----
917 J1 = ffunction(float j1(float), <math.h>,"");
918
919 //-----`ma.`Jn`-----
920 // Computes the Bessel function of the first kind of order n
921 // (first input signal) of the second input signal.
922 //
923 //
924 // #### Usage
925 //
926 // ```
927 // _ : Jn : _
928 // ```
929 //
930 // Where:
931 //
932 // * `n`: integer order
933 // * `x`: input value
934 //
935 // #### Test
936 // ```
937 // ma = library("maths.lib");
938 // Jn_test = (2, 1.0) : ma.Jn;
939 // ```
940 //-----
941 Jn = ffunction(float jn(int, float), <math.h>,"");
942
943 //-----`ma.`Y0`-----
944 // Computes the linearly independent Bessel function of the second kind
945 // of order 0 of the input signal.
946 //
947 //
948 // #### Usage
949 //
950 // ```

```

```

951 // _ : Y0 : _
952 // ...
953 //
954 // Where:
955 //
956 // * `x`: positive input value
957 //
958 // #### Test
959 // ...
960 // ma = library("maths.lib");
961 // Y0_test = 1.0 : ma.Y0;
962 // ...
963 //-----
964 Y0 = ffunction(float y0(float), <math.h>,"");
965
966 //-----`ma.`Y1`-----
967 // Computes the linearly independent Bessel function of the second kind
968 // of order 1 of the input signal.
969 //
970 //
971 // #### Usage
972 //
973 // ...
974 // _ : Y0 : _
975 // ...
976 //
977 // Where:
978 //
979 // * `x`: positive input value
980 //
981 // #### Test
982 // ...
983 // ma = library("maths.lib");
984 // Y1_test = 1.0 : ma.Y1;
985 // ...
986 //-----
987 Y1 = ffunction(float y1(float), <math.h>,"");
988
989 //-----`ma.`Yn`-----
990 // Computes the linearly independent Bessel function of the second kind
991 // of order n (first input signal) of the second input signal.
992 //
993 //
994 // #### Usage
995 //
996 // ...
997 // _ : Yn : _
998 // ...
999 //
1000 // Where:
1001 //
1002 // * `n`: integer order
1003 // * `x`: positive input value
1004 //
1005 // #### Test
1006 // ...
1007 // ma = library("maths.lib");
1008 // Yn_test = (2, 1.0) : ma.Yn;
1009 // ...
1010 //-----
1011 Yn = ffunction(float yn(int, float), <math.h>,"");
1012
1013 //-----`ma.`fabs`, `ma.`fmax`, `ma.`fmin
1014 //-----
1015 // Just for compatibility...
1016 //
1017 // ...

```

```

1018 // fabs = abs
1019 // fmax = max
1020 // fmin = min
1021 // ```
1022 //-----
1023 fabs = abs;
1024 fmax = max;
1025 fmin = min;
1026
1027 //-----`(ma.)np2`-----
1028 // Gives the next power of 2 of x.
1029 //
1030 // ### Usage
1031 //
1032 // ```
1033 // np2(n) : _
1034 // ```
1035 //
1036 // Where:
1037 //
1038 // * `n`: an integer
1039 //
1040 // ### Test
1041 //
1042 // ma = library("maths.lib");
1043 // np2_test = 5 : ma.np2;
1044 // ```
1045 //-----
1046 np2 = -(1) <: >>(1)|_ <: >>(2)|_ <: >>(4)|_ <: >>(8)|_ <: >>(16)|_ : +(1);
1047
1048
1049 //-----`(ma.)frac`-----
1050 // Gives the fractional part of n.
1051 //
1052 // ### Usage
1053 //
1054 // ```
1055 // frac(n) : _
1056 // ```
1057 //
1058 // Where:
1059 //
1060 // * `n`: a decimal number
1061 //
1062 // ### Test
1063 //
1064 // ma = library("maths.lib");
1065 // frac_test = 3.75 : ma.frac;
1066 // ```
1067 //-----
1068 frac(n) = n - floor(n);
1069 decimal = frac;
1070
1071 // NOTE: decimal does the same thing as frac but using floor instead. JOS uses frac a lot
1072 // in filters.lib so we decided to keep that one... decimal is declared though for
1073 // backward compatibility.
1074 // decimal(n) = n - floor(n);
1075
1076
1077 //-----`(ma.)modulo`-----
1078 // Modulus operation using the `(x%y)%y` formula to ensures the result is always non-
1079 // negative, even if `x` is negative.
1080 //
1081 // ### Usage
1082 //
1083 // ```
1084 // modulo(x,y) : _
1085 // ```

```

```

1085 //
1086 // Where:
1087 //
1088 // * `x`: the numerator
1089 // * `y`: the denominator
1090 //
1091 // #### Test
1092 // ```
1093 // ma = library("maths.lib");
1094 // modulo_test = (-3, 4) : ma.modulo;
1095 // ```
1096 //-----
1097 modulo(x,y) = (x % y + y) % y;
1098
1099
1100 //-----`(ma.)isnan`-----
1101 // Return non-zero if x is a NaN.
1102 //
1103 // #### Usage
1104 //
1105 // ```
1106 // isnan(x)
1107 // _ : isnan : _
1108 // ```
1109 //
1110 // Where:
1111 //
1112 // * `x`: signal to analyse
1113 //
1114 // #### Test
1115 // ```
1116 // ma = library("maths.lib");
1117 // os = library("oscillators.lib");
1118 // isnan_test = (os.osc(1) - 2.0) : sqrt : ma.isnan;
1119 // ```
1120 //-----
1121 isnan = ffunction(int isnanf|isnan|isnanl (float),<math.h>,"");
1122
1123
1124 //-----`(ma.)isinf`-----
1125 // Return non-zero if x is a positive or negative infinity.
1126 //
1127 // #### Usage
1128 //
1129 // ```
1130 // isinf(x)
1131 // _ : isinf : _
1132 // ```
1133 //
1134 // Where:
1135 //
1136 // * `x`: signal to analyse
1137 //
1138 // #### Test
1139 // ```
1140 // ma = library("maths.lib");
1141 // os = library("oscillators.lib");
1142 // isinf_test = (os.impulse - os.impulse) : log : ma.isinf;
1143 // ```
1144 //-----
1145 isinf = ffunction(int isinff|isinf|isinfl (float),<math.h>,"");
1146
1147 nextafter = ffunction(float nextafter(float, float),<math.h>,"");
1148
1149
1150 //-----`(ma.)chebychev`-----
1151 // Chebychev transformation of order N.
1152 //

```

```

1153 // #### Usage
1154 //
1155 // ...
1156 // _ : chebychev(N) : _
1157 // ...
1158 //
1159 // Where:
1160 //
1161 // * `N`: the order of the polynomial, a constant numerical expression
1162 //
1163 // #### Semantics
1164 //
1165 // ...
1166 //  $T[0](x) = 1$ ,
1167 //  $T[1](x) = x$ ,
1168 //  $T[n](x) = 2xT[n-1](x) - T[n-2](x)$ 
1169 // ...
1170 //
1171 // #### Test
1172 // ...
1173 // ma = library("maths.lib");
1174 // chebychev_test = 0.5 : ma.chebychev(3);
1175 // ...
1176 //
1177 // #### References
1178 //
1179 // * <http://en.wikipedia.org/wiki/Chebyshev\_polynomial>
1180 //-----
1181 chebychev(0,x) = 1;
1182 chebychev(1,x) = x;
1183 chebychev(n,x) = 2*x*chebychev(n-1, x) - chebychev(n-2, x);
1184
1185
1186 //-----`(ma.)chebychevpoly`-----
1187 // Linear combination of the first Chebyshev polynomials.
1188 //
1189 // #### Usage
1190 //
1191 // ...
1192 // _ : chebychevpoly((c0,c1,...,cn)) : _
1193 // ...
1194 //
1195 // Where:
1196 //
1197 // * `cn`: the different Chebyshev polynomials such that:
1198 //  $\text{chebychevpoly}((c0,c1,\dots,cn)) = \text{Sum of chebychev}(i)*c_i$ 
1199 //
1200 // #### Test
1201 // ...
1202 // ma = library("maths.lib");
1203 // chebychevpoly_test = 0.5 : ma.chebychevpoly((1, 0, 1));
1204 // ...
1205 //
1206 // #### References
1207 //
1208 // * <http://www.csounds.com/manual/html/chebyshevpoly.html>
1209 //-----
1210 chebychevpoly(lcoef) = _ <: L(0,lcoef) :> _
1211   with {
1212     L(n,(c,cs)) = chebychev(n)*c, L(n+1,cs);
1213     L(n,c)      = chebychev(n)*c;
1214   };
1215
1216
1217 //-----`(ma.)diffn`-----
1218 // Negated first-order difference.
1219 //
1220 // #### Usage

```

```

1221 //
1222 // ---
1223 // _ : diffn : _
1224 // ---
1225 //
1226 // Where:
1227 //
1228 // * `x`: input signal
1229 //
1230 // #### Test
1231 // ---
1232 // ma = library("maths.lib");
1233 // os = library("oscillators.lib");
1234 // diffn_test = os.osc(440) : ma.diffn;
1235 // ---
1236 //-----
1237 diffn(x) = x' - x; // negated first-order difference
1238
1239
1240 //-----`ma.`signum`-----
1241 // The signum function signum(x) is defined as
1242 // -1 for x<0, 0 for x==0, and 1 for x>0.
1243 //
1244 // #### Usage
1245 //
1246 // ---
1247 // _ : signum : _
1248 // ---
1249 //
1250 // Where:
1251 //
1252 // * `x`: input value
1253 //
1254 // #### Test
1255 // ---
1256 // ma = library("maths.lib");
1257 // signum_test = (-5.0) : ma.signum;
1258 // ---
1259 //-----
1260 signum(x) = (x>0)-(x<0);
1261
1262
1263 //-----`ma.`nextpow2`-----
1264 // The nextpow2(x) returns the lowest integer m such that
1265 // 2^m >= x.
1266 //
1267 // #### Usage
1268 //
1269 // ---
1270 // 2^nextpow2(n) : _
1271 // ---
1272 // Useful for allocating delay lines, e.g.,
1273 // ---
1274 // delay(2^nextpow2(maxDelayNeeded), currentDelay);
1275 // ---
1276 //
1277 // Where:
1278 //
1279 // * `n`: positive value whose next power-of-two exponent is computed
1280 //
1281 // #### Test
1282 // ---
1283 // ma = library("maths.lib");
1284 // nextpow2_test = 10.0 : ma.nextpow2;
1285 // ---
1286 //-----
1287 nextpow2(x) = ceil(log(x)/log(2.0));
1288

```

```

1289 //-----`(ma.)zc`-----
1290 // Indicator function for zero-crossing: it returns 1 if a zero-crossing
1291 // occurs, 0 otherwise.
1292 //
1293 //
1294 // #### Usage
1295 //
1296 // ```
1297 // _ : zc : _
1298 // ```
1299 //
1300 // Where:
1301 //
1302 // * `x`: input signal to monitor for zero crossings
1303 //
1304 // #### Test
1305 //
1306 // ma = library("maths.lib");
1307 // os = library("oscillators.lib");
1308 // zc_test = os.osc(440) : ma.zc;
1309 //
1310 //-----
1311 zc = ba.tAndH(!=(0)) <: *(mem) < 0;
1312
1313 //-----`(ma.)unwrap`-----
1314 // Unwrap the input signal so that successive output values never differ
1315 // by more than `pi`, switching to a 2*pi-complementary value when needed.
1316 //
1317 // #### Usage
1318 //
1319 // ```
1320 // _ : unwrap(pi) : _
1321 // ```
1322 //
1323 // Where:
1324 //
1325 // * `pi`: maximum discontinuity between the output values (typically `ma.PI`)
1326 //
1327 // #### Test
1328 //
1329 // ma = library("maths.lib");
1330 // os = library("oscillators.lib");
1331 // unwrap_test = os.oscsrc(100) : ma.unwrap(ma.PI);
1332 //
1333 //
1334 // #### Example test program
1335 //
1336 // ```
1337 // process = 0 - os.oscsrc(1000) // the true phase is either -PI or +PI
1338 //          : an.resonator(1,1000) : !, _ // oscillates between -PI and +PI
1339 //          : ma.unwrap(ma.PI); // oscillates near +PI
1340 // ```
1341 //
1342 //-----
1343 unwrap(pi,x) = (_ <: +(dz)) ~ _ with {
1344   dz(z) = wrap(x - z, 2*pi) <: ba.ifNc(1, >(pi), -(2*pi));
1345   wrap(x,d) = x - d * floor(x / d);
1346 };
1347
1348 //-----`(ma.)primes`-----
1349 // Return the n-th prime using a waveform primitive. Note that primes(0) is 2,
1350 // primes(1) is 3, and so on. The waveform is length 2048, so the largest
1351 // precomputed prime is primes(2047) which is 17863.
1352 //
1353 // #### Usage
1354 //
1355 // ```
1356 // _ : primes : _

```

```

1357 // ---
1358 //
1359 // Where:
1360 //
1361 // * `x`: index of the prime number sequence (0-based).
1362 //
1363 // #### Test
1364 // ---
1365 // ma = library("maths.lib");
1366 // primes_test = 10 : ma.primes;
1367 // ---
1368 //-----
1369 primes(x) = rdtbl(waveform{
1370 2,3,5,7,11,13,17,19,23,29,
1371 31,37,41,43,47,53,59,61,67,71,
1372 73,79,83,89,97,101,103,107,109,113,
1373 127,131,137,139,149,151,157,163,167,173,
1374 179,181,191,193,197,199,211,223,227,229,
1375 233,239,241,251,257,263,269,271,277,281,
1376 283,293,307,311,313,317,331,337,347,349,
1377 353,359,367,373,379,383,389,397,401,409,
1378 419,421,431,433,439,443,449,457,461,463,
1379 467,479,487,491,499,503,509,521,523,541,
1380 547,557,563,569,571,577,587,593,599,601,
1381 607,613,617,619,631,641,643,647,653,659,
1382 661,673,677,683,691,701,709,719,727,733,
1383 739,743,751,757,761,769,773,787,797,809,
1384 811,821,823,827,829,839,853,857,859,863,
1385 877,881,883,887,907,911,919,929,937,941,
1386 947,953,967,971,977,983,991,997,1009,1013,
1387 1019,1021,1031,1033,1039,1049,1051,1061,1063,1069,
1388 1087,1091,1093,1097,1103,1109,1117,1123,1129,1151,
1389 1153,1163,1171,1181,1187,1193,1201,1213,1217,1223,
1390 1229,1231,1237,1249,1259,1277,1279,1283,1289,1291,
1391 1297,1301,1303,1307,1319,1321,1327,1361,1367,1373,
1392 1381,1399,1409,1423,1427,1429,1433,1439,1447,1451,
1393 1453,1459,1471,1481,1483,1487,1489,1493,1499,1511,
1394 1523,1531,1543,1549,1553,1559,1567,1571,1579,1583,
1395 1597,1601,1607,1609,1613,1619,1621,1627,1637,1657,
1396 1663,1667,1669,1693,1697,1699,1709,1721,1723,1733,
1397 1741,1747,1753,1759,1777,1783,1787,1789,1801,1811,
1398 1823,1831,1847,1861,1867,1871,1873,1877,1879,1889,
1399 1901,1907,1913,1931,1933,1949,1951,1973,1979,1987,
1400 1993,1997,1999,2003,2011,2017,2027,2029,2039,2053,
1401 2063,2069,2081,2083,2087,2089,2099,2111,2113,2129,
1402 2131,2137,2141,2143,2153,2161,2179,2203,2207,2213,
1403 2221,2237,2239,2243,2251,2267,2269,2273,2281,2287,
1404 2293,2297,2309,2311,2333,2339,2341,2347,2351,2357,
1405 2371,2377,2381,2383,2389,2393,2399,2411,2417,2423,
1406 2437,2441,2447,2459,2467,2473,2477,2503,2521,2531,
1407 2539,2543,2549,2551,2557,2579,2591,2593,2609,2617,
1408 2621,2633,2647,2657,2659,2663,2671,2677,2683,2687,
1409 2689,2693,2699,2707,2711,2713,2719,2729,2731,2741,
1410 2749,2753,2767,2777,2789,2791,2797,2801,2803,2819,
1411 2833,2837,2843,2851,2857,2861,2879,2887,2897,2903,
1412 2909,2917,2927,2939,2953,2957,2963,2969,2971,2999,
1413 3001,3011,3019,3023,3037,3041,3049,3061,3067,3079,
1414 3083,3089,3109,3119,3121,3137,3163,3167,3169,3181,
1415 3187,3191,3203,3209,3217,3221,3229,3251,3253,3257,
1416 3259,3271,3299,3301,3307,3313,3319,3323,3329,3331,
1417 3343,3347,3359,3361,3371,3373,3389,3391,3407,3413,
1418 3433,3449,3457,3461,3463,3467,3469,3491,3499,3511,
1419 3517,3527,3529,3533,3539,3541,3547,3557,3559,3571,
1420 3581,3583,3593,3607,3613,3617,3623,3631,3637,3643,
1421 3659,3671,3673,3677,3691,3697,3701,3709,3719,3727,
1422 3733,3739,3761,3767,3769,3779,3793,3797,3803,3821,
1423 3823,3833,3847,3851,3853,3863,3877,3881,3889,3907,
1424 3911,3917,3919,3923,3929,3931,3943,3947,3967,3989,

```

1425 4001,4003,4007,4013,4019,4021,4027,4049,4051,4057,
 1426 4073,4079,4091,4093,4099,4111,4127,4129,4133,4139,
 1427 4153,4157,4159,4177,4201,4211,4217,4219,4229,4231,
 1428 4241,4243,4253,4259,4261,4271,4273,4283,4289,4297,
 1429 4327,4337,4339,4349,4357,4363,4373,4391,4397,4409,
 1430 4421,4423,4441,4447,4451,4457,4463,4481,4483,4493,
 1431 4507,4513,4517,4519,4523,4547,4549,4561,4567,4583,
 1432 4591,4597,4603,4621,4637,4639,4643,4649,4651,4657,
 1433 4663,4673,4679,4691,4703,4721,4723,4729,4733,4751,
 1434 4759,4783,4787,4789,4793,4799,4801,4813,4817,4831,
 1435 4861,4871,4877,4889,4903,4909,4919,4931,4933,4937,
 1436 4943,4951,4957,4967,4969,4973,4987,4993,4999,5003,
 1437 5009,5011,5021,5023,5039,5051,5059,5077,5081,5087,
 1438 5099,5101,5107,5113,5119,5147,5153,5167,5171,5179,
 1439 5189,5197,5209,5227,5231,5233,5237,5261,5273,5279,
 1440 5281,5297,5303,5309,5323,5333,5347,5351,5381,5387,
 1441 5393,5399,5407,5413,5417,5419,5431,5437,5441,5443,
 1442 5449,5471,5477,5479,5483,5501,5503,5507,5519,5521,
 1443 5527,5531,5557,5563,5569,5573,5581,5591,5623,5639,
 1444 5641,5647,5651,5653,5657,5659,5669,5683,5689,5693,
 1445 5701,5711,5717,5737,5741,5743,5749,5779,5783,5791,
 1446 5801,5807,5813,5821,5827,5839,5843,5849,5851,5857,
 1447 5861,5867,5869,5879,5881,5897,5903,5923,5927,5939,
 1448 5953,5981,5987,6007,6011,6029,6037,6043,6047,6053,
 1449 6067,6073,6079,6089,6091,6101,6113,6121,6131,6133,
 1450 6143,6151,6163,6173,6197,6199,6203,6211,6217,6221,
 1451 6229,6247,6257,6263,6269,6271,6277,6287,6299,6301,
 1452 6311,6317,6323,6329,6337,6343,6353,6359,6361,6367,
 1453 6373,6379,6389,6397,6421,6427,6449,6451,6469,6473,
 1454 6481,6491,6521,6529,6547,6551,6553,6563,6569,6571,
 1455 6577,6581,6599,6607,6619,6637,6653,6659,6661,6673,
 1456 6679,6689,6691,6701,6703,6709,6719,6733,6737,6761,
 1457 6763,6779,6781,6791,6793,6803,6823,6827,6829,6833,
 1458 6841,6857,6863,6869,6871,6883,6899,6907,6911,6917,
 1459 6947,6949,6959,6961,6967,6971,6977,6983,6991,6997,
 1460 7001,7013,7019,7027,7039,7043,7057,7069,7079,7103,
 1461 7109,7121,7127,7129,7151,7159,7177,7187,7193,7207,
 1462 7211,7213,7219,7229,7237,7243,7247,7253,7283,7297,
 1463 7307,7309,7321,7331,7333,7349,7351,7369,7393,7411,
 1464 7417,7433,7451,7457,7459,7477,7481,7487,7489,7499,
 1465 7507,7517,7523,7529,7537,7541,7547,7549,7559,7561,
 1466 7573,7577,7583,7589,7591,7603,7607,7621,7639,7643,
 1467 7649,7669,7673,7681,7687,7691,7699,7703,7717,7723,
 1468 7727,7741,7753,7757,7759,7789,7793,7817,7823,7829,
 1469 7841,7853,7867,7873,7877,7879,7883,7901,7907,7919,
 1470 7927,7933,7937,7949,7951,7963,7993,8009,8011,8017,
 1471 8039,8053,8059,8069,8081,8087,8089,8093,8101,8111,
 1472 8117,8123,8147,8161,8167,8171,8179,8191,8209,8219,
 1473 8221,8231,8233,8237,8243,8263,8269,8273,8287,8291,
 1474 8293,8297,8311,8317,8329,8353,8363,8369,8377,8387,
 1475 8389,8419,8423,8429,8431,8443,8447,8461,8467,8501,
 1476 8513,8521,8527,8537,8539,8543,8563,8573,8581,8597,
 1477 8599,8609,8623,8627,8629,8641,8647,8663,8669,8677,
 1478 8681,8689,8693,8699,8707,8713,8719,8731,8737,8741,
 1479 8747,8753,8761,8779,8783,8803,8807,8819,8821,8831,
 1480 8837,8839,8849,8861,8863,8867,8887,8893,8923,8929,
 1481 8933,8941,8951,8963,8969,8971,8999,9001,9007,9011,
 1482 9013,9029,9041,9043,9049,9059,9067,9091,9103,9109,
 1483 9127,9133,9137,9151,9157,9161,9173,9181,9187,9199,
 1484 9203,9209,9221,9227,9239,9241,9257,9277,9281,9283,
 1485 9293,9311,9319,9323,9337,9341,9343,9349,9371,9377,
 1486 9391,9397,9403,9413,9419,9421,9431,9433,9437,9439,
 1487 9461,9463,9467,9473,9479,9491,9497,9511,9521,9533,
 1488 9539,9547,9551,9587,9601,9613,9619,9623,9629,9631,
 1489 9643,9649,9661,9677,9679,9689,9697,9719,9721,9733,
 1490 9739,9743,9749,9767,9769,9781,9787,9791,9803,9811,
 1491 9817,9829,9833,9839,9851,9857,9859,9871,9883,9887,
 1492 9901,9907,9923,9929,9931,9941,9949,9967,9973,10007,

1493 10009,10037,10039,10061,10067,10069,10079,10091,10093,10099,
1494 10103,10111,10133,10139,10141,10151,10159,10163,10169,10177,
1495 10181,10193,10211,10223,10243,10247,10253,10259,10267,10271,
1496 10273,10289,10301,10303,10313,10321,10331,10333,10337,10343,
1497 10357,10369,10391,10399,10427,10429,10433,10453,10457,10459,
1498 10463,10477,10487,10499,10501,10513,10529,10531,10559,10567,
1499 10589,10597,10601,10607,10613,10627,10631,10639,10651,10657,
1500 10663,10667,10687,10691,10709,10711,10723,10729,10733,10739,
1501 10753,10771,10781,10789,10799,10831,10837,10847,10853,10859,
1502 10861,10867,10883,10889,10891,10903,10909,10937,10939,10949,
1503 10957,10973,10979,10987,10993,11003,11027,11047,11057,11059,
1504 11069,11071,11083,11087,11093,11113,11117,11119,11131,11149,
1505 11159,11161,11171,11173,11177,11197,11213,11239,11243,11251,
1506 11257,11261,11273,11279,11287,11299,11311,11317,11321,11329,
1507 11351,11353,11369,11383,11393,11399,11411,11423,11437,11443,
1508 11447,11467,11471,11483,11489,11491,11497,11503,11519,11527,
1509 11549,11551,11579,11587,11593,11597,11617,11621,11633,11657,
1510 11677,11681,11689,11699,11701,11717,11719,11731,11743,11777,
1511 11779,11783,11789,11801,11807,11813,11821,11827,11831,11833,
1512 11839,11863,11867,11887,11897,11903,11909,11923,11927,11933,
1513 11939,11941,11953,11959,11969,11971,11981,11987,12007,12011,
1514 12037,12041,12043,12049,12071,12073,12097,12101,12107,12109,
1515 12113,12119,12143,12149,12157,12161,12163,12197,12203,12211,
1516 12227,12239,12241,12251,12253,12263,12269,12277,12281,12289,
1517 12301,12323,12329,12343,12347,12373,12377,12379,12391,12401,
1518 12409,12413,12421,12433,12437,12451,12457,12473,12479,12487,
1519 12491,12497,12503,12511,12517,12527,12539,12541,12547,12553,
1520 12569,12577,12583,12589,12601,12611,12613,12619,12637,12641,
1521 12647,12653,12659,12671,12689,12697,12703,12713,12721,12739,
1522 12743,12757,12763,12781,12791,12799,12809,12821,12823,12829,
1523 12841,12853,12889,12893,12899,12907,12911,12917,12919,12923,
1524 12941,12953,12959,12967,12973,12979,12983,13001,13003,13007,
1525 13009,13033,13037,13043,13049,13063,13093,13099,13103,13109,
1526 13121,13127,13147,13151,13159,13163,13171,13177,13183,13187,
1527 13217,13219,13229,13241,13249,13259,13267,13291,13297,13309,
1528 13313,13327,13331,13337,13339,13367,13381,13397,13399,13411,
1529 13417,13421,13441,13451,13457,13463,13469,13477,13487,13499,
1530 13513,13523,13537,13553,13567,13577,13591,13597,13613,13619,
1531 13627,13633,13649,13669,13679,13681,13687,13691,13693,13697,
1532 13709,13711,13721,13723,13729,13751,13757,13759,13763,13781,
1533 13789,13799,13807,13829,13831,13841,13859,13873,13877,13879,
1534 13883,13901,13903,13907,13913,13921,13931,13933,13963,13967,
1535 13997,13999,14009,14011,14029,14033,14051,14057,14071,14081,
1536 14083,14087,14107,14143,14149,14153,14159,14173,14177,14197,
1537 14207,14221,14243,14249,14251,14281,14293,14303,14321,14323,
1538 14327,14341,14347,14369,14387,14389,14401,14407,14411,14419,
1539 14423,14431,14437,14447,14449,14461,14479,14489,14503,14519,
1540 14533,14537,14543,14549,14551,14557,14561,14563,14591,14593,
1541 14621,14627,14629,14633,14639,14653,14657,14669,14683,14699,
1542 14713,14717,14723,14731,14737,14741,14747,14753,14759,14767,
1543 14771,14779,14783,14797,14813,14821,14827,14831,14843,14851,
1544 14867,14869,14879,14887,14891,14897,14923,14929,14939,14947,
1545 14951,14957,14969,14983,15013,15017,15031,15053,15061,15073,
1546 15077,15083,15091,15101,15107,15121,15131,15137,15139,15149,
1547 15161,15173,15187,15193,15199,15217,15227,15233,15241,15259,
1548 15263,15269,15271,15277,15287,15289,15299,15307,15313,15319,
1549 15329,15331,15349,15359,15361,15373,15377,15383,15391,15401,
1550 15413,15427,15439,15443,15451,15461,15467,15473,15493,15497,
1551 15511,15527,15541,15551,15559,15569,15581,15583,15601,15607,
1552 15619,15629,15641,15643,15647,15649,15661,15667,15671,15679,
1553 15683,15727,15731,15733,15737,15739,15749,15761,15767,15773,
1554 15787,15791,15797,15803,15809,15817,15823,15859,15877,15881,
1555 15887,15889,15901,15907,15913,15919,15923,15937,15959,15971,
1556 15973,15991,16001,16007,16033,16057,16061,16063,16067,16069,
1557 16073,16087,16091,16097,16103,16111,16127,16139,16141,16183,
1558 16187,16189,16193,16217,16223,16229,16231,16249,16253,16267,
1559 16273,16301,16319,16333,16339,16349,16361,16363,16369,16381,
1560 16411,16417,16421,16427,16433,16447,16451,16453,16477,16481,

```

1561 16487,16493,16519,16529,16547,16553,16561,16567,16573,16603,
1562 16607,16619,16631,16633,16649,16651,16657,16661,16673,16691,
1563 16693,16699,16703,16729,16741,16747,16759,16763,16787,16811,
1564 16823,16829,16831,16843,16871,16879,16883,16889,16901,16903,
1565 16921,16927,16931,16937,16943,16963,16979,16981,16987,16993,
1566 17011,17021,17027,17029,17033,17041,17047,17053,17077,17093,
1567 17099,17107,17117,17123,17137,17159,17167,17183,17189,17191,
1568 17203,17207,17209,17231,17239,17257,17291,17293,17299,17317,
1569 17321,17327,17333,17341,17351,17359,17377,17383,17387,17389,
1570 17393,17401,17417,17419,17431,17443,17449,17467,17471,17477,
1571 17483,17489,17491,17497,17509,17519,17539,17551,17569,17573,
1572 17579,17581,17597,17599,17609,17623,17627,17657,17659,17669,
1573 17681,17683,17707,17713,17729,17737,17747,17749,17761,17783,
1574 17789,17791,17807,17827,17837,17839,17851,17863
1575 }, x);

```

Listing 8: platform.lib

```

1  //##### platform.lib #####
2  // A library to handle platform specific code in Faust. Its official prefix is `pl`.
3  //
4  // #### References
5  //
6  // * <https://github.com/grame-cncm/faustlibraries/blob/master/platform.lib>
7
8  //#####
9  // It can be reimplemented to globally change the SR and the tables size definitions
10
11
12  /*****
13  *****/
14  FAUST library file
15  Copyright (C) 2020 GRAME, Centre National de Creation Musicale
16  -----
17  This program is free software; you can redistribute it and/or modify
18  it under the terms of the GNU Lesser General Public License as
19  published by the Free Software Foundation; either version 2.1 of the
20  License, or (at your option) any later version.
21
22  This program is distributed in the hope that it will be useful,
23  but WITHOUT ANY WARRANTY; without even the implied warranty of
24  MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
25  GNU Lesser General Public License for more details.
26
27  You should have received a copy of the GNU Lesser General Public
28  License along with the GNU C Library; if not, write to the Free
29  Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
30  02111-1307 USA.
31
32  EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
33  larger FAUST program which directly or indirectly imports this library
34  file and still distribute the compiled code generated by the FAUST
35  compiler, or a modified version of this compiled code, under your own
36  copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
37  grants you the right to freely choose the license for the resulting
38  compiled code. In particular the resulting compiled code has no obligation
39  to be LGPL or GPL. For example you are free to choose a commercial or
40  closed source license or any other license if you decide so.
41  *****/
42  *****/
43
44  declare name "Generic Platform Library";
45  declare version "1.3.0";
46

```

```

47 //-----`(pl.)SR`-----
48 // Current sampling rate (between 1 and 192000Hz). Constant during
49 // program execution. Setting this value to a constant will allow the
50 // compiler to optimize the code by computing constant expressions at
51 // compile time, and can be valuable for performance, especially on
52 // embedded systems.
53 //-----
54 SR = min(192000.0, max(1.0, fconstant(int fSamplingFreq, <math.h>)));
55
56 //-----`(pl.)BS`-----
57 // Current block-size (between 1 and 16384 frames). Can change during the execution.
58 //-----
59 BS = min(16384.0, max(1.0, fvariable(int count, <math.h>)));
60
61 //-----`(pl.)tablesize`-----
62 // Oscillator table size. This value is used to define the size of the
63 // table used by the oscillators. It is usually a power of 2 and can be lowered
64 // to save memory. The default value is 65536.
65 //-----
66 tablesize = 1 << 16;

```

Listing 9: filters.lib

```

1 //##### filters.lib #####
2 // Filters library. Its official prefix is `fi`.
3 //
4 // This library provides a comprehensive collection of linear and nonlinear
5 // filters used in audio and signal processing. It includes low-pass, high-pass,
6 // band-pass, allpass, shelving, equalizer, and crossover filters, as well as advanced
7 // analog and digital filter design sections for both educational and production use.
8 //
9 // The Filters library is organized into 23 sections:
10 //
11 // * [Basic Filters](#basic-filters)
12 // * [Comb Filters](#comb-filters)
13 // * [Direct-Form Digital Filter Sections](#direct-form-digital-filter-sections)
14 // * [Direct-Form Second-Order Biquad Sections](#direct-form-second-order-biquad-sections)
15 // * [Ladder/Lattice Digital Filters](#ladderlattice-digital-filters)
16 // * [Useful Special Cases](#useful-special-cases)
17 // * [Ladder/Lattice Allpass Filters](#ladderlattice-allpass-filters)
18 // * [Digital Filter Sections Specified as Analog Filter Sections](#digital-filter-sections-
19 //   specified-as-analog-filter-sections)
20 // * [Simple Resonator Filters](#simple-resonator-filters)
21 // * [Butterworth Lowpass/Highpass Filters](#butterworth-lowpasshighpass-filters)
22 // * [Special Filter-Bank Delay-Equalizing Allpass Filters](#special-filter-bank-delay-
23 //   equalizing-allpass-filters)
24 // * [Elliptic (Cauer) Lowpass Filters](#elliptic-cauer-lowpass-filters)
25 // * [Elliptic Highpass Filters](#elliptic-highpass-filters)
26 // * [Butterworth Bandpass/Bandstop Filters](#butterworth-bandpassbandstop-filters)
27 // * [Elliptic Bandpass Filters](#elliptic-bandpass-filters)
28 // * [Parametric Equalizers (Shelf, Peaking)](#parametric-equalizers-shelf-peaking)
29 // * [Mth-Octave Filter-Banks](#mth-octave-filter-banks)
30 // * [Arbitrary-Crossover Filter-Banks and Spectrum Analyzers](#arbitrary-crossover-filter-
31 //   banks-and-spectrum-analyzers)
32 // * [State Variable Filters (SVF)](#state-variable-filters)
33 // * [Linkwitz-Riley 4th-order 2-way, 3-way, and 4-way crossovers](#linkwitz-riley-4th-order
34 //   -2-way-3-way-and-4-way-crossovers)
35 // * [Standardized Filters](#standardized-filters)
36 // * [Averaging Functions](#averaging-functions)
37 // * [Kalman Filters](#kalman-filters)
38 //
39 // #### References
40 //
41 // * <https://github.com/grame-cncm/faustlibraries/blob/master/filters.lib>

```

```

38 //#####
39
40 // NOTE ABOUT LICENSES:
41 // Each function in this library has its own license. Licenses are declared
42 // before each function. Corresponding license terms can be found at the
43 // bottom of this file or in the Faust libraries documentation.
44
45 ma = library("maths.lib");
46 ba = library("basics.lib");
47 ro = library("routes.lib");
48 de = library("delays.lib");
49 an = library("analyzers.lib");
50 ef = library("misceffects.lib");
51 si = library("signals.lib");
52 fi = library("filters.lib"); // for compatible copy/paste out of this file
53 la = library("linearalgebra.lib");
54
55 declare name "Faust Filters Library";
56 declare version "1.7.1";
57
58 //=====Basic Filters=====
59 //=====
60
61 //-----`(fi.)zero`-----
62 // One zero filter. Difference equation:  $(y(n) = x(n) - zx(n-1))$ .
63 //
64 // #### Usage
65 //
66 // ```
67 // _ : zero(z) : _
68 // ```
69 //
70 // Where:
71 //
72 // * `z`: location of zero along real axis in z-plane
73 //
74 // #### Test
75 // ```
76 // fi = library("filters.lib");
77 // os = library("oscillators.lib");
78 // src = os.osc(440);
79 // zero_test = src : fi.zero(0.5);
80 // ```
81 //
82 // #### References
83 // <https://ccrma.stanford.edu/~jos/filters/One\_Zero.html>
84 //-----
85 declare zero author "Julius O. Smith III";
86 declare zero copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
87 declare zero license "MIT-style STK-4.3 license";
88 zero(z) = _ <: _,mem : _,*(z) : -;
89
90 //-----`(fi.)pole`-----
91 // One pole filter. Could also be called a "leaky integrator".
92 // Difference equation:  $(y(n) = x(n) + py(n-1))$ .
93 //
94 // #### Usage
95 //
96 // ```
97 // _ : pole(p) : _
98 // ```
99 //
100 // Where:
101 //
102 // * `p`: pole location = feedback coefficient
103 //
104 // #### Test

```

```

105 // ```
106 // fi = library("filters.lib");
107 // os = library("oscillators.lib");
108 // src = os.osc(440);
109 // pole_test = src : fi.pole(0.9);
110 // ```
111 //
112 // #### References
113 // <https://ccrma.stanford.edu/~jos/filters/One\_Pole.html>
114 //-----
115 declare pole author "Julius O. Smith III";
116 declare pole copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
117 declare pole license "MIT-style STK-4.3 license";
118 pole(p) = + - *(p);
119
120 //-----`fi.)integrator`-----
121 // Same as `pole(1)` [implemented separately for block-diagram clarity].
122 //
123 // #### Usage
124 //
125 // ```
126 // _ : integrator : _
127 // ```
128 //
129 // #### Test
130 // ```
131 // fi = library("filters.lib");
132 // os = library("oscillators.lib");
133 // src = os.osc(440);
134 // integrator_test = src : fi.integrator;
135 // ```
136 //-----
137 declare integrator author "Julius O. Smith III";
138 declare integrator copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
139 declare integrator license "MIT-style STK-4.3 license";
140 integrator = + - _;
141
142 //-----`fi.)dcblockerat`-----
143 // DC blocker with configurable "break frequency".
144 // The amplitude response is substantially flat above `fb`,
145 // and sloped at about +6 dB/octave below `fb`.
146 // Derived from the analog transfer function:
147 //  $H(s) = \frac{s}{s + 2\pi f_b}$ 
148 // (which can be seen as a 1st-order Butterworth highpass filter)
149 // by the low-frequency-matching bilinear transform method
150 // (i.e., using the typical frequency-scaling constant `2*SR`).
151 //
152 // #### Usage
153 //
154 // ```
155 // _ : dcblockerat(fb) : _
156 // ```
157 //
158 // Where:
159 //
160 // * `fb`: "break frequency" in Hz, i.e., -3 dB gain frequency (see 2nd reference below)
161 //
162 // #### Test
163 // ```
164 // fi = library("filters.lib");
165 // os = library("oscillators.lib");
166 // src = os.osc(440);
167 // dcblockerat_test = src : fi.dcblockerat(30);
168 // ```
169 //
170 // #### References

```

```

171 //
172 // * <https://ccrma.stanford.edu/~jos/pasp/Bilinear\_Transformation.html>
173 // * <https://ccrma.stanford.edu/~jos/spectilt/Bode\_Plots.html>
174
175 //-----
176 declare dcblokerat author "Julius O. Smith III";
177 declare dcblokerat copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
178 declare dcblokerat license "MIT-style STK-4.3 license";
179 dcblokerat(fb) = *(b0) : zero(1) : pole(p)
180 with {
181     wn = ma.PI*fb/ma.SR;
182     b0 = 1.0 / (1 + wn);
183     p = (1 - wn) * b0;
184 };
185
186 //-----`fi.)dcbloker`-----
187 // DC blocker. Default dc blocker has -3dB point near 35 Hz (at 44.1 kHz)
188 // and high-frequency gain near 1.0025 (due to no scaling).
189 // `dcbloker` is as standard Faust function.
190 //
191 // #### Usage
192 //
193 // ...
194 // _ : dcbloker : _
195 // ...
196 //
197 // #### Test
198 // ...
199 // fi = library("filters.lib");
200 // os = library("oscillators.lib");
201 // src = os.osc(440);
202 // dcbloker_test = src : fi.dcbloker;
203 // ...
204 //-----
205 declare dcbloker author "Julius O. Smith III";
206 declare dcbloker copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
207 declare dcbloker license "MIT-style STK-4.3 license";
208 dcbloker = zero(1) : pole(0.995);
209
210 //-----`fi.)lptN`-----
211 // One-pole lowpass filter with arbitrary dis/charging factors set in dB and
212 // times set in seconds.
213 //
214 // #### Usage
215 //
216 // ...
217 // _ : lptN(N, tN) : _
218 // ...
219 //
220 // Where:
221 //
222 // * `N`: is the attenuation factor in dB
223 // * `tN`: is the filter period in seconds, that is, the time for the
224 // impulse response to decay by `N` dB
225 //
226 // #### Test
227 // ...
228 // fi = library("filters.lib");
229 // os = library("oscillators.lib");
230 // src = os.osc(440);
231 // lptN_test = src : fi.lptN(60, 0.1);
232 // ...
233 //
234 // #### References
235 //
236 // <https://ccrma.stanford.edu/~jos/mdft/Exponentials.html>

```

```

237 //-----
238 declare lptN author "Julius O. Smith III";
239 declare lptN copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
240 declare lptN license "MIT-style STK-4.3 license";
241 lptN(N, tN, x) = x : si.smooth(ba.tau2pole(tN / log(10.0^(float(N)/20.0))));
242 // Special cases of lptN
243 //-----` (fi.)lptau`-----
244 // One-pole lowpass with a tau time constant (1/e attenuation after `tN` seconds).
245 //
246 // #### Usage
247 //
248 // ```
249 // _ : lptau(tN) : _
250 // ```
251 //
252 // Where:
253 //
254 // * `tN`: tau time constant in seconds (1/e attenuation after `tN` seconds)
255 //
256 // #### Test
257 // ```
258 // fi = library("filters.lib");
259 // os = library("oscillators.lib");
260 // src = os.osc(440);
261 // lptau_test = src : fi.lptau(0.1);
262 // ```
263 //-----
264 lptau(tN, x) = lptN(8.6858896381, tN, x); // Tau time constant, i.e., 1/e atten. after tN
    secs
265
266 //-----` (fi.)lpt60`-----
267 // One-pole lowpass with a T60 time constant (60 dB attenuation after `tN` seconds).
268 //
269 // #### Usage
270 //
271 // ```
272 // _ : lpt60(tN) : _
273 // ```
274 //
275 // Where:
276 //
277 // * `tN`: T60 time constant in seconds (60 dB attenuation after `tN` seconds)
278 //
279 // #### Test
280 // ```
281 // fi = library("filters.lib");
282 // os = library("oscillators.lib");
283 // src = os.osc(440);
284 // lpt60_test = src : fi.lpt60(0.3);
285 // ```
286 //-----
287 lpt60(tN, x) = lptN(60, tN, x); // T60 constant, i.e., 1/1000 atten. after tN secs
288
289 //-----` (fi.)lpt19`-----
290 // One-pole lowpass with a T19 time constant (approx. 19 dB attenuation after `tN` seconds).
291 //
292 // #### Usage
293 //
294 // ```
295 // _ : lpt19(tN) : _
296 // ```
297 //
298 // Where:
299 //
300 // * `tN`: T19 time constant in seconds (approximately 19 dB attenuation after `tN` seconds)
301 //
302 // #### Test

```

```

303 // ```
304 // fi = library("filters.lib");
305 // os = library("oscillators.lib");
306 // src = os.osc(440);
307 // lpt19_test = src : fi.lpt19(0.2);
308 // ```
309 //-----
310 lpt19(tN, x) = lptN(19, tN, x); // T19 constant, i.e., 1/e^2.2 atten. after tN secs
311
312 //=====Comb Filters=====
313 //=====
314
315 //-----`(fi.)ff_comb`-----
316 // Feed-Forward Comb Filter. Note that `ff_comb` requires integer delays
317 // (uses `delay` internally).
318 // `ff_comb` is a standard Faust function.
319 //
320 // #### Usage
321 //
322 // ```
323 // _ : ff_comb(maxdel,intdel,b0,bM) : _
324 // ```
325 //
326 // Where:
327 //
328 // * `maxdel`: maximum delay (a power of 2)
329 // * `intdel`: current (integer) comb-filter delay between 0 and maxdel
330 // * `del`: current (float) comb-filter delay between 0 and maxdel
331 // * `b0`: gain applied to delay-line input
332 // * `bM`: gain applied to delay-line output and then summed with input
333 //
334 // #### Test
335 // ```
336 // fi = library("filters.lib");
337 // os = library("oscillators.lib");
338 // src = os.osc(440);
339 // ff_comb_test = src : fi.ff_comb(2048, 64, 1, 0.7);
340 // ```
341 //
342 // #### References
343 //
344 // <https://ccrma.stanford.edu/~jos/pasp/Feedforward\_Comb\_Filters.html>
345 //-----
346 declare ff_comb author "Julius O. Smith III";
347 declare ff_comb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
348 declare ff_comb license "MIT-style STK-4.3 license";
349 ff_comb(maxdel,M,b0,bM) = _ <: *(b0), bM * de.delay(maxdel,M) : +;
350
351 //-----`(fi.)ff_fcomb`-----
352 // Feed-Forward Comb Filter. Note that `ff_fcomb` takes floating-point delays
353 // (uses `fdelay` internally).
354 // `ff_fcomb` is a standard Faust function.
355 //
356 // #### Usage
357 //
358 // ```
359 // _ : ff_fcomb(maxdel,del,b0,bM) : _
360 // ```
361 //
362 // Where:
363 //
364 // * `maxdel`: maximum delay (a power of 2)
365 // * `del`: current (float) comb-filter delay between 0 and maxdel
366 // * `b0`: gain applied to delay-line input
367 // * `bM`: gain applied to delay-line output and then summed with input
368 //
369 // #### Test

```

```

370 // ```
371 // fi = library("filters.lib");
372 // os = library("oscillators.lib");
373 // src = os.osc(440);
374 // ff_fcomb_test = src : fi.ff_fcomb(2048, 64.5, 1, 0.7);
375 // ```
376 //
377 // #### References
378 //
379 // <https://ccrma.stanford.edu/~jos/pasp/Feedforward_Comb_Filters.html>
380 //-----
381 declare ff_fcomb author "Julius O. Smith III";
382 declare ff_fcomb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
383 declare ff_fcomb license "MIT-style STK-4.3 license";
384 ff_fcomb(maxdel,del,b0,bM) = _ <: *(b0), bM * de.fdelay(maxdel,del) : +;
385
386 //-----`(fi.)ff_fcombfilter`-----
387 // Typical special case of `ff_fcomb()` where: `b0 = 1`.
388 //
389 // #### Test
390 // ```
391 // fi = library("filters.lib");
392 // os = library("oscillators.lib");
393 // src = os.osc(440);
394 // ff_fcombfilter_test = src : fi.ff_fcombfilter(2048, 64, 0.7);
395 // ```
396 //-----
397 declare ff_fcombfilter author "Julius O. Smith III";
398 declare ff_fcombfilter copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
399 declare ff_fcombfilter license "MIT-style STK-4.3 license";
400 ff_fcombfilter(maxdel,del,g) = ff_fcomb(maxdel,del,1,g);
401
402 //-----`(fi.)fb_comb_common`-----
403 // A generic feedback comb filter.
404 //
405 // #### Usage
406 //
407 // ```
408 // _ : fb_comb_common(dop,N,b0,aN) : _
409 // ```
410 //
411 // Where
412 //
413 // * `dop`: delay operator, e.g. `@` or `de.fdelay4a(2048)`
414 // * `N`: current delay
415 // * `b0`: gain applied to input
416 // * `aN`: gain applied to delay-line output
417 //
418 // #### Example test program
419 //
420 // ```
421 // process = fb_comb_common(@,N,b0,aN);
422 // ```
423 // implements the following difference equation:
424 //
425 // 
$$y[n] = b_0 x[n] + a_N y[n - N]$$

426 //
427 //
428 // See more examples in `filters.lib` below.
429 //
430 // #### Test
431 // ```
432 // fi = library("filters.lib");
433 // os = library("oscillators.lib");
434 // src = os.osc(440);
435 // fb_comb_common_test = src : fi.fb_comb_common(@, 64, 0.8, 0.6);

```

```

436 // ``
437 // -----
438 declare fb_comb_common author "Oleg Nesterov";
439 fb_comb_common(dop,N,b0,aN) = + ~ aN * dop(N-1) : *(b0);
440
441 //-----`(fi.)fb_comb`-----
442 // Feed-Back Comb Filter (integer delay).
443 //
444 // #### Usage
445 //
446 // ``
447 // _ : fb_comb(maxdel,del,b0,aN) : _
448 // ``
449 //
450 // Where:
451 //
452 // * `maxdel`: maximum delay (a power of 2)
453 // * `del`: current (float) comb-filter delay between 0 and maxdel
454 // * `b0`: gain applied to delay-line input and forwarded to output
455 // * `aN`: minus the gain applied to delay-line output before summing with the input
456 // and feeding to the delay line
457 //
458 // #### Test
459 // ``
460 // fi = library("filters.lib");
461 // os = library("oscillators.lib");
462 // src = os.osc(440);
463 // fb_comb_test = src : fi.fb_comb(2048, 64, 0.7, 0.6);
464 // ``
465 //
466 // #### References
467 //
468 // <https://ccrma.stanford.edu/~jos/pasp/Feedback\_Comb\_Filters.html>
469 //-----
470 declare fb_comb author "Julius O. Smith III";
471 declare fb_comb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>, revised by Oleg Nesterov";
472 declare fb_comb license "MIT-style STK-4.3 license";
473 fb_comb(maxdel,del,b0,aN) = fb_comb_common(de.delay(maxdel),del,b0,-aN) : mem;
474
475 //-----`(fi.)fb_fcomb`-----
476 // Feed-Back Comb Filter (floating point delay).
477 //
478 // #### Usage
479 //
480 // ``
481 // _ : fb_fcomb(maxdel,del,b0,aN) : _
482 // ``
483 //
484 // Where:
485 //
486 // * `maxdel`: maximum delay (a power of 2)
487 // * `del`: current (float) comb-filter delay between 0 and maxdel
488 // * `b0`: gain applied to delay-line input and forwarded to output
489 // * `aN`: minus the gain applied to delay-line output before summing with the input
490 // and feeding to the delay line
491 //
492 // #### Test
493 // ``
494 // fi = library("filters.lib");
495 // os = library("oscillators.lib");
496 // src = os.osc(440);
497 // fb_fcomb_test = src : fi.fb_fcomb(2048, 64.5, 0.7, 0.6);
498 // ``
499 //
500 // #### References
501 //
502 // <https://ccrma.stanford.edu/~jos/pasp/Feedback\_Comb\_Filters.html>

```

```

503 //-----
504 declare fb_fcomb author "Julius O. Smith III";
505 declare fb_fcomb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>, revised by Oleg Nesterov";
506 declare fb_fcomb license "MIT-style STK-4.3 license";
507 fb_fcomb(maxdel,del,b0,aN) = fb_comb_common(de.fdelay(maxdel),del,b0,-aN) : mem;
508
509 //-----`(fi.)rev1`-----
510 // Special case of `fb_comb` (`rev1(maxdel,N,g)`).
511 // The "rev1 section" dates back to the 1960s in computer-music reverberation.
512 // See the `jcrev` and `brassrev` in `reverbs.lib` for usage examples.
513 //
514 // #### Test
515 // ```
516 // fi = library("filters.lib");
517 // os = library("oscillators.lib");
518 // src = os.osc(440);
519 // rev1_test = src : fi.rev1(2048, 64, 0.6);
520 // ```
521 //-----
522 declare rev1 author "Julius O. Smith III";
523 declare rev1 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
524 declare rev1 license "MIT-style STK-4.3 license";
525 rev1(maxdel,N,g) = fb_comb(maxdel,N,1,-g);
526
527 //----`(fi.)fbcombfilter` and `(fi.)ffbcombfilter`-----
528 // Other special cases of Feed-Back Comb Filter.
529 //
530 // #### Usage
531 //
532 // ```
533 // _ : fbcombfilter(maxdel,intdel,g) : _
534 // _ : ffbcombfilter(maxdel,del,g) : _
535 // ```
536 //
537 // Where:
538 //
539 // * `maxdel`: maximum delay (a power of 2)
540 // * `intdel`: current (integer) comb-filter delay between 0 and maxdel
541 // * `del`: current (float) comb-filter delay between 0 and maxdel
542 // * `g`: feedback gain
543 //
544 // #### Test
545 // ```
546 // fi = library("filters.lib");
547 // os = library("oscillators.lib");
548 // src = os.osc(440);
549 // fbcombfilter_test = src : fi.fbcombfilter(2048, 64, 0.6);
550 // ```
551 //
552 // #### References
553 //
554 // <https://ccrma.stanford.edu/~jos/pasp/Feedback\_Comb\_Filters.html>
555 //-----
556 declare fbcombfilter author "Julius O. Smith III";
557 declare fbcombfilter copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
558 declare fbcombfilter license "MIT-style STK-4.3 license";
559 fbcombfilter(maxdel,intdel,g) = (+ : de.delay(maxdel,intdel)) ~ *(g);
560
561 declare ffbcombfilter author "Julius O. Smith III";
562 declare ffbcombfilter copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
563 declare ffbcombfilter license "MIT-style STK-4.3 license";
564 ffbcombfilter(maxdel,del,g) = (+ : de.fdelay(maxdel,del)) ~ *(g);
565
566

```

```

567 //-----`(fi.)allpass_comb`-----
568 // Schroeder Allpass Comb Filter. Note that:
569 //
570 // ...
571 // allpass_comb(maxlen,len,aN) = ff_comb(maxlen,len,aN,1) : fb_comb(maxlen,len-1,1,aN);
572 // ...
573 //
574 // which is a direct-form-1 implementation, requiring two delay lines.
575 // The implementation here is direct-form-2 requiring only one delay line.
576 //
577 // #### Usage
578 //
579 // ...
580 // _ : allpass_comb(maxdel,intdel,aN) : _
581 // ...
582 //
583 // Where:
584 //
585 // * `maxdel`: maximum delay (a power of 2)
586 // * `intdel`: current (integer) comb-filter delay between 0 and maxdel
587 // * `del`: current (float) comb-filter delay between 0 and maxdel
588 // * `aN`: minus the feedback gain
589 //
590 // #### Test
591 // ...
592 // fi = library("filters.lib");
593 // os = library("oscillators.lib");
594 // src = os.osc(440);
595 // allpass_comb_test = src : fi.allpass_comb(2048, 64, 0.6);
596 // ...
597 //
598 // #### References
599 //
600 // * <https://ccrma.stanford.edu/~jos/pasp/Allpass\_Two\_Combs.html>
601 // * <https://ccrma.stanford.edu/~jos/pasp/Schroeder\_Allpass\_Sections.html>
602 // * <https://ccrma.stanford.edu/~jos/filters/Four\_Direct\_Forms.html>
603 //-----
604 declare allpass_comb author "Julius O. Smith III";
605 declare allpass_comb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
606 declare allpass_comb license "MIT-style STK-4.3 license";
607 allpass_comb(maxdel,N,aN) = (+ <: de.delay(maxdel,N-1),*(aN)) ~ *(-aN) : mem,_ : +;
608
609
610 //-----`(fi.)allpass_fcomb`-----
611 // Schroeder Allpass Comb Filter. Note that:
612 //
613 // ...
614 // allpass_comb(maxlen,len,aN) = ff_comb(maxlen,len,aN,1) : fb_comb(maxlen,len-1,1,aN);
615 // ...
616 //
617 // which is a direct-form-1 implementation, requiring two delay lines.
618 // The implementation here is direct-form-2 requiring only one delay line.
619 //
620 // `allpass_fcomb` is a standard Faust library.
621 //
622 // #### Usage
623 //
624 // ...
625 // _ : allpass_comb(maxdel,intdel,aN) : _
626 // _ : allpass_fcomb(maxdel,del,aN) : _
627 // ...
628 //
629 // Where:
630 //
631 // * `maxdel`: maximum delay (a power of 2)
632 // * `intdel`: current (float) comb-filter delay between 0 and maxdel
633 // * `del`: current (float) comb-filter delay between 0 and maxdel

```

```

634 // * `aN`: minus the feedback gain
635 //
636 // #### Test
637 // ...
638 // fi = library("filters.lib");
639 // os = library("oscillators.lib");
640 // src = os.osc(440);
641 // allpass_fcomb_test = src : fi.allpass_fcomb(2048, 64.5, 0.6);
642 // ...
643 //
644 // #### References
645 //
646 // * <https://ccrma.stanford.edu/~jos/pasp/Allpass_Two_Combs.html>
647 // * <https://ccrma.stanford.edu/~jos/pasp/Schroeder_Allpass_Sections.html>
648 // * <https://ccrma.stanford.edu/~jos/filters/Four_Direct_Forms.html>
649 //-----
650 declare allpass_fcomb author "Julius O. Smith III";
651 declare allpass_fcomb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
652 declare allpass_fcomb license "MIT-style STK-4.3 license";
653 allpass_fcomb(maxdel,N,aN) = (+ <: de.fdelay(maxdel,N-1),*(aN)) ~ *(-aN) : mem, _ : +;
654
655 //-----`(fi).rev2`-----
656 // Special case of `allpass_comb` (`rev2(maxlen,len,g)`).
657 // The "rev2 section" dates back to the 1960s in computer-music reverberation.
658 // See the `jcrev` and `brassrev` in `reverbs.lib` for usage examples.
659 //
660 // #### Test
661 // ...
662 // fi = library("filters.lib");
663 // os = library("oscillators.lib");
664 // src = os.osc(440);
665 // rev2_test = src : fi.rev2(2048, 64, 0.6);
666 // ...
667 //-----
668
669 declare rev2 author "Julius O. Smith III";
670 declare rev2 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
671 declare rev2 license "MIT-style STK-4.3 license";
672 rev2(maxlen,len,g) = allpass_comb(maxlen,len,-g);
673
674 //-----`(fi.)allpass_fcomb5` and `(fi.)allpass_fcomb1a`-----
675 // Same as `allpass_fcomb` but use `fdelay5` and `fdelay1a` internally
676 // (Interpolation helps - look at an fft of faust2octave on:
677 // `1-1' <: allpass_fcomb(1024,10.5,0.95), allpass_fcomb5(1024,10.5,0.95);`)
678 //
679 // #### Test
680 // ...
681 // fi = library("filters.lib");
682 // os = library("oscillators.lib");
683 // src = os.osc(440);
684 // allpass_fcomb5_test = src : fi.allpass_fcomb5(2048, 64.5, 0.6);
685 // ...
686 //-----
687 declare allpass_fcomb5 author "Julius O. Smith III";
688 declare allpass_fcomb5 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
689 declare allpass_fcomb5 license "MIT-style STK-4.3 license";
690 allpass_fcomb5(maxdel,N,aN) = (+ <: de.fdelay5(maxdel,N-1),*(aN)) ~ *(-aN) : mem, _ : +;
691
692 declare allpass_fcomb1a author "Julius O. Smith III";
693 declare allpass_fcomb1a copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
694 declare allpass_fcomb1a license "MIT-style STK-4.3 license";
695 allpass_fcomb1a(maxdel,N,aN) = (+ <: de.fdelay1a(maxdel,N-1),*(aN)) ~ *(-aN) : mem, _ : +;
696
697

```

```

698 //=====Direct-Form Digital Filter Sections=====
699 //=====
700
701 // Specified by transfer-function polynomials B(z)/A(z) as in matlab
702
703 //-----`(fi.)iir`-----
704 // Nth-order Infinite-Impulse-Response (IIR) digital filter,
705 // implemented in terms of the Transfer-Function (TF) coefficients.
706 // Such filter structures are termed "direct form".
707 //
708 // `iir` is a standard Faust function.
709 //
710 // #### Usage
711 //
712 // ```
713 // _ : iir(bcoeffs,acoeffs) : _
714 // ```
715 //
716 // Where:
717 //
718 // * `bcoeffs`: (b0,b1,...,b_order) = TF numerator coefficients
719 // * `acoeffs`: (a1,...,a_order) = TF denominator coeffs (a0=1)
720 //
721 // #### Test
722 // ```
723 // fi = library("filters.lib");
724 // os = library("oscillators.lib");
725 // src = os.osc(440);
726 // iir_test = src : fi.iir((0.5, 0.5), (0.3));
727 // ```
728 //
729 // #### References
730 //
731 // <https://ccrma.stanford.edu/~jos/filters/Four\_Direct\_Forms.html>
732 //-----
733 declare iir author "Julius O. Smith III";
734 declare iir copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
735 declare iir license "MIT-style STK-4.3 license";
736 iir(bv,av) = ma.sub ~ fir(av) : fir(bv);
737
738 //-----`(fi.)fir`-----
739 // FIR filter (convolution of FIR filter coefficients with a signal). `fir` is standard Faust
    function.
740 //
741 // #### Usage
742 //
743 // ```
744 // _ : fir(bv) : _
745 // ```
746 //
747 // Where:
748 //
749 // * `bv`: `b0,b1,...,bn`, a parallel bank of coefficient signals
750 //
751 // #### Note
752 //
753 // `bv` is processed using pattern-matching at compile time,
754 // so it must have this normal form (parallel signals).
755 //
756 // #### Example test program
757 //
758 // Smoothing white noise with a five-point moving average:
759 //
760 // ```
761 // bv = .2,.2,.2,.2,.2;
762 // process = noise : fir(bv);
763 // ```

```

```

764 //
765 // Equivalent (note double parens):
766 //
767 // ...
768 // process = noise : fir((.2,.2,.2,.2,.2));
769 // ...
770 //-----
771 //fir(bv) = conv(bv);
772 //
773 // #### Test
774 // ...
775 // fi = library("filters.lib");
776 // os = library("oscillators.lib");
777 // src = os.osc(440);
778 // fir_test = src : fi.fir((0.2, 0.2, 0.2, 0.2, 0.2));
779 // ...
780 //-----
781 declare fir author "Julius O. Smith III";
782 declare fir copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
783 declare fir license "MIT-style STK-4.3 license";
784 fir((b0,bv)) = _ <: *(b0), R(1,bv) :> _ with {
785     R(n,(bn,bv)) = (@(n):*(bn)), R(n+1,bv);
786     R(n, bn) = (@(n):*(bn)); };
787 fir(b0) = *(b0);
788
789 //-----`(fi.)conv` and `(fi.)convN`-----
790 // Convolution of input signal with given coefficients.
791 //
792 // #### Usage
793 //
794 // ...
795 // _ : conv((k1,k2,k3,...,kN)) : _ // Argument = one signal bank
796 // _ : convN(N,(k1,k2,k3,...)) : _ // Useful when N < count((k1,...))
797 // ...
798 //-----
799 //convN(N,kv,x) = sum(i,N,ba.take(i+1,kv) * x@i); // take() defined in basics.lib
800 //
801 // #### Test
802 // ...
803 // fi = library("filters.lib");
804 // os = library("oscillators.lib");
805 // si = library("signals.lib");
806 // src = os.osc(440);
807 // convN_test = (src <: si.bus(3)) : fi.convN(3, (0.3, 0.2, 0.1, 0.05));
808 // ...
809
810 declare convN author "Julius O. Smith III";
811 declare convN copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
812 declare convN license "MIT-style STK-4.3 license";
813 convN(N,kv) = sum(i,N, @(i)*ba.take(i+1,kv)); // take() defined in basics.lib
814 //conv(kv,x) = sum(i,count(kv),take(i+1,kv) * x@i); // count() from math.lib
815
816 declare conv author "Julius O. Smith III";
817 declare conv copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
818 declare conv license "MIT-style STK-4.3 license";
819 conv(kv) = fir(kv);
820
821 //-----`(fi.)tf1`, `(fi.)tf2` and `(fi.)tf3`-----
822 // tfN = N'th-order direct-form digital filter.
823 //
824 // #### Usage
825 //
826 // ...
827 // _ : tf1(b0,b1,a1) : _
828 // _ : tf2(b0,b1,b2,a1,a2) : _

```

```

829 // _ : tf3(b0,b1,b2,b3,a1,a2,a3) : _
830 // ...
831 //
832 // Where:
833 //
834 // * `b`: transfer-function numerator
835 // * `a`: transfer-function denominator (monic)
836 //
837 // #### Test
838 // ...
839 // fi = library("filters.lib");
840 // os = library("oscillators.lib");
841 // src = os.osc(440);
842 // tf1_test = src : fi.tf1(0.5, 0.25, -0.4);
843 // ...
844 //
845 // #### References
846 //
847 // <https://ccrma.stanford.edu/~jos/fp/Direct_Form_I.html>
848 //-----
849 declare tf1 author "Julius O. Smith III";
850 declare tf1 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
      edu>";
851 declare tf1 license "MIT-style STK-4.3 license";
852 tf1(b0,b1,a1) = _ <: *(b0), (mem : *(b1)) :> + ~ *(0-a1);
853
854 declare tf2 author "Julius O. Smith III";
855 declare tf2 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
      edu>";
856 declare tf2 license "MIT-style STK-4.3 license";
857 tf2(b0,b1,b2,a1,a2) = iir((b0,b1,b2),(a1,a2));
858 // tf2 is a variant of tf22 below with duplicated mems
859
860 declare tf3 author "Julius O. Smith III";
861 declare tf3 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
      edu>";
862 declare tf3 license "MIT-style STK-4.3 license";
863 tf3(b0,b1,b2,b3,a1,a2,a3) = iir((b0,b1,b2,b3),(a1,a2,a3));
864
865 // "Original" version for music.lib. This is here for comparison but people should
866 // use tf2 instead
867 TF2(b0,b1,b2,a1,a2) = sub ~ conv2(a1,a2) : conv3(b0,b1,b2)
868 with {
869     conv3(k0,k1,k2,x) = k0*x + k1*x' + k2*x'';
870     conv2(k0,k1,x) = k0*x + k1*x';
871     sub(x,y) = y-x;
872 };
873
874 //-----`fi.)notchw`-----
875 // Simple notch filter based on a biquad (`tf2`).
876 // `notchw` is a standard Faust function.
877 //
878 // #### Usage:
879 //
880 // ...
881 // _ : notchw(width,freq) : _
882 // ...
883 //
884 // Where:
885 //
886 // * `width`: "notch width" in Hz (approximate)
887 // * `freq`: "notch frequency" in Hz
888 //
889 // #### Test
890 // ...
891 // fi = library("filters.lib");
892 // os = library("oscillators.lib");
893 // src = os.osc(440);

```

```

894 // notchw_test = src : fi.notchw(200, 1000);
895 // ```
896 //
897 // #### References
898 //
899 // <https://ccrma.stanford.edu/~jos/pasp/Phasing\_2nd\_Order\_Allpass\_Filters.html>
900 //-----
901 declare notchw author "Julius O. Smith III";
902 declare notchw copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
903 declare notchw license "MIT-style STK-4.3 license";
904 notchw(width,freq) = tf2(b0,b1,b2,a1,a2)
905 with {
906   fb = 0.5*width; // First design a dcblokerat(width/2)
907   wn = ma.PI*fb/ma.SR;
908   b0db = 1.0 / (1 + wn);
909   p = (1 - wn) * b0db; // This is our pole radius.
910   // Now place unit-circle zeros at desired angles:
911   tn = 2*ma.PI*freq/ma.SR;
912   a2 = p * p;
913   a2p1 = 1+a2;
914   a1 = -a2p1*cos(tn);
915   b1 = a1;
916   b0 = 0.5*a2p1;
917   b2 = b0;
918 };
919
920 //=====Direct-Form Second-Order Biquad Sections=====
921 // Direct-Form Second-Order Biquad Sections
922 //
923 // #### References
924 //
925 // <https://ccrma.stanford.edu/~jos/filters/Four\_Direct\_Forms.html>
926 //=====
927
928 //-----`(fi.)tf21`, `(fi.)tf22`, `(fi.)tf22t` and `(fi.)tf21t`
929 //-----
930 // tfN = N'th-order direct-form digital filter where:
931 //
932 // * `tf21` is tf2, direct-form 1
933 // * `tf22` is tf2, direct-form 2
934 // * `tf22t` is tf2, direct-form 2 transposed
935 // * `tf21t` is tf2, direct-form 1 transposed
936 //
937 // #### Usage
938 //
939 // _ : tf21(b0,b1,b2,a1,a2) : _
940 // _ : tf22(b0,b1,b2,a1,a2) : _
941 // _ : tf22t(b0,b1,b2,a1,a2) : _
942 // _ : tf21t(b0,b1,b2,a1,a2) : _
943 //
944 //
945 // Where:
946 //
947 // * `b`: transfer-function numerator
948 // * `a`: transfer-function denominator (monic)
949 //
950 // #### Test
951 //
952 // fi = library("filters.lib");
953 // os = library("oscillators.lib");
954 // src = os.osc(440);
955 // tf21_test = src : fi.tf21(0.1, 0.2, 0.1, -0.5, 0.06);
956 //
957 //
958 // #### References
959 //

```

```

960 // <https://ccrma.stanford.edu/~jos/fp/Direct_Form_I.html>
961 //-----
962 declare tf21 author "Julius O. Smith III";
963 declare tf21 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
964 declare tf21 license "MIT-style STK-4.3 license";
965 tf21(b0,b1,b2,a1,a2) = // tf2, direct-form 1:
966   _ <:(mem<:(mem:*(b2)),*(b1)),*(b0) :>_
967   : ((_,_,_>_) ~(_<:*(-a1),(mem:*(-a2))));
968
969 declare tf22 author "Julius O. Smith III";
970 declare tf22 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
971 declare tf22 license "MIT-style STK-4.3 license";
972 tf22(b0,b1,b2,a1,a2) = // tf2, direct-form 2:
973   _ : ((_,_,_>_)~*(-a1)<:mem,*(b0))~*(-a2))
974   : (_<:mem,*(b1)),_ : *(b2),_,_ :> _;
975
976 declare tf22t author "Julius O. Smith III";
977 declare tf22t copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
978 declare tf22t license "MIT-style STK-4.3 license";
979 tf22t(b0,b1,b2,a1,a2) = // tf2, direct-form 2 transposed:
980   _ : (_,_,_<: *(b2)',*(b1)',*(b0))
981   : _,+',_,_ :> _)*(-a1)~*(-a2) : _;
982
983 declare tf21t author "Julius O. Smith III";
984 declare tf21t copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
985 declare tf21t license "MIT-style STK-4.3 license";
986 tf21t(b0,b1,b2,a1,a2) = // tf2, direct-form 1 transposed:
987   tf22t(1,0,0,a1,a2) : tf22t(b0,b1,b2,0,0); // or write it out if you want
988
989 //===== Ladder/Lattice Digital Filters =====
990 // Ladder and lattice digital filters generally have superior numerical
991 // properties relative to direct-form digital filters. They can be derived
992 // from digital waveguide filters, which gives them a physical interpretation.
993
994 // #### References
995 //
996 // * F. Itakura and S. Saito: "Digital Filtering Techniques for Speech Analysis and
    Synthesis",
997 //   7th Int. Cong. Acoustics, Budapest, 25 C 1, 1971.
998 // * J. D. Markel and A. H. Gray: Linear Prediction of Speech, New York: Springer Verlag,
    1976.
999 // * <https://ccrma.stanford.edu/~jos/pasp/Conventional_Ladder_Filters.html>
1000 //=====
1001
1002 //-----` (fi.)av2sv`-----
1003 // Compute reflection coefficients sv from transfer-function denominator av.
1004 //
1005 // #### Usage
1006 //
1007 // ```
1008 // sv = av2sv(av)
1009 // ```
1010 //
1011 // Where:
1012 //
1013 // * `av`: parallel signal bank `a1,...,aN`
1014 // * `sv`: parallel signal bank `s1,...,sN`
1015 //
1016 // where `ro` = ith` reflection coefficient, and
1017 // `ai` = coefficient of `z^(-i)` in the filter
1018 // transfer-function denominator `A(z)`.
1019 //
1020 // #### Test
1021 // ```

```

```

1022 // fi = library("filters.lib");
1023 // si = library("signals.lib");
1024 // av2sv_test = fi.av2sv((-0.4, 0.1)) : si.bus(2);
1025 // ```
1026 //
1027 // #### References
1028 //
1029 // <https://ccrma.stanford.edu/~jos/filters/Step_Down_Procedure.html>
1030 // (where reflection coefficients are denoted by k rather than s).
1031 //-----
1032 declare av2sv author "Julius O. Smith III";
1033 declare av2sv copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
1034 declare av2sv license "MIT-style STK-4.3 license";
1035 av2sv(av) = par(i,M,s(i+1)) with {
1036   M = ba.count(av);
1037   s(m) = sr(M-m+1); // m=1..M
1038   sr(m) = Ari(m,M-m+1); // s_{M-1-m}
1039   Ari(m,i) = ba.take(i+1,Ar(m-1));
1040   //step-down recursion for lattice/ladder digital filters:
1041   Ar(0) = (1,av); // Ar(m) is order M-m (i.e. "reverse-indexed")
1042   Ar(m) = 1,par(i,M-m, (Ari(m,i+1) - sr(m)*Ari(m,M-m-i))/(1-sr(m)*sr(m)));
1043 };
1044
1045 //-----`fi.)bvav2nuv`-----
1046 // Compute lattice tap coefficients from transfer-function coefficients.
1047 //
1048 // #### Usage
1049 //
1050 // ```
1051 // nuv = bvav2nuv(bv,av)
1052 // ```
1053 //
1054 // Where:
1055 //
1056 // * `av`: parallel signal bank `a1,...,aN`
1057 // * `bv`: parallel signal bank `b0,b1,...,aN`
1058 // * `nuv`: parallel signal bank `nu1,...,nuN`
1059 //
1060 // where `nui` is the i'th tap coefficient,
1061 //   `bi` is the coefficient of `z^{-i}` in the filter numerator,
1062 //   `ai` is the coefficient of `z^{-i}` in the filter denominator
1063 //
1064 // #### Test
1065 // ```
1066 // fi = library("filters.lib");
1067 // si = library("signals.lib");
1068 // bvav2nuv_test = fi.bvav2nuv((0.1, 0.2, 0.3), (-0.4, 0.1)) : si.bus(3);
1069 // ```
1070 //-----
1071 declare bvav2nuv author "Julius O. Smith III";
1072 declare bvav2nuv copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
1073 declare bvav2nuv license "MIT-style STK-4.3 license";
1074 bvav2nuv(bv,av) = par(m,M+1,nu(m)) with {
1075   M = ba.count(av);
1076   nu(m) = ba.take(m+1,Pr(M-m)); // m=0..M
1077   // lattice/ladder tap parameters:
1078   Pr(0) = bv; // Pr(m) is order M-m, 'r' means "reversed"
1079   Pr(m) = par(i,M-m+1, (Pri(m,i) - nu(M-m+1)*Ari(m,M-m-i+1)));
1080   Pri(m,i) = ba.take(i+1,Pr(m-1));
1081   Ari(m,i) = ba.take(i+1,Ar(m-1));
1082   //step-down recursion for lattice/ladder digital filters:
1083   Ar(0) = (1,av); // Ar(m) is order M-m (recursion index must start at constant)
1084   Ar(m) = 1,par(i,M-m, (Ari(m,i+1) - sr(m)*Ari(m,M-m-i))/(1-sr(m)*sr(m)));
1085   sr(m) = Ari(m,M-m+1); // s_{M-1-m}
1086 };
1087

```

```

1088 //-----`(fi.)iir_lat2`-----
1089 // Two-multiply lattice IIR filter of arbitrary order.
1090 //
1091 // #### Usage
1092 //
1093 // ...
1094 // _ : iir_lat2(bv,av) : _
1095 // ...
1096 //
1097 // Where:
1098 //
1099 // * `bv`: transfer-function numerator
1100 // * `av`: transfer-function denominator (monic)
1101 //
1102 // #### Test
1103 // ...
1104 // fi = library("filters.lib");
1105 // os = library("oscillators.lib");
1106 // src = os.osc(440);
1107 // iir_lat2_test = src : fi.iir_lat2((0.1, 0.2, 0.3), (-0.4, 0.1));
1108 // ...
1109 //-----
1110 declare iir_lat2 author "Julius O. Smith III";
1111 declare iir_lat2 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
1112 declare iir_lat2 license "MIT-style STK-4.3 license";
1113 iir_lat2(bv,av) = allpassnt(M,sv) : sum(i,M+1,*(ba.take(M-i+1,tg)))
1114 with {
1115     M = ba.count(av);
1116     sv = av2sv(av); // sv = vector of sin(theta) reflection coefficients
1117     tg = bvav2nuv(bv,av); // tg = vector of tap gains
1118 };
1119
1120 //-----`(fi.)allpassnt`-----
1121 // Two-multiply lattice allpass (nested order-1 direct-form-ii allpasses), with taps.
1122 //
1123 // #### Usage
1124 //
1125 // ...
1126 // _ : allpassnt(n,sv) : si.bus(n+1)
1127 // ...
1128 //
1129 // Where:
1130 //
1131 // * `n`: the order of the filter
1132 // * `sv`: the reflection coefficients (-1 1)
1133 //
1134 // The first output is the n-th order allpass output,
1135 // while the remaining outputs are taps taken from the
1136 // input of each delay element from the input to the output.
1137 // See (fi.)allpassn for the single-output case.
1138 //
1139 // #### Test
1140 // ...
1141 // fi = library("filters.lib");
1142 // os = library("oscillators.lib");
1143 // si = library("signals.lib");
1144 // src = os.osc(440);
1145 // allpassnt_test = src : fi.allpassnt(2, (0.3, -0.2)) : si.bus(3);
1146 // ...
1147 //-----
1148 declare allpassnt author "Julius O. Smith III";
1149 declare allpassnt copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
1150 declare allpassnt license "MIT-style STK-4.3 license";
1151 allpassnt(0,sv) = _;
1152 allpassnt(n,sv) = _ : ((+ <: (allpassnt(n-1,sv),*(s)))-*(-s)) : fsec(n)
1153 with {

```

```

1154 fsec(1) = ro.crossnn(1) : _, (_<:mem,_) : +,_;
1155 fsec(n) = ro.crossnn(1)(n) : _, (_<:mem,_) : +, par(i,n-1,_);
1156 innertaps(n) = par(i,n,_);
1157 s = ba.take(n,sv); // reflection coefficient s = sin(theta)
1158 };
1159
1160 //-----`(fi.)iir_kl`-----
1161 // Kelly-Lochbaum ladder IIR filter of arbitrary order.
1162 //
1163 // #### Usage
1164 //
1165 // ...
1166 // _ : iir_kl(bv,av) : _
1167 // ...
1168 //
1169 // Where:
1170 //
1171 // * `bv`: transfer-function numerator
1172 // * `av`: transfer-function denominator (monic)
1173 //
1174 // #### Test
1175 // ...
1176 // fi = library("filters.lib");
1177 // os = library("oscillators.lib");
1178 // src = os.osc(440);
1179 // iir_kl_test = src : fi.iir_kl((0.1, 0.2, 0.3), (-0.4, 0.1));
1180 // ...
1181 //-----
1182 declare iir_kl author "Julius O. Smith III";
1183 declare iir_kl copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
1184 declare iir_kl license "MIT-style STK-4.3 license";
1185 iir_kl(bv,av) = allpassnkl(M,sv) : sum(i,M+1,*(tghr(i)))
1186 with {
1187   M = ba.count(av);
1188   sv = av2sv(av); // sv = vector of sin(theta) reflection coefficients
1189   tg = bvav2nuv(bv,av); // tg = vector of tap gains for 2mul case
1190   tgr(i) = ba.take(M+1-i,tg);
1191   tghr(n) = tgr(n)/pi(n);
1192   pi(0) = 1;
1193   pi(n) = pi(n-1)*(1+ba.take(M-n+1,sv)); // all sign parameters '+'
1194 };
1195
1196 //-----`(fi.)allpassnkl`-----
1197 // Kelly-Lochbaum ladder allpass.
1198 //
1199 // #### Usage:
1200 //
1201 // ...
1202 // _ : allpassnkl(n,sv) : _
1203 // ...
1204 //
1205 // Where:
1206 //
1207 // * `n`: the order of the filter
1208 // * `sv`: the reflection coefficients (-1 1)
1209 //
1210 // #### Test
1211 // ...
1212 // fi = library("filters.lib");
1213 // os = library("oscillators.lib");
1214 // si = library("signals.lib");
1215 // src = os.osc(440);
1216 // allpassnkl_test = src : fi.allpassnkl(2, (0.3, -0.2)) : si.bus(3);
1217 // ...
1218 //-----
1219 declare allpassnkl author "Julius O. Smith III";
1220 declare allpassnkl copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.

```

```

stanford.edu>";
1221 declare allpassnkl license "MIT-style STK-4.3 license";
1222 allpassnkl(0,sv) = _;
1223 allpassnkl(n,sv) = _ <: *(s),*(1+s) : (+
1224           : allpassnkl(n-1,sv))-*(-s)) : fsec(n)
1225 with {
1226   fsec(1) = _, (<:mem*(1-s),_) : sumandtaps(n);
1227   fsec(n) = _, (<:mem*(1-s),_), par(i,n-1,_) : sumandtaps(n);
1228   s = ba.take(n,sv);
1229   sumandtaps(n) = +,par(i,n,_)
1230 };
1231
1232 //-----`(fi.)iir_lat1`-----
1233 // One-multiply lattice IIR filter of arbitrary order.
1234 //
1235 // ### Usage
1236 //
1237 // ```
1238 // _ : iir_lat1(bv,av) : _
1239 // ```
1240 //
1241 // Where:
1242 //
1243 // * `bv`: transfer-function numerator as a bank of parallel signals
1244 // * `av`: transfer-function denominator as a bank of parallel signals
1245 //
1246 // ### Test
1247 // ```
1248 // fi = library("filters.lib");
1249 // os = library("oscillators.lib");
1250 // src = os.osc(440);
1251 // iir_lat1_test = src : fi.iir_lat1((0.1, 0.2, 0.3), (-0.4, 0.1));
1252 // ```
1253 //-----
1254 declare iir_lat1 author "Julius O. Smith III";
1255 declare iir_lat1 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
stanford.edu>";
1256 declare iir_lat1 license "MIT-style STK-4.3 license";
1257 iir_lat1(bv,av) = allpassn1mt(M,sv) : sum(i,M+1,(tghr(i+1)))
1258 with {
1259   M = ba.count(av);
1260   sv = av2sv(av); // sv = vector of sin(theta) reflection coefficients
1261   tg = bvav2nuv(bv,av); // tg = vector of tap gains
1262   tgr(i) = ba.take(M+2-i,tg); // i=1..M+1 (for "takability")
1263   tghr(n) = tgr(n)/pi(n);
1264   pi(1) = 1;
1265   pi(n) = pi(n-1)*(1+ba.take(M-n+2,sv)); // all sign parameters '+'
1266 };
1267
1268 //-----`(fi.)allpassn1mt`-----
1269 // One-multiply lattice allpass with tap lines.
1270 //
1271 // ### Usage
1272 //
1273 // ```
1274 // _ : allpassn1mt(N,sv) : _
1275 // ```
1276 //
1277 // Where:
1278 //
1279 // * `N`: the order of the filter (fixed at compile time)
1280 // * `sv`: the reflection coefficients (-1 1)
1281 //
1282 // ### Test
1283 // ```
1284 // fi = library("filters.lib");
1285 // os = library("oscillators.lib");
1286 // si = library("signals.lib");

```

```

1287 // src = os.osc(440);
1288 // allpassn1mt_test = src : fi.allpassn1mt(2, (0.3, -0.2)) : si.bus(3);
1289 // ...
1290 //-----
1291 declare allpassn1mt author "Julius O. Smith III";
1292 declare allpassn1mt copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
1293 declare allpassn1mt license "MIT-style STK-4.3 license";
1294 allpassn1mt(0,sv) = _;
1295 allpassn1mt(n,sv) = _ <: _,_ : ((+:(s) <: _,_) ,_ : _,+ : ro.crossnn(1)
1296     : allpassn1mt(n-1,sv),_)*(-1)) : fsec(n)
1297 with {
1298     fsec(1) = ro.crossnn(1) : _, (_<:mem,_) : +,_;
1299     fsec(n) = ro.crossnn(1) : _, (_<:mem,_) ,par(i,n-1,_) : +, par(i,n,_) ;
1300     innertaps(n) = par(i,n,_) ;
1301     s = ba.take(n,sv); // reflection coefficient s = sin(theta)
1302 };
1303
1304 //-----`fi.)iir_n1`-----
1305 // Normalized ladder filter of arbitrary order.
1306 //
1307 // #### Usage
1308 //
1309 // ...
1310 // _ : iir_n1(bv,av) : _
1311 // ...
1312 //
1313 // Where:
1314 //
1315 // * `bv`: transfer-function numerator
1316 // * `av`: transfer-function denominator (monic)
1317 //
1318 // #### Test
1319 // ...
1320 // fi = library("filters.lib");
1321 // os = library("oscillators.lib");
1322 // src = os.osc(440);
1323 // iir_n1_test = src : fi.iir_n1((0.1, 0.2, 0.3), (-0.4, 0.1));
1324 // ...
1325 //
1326 // #### References
1327 //
1328 // * J. D. Markel and A. H. Gray, Linear Prediction of Speech, New York: Springer Verlag,
    1976.
1329 // * <https://ccrma.stanford.edu/~jos/pasp/Normalized\_Scattering\_Junctions.html>
1330 //-----
1331 declare iir_n1 author "Julius O. Smith III";
1332 declare iir_n1 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
1333 declare iir_n1 license "MIT-style STK-4.3 license";
1334 iir_n1(bv,av) = allpassnnlt(M,sv) : sum(i,M+1,*(tghr(i)))
1335 with {
1336     M = ba.count(av);
1337     sv = av2sv(av); // sv = vector of sin(theta) reflection coefficients
1338     tg = bvav2nuv(bv,av); // tg = vector of tap gains for 2mul case
1339     tgr(i) = ba.take(M+1-i,tg);
1340     tghr(n) = tgr(n)/pi(n);
1341     pi(0) = 1;
1342     s(n) = ba.take(M-n+1,sv); // reflection coefficient = sin(theta)
1343     c(n) = sqrt(max(0,1-s(n)*s(n))); // compiler crashes on sqrt(-)
1344     pi(n) = pi(n-1)*c(n);
1345 };
1346
1347 //-----`fi.)allpassnnlt`-----
1348 // Normalized ladder allpass filter of arbitrary order.
1349 //
1350 // #### Usage:
1351 //

```

```

1352 // ``
1353 // _ : allpassnlt(N,sv) : _
1354 // ``
1355 //
1356 // Where:
1357 //
1358 // * `N`: the order of the filter (fixed at compile time)
1359 // * `sv`: the reflection coefficients (-1 1)
1360 //
1361 // #### Test
1362 // ``
1363 // fi = library("filters.lib");
1364 // os = library("oscillators.lib");
1365 // si = library("signals.lib");
1366 // src = os.osc(440);
1367 // allpassnlt_test = src : fi.allpassnlt(2, (0.3, -0.2)) : si.bus(3);
1368 // ``
1369 //
1370 // #### References
1371 //
1372 // * J. D. Markel and A. H. Gray, Linear Prediction of Speech, New York: Springer Verlag,
1373 //   1976.
1374 // * <https://ccrma.stanford.edu/~jos/pasp/Normalized\_Scattering\_Junctions.html>
1375 //-----
1376 declare allpassnlt author "Julius O. Smith III";
1377 declare allpassnlt copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
1378   stanford.edu>";
1379 declare allpassnlt license "MIT-style STK-4.3 license";
1380 allpassnlt(0,sv) = _;
1381 allpassnlt(n,scl*(sv)) = allpassnlt(n,par(i,count(sv),scl*(sv(i))));
1382 allpassnlt(n,sv) = _ <: *(s),*(c) : (+
1383   : allpassnlt(n-1,sv))-(*(-s))) : fsec(n)
1384 with {
1385   fsec(1) = _, (<:mem*(c),_) : sumandtaps(n);
1386   fsec(n) = _, (<:mem*(c),_), par(i,n-1,_) : sumandtaps(n);
1387   s = ba.take(n,sv);
1388   c = sqrt(max(0,1-s*s));
1389   sumandtaps(n) = +,par(i,n,_)
1390 };
1391
1392 //=====Useful Special Cases=====
1393 //-----
1394 //-----` (fi.)tf2np`-----
1395 // Biquad based on a stable second-order Normalized Ladder Filter
1396 // (more robust to modulation than `tf2` and protected against instability).
1397 //
1398 // #### Usage
1399 //
1400 // _ : tf2np(b0,b1,b2,a1,a2) : _
1401 // ``
1402 //
1403 // Where:
1404 //
1405 // * `b`: transfer-function numerator
1406 // * `a`: transfer-function denominator (monic)
1407 //
1408 // #### Test
1409 // ``
1410 // fi = library("filters.lib");
1411 // os = library("oscillators.lib");
1412 // src = os.osc(440);
1413 // tf2np_test = src : fi.tf2np(0.6, 0.3, 0.2, -0.5, 0.2);
1414 // ``
1415 //-----
1416 declare tf2np author "Julius O. Smith III";
1417 declare tf2np copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.

```

```

1418     edu>";
1419 declare tf2np license "MIT-style STK-4.3 license";
1420
1421 tf2np(b0,b1,b2,a1,a2) = allpassnlt(M,sv) : sum(i,M+1,*(tghr(i)))
1422 with {
1423     smax = 1.0-ma.EPSILON; // maximum reflection-coefficient magnitude allowed
1424     s2 = max(-smax, min(smax,a2)); // Project both reflection-coefficients
1425     s1 = max(-smax, min(smax,a1/(1+a2))); // into the defined stability-region.
1426     sv = (s1,s2); // vector of sin(theta) reflection coefficients
1427
1428     M = 2;
1429     nu(2) = b2;
1430     nu(1) = b1 - b2*a1;
1431     nu(0) = (b0-b2*a2) - nu(1)*s1;
1432     tg = (nu(0),nu(1),nu(2));
1433     tgr(i) = ba.take(M+1-i,tg); // vector of tap gains for 2mul case
1434     tghr(n) = tgr(n)/pi(n); // apply pi parameters for NLF case
1435     pi(0) = 1;
1436     s(n) = ba.take(M-n+1,sv);
1437     c(n) = sqrt(1-s(n)*s(n));
1438     pi(n) = pi(n-1)*c(n);
1439 };
1440
1441 //-----`fi.)wgr`-----
1442 // Second-order transformer-normalized digital waveguide resonator.
1443 //
1444 // ### Usage
1445 //
1446 // ...
1447 // : wgr(f,r) : _
1448 // ...
1449 //
1450 // Where:
1451 //
1452 // * `f`: resonance frequency (Hz)
1453 // * `r`: loss factor for exponential decay (set to 1 to make a numerically stable oscillator
1454 // )
1455 //
1456 // ### Test
1457 //
1458 // ...
1459 // fi = library("filters.lib");
1460 // os = library("oscillators.lib");
1461 // src = os.osc(440);
1462 // wgr_test = fi.wgr(440, 0.995, src);
1463 // ...
1464 //
1465 // ### References
1466 //
1467 // * <https://ccrma.stanford.edu/~jos/pasp/Power_Normalized_Waveguide_Filters.html>
1468 // * <https://ccrma.stanford.edu/~jos/pasp/Digital_Waveguide_Oscillator.html>
1469 //-----
1470 declare wgr author "Julius O. Smith III";
1471 declare wgr copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
1472     edu>";
1473 declare wgr license "MIT-style STK-4.3 license";
1474 wgr(f,r,x) = (*(G),_<:_,(+:(*(C))<:_,_):+,_,-:(x),-) ~ cross : _,(0-gi)
1475 with {
1476     C = cos(2*ma.PI*f/ma.SR);
1477     gi = sqrt(max(0,(1+C)/(1-C))); // compensate amplitude (only needed when
1478     G = r*(1-1' + gi')/gi; // frequency changes substantially)
1479     cross = _,- <: !,_,-,!;
1480 };
1481
1482 //-----`fi.)nlf2`-----
1483 // Second order normalized digital waveguide resonator.
1484 //
1485 // ### Usage
1486 //
1487 // ...
1488 // ...

```

```

1483 // _ : nlf2(f,r) : _
1484 // ...
1485 //
1486 // Where:
1487 //
1488 // * `f`: resonance frequency (Hz)
1489 // * `r`: loss factor for exponential decay (set to 1 to make a sinusoidal oscillator)
1490 //
1491 // #### Test
1492 // ...
1493 // fi = library("filters.lib");
1494 // os = library("oscillators.lib");
1495 // src = os.osc(440);
1496 // nlf2_test = fi.nlf2(440, 0.995, src);
1497 // ...
1498 //
1499 // #### References
1500 //
1501 // <https://ccrma.stanford.edu/~jos/pasp/Power_Normalized_Waveguide_Filters.html>
1502 //-----
1503 declare nlf2 author "Julius O. Smith III";
1504 declare nlf2 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
1505 declare nlf2 license "MIT-style STK-4.3 license";
1506 nlf2(f,r,x) = ((_<:_),(_<:_)) : (*s),*(c),*(c),*(0-s)) :>
1507     (*r),+(x))) ~ cross
1508 with {
1509     th = 2*ma.PI*f/ma.SR;
1510     c = cos(th);
1511     s = sin(th);
1512     cross = _<:_ <: !,_,_!;
1513 };
1514
1515 //-----`fi.)apnl`-----
1516 // Passive Nonlinear Allpass based on Pierce switching springs idea.
1517 // Switch between allpass coefficient `a1` and `a2` at signal zero crossings.
1518 //
1519 // #### Usage
1520 //
1521 // ...
1522 // _ : apnl(a1,a2) : _
1523 // ...
1524 //
1525 // Where:
1526 //
1527 // * `a1`: first allpass coefficient
1528 // * `a2`: second allpass coefficient
1529 //
1530 // #### Test
1531 // ...
1532 //
1533 // fi = library("filters.lib");
1534 // os = library("oscillators.lib");
1535 // src = os.osc(440);
1536 // apnl_test = fi.apnl(0.5, -0.5, src);
1537 // ...
1538 //
1539 // #### References
1540 //
1541 // * "A Passive Nonlinear Digital Filter Design ..." by John R. Pierce and Scott
1542 //   A. Van Duyne, JASA, vol. 101, no. 2, pp. 1120-1126, 1997
1543 //-----
1544 declare apnl author "Julius O. Smith III";
1545 declare apnl copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
1546 declare apnl license "MIT-style STK-4.3 license";
1547 apnl(a1,a2,x) = nonLinFilter
1548 with {

```

```

1549     condition = _>0;
1550     nonLinFilter = (x - _ <: _*(condition*a1 + (1-condition)*a2),_')~_ :> +;
1551 };
1552
1553
1554 //=====Ladder/Lattice Allpass Filters=====
1555 // An allpass filter has gain 1 at every frequency, but variable phase.
1556 // Ladder/lattice allpass filters are specified by reflection coefficients.
1557 // They are defined here as nested allpass filters, hence the names `allpassn*`.
1558 //
1559 // ##### References
1560 //
1561 // * <https://ccrma.stanford.edu/~jos/pasp/Conventional_Ladder_Filters.html>
1562 // * <https://ccrma.stanford.edu/~jos/pasp/Nested_Allpass_Filters.html>
1563 // * Linear Prediction of Speech, Markel and Gray, Springer Verlag, 1976
1564 //=====
1565
1566 //-----`(fi).scatN`-----
1567 // N-port scattering junction.
1568 //
1569 // ##### Usage
1570 //
1571 // ...
1572 // si.bus(N) : scatN(N,av,filter) : si.bus(N)
1573 // ...
1574 //
1575 // Where:
1576 //
1577 // * `N`: number of incoming/outgoing waves
1578 // * `av`: vector (list) of `N` alpha parameters (each between 0 and 2, and normally summing
1579 //   to 2): <https://ccrma.stanford.edu/~jos/pasp/Alpha_Parameters.html>
1580 // * `filter` : optional junction filter to apply (`_` for none, see below)
1581 //
1582 // With no filter:
1583 //
1584 // - The junction is _lossless_ when the alpha parameters sum to 2 ("allpass").
1585 // - The junction is _passive_ but lossy when the alpha parameters sum to less than 2 ("
1586 //   resistive loss").
1587 // - Dynamic and reactive junctions are obtained using the `filter` argument.
1588 // For guaranteed stability, the filter should be _positive real_. (See 2nd ref. below).
1589 //
1590 // For \((N=2)\) (two-port scattering), the reflection coefficient \(\(\rho\)\) corresponds
1591 // to alpha parameters \(\(1\pm\rho\)\).
1592 //
1593 // ##### Example: Whacky echo chamber made of 16 lossless "acoustic tubes":
1594 //
1595 // ...
1596 // process = _ : *(1.0/sqrt(N)) <: daisyRev(16,2,0.9999) :> _,_ with {
1597 //   daisyRev(N,Dp2,G) = si.bus(N) : (si.bus(2*N) :> si.bus(N)
1598 //     : fi.scatN(N, par(i,N,2*G/float(N)), fi.lowpass(1,5000.0))
1599 //     : par(i,N,de.delay(DS(i),DS(i)-1))) ~ si.bus(N) with { DS(i) = 2^(Dp2+i); };
1600 // };
1601 //
1602 // ##### Test
1603 //
1604 // ...
1605 // fi = library("filters.lib");
1606 // os = library("oscillators.lib");
1607 // dual_src = os.osc(440), os.osc(660);
1608 // scatN_test = dual_src : fi.scatN(2, (1, 1), _);
1609 //
1610 // ##### References
1611 //
1612 // * <https://ccrma.stanford.edu/~jos/pasp/Loaded_Waveguide_Junctions.html>
1613 // * <https://ccrma.stanford.edu/~jos/pasp/Passive_String_Terminations.html>
1614 // * <https://ccrma.stanford.edu/~jos/pasp/Unloaded_Junctions_Alpha_Parameters.html>
1615 //-----

```

```

1615 declare scatN author "Julius O. Smith III";
1616 declare scatN copyright "Copyright (C) 2024 by Julius O. Smith III <jos@ccrma.stanford.edu
>";
1617 declare scatN license "MIT-style STK-4.3 license";
1618
1619 scatN(0,av,filter) = !;
1620 scatN(N,av,filter) = incomingWaves <: (junctionSum : filter <: si.bus(N)), par(i,N,*(-1)) :>
    si.bus(N)
1621 with {
1622     incomingWaves = si.bus(N);
1623     alpha(i) = ba.take(i+1,av);
1624     junctionSum = par(i,N,*(alpha(i))) :> _; // Junction velocity/pressure for series/parallel
        junction
1625     outgoingWaves = junctionSum <: si.bus(N), negatedIncoming :> si.bus(N);
1626     alphaSum = sum(i,N,alpha(i));
1627 };
1628
1629 //-----`fi.`scat`-----
1630 // Scatter off of reflectance r with reflection coefficient s.
1631 //
1632 // #### Usage:
1633 //
1634 // ```
1635 // _ : scat(s,r) : _
1636 // ```
1637 // Where:
1638 //
1639 // * `s`: reflection coefficient between -1 and 1 for stability
1640 // * `r`: single-input, single-output block diagram,
1641 //     having gain less than 1 at all frequencies for stability.
1642 //
1643 // #### Example: the following program should produce all zeros:
1644 //
1645 // ```
1646 // process = fi.allpassn(3,(.3,.2,.1)), fi.scat(.1, fi.scat(.2, fi.scat(.3, _)))
1647 // :> - : ^2) : +~;
1648 // ```
1649 //
1650 // #### Test
1651 //
1652 // ```
1653 // fi = library("filters.lib");
1654 // os = library("oscillators.lib");
1655 // src = os.osc(440);
1656 // scat_test = src : fi.scat(0.5, _);
1657 // ```
1658 //
1659 // #### References
1660 //
1661 // * <https://ccrma.stanford.edu/~jos/pasp/Scattering_Impedance_Changes.html>
1662 //-----
1663 declare scat author "Julius O. Smith III";
1664 declare scat copyright "Copyright (C) 2024 by Julius O. Smith III <jos@ccrma.stanford.edu>";
1665 declare scat license "MIT-style STK-4.3 license";
1666
1667 scat(s,r) = _ <: ((+ <: (r,*(s)))-(*(-s))) : _',_ :+;
1668
1669 //-----`fi.`allpassn`-----
1670 // Two-multiply lattice filter.
1671 //
1672 // #### Usage:
1673 //
1674 // ```
1675 // _ : allpassn(n,sv) : _
1676 // ```
1677 // Where:
1678 //
1679 // * `n`: the order of the filter
1680 // * `sv`: the reflection coefficients `(s1,s2,...,sN)`, each between -1 and 1

```

```

1680 //
1681 // Equivalent to `fi.allpassnt(n,sv) : _, par(i,n,!);`
1682 //
1683 // Equivalent to `fi.scats( s(n), fi.scats( s(n-1), ..., fi.scats( s(1), _ )))`
1684 // with { s(k) = ba.take(k,sv); } ;`
1685 //
1686 // Identical to `allpassn` in `old/filter.lib`.
1687 //
1688 // #### Test
1689 // ```
1690 // fi = library("filters.lib");
1691 // os = library("oscillators.lib");
1692 // src = os.osc(440);
1693 // allpassn_test = src : fi.allpassn(3, (0.3, 0.2, 0.1));
1694 // ```
1695 //
1696 // #### References
1697 //
1698 // * J. D. Markel and A. H. Gray: Linear Prediction of Speech, New York: Springer Verlag,
1699 //   1976.
1700 // <https://ccrma.stanford.edu/~jos/pasp/Conventional\_Ladder\_Filters.html>
1701 //-----
1702 declare allpassn author "Julius O. Smith III";
1703 declare allpassn copyright "Copyright (c) 2003-2024 by Julius O. Smith III <jos@ccrma.
1704   stanford.edu>";
1705 declare allpassn license "MIT-style STK-4.3 license";
1706 allpassn(0,sv) = _;
1707 allpassn(n,sv) = _ <: ((+ <: (allpassn(n-1,sv)),*(s))-*(-s)) : _,_ :+
1708 with { s = ba.take(n,sv); };
1709
1710 //-----`(fi.)allpassnn`-----
1711 // Normalized form - four multiplies and two adds per section,
1712 // but coefficients can be time varying and nonlinear without
1713 // "parametric amplification" (modulation of signal energy).
1714 //
1715 // #### Usage:
1716 //
1717 // _ : allpassnn(n,tv) : _
1718 //
1719 // Where:
1720 //
1721 // * `n`: the order of the filter
1722 // * `tv`: the reflection coefficients (-PI PI)
1723 //
1724 // #### Test
1725 // ```
1726 // fi = library("filters.lib");
1727 // os = library("oscillators.lib");
1728 // src = os.osc(440);
1729 // allpassnn_test = src : fi.allpassnn(3, (0.3, 0.2, 0.1));
1730 // ```
1731 //-----
1732 // power-normalized (reflection coefficients s = sin(t)):
1733 declare allpassnn author "Julius O. Smith III";
1734 declare allpassnn copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
1735   stanford.edu>";
1736 declare allpassnn license "MIT-style STK-4.3 license";
1737
1738 allpassnn(0,tv) = _;
1739 allpassnn(n,tv) = _ <: *(s), *(c) : (+
1740   : allpassnn(n-1,tv))-*(-s)) : _, mem*c : +
1741 with { c = cos(ba.take(n,tv)); s = sin(ba.take(n,tv)); };
1742
1743 //-----`(fi.)allpassnkl`-----
1744 // Kelly-Lochbaum form - four multiplies and two adds per
1745 // section, but all signals have an immediate physical

```

```

1745 // interpretation as traveling pressure waves, etc.
1746 //
1747 // #### Usage:
1748 //
1749 // ---
1750 // _ : allpassnkl(n,sv) : _
1751 // ---
1752 //
1753 // Where:
1754 //
1755 // * `n`: the order of the filter
1756 // * `sv`: the reflection coefficients (-1 1)
1757 //
1758 // #### Test
1759 // ---
1760 // fi = library("filters.lib");
1761 // os = library("oscillators.lib");
1762 // src = os.osc(440);
1763 // allpassnkl_test = src : fi.allpassnkl(3, (0.3, 0.2, 0.1));
1764 // ---
1765 //-----
1766 // Kelly-Lochbaum:
1767 declare allpassnkl author "Julius O. Smith III";
1768 declare allpassnkl copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
1769 declare allpassnkl license "MIT-style STK-4.3 license";
1770
1771 allpassnkl(0,sv) = _;
1772 allpassnkl(n,sv) = _ <: *(s),*(1+s) : (+
    : allpassnkl(n-1,sv))-*(-s)) : _, mem*(1-s) : +
1773 with { s = ba.take(n,sv); };
1774
1775 //-----^(fi.)allpass1m-----
1776 // One-multiply form - one multiply and three adds per section.
1777 // Normally the most efficient in special-purpose hardware.
1778 //
1779 // #### Usage:
1780 //
1781 // ---
1782 // _ : allpassn1m(n,sv) : _
1783 // ---
1784 //
1785 // Where:
1786 //
1787 // * `n`: the order of the filter
1788 // * `sv`: the reflection coefficients (-1 1)
1789 //
1790 // #### Test
1791 // ---
1792 // fi = library("filters.lib");
1793 // os = library("oscillators.lib");
1794 // src = os.osc(440);
1795 // allpassn1m_test = src : fi.allpassn1m(3, (0.3, 0.2, 0.1));
1796 // ---
1797 //-----
1798 // one-multiply:
1799 declare allpassn1m author "Julius O. Smith III";
1800 declare allpassn1m copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
1801 declare allpassn1m license "MIT-style STK-4.3 license";
1802
1803 allpassn1m(0,sv) = _;
1804 allpassn1m(n,sv) = _ <: _,_ : ((+:(s) <: _,_) : _,+ : cross
    : allpassn1m(n-1,sv),_)-*(-1)) : _,_ : +
1805 with { s = ba.take(n,sv); cross = _,_ <: !,_,_,!; };
1806
1807 //=====Digital Filter Sections Specified as Analog Filter Sections=====
1808 //=====
1809
1810

```

```

1811 //-----`(fi.)tf2s` and `(fi.)tf2snp`-----
1812 // Second-order direct-form digital filter,
1813 // specified by ANALOG transfer-function polynomials B(s)/A(s),
1814 // and a frequency-scaling parameter. Digitization via the
1815 // bilinear transform is built in.
1816 //
1817 //
1818 // #### Usage
1819 //
1820 // ```
1821 // _ : tf2s(b2,b1,b0,a1,a0,w1) : _
1822 // ```
1823 // Where:
1824 //
1825 // ```
1826 //      b2 s^2 + b1 s + b0
1827 // H(s) = -----
1828 //      s^2 + a1 s + a0
1829 // ```
1830 //
1831 // and `w1` is the desired digital frequency (in radians/second)
1832 // corresponding to analog frequency 1 rad/sec (i.e., `s = j`).
1833 //
1834 // #### Example test program
1835 //
1836 // A second-order ANALOG Butterworth lowpass filter,
1837 // normalized to have cutoff frequency at 1 rad/sec,
1838 // has transfer function:
1839 //
1840 // ```
1841 //      1
1842 // H(s) = -----
1843 //      s^2 + a1 s + 1
1844 // ```
1845 //
1846 // where `a1 = sqrt(2)`. Therefore, a DIGITAL Butterworth lowpass
1847 // cutting off at `SR/4` is specified as `tf2s(0,0,1,sqrt(2),1,PI*SR/2);`
1848 //
1849 // #### Method
1850 //
1851 // Bilinear transform scaled for exact mapping of w1.
1852 //
1853 // #### Test
1854 //
1855 // fi = library("filters.lib");
1856 // os = library("oscillators.lib");
1857 // ma = library("maths.lib");
1858 // src = os.osc(440);
1859 // tf2s_test = src : fi.tf2s(0, 0, 1, sqrt(2), 1, ma.PI*ma.SR/2);
1860 //
1861 //
1862 // #### References
1863 //
1864 // * <https://ccrma.stanford.edu/~jos/pasp/Bilinear\_Transformation.html>
1865 //-----
1866 declare tf2s author "Julius O. Smith III";
1867 declare tf2s copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
1868 declare tf2s license "MIT-style STK-4.3 license";
1869 tf2s(b2,b1,b0,a1,a0,w1) = tf2(b0d,b1d,b2d,a1d,a2d)
1870 with {
1871   c = 1/tan(w1*0.5/ma.SR); // bilinear-transform scale-factor
1872   csq = c*c;
1873   d = a0 + a1 * c + csq;
1874   b0d = (b0 + b1 * c + b2 * csq)/d;
1875   b1d = 2 * (b0 - b2 * csq)/d;
1876   b2d = (b0 - b1 * c + b2 * csq)/d;
1877   a1d = 2 * (a0 - csq)/d;

```

```

1878     a2d = (a0 - a1*c + csq)/d;
1879 };
1880
1881 // tf2snp = tf2s but using a protected normalized ladder filter for tf2:
1882 tf2snp(b2,b1,b0,a1,a0,w1) = tf2np(b0d,b1d,b2d,a1d,a2d)
1883 with {
1884     c = 1/tan(w1*0.5/ma.SR); // bilinear-transform scale-factor
1885     csq = c*c;
1886     d = a0 + a1 * c + csq;
1887     b0d = (b0 + b1 * c + b2 * csq)/d;
1888     b1d = 2 * (b0 - b2 * csq)/d;
1889     b2d = (b0 - b1 * c + b2 * csq)/d;
1890     a1d = 2 * (a0 - csq)/d;
1891     a2d = (a0 - a1*c + csq)/d;
1892 };
1893
1894 //-----
1895 // #### Test
1896 // ```
1897 // fi = library("filters.lib");
1898 // os = library("oscillators.lib");
1899 // ma = library("maths.lib");
1900 // src = os.osc(440);
1901 // tf2snp_test = src : fi.tf2snp(0, 0, 1, sqrt(2), 1, ma.PI*ma.SR/2);
1902 // ```
1903 //-----
1904
1905 //-----`fi.)tf1snp`-----
1906 // First-order special case of tf2snp above.
1907 //
1908 // #### Usage
1909 //
1910 // ```
1911 // _ : tf1snp(b1,b0,a0,w1) : _
1912 // ```
1913 //
1914 // Where:
1915 //
1916 // * `b1`: analog numerator coefficient for `s`
1917 // * `b0`: analog numerator constant coefficient
1918 // * `a0`: analog denominator constant coefficient
1919 // * `w1`: desired digital frequency (radians/second) corresponding to analog frequency `1
1920 //         rad/sec`
1921 //
1922 // #### Test
1923 // ```
1924 // fi = library("filters.lib");
1925 // os = library("oscillators.lib");
1926 // ma = library("maths.lib");
1927 // src = os.osc(440);
1928 // tf1snp_test = src : fi.tf1snp(0, 1, 1, ma.PI*ma.SR/2);
1929 // ```
1930 //-----
1931 declare tf1snp author "Julius O. Smith III";
1932 declare tf1snp copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford
1933 .edu>";
1934
1935 declare tf1snp license "MIT-style STK-4.3 license";
1936 tf1snp(b1,b0,a0,w1) = fi.tf2snp(b1,b0,0,a0,0,w1); // FIXME: Faust compiler does not fully
1937 // optimize - does C++?
1938
1939 //-----`fi.)tf3snp`-----
1940 // Analogous to `tf2s` above, but third order, and using the typical
1941 // low-frequency-matching bilinear-transform constant 2/T ("1f" series)
1942 // instead of the specific-frequency-matching value used in `tf2s` and `tf1s`.
1943 // Note the lack of a "w1" argument.
1944 //
1945 // #### Usage

```

```

1943 //
1944 // ```
1945 // _ : tf3slf(b3,b2,b1,b0,a3,a2,a1,a0) : _
1946 // ```
1947 //
1948 // Where:
1949 //
1950 // * `b3`: analog numerator coefficient of `s^3`
1951 // * `b2`: analog numerator coefficient of `s^2`
1952 // * `b1`: analog numerator coefficient of `s`
1953 // * `b0`: analog numerator constant term
1954 // * `a3`: analog denominator coefficient of `s^3`
1955 // * `a2`: analog denominator coefficient of `s^2`
1956 // * `a1`: analog denominator coefficient of `s`
1957 // * `a0`: analog denominator constant term
1958 //
1959 // #### Test
1960 // ```
1961 // fi = library("filters.lib");
1962 // os = library("oscillators.lib");
1963 // src = os.osc(440);
1964 // tf3slf_test = src : fi.tf3slf(0, 0, 0, 1, 1, 2, 2, 1);
1965 // ```
1966 //-----
1967 declare tf3slf author "Julius O. Smith III";
1968 declare tf3slf copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
1969 declare tf3slf license "MIT-style STK-4.3 license";
1970 tf3slf(b3,b2,b1,b0,a3,a2,a1,a0) = tf3(b0d,b1d,b2d,b3d,a1d,a2d,a3d) with {
1971   c = 2.0 * ma.SR; // bilinear-transform scale-factor ("lf" case)
1972   csq = c*c;
1973   cc = csq*c;
1974   // Thank you maxima:
1975   b3d = (b3*c^3-b2*c^2+b1*c-b0)/d;
1976   b2d = (-3*b3*c^3+b2*c^2+b1*c-3*b0)/d;
1977   b1d = (3*b3*c^3+b2*c^2-b1*c-3*b0)/d;
1978   b0d = (-b3*c^3-b2*c^2-b1*c-b0)/d;
1979   a3d = (a3*c^3-a2*c^2+a1*c-a0)/d;
1980   a2d = (-3*a3*c^3+a2*c^2+a1*c-3*a0)/d;
1981   a1d = (3*a3*c^3+a2*c^2-a1*c-3*a0)/d;
1982   d = (-a3*c^3-a2*c^2-a1*c-a0);
1983 };
1984
1985 //-----`(fi.)tf1s`-----
1986 // First-order direct-form digital filter,
1987 // specified by ANALOG transfer-function polynomials B(s)/A(s),
1988 // and a frequency-scaling parameter.
1989 //
1990 // #### Usage
1991 //
1992 // ```
1993 // _ : tf1s(b1,b0,a0,w1) : _
1994 // ```
1995 //
1996 // Where:
1997 //
1998 // * `b1`: analog numerator coefficient of `s`
1999 // * `b0`: analog numerator constant term
2000 // * `a0`: analog denominator constant term in `s + a0`
2001 // * `w1`: desired digital frequency in radians per second corresponding to analog frequency
2002 //       `1 rad/s`
2003 //
2004 // Where:
2005 //
2006 // 
$$H(s) = \frac{b1 s + b0}{s + a0}$$

2007 //
2008 //

```

```

2009 // and `w1` is the desired digital frequency (in radians/second)
2010 // corresponding to analog frequency 1 rad/sec (i.e., `s = j`).
2011 //
2012 // #### Example test program
2013 //
2014 // A first-order ANALOG Butterworth lowpass filter,
2015 // normalized to have cutoff frequency at 1 rad/sec,
2016 // has transfer function:
2017 //
2018 //      1
2019 // H(s) = -----
2020 //      s + 1
2021 //
2022 // so `b0 = a0 = 1` and `b1 = 0`. Therefore, a DIGITAL first-order
2023 // Butterworth lowpass with gain -3dB at `SR/4` is specified as
2024 //
2025 // ```
2026 // tf1s(0,1,1,PI*SR/2); // digital half-band order 1 Butterworth
2027 // ```
2028 //
2029 // #### Method
2030 //
2031 // Bilinear transform scaled for exact mapping of w1.
2032 //
2033 // #### Test
2034 // ```
2035 // fi = library("filters.lib");
2036 // os = library("oscillators.lib");
2037 // ma = library("maths.lib");
2038 // src = os.osc(440);
2039 // tf1s_test = src : fi.tf1s(0, 1, 1, ma.PI*ma.SR/2);
2040 // ```
2041 //
2042 // #### References
2043 //
2044 // * <https://ccrma.stanford.edu/~jos/pasp/Bilinear\_Transformation.html>
2045 //-----
2046 declare tf1s author "Julius O. Smith III";
2047 declare tf1s copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.edu>";
2048 declare tf1s license "MIT-style STK-4.3 license";
2049 tf1s(b1,b0,a0,w1) = tf1(b0d,b1d,a1d)
2050 with {
2051   c = 1/tan(w1*0.5/ma.SR); // bilinear-transform scale-factor
2052   d = a0 + c;
2053   b1d = (b0 - b1*c) / d;
2054   b0d = (b0 + b1*c) / d;
2055   a1d = (a0 - c) / d;
2056 };
2057
2058 //-----`(fi.)tf2sb`-----
2059 // Bandpass mapping of `tf2s`: In addition to a frequency-scaling parameter
2060 // `w1` (set to HALF the desired passband width in rad/sec),
2061 // there is a desired center-frequency parameter wc (also in rad/s).
2062 // Thus, `tf2sb` implements a fourth-order digital bandpass filter section
2063 // specified by the coefficients of a second-order analog lowpass prototype
2064 // section. Such sections can be combined in series for higher orders.
2065 // The order of mappings is (1) frequency scaling (to set lowpass cutoff w1),
2066 // (2) bandpass mapping to wc, then (3) the bilinear transform, with the
2067 // usual scale parameter `2*SR`. Algebra carried out in maxima and pasted here.
2068 //
2069 // #### Usage
2070 //
2071 // ```
2072 // _ : tf2sb(b2,b1,b0,a1,a0,w1,wc) : _
2073 // ```
2074 //
2075 // Where:

```

```

2076 //
2077 // * `b2`, `b1`, `b0`: analog lowpass numerator coefficients
2078 // * `a1`, `a0`: analog lowpass denominator coefficients
2079 // * `w1`: half the desired passband width in radians/second
2080 // * `wc`: desired center frequency in radians/second
2081 //
2082 // #### Test
2083 // ---
2084 // fi = library("filters.lib");
2085 // os = library("oscillators.lib");
2086 // ma = library("maths.lib");
2087 // src = os.osc(440);
2088 // tf2sb_test = src : fi.tf2sb(0, 0, 1, sqrt(2), 1, 2*ma.PI*200, 2*ma.PI*1000);
2089 // ---
2090 //-----
2091 declare tf2sb author "Julius O. Smith III";
2092 declare tf2sb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
2093 declare tf2sb license "MIT-style STK-4.3 license";
2094 tf2sb(b2,b1,b0,a1,a0,w1,wc) =
2095   iir((b0d/a0d,b1d/a0d,b2d/a0d,b3d/a0d,b4d/a0d),(a1d/a0d,a2d/a0d,a3d/a0d,a4d/a0d)) with {
2096     T = 1.0/float(ma.SR);
2097     b0d = (4*b0*w1^2+8*b2*wc^2)*T^2+8*b1*w1*T+16*b2;
2098     b1d = 4*b2*wc^4*T^4+4*b1*wc^2*w1*T^3-16*b1*w1*T-64*b2;
2099     b2d = 6*b2*wc^4*T^4+(-8*b0*w1^2-16*b2*wc^2)*T^2+96*b2;
2100     b3d = 4*b2*wc^4*T^4-4*b1*wc^2*w1*T^3+16*b1*w1*T-64*b2;
2101     b4d = (b2*wc^4*T^4-2*b1*wc^2*w1*T^3+(4*b0*w1^2+8*b2*wc^2)*T^2-8*b1*w1*T+16*b2)
2102           + b2*wc^4*T^4+2*b1*wc^2*w1*T^3;
2103     a0d = wc^4*T^4+2*a1*wc^2*w1*T^3+(4*a0*w1^2+8*wc^2)*T^2+8*a1*w1*T+16;
2104     a1d = 4*wc^4*T^4+4*a1*wc^2*w1*T^3-16*a1*w1*T-64;
2105     a2d = 6*wc^4*T^4+(-8*a0*w1^2-16*wc^2)*T^2+96;
2106     a3d = 4*wc^4*T^4-4*a1*wc^2*w1*T^3+16*a1*w1*T-64;
2107     a4d = wc^4*T^4-2*a1*wc^2*w1*T^3+(4*a0*w1^2+8*wc^2)*T^2-8*a1*w1*T+16;
2108   };
2109
2110 //-----`fi.tf1sb`-----
2111 // First-to-second-order lowpass-to-bandpass section mapping,
2112 // analogous to tf2sb above.
2113 //
2114 // #### Usage
2115 // ---
2116 // _ : tf1sb(b1,b0,a0,w1,wc) : _
2117 // ---
2118 //
2119 // Where:
2120 //
2121 // * `b1`, `b0`: analog numerator coefficients
2122 // * `a0`: analog denominator constant coefficient
2123 // * `w1`: half the desired passband width in radians/second
2124 // * `wc`: desired center frequency in radians/second
2125 //
2126 // #### Test
2127 // ---
2128 //
2129 // fi = library("filters.lib");
2130 // os = library("oscillators.lib");
2131 // ma = library("maths.lib");
2132 // src = os.osc(440);
2133 // tf1sb_test = src : fi.tf1sb(0, 1, 1, 2*ma.PI*200, 2*ma.PI*1000);
2134 // ---
2135 //-----
2136 declare tf1sb author "Julius O. Smith III";
2137 declare tf1sb copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.stanford.
    edu>";
2138 declare tf1sb license "MIT-style STK-4.3 license";
2139 tf1sb(b1,b0,a0,w1,wc) = tf2(b0d/a0d,b1d/a0d,b2d/a0d,a1d/a0d,a2d/a0d) with {
2140   T = 1.0/float(ma.SR);
2141   a0d = wc^2*T^2+2*a0*w1*T+4;

```

```

2142 b0d = b1*wc^2*T^2 + 2*b0*w1*T+4*b1;
2143 b1d = 2*b1*wc^2*T^2-8*b1;
2144 b2d = b1*wc^2*T^2-2*b0*w1*T+4*b1;
2145 a1d = 2*wc^2*T^2-8;
2146 a2d = wc^2*T^2-2*a0*w1*T+4;
2147 };
2148
2149
2150 //=====Simple Resonator Filters=====
2151 //=====
2152
2153 //-----`(fi.)resonlp`-----
2154 // Simple resonant lowpass filter based on `tf2s` (virtual analog).
2155 // `resonlp` is a standard Faust function.
2156 //
2157 // #### Usage
2158 //
2159 // ```
2160 // _ : resonlp(fc,Q,gain) : _
2161 // _ : resonhp(fc,Q,gain) : _
2162 // _ : resonbp(fc,Q,gain) : _
2163 //
2164 // ```
2165 //
2166 // Where:
2167 //
2168 // * `fc`: center frequency (Hz)
2169 // * `Q`: q
2170 // * `gain`: gain (0-1)
2171 //
2172 // #### Test
2173 // ```
2174 // fi = library("filters.lib");
2175 // os = library("oscillators.lib");
2176 // src = os.osc(440);
2177 // resonlp_test = src : fi.resonlp(1000, 2, 0.8);
2178 // ```
2179 //-----
2180 // resonlp = 2nd-order lowpass with corner resonance:
2181 declare resonlp author "Julius O. Smith III";
2182 declare resonlp copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
stanford.edu>";
2183 declare resonlp license "MIT-style STK-4.3 license";
2184
2185 resonlp(fc,Q,gain) = tf2s(b2,b1,b0,a1,a0,wc)
2186 with {
2187   wc = 2*ma.PI*fc;
2188   a1 = 1/Q;
2189   a0 = 1;
2190   b2 = 0;
2191   b1 = 0;
2192   b0 = gain;
2193 };
2194
2195
2196 //-----`(fi.)resonhp`-----
2197 // Simple resonant highpass filters based on `tf2s` (virtual analog).
2198 // `resonhp` is a standard Faust function.
2199 //
2200 // #### Usage
2201 //
2202 // ```
2203 // _ : resonlp(fc,Q,gain) : _
2204 // _ : resonhp(fc,Q,gain) : _
2205 // _ : resonbp(fc,Q,gain) : _
2206 //
2207 // ```
2208 //

```

```

2209 // Where:
2210 //
2211 // * `fc`: center frequency (Hz)
2212 // * `Q`: q
2213 // * `gain`: gain (0-1)
2214 //
2215 // #### Test
2216 // ```
2217 // fi = library("filters.lib");
2218 // os = library("oscillators.lib");
2219 // src = os.osc(440);
2220 // resonhp_test = fi.resonhp(1000, 2, 0.8, src);
2221 // ```
2222 //-----
2223 // resonhp = 2nd-order highpass with corner resonance:
2224 declare resonhp author "Julius O. Smith III";
2225 declare resonhp copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2226 declare resonhp license "MIT-style STK-4.3 license";
2227
2228 resonhp(fc,Q,gain,x) = gain*x-resonlp(fc,Q,gain,x);
2229
2230
2231 //-----`(fi.)resonbp`-----
2232 // Simple resonant bandpass filters based on `tf2s` (virtual analog).
2233 // `resonbp` is a standard Faust function.
2234 //
2235 // #### Usage
2236 //
2237 // ```
2238 // _ : resonlp(fc,Q,gain) : _
2239 // _ : resonhp(fc,Q,gain) : _
2240 // _ : resonbp(fc,Q,gain) : _
2241 //
2242 // ```
2243 //
2244 // Where:
2245 //
2246 // * `fc`: center frequency (Hz)
2247 // * `Q`: q
2248 // * `gain`: gain (0-1)
2249 //
2250 // #### Test
2251 // ```
2252 // fi = library("filters.lib");
2253 // os = library("oscillators.lib");
2254 // src = os.osc(440);
2255 // resonbp_test = src : fi.resonbp(1000, 2, 0.8);
2256 // ```
2257 //-----
2258 // resonbp = 2nd-order bandpass
2259 declare resonbp author "Julius O. Smith III";
2260 declare resonbp copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2261 declare resonbp license "MIT-style STK-4.3 license";
2262
2263 resonbp(fc,Q,gain) = tf2s(b2,b1,b0,a1,a0,wc)
2264 with {
2265     wc = 2*ma.PI*fc;
2266     a1 = 1/Q;
2267     a0 = 1;
2268     b2 = 0;
2269     b1 = gain;
2270     b0 = 0;
2271 };
2272
2273
2274 //=====Butterworth Lowpass/Highpass Filters=====

```

```

2275 //=====
2276
2277 //-----`(fi.)lowpass`-----
2278 // Nth-order Butterworth lowpass filter.
2279 // `lowpass` is a standard Faust function.
2280 //
2281 // #### Usage
2282 //
2283 // ```
2284 // _ : lowpass(N,fc) : _
2285 // ```
2286 //
2287 // Where:
2288 //
2289 // * `N`: filter order (number of poles), nonnegative constant numerical expression
2290 // * `fc`: desired cut-off frequency (-3dB frequency) in Hz
2291 //
2292 // #### Test
2293 // ```
2294 // fi = library("filters.lib");
2295 // os = library("oscillators.lib");
2296 // src = os.osc(440);
2297 // lowpass_test = src : fi.lowpass(4, 2000);
2298 // ```
2299 //
2300 // #### References
2301 //
2302 // * <https://ccrma.stanford.edu/~jos/filters/Butterworth\_Lowpass\_Design.html>
2303 // * `butter` function in Octave `("[z,p,g] = butter(N,1,'s');")`
2304 //-----
2305 declare lowpass author "Julius O. Smith III";
2306 declare lowpass copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2307 declare lowpass license "MIT-style STK-4.3 license";
2308
2309 lowpass(N,fc) = lowpass0_highpass1(0,N,fc);
2310
2311
2312 //-----`(fi.)highpass`-----
2313 // Nth-order Butterworth highpass filter.
2314 // `highpass` is a standard Faust function.
2315 //
2316 // #### Usage
2317 //
2318 // ```
2319 // _ : highpass(N,fc) : _
2320 // ```
2321 //
2322 // Where:
2323 //
2324 // * `N`: filter order (number of poles), nonnegative constant numerical expression
2325 // * `fc`: desired cut-off frequency (-3dB frequency) in Hz
2326 //
2327 // #### Test
2328 // ```
2329 // fi = library("filters.lib");
2330 // os = library("oscillators.lib");
2331 // src = os.osc(440);
2332 // highpass_test = src : fi.highpass(4, 500);
2333 // ```
2334 //
2335 // #### References
2336 //
2337 // * <https://ccrma.stanford.edu/~jos/filters/Butterworth\_Lowpass\_Design.html>
2338 // * `butter` function in Octave `("[z,p,g] = butter(N,1,'s');")`
2339 //-----
2340 declare highpass author "Julius O. Smith III";
2341 declare highpass copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.

```

```

2342 stanford.edu>";
2343 declare higpass license "MIT-style STK-4.3 license";
2344
2345 higpass(N,fc) = lowpass0_highpass1(1,N,fc);
2346
2347 //-----`(fi.)lowpass0_highpass1`-----
2348 //
2349 // #### Test
2350 // ```
2351 // fi = library("filters.lib");
2352 // os = library("oscillators.lib");
2353 // src = os.osc(440);
2354 // lowpass0_highpass1_test = src : fi.lowpass0_highpass1(0, 2, 1000);
2355 // ```
2356 //-----
2357 declare lowpass0_highpass1 author "Julius O. Smith III";
2358 declare lowpass0_highpass1 "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
2359 stanford.edu>";
2360 declare lowpass0_highpass1 "MIT-style STK-4.3 license";
2361
2362 lowpass0_highpass1(s,N,fc) = lphpr(s,N,N,fc)
2363 with {
2364   lphpr(s,0,N,fc) = _;
2365   lphpr(s,1,N,fc) = tf1s(s,1-s,1,2*ma.PI*fc);
2366   lphpr(s,0,N,fc) = lphpr(s,(0-2),N,fc) : tf2s(s,0,1-s,ais,1,w1) with {
2367     parity = N % 2;
2368     S = (0-parity)/2; // current section number
2369     ais = -2*cos((ma.PI)*-1 + (1-parity)*ma.PI/(2*N) + (S-1+parity)*ma.PI/N);
2370     w1 = 2*ma.PI*fc;
2371   };
2372 };
2373
2374 //=====Special Filter-Bank Delay-Equalizing Allpass Filters=====
2375 // These special allpass filters are needed by filterbank et al. below.
2376 // They are equivalent to `(lowpass(N,fc)` +|- `highpass(N,fc))/2`, but with
2377 // canceling pole-zero pairs removed (which occurs for odd N).
2378 //=====
2379
2380 //-----`(fi.)highpass_plus_lowpass`-----
2381 //
2382 // #### Test
2383 // ```
2384 // fi = library("filters.lib");
2385 // os = library("oscillators.lib");
2386 // highpass_plus_lowpass_test = os.osc(440) : fi.highpass_plus_lowpass(3, 1000);
2387 // ```
2388 //-----
2389 declare highpass_plus_lowpass author "Julius O. Smith III";
2390 declare highpass_plus_lowpass copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
2391 jos@ccrma.stanford.edu>";
2392 declare highpass_plus_lowpass license "MIT-style STK-4.3 license";
2393
2394 highpass_plus_lowpass(1,fc) = _;
2395 highpass_plus_lowpass(3,fc) = tf2s(1,-1,1,1,w1) with { w1 = 2*ma.PI*fc; };
2396 highpass_plus_lowpass(5,fc) = tf2s(1,-a11,1,a11,1,w1)
2397 with {
2398   a11 = 1.618033988749895;
2399   w1 = 2*ma.PI*fc;
2400 };
2401
2402 // Catch-all definitions for generality - even order is done:
2403 highpass_plus_lowpass(N,fc) = _ <: switch_odd_even(N%2,N,fc) with {
2404   switch_odd_even(0,N,fc) = highpass_plus_lowpass_even(N,fc);
2405   switch_odd_even(1,N,fc) = highpass_plus_lowpass_odd(N,fc);
2406 };

```

```

2407 //-----`(fi.)highpass_minus_lowpass`-----
2408 //
2409 // #### Test
2410 // ...
2411 // fi = library("filters.lib");
2412 // os = library("oscillators.lib");
2413 // highpass_minus_lowpass_test = os.osc(440) : fi.highpass_minus_lowpass(3, 1000);
2414 // ...
2415 //-----
2416 declare highpass_minus_lowpass author "Julius O. Smith III";
2417 declare highpass_minus_lowpass copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
      jos@ccrma.stanford.edu>";
2418 declare highpass_minus_lowpass license "MIT-style STK-4.3 license";
2419 highpass_minus_lowpass(3,fc) = tf1s(-1,1,1,w1) with { w1 = 2*ma.PI*fc; };
2420 highpass_minus_lowpass(5,fc) = tf1s(1,-1,1,w1) : tf2s(1,-a12,1,a12,1,w1)
2421 with {
2422   a12 = 0.618033988749895;
2423   w1 = 2*ma.PI*fc;
2424 };
2425
2426 // Catch-all definitions for generality - even order is done:
2427 highpass_minus_lowpass(N,fc) = _ <: switch_odd_even(N%2,N,fc) with {
2428   switch_odd_even(0,N,fc) = highpass_minus_lowpass_even(N,fc);
2429   switch_odd_even(1,N,fc) = highpass_minus_lowpass_odd(N,fc);
2430 };
2431
2432 //-----`(fi.)highpass_minus_lowpass_even`-----
2433 //
2434 // #### Test
2435 // ...
2436 // fi = library("filters.lib");
2437 // os = library("oscillators.lib");
2438 // highpass_plus_lowpass_even_test = os.osc(440), os.osc(440) : fi.highpass_plus_lowpass_even
      (4, 1000);
2439 // ...
2440 //-----
2441 declare highpass_plus_lowpass_even author "Julius O. Smith III";
2442 declare highpass_plus_lowpass_even copyright "Copyright (C) 2003-2019 by Julius O. Smith III
      <jos@ccrma.stanford.edu>";
2443 declare highpass_plus_lowpass_even license "MIT-style STK-4.3 license";
2444 highpass_plus_lowpass_even(N,fc) = highpass(N,fc) + lowpass(N,fc);
2445
2446 //-----`(fi.)highpass_plus_lowpass_even`-----
2447 //
2448 // #### Test
2449 // ...
2450 // fi = library("filters.lib");
2451 // os = library("oscillators.lib");
2452 // highpass_minus_lowpass_even_test = os.osc(440), os.osc(440) : fi.
      highpass_minus_lowpass_even(4, 1000);
2453 // ...
2454 //-----
2455 declare highpass_minus_lowpass_even author "Julius O. Smith III";
2456 declare highpass_minus_lowpass_even copyright "Copyright (C) 2003-2019 by Julius O. Smith
      III <jos@ccrma.stanford.edu>";
2457 declare highpass_minus_lowpass_even license "MIT-style STK-4.3 license";
2458 highpass_minus_lowpass_even(N,fc) = highpass(N,fc) - lowpass(N,fc);
2459
2460
2461 //-----`(fi.)highpass_plus_lowpass_odd`-----
2462 // Sum of the odd-order highpass and lowpass responses.
2463 //
2464 // #### Usage
2465 //
2466 // ...
2467 // _ : highpass_plus_lowpass_odd(N,fc) : _
2468 // ...
2469 //

```

```

2470 // Where:
2471 //
2472 // * `N`: odd Butterworth order
2473 // * `fc`: cutoff frequency in Hz
2474 //
2475 // #### Test
2476 // ```
2477 // fi = library("filters.lib");
2478 // os = library("oscillators.lib");
2479 // highpass_plus_lowpass_odd_test = os.osc(440), os.osc(440) : fi.highpass_plus_lowpass_odd
2480 // (3, 1000);
2481 // ```
2482 // FIXME: Rewrite the following, as for orders 3 and 5 above,
2483 // to eliminate pole-zero cancellations:
2484 //-----
2485 declare highpass_plus_lowpass_odd author "Julius O. Smith III";
2486 declare highpass_plus_lowpass_odd copyright "Copyright (C) 2003-2019 by Julius O. Smith III
2487 <jos@ccrma.stanford.edu>";
2488 declare highpass_plus_lowpass_odd license "MIT-style STK-4.3 license";
2489 highpass_plus_lowpass_odd(N,fc) = highpass(N,fc) + lowpass(N,fc);
2490
2491 //-----`(fi.)highpass_minus_lowpass_odd`-----
2492 // Difference between the odd-order highpass and lowpass responses.
2493 //
2494 // #### Usage
2495 //
2496 // ```
2497 // _ : highpass_minus_lowpass_odd(N,fc) : _
2498 // ```
2499 //
2500 // Where:
2501 //
2502 // * `N`: odd Butterworth order
2503 // * `fc`: cutoff frequency in Hz
2504 //
2505 // #### Test
2506 // ```
2507 // fi = library("filters.lib");
2508 // os = library("oscillators.lib");
2509 // highpass_minus_lowpass_odd_test = os.osc(440), os.osc(440) : fi.highpass_minus_lowpass_odd
2510 // (3, 1000);
2511 // ```
2512 // FIXME: Rewrite the following, as for orders 3 and 5 above,
2513 // to eliminate pole-zero cancellations/
2514 //-----
2515 highpass_minus_lowpass_odd(N,fc) = highpass(N,fc) - lowpass(N,fc);
2516 declare highpass_minus_lowpass_odd author "Julius O. Smith III";
2517 declare highpass_minus_lowpass_odd copyright "Copyright (C) 2003-2019 by Julius O. Smith III
2518 <jos@ccrma.stanford.edu>";
2519 declare highpass_plus_lowpass_odd license "MIT-style STK-4.3 license";
2520
2521 //=====Elliptic (Cauer) Lowpass Filters=====
2522 // Elliptic (Cauer) Lowpass Filters
2523 //
2524 // #### References
2525 //
2526 // * <http://en.wikipedia.org/wiki/Elliptic\_filter>
2527 // * functions `ncauer` and `ellip` in Octave.
2528 //=====
2529 //-----`(fi.)lowpass3e`-----
2530 // Third-order Elliptic (Cauer) lowpass filter.
2531 //
2532 // #### Usage
2533 //
2534 // ```
2535 // _ : lowpass3e(fc) : _

```

```

2534 // ``
2535 //
2536 // Where:
2537 //
2538 // * `fc`: -3dB frequency in Hz
2539 //
2540 // #### Test
2541 // ``
2542 // fi = library("filters.lib");
2543 // os = library("oscillators.lib");
2544 // src = os.osc(440);
2545 // lowpass3e_test = src : fi.lowpass3e(1000);
2546 // ``
2547 //
2548 // #### Design
2549 //
2550 // For spectral band-slice level display (see `octave_analyzer3e`):
2551 //
2552 // ``
2553 // [z,p,g] = ncauer(Rp,Rs,3); % analog zeros, poles, and gain, where
2554 // Rp = 60 % dB ripple in stopband
2555 // Rs = 0.2 % dB ripple in passband
2556 // ``
2557 //-----
2558 declare lowpass3e author "Julius O. Smith III";
2559 declare lowpass3e copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2560 declare lowpass3e license "MIT-style STK-4.3 license";
2561
2562 lowpass3e(fc) = tf2s(b21,b11,b01,a11,a01,w1) : tf1s(0,1,a02,w1)
2563 with {
2564   a11 = 0.802636764161030; // format long; poly(p(1:2)) % in octave
2565   a01 = 1.412270893774204;
2566   a02 = 0.822445908998816; // poly(p(3)) % in octave
2567   b21 = 0.019809144837789; // poly(z)
2568   b11 = 0;
2569   b01 = 1.161516418982696;
2570   w1 = 2*ma.PI*fc;
2571 };
2572
2573 //-----`fi().lowpass6e`-----
2574 // Sixth-order Elliptic/Cauer lowpass filter.
2575 //
2576 // #### Usage
2577 //
2578 // ``
2579 // _ : lowpass6e(fc) : _
2580 // ``
2581 //
2582 // Where:
2583 //
2584 // * `fc`: -3dB frequency in Hz
2585 //
2586 // #### Test
2587 // ``
2588 // fi = library("filters.lib");
2589 // os = library("oscillators.lib");
2590 // src = os.osc(440);
2591 // lowpass6e_test = src : fi.lowpass6e(1000);
2592 // ``
2593 //
2594 // #### Design
2595 //
2596 // For spectral band-slice level display (see octave_analyzer6e):
2597 //
2598 // ``
2599 // [z,p,g] = ncauer(Rp,Rs,6); % analog zeros, poles, and gain, where
2600 // Rp = 80 % dB ripple in stopband

```

```

2601 // Rs = 0.2 % dB ripple in passband
2602 // ...
2603 //-----
2604 declare lowpass6e author "Julius O. Smith III";
2605 declare lowpass6e copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2606 declare lowpass6e license "MIT-style STK-4.3 license";
2607
2608 lowpass6e(fc) =
2609     tf2s(b21,b11,b01,a11,a01,w1) :
2610     tf2s(b22,b12,b02,a12,a02,w1) :
2611     tf2s(b23,b13,b03,a13,a03,w1)
2612 with {
2613     b21 = 0.000099999997055;
2614     a21 = 1;
2615     b11 = 0;
2616     a11 = 0.782413046821645;
2617     b01 = 0.000433227200555;
2618     a01 = 0.245291508706160;
2619     b22 = 1;
2620     a22 = 1;
2621     b12 = 0;
2622     a12 = 0.512478641889141;
2623     b02 = 7.621731298870603;
2624     a02 = 0.689621364484675;
2625     b23 = 1;
2626     a23 = 1;
2627     b13 = 0;
2628     a13 = 0.168404871113589;
2629     b03 = 53.536152954556727;
2630     a03 = 1.069358407707312;
2631     w1 = 2*ma.PI*fc;
2632 };
2633
2634
2635 //=====Elliptic Highpass Filters=====
2636 //=====
2637
2638 //-----`(fi.)highpass3e`-----
2639 // Third-order Elliptic (Cauer) highpass filter. Inversion of `lowpass3e` wrt unit
2640 // circle in s plane (s <- 1/s).
2641 //
2642 // #### Usage
2643 //
2644 // ...
2645 // _ : highpass3e(fc) : _
2646 // ...
2647 //
2648 // Where:
2649 //
2650 // * `fc`: -3dB frequency in Hz
2651 //
2652 // #### Test
2653 // ...
2654 // fi = library("filters.lib");
2655 // os = library("oscillators.lib");
2656 // src = os.osc(440);
2657 // highpass3e_test = src : fi.highpass3e(1000);
2658 // ...
2659 //-----
2660 declare highpass3e author "Julius O. Smith III";
2661 declare highpass3e copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2662 declare highpass3e license "MIT-style STK-4.3 license";
2663 highpass3e(fc) = tf2s(b01/a01,b11/a01,b21/a01,a11/a01,1/a01,w1) :
2664     tf1s(1/a02,0,1/a02,w1)
2665 with {
2666     a11 = 0.802636764161030;

```

```

2667     a01 = 1.412270893774204;
2668     a02 = 0.822445908998816;
2669     b21 = 0.019809144837789;
2670     b11 = 0;
2671     b01 = 1.161516418982696;
2672     w1 = 2*ma.PI*fc;
2673 };
2674
2675 //-----`(fi.)highpass6e`-----
2676 // Sixth-order Elliptic/Cauer highpass filter. Inversion of `lowpass3e` wrt unit
2677 // circle in s plane (s <- 1/s).
2678 //
2679 // ### Usage
2680 //
2681 // ...
2682 // _ : highpass6e(fc) : _
2683 // ...
2684 //
2685 // Where:
2686 //
2687 // * `fc`: -3dB frequency in Hz
2688 //
2689 // ### Test
2690 // ...
2691 // fi = library("filters.lib");
2692 // os = library("oscillators.lib");
2693 // src = os.osc(440);
2694 // highpass6e_test = src : fi.highpass6e(1000);
2695 // ...
2696 //-----
2697 declare highpass6e author "Julius O. Smith III";
2698 declare highpass6e copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2699 declare highpass6e license "MIT-style STK-4.3 license";
2700 highpass6e(fc) =
2701     tf2s(b01/a01,b11/a01,b21/a01,a11/a01,1/a01,w1) :
2702     tf2s(b02/a02,b12/a02,b22/a02,a12/a02,1/a02,w1) :
2703     tf2s(b03/a03,b13/a03,b23/a03,a13/a03,1/a03,w1)
2704 with {
2705     b21 = 0.000099999997055;
2706     a21 = 1;
2707     b11 = 0;
2708     a11 = 0.782413046821645;
2709     b01 = 0.000433227200555;
2710     a01 = 0.245291508706160;
2711     b22 = 1;
2712     a22 = 1;
2713     b12 = 0;
2714     a12 = 0.512478641889141;
2715     b02 = 7.621731298870603;
2716     a02 = 0.689621364484675;
2717     b23 = 1;
2718     a23 = 1;
2719     b13 = 0;
2720     a13 = 0.168404871113589;
2721     b03 = 53.536152954556727;
2722     a03 = 1.069358407707312;
2723     w1 = 2*ma.PI*fc;
2724 };
2725
2726 //=====Butterworth Bandpass/Bandstop Filters=====
2727 //=====
2728
2729 //-----`(fi.)bandpass`-----
2730 // Order 2*Nh Butterworth bandpass filter made using the transformation
2731 // `s <- s + wc^2/s` on `lowpass(Nh)`, where `wc` is the desired bandpass center
2732 // frequency. The `lowpass(Nh)` cutoff `w1` is half the desired bandpass width.

```

```

2734 // `bandpass` is a standard Faust function.
2735 //
2736 // ##### Usage
2737 //
2738 // ```
2739 // _ : bandpass(Nh,fl,fu) : _
2740 // ```
2741 //
2742 // Where:
2743 //
2744 // * `Nh`: HALF the desired bandpass order (which is therefore even)
2745 // * `fl`: lower -3dB frequency in Hz
2746 // * `fu`: upper -3dB frequency in Hz
2747 // Thus, the passband width is `fu-fl`,
2748 // and its center frequency is `(fl+fu)/2`.
2749 //
2750 // ##### Test
2751 // ```
2752 // fi = library("filters.lib");
2753 // os = library("oscillators.lib");
2754 // src = os.osc(440);
2755 // bandpass_test = src : fi.bandpass(2, 500, 1500);
2756 // ```
2757 //-----
2758 declare bandpass author "Julius O. Smith III";
2759 declare bandpass copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
stanford.edu>";
2760 declare bandpass license "MIT-style STK-4.3 license";
2761 bandpass(Nh,fl,fu) = bandpass0_bandstop1(0,Nh,fl,fu);
2762
2763
2764 //-----`(fi.)bandpass0_bandstop1`-----
2765 // Order 2*Nh Butterworth bandstop filter made using the transformation
2766 // `s <- s + wc^2/s` on `highpass(Nh)`, where `wc` is the desired bandpass center
2767 // frequency. The `highpass(Nh)` cutoff `w1` is half the desired bandpass width.
2768 // `bandstop` is a standard Faust function.
2769 //
2770 // ##### Usage
2771 //
2772 // ```
2773 // _ : bandstop(Nh,fl,fu) : _
2774 // ```
2775 // Where:
2776 //
2777 // * `Nh`: HALF the desired bandstop order (which is therefore even)
2778 // * `fl`: lower -3dB frequency in Hz
2779 // * `fu`: upper -3dB frequency in Hz
2780 // Thus, the passband (stopband) width is `fu-fl`,
2781 // and its center frequency is `(fl+fu)/2`.
2782 //
2783 // ##### Test
2784 // ```
2785 // fi = library("filters.lib");
2786 // os = library("oscillators.lib");
2787 // src = os.osc(440);
2788 // bandstop_test = src : fi.bandstop(2, 500, 1500);
2789 // ```
2790 //-----
2791 declare bandstop author "Julius O. Smith III";
2792 declare bandstop copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
stanford.edu>";
2793 declare bandstop license "MIT-style STK-4.3 license";
2794 bandstop(Nh,fl,fu) = bandpass0_bandstop1(1,Nh,fl,fu);
2795
2796 //-----`(fi.)bandstop`-----
2797 //
2798 // ##### Test
2799 // ```

```

```

2800 // fi = library("filters.lib");
2801 // os = library("oscillators.lib");
2802 // src = os.osc(440);
2803 // bandpass0_bandstop1_test = src : fi.bandpass0_bandstop1(0, 2, 500, 1500);
2804 // ...
2805 //-----
2806 declare bandpass0_bandstop1 author "Julius O. Smith III";
2807 declare bandpass0_bandstop1 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
      jos@ccrma.stanford.edu>";
2808 declare bandpass0_bandstop1 license "MIT-style STK-4.3 license";
2809 bandpass0_bandstop1(s,Nh,fl,fu) = bpbsr(s,Nh,Nh,fl,fu)
2810 with {
2811   wl = 2*ma.PI*fl; // digital (z-plane) lower passband edge
2812   wu = 2*ma.PI*fu; // digital (z-plane) upper passband edge
2813
2814   c = 2.0*ma.SR; // bilinear transform scaling used in tf2sb, tf1sb
2815   wla = c*tan(wl/c); // analog (s-plane) lower cutoff
2816   wua = c*tan(wu/c); // analog (s-plane) upper cutoff
2817
2818   wc = sqrt(wla*wua); // s-plane center frequency
2819   w1 = wua - wc^2/wua; // s-plane lowpass prototype cutoff
2820
2821   bpbsr(s,0,Nh,fl,fu) = _;
2822   bpbsr(s,1,Nh,fl,fu) = tf1sb(s,1-s,1,w1,wc);
2823   bpbsr(s,0,Nh,fl,fu) = bpbsr(s,0-2,Nh,fl,fu) : tf2sb(s,0,(1-s),a1s,1,w1,wc)
2824   with {
2825     parity = Nh % 2;
2826     S = (0-parity)/2; // current section number
2827     a1s = -2*cos(-1*ma.PI + (1-parity)*ma.PI/(2*Nh) + (S-1+parity)*ma.PI/Nh);
2828   };
2829 };
2830
2831
2832 //-----Elliptic Bandpass Filters-----
2833 //-----
2834
2835 //-----`(fi.)bandpass6e`-----
2836 // Order 12 elliptic bandpass filter analogous to `bandpass(6)`.
2837 //-----
2838 declare bandpass6e author "Julius O. Smith III";
2839 declare bandpass6e copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
      stanford.edu>";
2840 declare bandpass6e license "MIT-style STK-4.3 license";
2841 bandpass6e(fl,fu) = tf2sb(b21,b11,b01,a11,a01,w1,wc) : tf1sb(0,1,a02,w1,wc)
2842 with {
2843   a11 = 0.802636764161030; // In octave: format long; poly(p(1:2))
2844   a01 = 1.412270893774204;
2845   a02 = 0.822445908998816; // poly(p(3))
2846   b21 = 0.019809144837789; // poly(z)
2847   b11 = 0;
2848   b01 = 1.161516418982696;
2849
2850   wl = 2*ma.PI*fl; // digital (z-plane) lower passband edge
2851   wu = 2*ma.PI*fu; // digital (z-plane) upper passband edge
2852
2853   c = 2.0*ma.SR; // bilinear transform scaling used in tf2sb, tf1sb
2854   wla = c*tan(wl/c); // analog (s-plane) lower cutoff
2855   wua = c*tan(wu/c); // analog (s-plane) upper cutoff
2856
2857   wc = sqrt(wla*wua); // s-plane center frequency
2858   w1 = wua - wc^2/wua; // s-plane lowpass cutoff
2859 };
2860
2861 //-----`(fi.)bandpass12e`-----
2862 // Order 24 elliptic bandpass filter analogous to `bandpass(6)`.
2863 //-----
2864 declare bandpass12e author "Julius O. Smith III";
2865 declare bandpass12e copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.

```

```

stanford.edu>";
2866 declare bandpass12e license "MIT-style STK-4.3 license";
2867 bandpass12e(fl,fu) =
2868     tf2sb(b21,b11,b01,a11,a01,w1,wc) :
2869     tf2sb(b22,b12,b02,a12,a02,w1,wc) :
2870     tf2sb(b23,b13,b03,a13,a03,w1,wc)
2871 with { // octave script output:
2872     b21 = 0.000099999997055;
2873     a21 = 1;
2874     b11 = 0;
2875     a11 = 0.782413046821645;
2876     b01 = 0.000433227200555;
2877     a01 = 0.245291508706160;
2878     b22 = 1;
2879     a22 = 1;
2880     b12 = 0;
2881     a12 = 0.512478641889141;
2882     b02 = 7.621731298870603;
2883     a02 = 0.689621364484675;
2884     b23 = 1;
2885     a23 = 1;
2886     b13 = 0;
2887     a13 = 0.168404871113589;
2888     b03 = 53.536152954556727;
2889     a03 = 1.069358407707312;
2890
2891     wl = 2*ma.PI*fl; // digital (z-plane) lower passband edge
2892     wu = 2*ma.PI*fu; // digital (z-plane) upper passband edge
2893
2894     c = 2.0*ma.SR; // bilinear transform scaling used in tf2sb, tf1sb
2895     wla = c*tan(wl/c); // analog (s-plane) lower cutoff
2896     wua = c*tan(wu/c); // analog (s-plane) upper cutoff
2897
2898     wc = sqrt(wla*wua); // s-plane center frequency
2899     w1 = wua - wc^2/wua; // s-plane lowpass cutoff
2900 };
2901
2902 //-----`(fi.)pospass`-----
2903 // Positive-Pass Filter (single-side-band filter).
2904 //
2905 // #### Usage
2906 //
2907 // ...
2908 // _ : pospass(N,fc) : _,_
2909 // ...
2910 //
2911 // where
2912 //
2913 // * `N`: filter order (Butterworth bandpass for positive frequencies).
2914 // * `fc`: lower bandpass cutoff frequency in Hz.
2915 // - Highpass cutoff frequency at ma.SR/2 - fc Hz.
2916 //
2917 // #### Example test program
2918 //
2919 // * See `dm.pospass_demo`
2920 // * Look at frequency response
2921 //
2922 // #### Method
2923 //
2924 // A filter passing only positive frequencies can be made from a
2925 // half-band lowpass by modulating it up to the positive-frequency range.
2926 // Equivalently, down-modulate the input signal using a complex sinusoid at -SR/4 Hz,
2927 // lowpass it with a half-band filter, and modulate back up by SR/4 Hz.
2928 // In Faust/math notation:
2929 // $$pospass(N) = \ast(e^{-j\frac{\pi}{2}n}) : \boxed{\text{lowpass}(N,SR/4)} : \ast(e^{j\frac{\pi}{2}n})$$
2930 //
2931 // An approximation to the Hilbert transform is given by the

```

```

2932 // imaginary output signal:
2933 //
2934 // ...
2935 // hilbert(N) = pospass(N) : !,*(2);
2936 // ...
2937 //
2938 // #### References
2939 //
2940 // * <https://ccrma.stanford.edu/~jos/mdft/Analytic_Signals_Hilbert_Transform.html>
2941 // * <https://ccrma.stanford.edu/~jos/sasp/Comparison_Optimal_Chebyshev_FIR_I.html>
2942 // * <https://ccrma.stanford.edu/~jos/sasp/Hilbert_Transform.html>
2943 //-----
2944 declare pospass author "Julius O. Smith III";
2945 declare pospass copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2946 declare pospass license "MIT-style STK-4.3 license";
2947 pospass(N,fc) = fi.pospass0(lpf) with {
2948     fcs = ma.SR/4 - fc; // Upper lowpass cutoff = (SR/2 - fc) - SR/4
2949     lpf = fi.lowpass(N,fcs); // Butterworth lowpass
2950 };
2951
2952 declare pospass6e author "Julius O. Smith III";
2953 declare pospass6e copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2954 declare pospass6e license "MIT-style STK-4.3 license";
2955 pospass6e(fc) = fi.pospass0(lpf) with {
2956     lpf = fi.lowpass6e(ma.SR/4 - fc); // Elliptic lowpass, order 6
2957 };
2958
2959 declare pospass0 author "Julius O. Smith III";
2960 declare pospass0 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
2961 declare pospass0 license "MIT-style STK-4.3 license";
2962 pospass0(lpf) = unmodulate : lpf, lpf : modulate with {
2963     c = 1-1' : +*(-1):mem); // complex sinusoid rotating at SR/4
2964     s = c'; // ||: 0, 1, 0, -1 :||
2965     unmodulate = _ <: *(c),*(-s); // subtract SR/4 from all input frequencies
2966     modulate(x,y) = c*x-s*y, c*y + s*x; // add SR/4 to all frequencies
2967 };
2968
2969
2970 //=====Parametric Equalizers (Shelf, Peaking)=====
2971 // Parametric Equalizers (Shelf, Peaking).
2972 //
2973 // #### References
2974 //
2975 // * <http://en.wikipedia.org/wiki/Equalization>
2976 // * <https://webaudio.github.io/Audio-EQ-Cookbook/Audio-EQ-Cookbook.txt>
2977 // * Digital Audio Signal Processing, Udo Zolzer, Wiley, 1999, p. 124
2978 // * <https://ccrma.stanford.edu/~jos/filters/Low_High_Shelving_Filters.html>
2979 // * <https://ccrma.stanford.edu/~jos/filters/Peaking_Equalizers.html>
2980 // * maxmsp.lib in the Faust distribution
2981 // * bandfilter.dsp in the faust2pd distribution
2982 //=====
2983
2984 //-----`fi).lowshelf`-----
2985 // First-order "low shelf" filter (gain boost/cut between dc and some frequency)
2986 // `low_shelf` is a standard Faust function.
2987 //
2988 // #### Usage
2989 //
2990 // ...
2991 // _ : lowshelf(N,L0,fx) : _
2992 // _ : low_shelf(L0,fx) : _ // default case (order 3)
2993 // _ : lowshelf_other_freq(N,L0,fx) : _
2994 // ...
2995 //
2996 // Where:

```

```

2997 //
2998 // * `N`: filter order 1, 3, 5, ... (odd only, default should be 3, a constant numerical
      expression)
2999 // * `L0`: desired level (dB) between dc and fx (boost `L0>0` or cut `L0<0`)
3000 // * `fx`: -3dB frequency of lowpass band (`L0>0`) or upper band (`L0<0`)
3001 //      (see "SHELF SHAPE" below).
3002 //
3003 // The gain at SR/2 is constrained to be 1.
3004 // The generalization to arbitrary odd orders is based on the well known
3005 // fact that odd-order Butterworth band-splits are allpass-complementary
3006 // (see filterbank documentation below for references).
3007 //
3008 // #### Test
3009 // ```
3010 // fi = library("filters.lib");
3011 // os = library("oscillators.lib");
3012 // src = os.osc(440);
3013 // lowshelf_test = src : fi.lowshelf(3, 6, 500);
3014 // ```
3015 //
3016 // #### Shelf Shape
3017 // The magnitude frequency response is approximately piecewise-linear
3018 // on a log-log plot ("BODE PLOT"). The Bode "stick diagram" approximation
3019 // L(lf) is easy to state in dB versus dB-frequency lf = dB(f):
3020 //
3021 // * L0 > 0:
3022 // * L(lf) = L0, f between 0 and fx = 1st corner frequency;
3023 // * L(lf) = L0 - N * (lf - lfx), f between fx and f2 = 2nd corner frequency;
3024 // * L(lf) = 0, lf > lf2.
3025 // * lf2 = lfx + L0/N = dB-frequency at which level gets back to 0 dB.
3026 // * L0 < 0:
3027 // * L(lf) = L0, f between 0 and f1 = 1st corner frequency;
3028 // * L(lf) = -N * (lfx - lf), f between f1 and lfx = 2nd corner frequency;
3029 // * L(lf) = 0, lf > lfx.
3030 // * lf1 = lfx + L0/N = dB-frequency at which level goes up from L0.
3031 //
3032 // See `lowshelf_other_freq`.
3033 //
3034 // #### References
3035 //
3036 // * See "Parametric Equalizers" above for references regarding `low_shelf`, `high_shelf`,
      and `peak_eq`.
3037 //-----
3038 declare lowshelf author "Julius O. Smith III";
3039 declare lowshelf copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
      stanford.edu>";
3040 declare lowshelf license "MIT-style STK-4.3 license";
3041 lowshelf(N,L0,fx) = filterbank(N,(fx)) : _, *(ba.db2linear(L0)) :> _;
3042
3043 // Special cases and optimization:
3044
3045 //-----`fi.`low_shelf`-----
3046 // #### Test
3047 // ```
3048 // fi = library("filters.lib");
3049 // os = library("oscillators.lib");
3050 // src = os.osc(440);
3051 // low_shelf_test = src : fi.low_shelf(6, 500);
3052 // ```
3053 //-----
3054 declare low_shelf author "Julius O. Smith III";
3055 declare low_shelf copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
      stanford.edu>";
3056 declare low_shelf license "MIT-style STK-4.3 license";
3057 low_shelf = lowshelf(3); // default = 3rd order Butterworth
3058
3059 //-----`fi.`low_shelf1`-----
3060

```

```

3061 // #### Test
3062 // ...
3063 // fi = library("filters.lib");
3064 // os = library("oscillators.lib");
3065 // src = os.osc(440);
3066 // low_shelf1_l_test = fi.low_shelf1_l(2, 500, src);
3067 // ...
3068 //-----
3069 declare low_shelf1 author "Julius O. Smith III";
3070 declare low_shelf1 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
3071 declare low_shelf1 license "MIT-style STK-4.3 license";
3072 low_shelf1(L0,fx,x) = x + (ba.db2linear(L0)-1)*lowpass(1,fx,x); // optimized
3073
3074
3075 //-----`fi.`low_shelf1_l`-----
3076 // #### Test
3077 // ...
3078 // fi = library("filters.lib");
3079 // os = library("oscillators.lib");
3080 // src = os.osc(440);
3081 // low_shelf1_l_test = fi.low_shelf1_l(2, 500, src);
3082 // ...
3083 //-----
3084 declare low_shelf1_l author "Julius O. Smith III";
3085 declare low_shelf1_l copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
3086 declare low_shelf1_l license "MIT-style STK-4.3 license";
3087 low_shelf1_l(G0,fx,x) = x + (G0-1)*lowpass(1,fx,x); // optimized
3088
3089
3090 //-----`fi.`lowshelf_other_freq`-----
3091 // #### Test
3092 // ...
3093 // fi = library("filters.lib");
3094 // lowshelf_other_freq_test = fi.lowshelf_other_freq(3, 6, 500);
3095 // ...
3096 //-----
3097 declare lowshelf_other_freq author "Julius O. Smith III";
3098 declare lowshelf_other_freq copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
    jos@ccrma.stanford.edu>";
3099 declare lowshelf_other_freq license "MIT-style STK-4.3 license";
3100 lowshelf_other_freq(N, L0, fx) = ba.db2linear(ba.linear2db(fx) + L0/N); // convenience
3101
3102 //-----`fi.`highshelf`-----
3103 // First-order "high shelf" filter (gain boost/cut above some frequency).
3104 // `high_shelf` is a standard Faust function.
3105 //
3106 // #### Usage
3107 //
3108 // ...
3109 // _ : highshelf(N,Lpi,fx) : _
3110 // _ : high_shelf(L0,fx) : _ // default case (order 3)
3111 // _ : highshelf_other_freq(N,Lpi,fx) : _
3112 // ...
3113 //
3114 // Where:
3115 //
3116 // * `N`: filter order 1, 3, 5, ... (odd only, a constant numerical expression).
3117 // * `Lpi`: desired level (dB) between fx and SR/2 (boost Lpi>0 or cut Lpi<0)
3118 // * `fx`: -3dB frequency of highpass band (L0>0) or lower band (L0<0)
3119 //       (Use highshelf_other_freq() below to find the other one.)
3120 //
3121 // The gain at dc is constrained to be 1.
3122 // See `lowshelf` documentation above for more details on shelf shape.
3123 //
3124 // #### Test
3125 // ...

```

```

3126 // fi = library("filters.lib");
3127 // os = library("oscillators.lib");
3128 // src = os.osc(440);
3129 // highshelf_test = src : fi.highshelf(3, 6, 2000);
3130 // ```
3131 //
3132 // ##### References
3133 //
3134 // * See "Parametric Equalizers" above for references regarding `low_shelf`, `high_shelf`,
    and `peak_eq`.
3135 //-----
3136 declare highshelf author "Julius O. Smith III";
3137 declare highshelf copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
3138 declare highshelf license "MIT-style STK-4.3 license";
3139 highshelf(N,Lpi,fx) = filterbank(N,(fx)) : *(ba.db2linear(Lpi)), _ :> _;
3140
3141 // Special cases and optimization:
3142
3143 //-----`fi.)high_shelf`-----
3144 // ##### Test
3145 // ```
3146 // fi = library("filters.lib");
3147 // os = library("oscillators.lib");
3148 // src = os.osc(440);
3149 // high_shelf_test = src : fi.high_shelf(6, 2000);
3150 // ```
3151 //-----
3152 high_shelf = highshelf(3); // default = 3rd order Butterworth
3153
3154 //-----`fi.)high_shelf1`-----
3155 // ##### Test
3156 // ```
3157 // fi = library("filters.lib");
3158 // os = library("oscillators.lib");
3159 // src = os.osc(440);
3160 // high_shelf1_test = fi.high_shelf1(6, 2000, src);
3161 // ```
3162 //-----
3163 declare high_shelf1 author "Julius O. Smith III";
3164 declare high_shelf1 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
3165 declare high_shelf1 license "MIT-style STK-4.3 license";
3166 high_shelf1(Lpi,fx,x) = x + (ba.db2linear(Lpi)-1)*highpass(1,fx,x); // optimized
3167
3168 //-----`fi.)high_shelf1_l`-----
3169 // ##### Test
3170 // ```
3171 // fi = library("filters.lib");
3172 // os = library("oscillators.lib");
3173 // src = os.osc(440);
3174 // high_shelf1_l_test = fi.high_shelf1_l(2, 2000, src);
3175 // ```
3176 //-----
3177 declare high_shelf1_l author "Julius O. Smith III";
3178 declare high_shelf1_l copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
3179 declare high_shelf1_l license "MIT-style STK-4.3 license";
3180 high_shelf1_l(Gpi,fx,x) = x + (Gpi-1)*highpass(1,fx,x); //optimized
3181
3182 // shelf transitions between frequency fx and this one:
3183
3184 //-----`fi.)highshelf_other_freq`-----
3185 // ##### Test
3186 // ```
3187 // fi = library("filters.lib");
3188 // highshelf_other_freq_test = fi.highshelf_other_freq(3, 6, 2000);
3189 // ```

```

```

3190 //-----
3191 declare highshelf_other_freq author "Julius O. Smith III";
3192 declare highshelf_other_freq copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
      jos@ccrma.stanford.edu>";
3193 declare highshelf_other_freq license "MIT-style STK-4.3 license";
3194 highshelf_other_freq(N, Lpi, fx) = ba.db2linear(ba.linear2db(fx) - Lpi/N);
3195
3196
3197 //-----`fi.`peak_eq`-----
3198 // Second order "peaking equalizer" section (gain boost or cut near some frequency)
3199 // Also called a "parametric equalizer" section.
3200 // `peak_eq` is a standard Faust function.
3201 //
3202 // #### Usage
3203 //
3204 // ```
3205 // _ : peak_eq(Lfx,fx,B) : _
3206 // ```
3207 //
3208 // Where:
3209 //
3210 // * `Lfx`: level (dB) at fx (boost Lfx>0 or cut Lfx<0)
3211 // * `fx`: peak frequency (Hz)
3212 // * `B`: bandwidth (B) of peak in Hz
3213 //
3214 // #### Test
3215 // ```
3216 // fi = library("filters.lib");
3217 // os = library("oscillators.lib");
3218 // src = os.osc(440);
3219 // peak_eq_test = src : fi.peak_eq(6, 1000, 200);
3220 // ```
3221 //
3222 // #### References
3223 //
3224 // * See "Parametric Equalizers" above for references regarding `low_shelf`, `high_shelf`,
      and `peak_eq`.
3225 //-----
3226 declare peak_eq author "Julius O. Smith III";
3227 declare peak_eq copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
      stanford.edu>";
3228 declare peak_eq license "MIT-style STK-4.3 license";
3229 peak_eq(Lfx,fx,B) = tf2s(1,b1s,1,a1s,1,wx) with {
3230   T = float(1.0/ma.SR);
3231   Bw = B*T/sin(wx*T); // prewarp s-bandwidth for more accuracy in z-plane
3232   a1 = ma.PI*Bw;
3233   b1 = g*a1;
3234   g = ba.db2linear(abs(Lfx));
3235   b1s = select2(Lfx>0,a1,b1); // When Lfx>0, pole dominates bandwidth
3236   a1s = select2(Lfx>0,b1,a1); // When Lfx<0, zero dominates
3237   wx = 2*ma.PI*fx;
3238 };
3239
3240 //-----`fi.`peak_eq_cq`-----
3241 // Constant-Q second order peaking equalizer section.
3242 //
3243 // #### Usage
3244 //
3245 // ```
3246 // _ : peak_eq_cq(Lfx,fx,Q) : _
3247 // ```
3248 //
3249 // Where:
3250 //
3251 // * `Lfx`: level (dB) at fx
3252 // * `fx`: boost or cut frequency (Hz)
3253 // * `Q`: "Quality factor" = fx/B where B = bandwidth of peak in Hz
3254 //

```

```

3255 // #### Test
3256 // ```
3257 // fi = library("filters.lib");
3258 // os = library("oscillators.lib");
3259 // src = os.osc(440);
3260 // peak_eq_cq_test = src : fi.peak_eq_cq(6, 1000, 4);
3261 // ```
3262 //
3263 // #### References
3264 //
3265 // * See "Parametric Equalizers" above for references regarding `low_shelf`, `high_shelf`,
    and `peak_eq`.
3266 //-----
3267 declare peak_eq_cq author "Julius O. Smith III";
3268 declare peak_eq_cq copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
3269 declare peak_eq_cq license "MIT-style STK-4.3 license";
3270 peak_eq_cq(Lfx,fx,Q) = peak_eq(Lfx,fx,fx/Q);
3271
3272 //-----`(fi.)peak_eq_rm`-----
3273 // Regalia-Mitra second order peaking equalizer section.
3274 //
3275 // #### Usage
3276 //
3277 // ```
3278 // _ : peak_eq_rm(Lfx,fx,tanPiBT) : _
3279 // ```
3280 //
3281 // Where:
3282 //
3283 // * `Lfx`: level (dB) at fx
3284 // * `fx`: boost or cut frequency (Hz)
3285 // * `tanPiBT`:  $\tan(\text{PI} \cdot B / \text{SR})$ , where  $B = -3\text{dB bandwidth (Hz) when } 10^{(Lfx/20)} = 0$ 
3286 // ~  $\text{PI} \cdot B / \text{SR}$  for narrow bandwidths B
3287 //
3288 // #### Test
3289 // ```
3290 // fi = library("filters.lib");
3291 // os = library("oscillators.lib");
3292 // ma = library("maths.lib");
3293 // src = os.osc(440);
3294 // peak_eq_rm_test = src : fi.peak_eq_rm(6, 1000, tan(ma.PI*200/ma.SR));
3295 // ```
3296 //
3297 // #### References
3298 //
3299 // P.A. Regalia, S.K. Mitra, and P.P. Vaidyanathan,
3300 // "The Digital All-Pass Filter: A Versatile Signal Processing Building Block"
3301 // Proceedings of the IEEE, 76(1):19-37, Jan. 1988. (See pp. 29-30.)
3302 // See also "Parametric Equalizers" above for references on shelf
3303 // and peaking equalizers in general.
3304 //-----
3305 declare peak_eq_rm author "Julius O. Smith III";
3306 declare peak_eq_rm copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
3307 declare peak_eq_rm license "MIT-style STK-4.3 license";
3308 peak_eq_rm(Lfx,fx,tanPiBT) = _ <: _,A,_ : +,- : *(0.5),*(K/2.0) : + with {
3309   A = tf2(k2, k1*(1+k2), 1, k1*(1+k2), k2) <: _,_; // allpass
3310   k1 = 0.0 - cos(2.0*ma.PI*fx/ma.SR);
3311   k2 = (1.0 - tanPiBT)/(1.0 + tanPiBT);
3312   K = ba.db2linear(Lfx);
3313 };
3314
3315 //-----`(fi.)spectral_tilt`-----
3316 // Spectral tilt filter, providing an arbitrary spectral rolloff factor
3317 // alpha in (-1,1), where
3318 // -1 corresponds to one pole (-6 dB per octave), and

```

```

3320 // +1 corresponds to one zero (+6 dB per octave).
3321 // In other words, alpha is the slope of the ln magnitude versus ln frequency.
3322 // For a "pinking filter" (e.g., to generate 1/f noise from white noise),
3323 // set alpha to -1/2.
3324 //
3325 // #### Usage
3326 //
3327 // ...
3328 // _ : spectral_tilt(N,f0,bw,alpha) : _
3329 // ...
3330 // Where:
3331 //
3332 // * `N`: desired integer filter order (fixed at compile time)
3333 // * `f0`: lower frequency limit for desired roll-off band > 0
3334 // * `bw`: bandwidth of desired roll-off band
3335 // * `alpha`: slope of roll-off desired in nepers per neper,
3336 //             between -1 and 1 (ln mag / ln radian freq)
3337 //
3338 // #### Test
3339 // ...
3340 // fi = library("filters.lib");
3341 // os = library("oscillators.lib");
3342 // src = os.osc(440);
3343 // spectral_tilt_test = src : fi.spectral_tilt(4, 200, 2000, -0.5);
3344 // ...
3345 //
3346 // #### Example test program
3347 //
3348 // See `dm.spectral_tilt_demo` and the documentation for `no.pink_noise`.
3349 //
3350 // #### References
3351 //
3352 // J.O. Smith and H.F. Smith,
3353 // "Closed Form Fractional Integration and Differentiation via Real Exponentially Spaced Pole
3354 //   -Zero Pairs",
3355 // arXiv.org publication arXiv:1606.06154 [cs.CE], June 7, 2016, <http://arxiv.org/abs
3356 //   /1606.06154>
3357 //-----
3358 declare spectral_tilt author "Julius O. Smith III";
3359 declare spectral_tilt copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
3360   stanford.edu>";
3361 declare spectral_tilt license "MIT-style STK-4.3 license";
3362 spectral_tilt(N,f0,bw,alpha) = seq(i,N,sec(i)) with {
3363   sec(i) = g * tfis(b1,b0,a0,1) with {
3364     g = a0/b0; // unity dc-gain scaling
3365     b1 = 1.0;
3366     b0 = mzh(i);
3367     a0 = mph(i);
3368     mzh(i) = prewarp(mz(i),ma.SR,w0); // prewarping for bilinear transform
3369     mph(i) = prewarp(mp(i),ma.SR,w0);
3370     prewarp(w,SR,wp) = wp * tan(w*T/2) / tan(wp*T/2) with { T = 1/ma.SR; };
3371     mz(i) = w0 * r ^ (-alpha+i); // minus zero i in s plane
3372     mp(i) = w0 * r ^ i; // minus pole i in s plane
3373     f0p = max(f0,ma.EPSILON); // cannot go to zero
3374     w0 = 2 * ma.PI * f0p; // radian frequency of first pole
3375     f1 = f0p + bw; // upper band limit
3376     r = (f1/f0p)^(1.0/float(N-1)); // pole ratio (2 => octave spacing)
3377   };
3378 };
3379 //-----
3380 //-----`(fi.)levelfilter`-----
3381 // Dynamic level lowpass filter.
3382 // `levelfilter` is a standard Faust function.
3383 //
3384 // #### Usage
3385 //
3386 // ...

```

```

3385 // _ : levelfilter(L,freq) : _
3386 // ...
3387 //
3388 // Where:
3389 //
3390 // * `L`: desired level (in dB) at Nyquist limit (SR/2), e.g., -60
3391 // * `freq`: corner frequency (-3dB point) usually set to fundamental freq
3392 // * `N`: Number of filters in series where L = L/N
3393 //
3394 // #### Test
3395 // ...
3396 // fi = library("filters.lib");
3397 // os = library("oscillators.lib");
3398 // src = os.osc(440);
3399 // levelfilter_test = fi.levelfilter(0.1, 200, src);
3400 // ...
3401 //
3402 // #### References
3403 //
3404 // * <https://ccrma.stanford.edu/realsimple/faust\_strings/Dynamic\_Level\_Lowpass\_Filter.html>
3405 //-----
3406 declare levelfilter author "Julius O. Smith III";
3407 declare levelfilter copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
3408 declare levelfilter license "MIT-style STK-4.3 license";
3409 levelfilter(L,freq,x) = (L * L0 * x) + ((1.0-L) * lp2out(x))
3410 with {
3411     L0 = pow(L,1/3);
3412     Lw = ma.PI*freq/ma.SR; // = w1 T / 2
3413     Lgain = Lw / (1.0 + Lw);
3414     Lpole2 = (1.0 - Lw) / (1.0 + Lw);
3415     lp2out = *(Lgain) : + ~ *(Lpole2);
3416 };
3417
3418
3419 //-----`(fi.)levelfilterN`-----
3420 // Dynamic level lowpass filter.
3421 //
3422 // #### Usage
3423 //
3424 // ...
3425 // _ : levelfilterN(N,freq,L) : _
3426 // ...
3427 //
3428 // Where:
3429 //
3430 // * `N`: Number of filters in series where L = L/N, a constant numerical expression
3431 // * `freq`: corner frequency (-3dB point) usually set to fundamental freq
3432 // * `L`: desired level (in dB) at Nyquist limit (SR/2), e.g., -60
3433 //
3434 // #### Test
3435 // ...
3436 // fi = library("filters.lib");
3437 // os = library("oscillators.lib");
3438 // src = os.osc(440);
3439 // levelfilterN_test = src : fi.levelfilterN(3, 200, 0.1);
3440 // ...
3441 //
3442 // #### References
3443 //
3444 // * <https://ccrma.stanford.edu/realsimple/faust\_strings/Dynamic\_Level\_Lowpass\_Filter.html>
3445 //-----
3446 declare levelfilterN author "Julius O. Smith III";
3447 declare levelfilterN copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
3448 declare levelfilterN license "MIT-style STK-4.3 license";
3449 levelfilterN(N,freq,L) = seq(i,N,levelfilter((L/N),freq));
3450

```

```

3451 //=====Mth-Octave Filter-Banks=====
3452 // Mth-octave filter-banks split the input signal into a bank of parallel signals, one
3453 // for each spectral band. They are related to the Mth-Octave Spectrum-Analyzers in
3454 // `analysis.lib`.
3455 // The documentation of this library contains more details about the implementation.
3456 // The parameters are:
3457 //
3458 // * `M`: number of band-slices per octave (>1), a constant numerical expression
3459 // * `N`: total number of bands (>2), a constant numerical expression
3460 // * `ftop`: upper bandlimit of the Mth-octave bands (<SR/2)
3461 //
3462 // In addition to the Mth-octave output signals, there is a highpass signal
3463 // containing frequencies from ftop to SR/2, and a "dc band" lowpass signal
3464 // containing frequencies from 0 (dc) up to the start of the Mth-octave bands.
3465 // Thus, the N output signals are
3466 //
3467 // ```
3468 // highpass(ftop), MthOctaveBands(M,N-2,ftop), dcBand(ftop*2^(-(M*(N-1))))
3469 // ```
3470 //
3471 // A Filter-Bank is defined here as a signal bandsplitter having the
3472 // property that summing its output signals gives an allpass-filtered
3473 // version of the filter-bank input signal. A more conventional term for
3474 // this is an "allpass-complementary filter bank". If the allpass filter
3475 // is a pure delay (and possible scaling), the filter bank is said to be
3476 // a "perfect-reconstruction filter bank" (see Vaidyanathan-1993 cited
3477 // below for details). A "graphic equalizer", in which band signals
3478 // are scaled by gains and summed, should be based on a filter bank.
3479 //
3480 // The filter-banks below are implemented as Butterworth or Elliptic
3481 // spectrum-analyzers followed by delay equalizers that make them
3482 // allpass-complementary.
3483 //
3484 // #### Increasing Channel Isolation
3485 //
3486 // Go to higher filter orders - see Regalia et al. or Vaidyanathan (cited
3487 // below) regarding the construction of more aggressive recursive
3488 // filter-banks using elliptic or Chebyshev prototype filters.
3489 //
3490 // #### References
3491 //
3492 // * "Tree-structured complementary filter banks using all-pass sections", Regalia et al.,
3493 //   IEEE Trans. Circuits & Systems, CAS-34:1470-1484, Dec. 1987
3494 // * "Multirate Systems and Filter Banks", P. Vaidyanathan, Prentice-Hall, 1993
3495 // * Elementary filter theory: <https://ccrma.stanford.edu/~jos/filters/>
3496 //=====
3497 //-----`(fi.)mth_octave_filterbank[n]`-----
3498 // Allpass-complementary filter banks based on Butterworth band-splitting.
3499 // For Butterworth band-splits, the needed delay equalizer is easily found.
3500 //
3501 // #### Usage
3502 //
3503 // ```
3504 // _ : mth_octave_filterbank(0,M,ftop,N) : par(i,N,_) // 0th-order
3505 // _ : mth_octave_filterbank_alt(0,M,ftop,N) : par(i,N,_) // dc-inverted version
3506 // ```
3507 //
3508 // Also for convenience:
3509 //
3510 // ```
3511 // _ : mth_octave_filterbank3(M,ftop,N) : par(i,N,_) // 3rd-order Butterworth
3512 // _ : mth_octave_filterbank5(M,ftop,N) : par(i,N,_) // 5th-order Butterworth
3513 // mth_octave_filterbank_default = mth_octave_filterbank5;
3514 // ```
3515 //
3516 // Where:

```

```

3518 //
3519 // * `O`: order of filter used to split each frequency band into two, a constant numerical
      expression
3520 // * `M`: number of band-slices per octave, a constant numerical expression
3521 // * `ftop`: highest band-split crossover frequency (e.g., 20 kHz)
3522 // * `N`: total number of bands (including dc and Nyquist), a constant numerical expression
3523 //
3524 // #### Test
3525 // ...
3526 // fi = library("filters.lib");
3527 // os = library("oscillators.lib");
3528 // sig = os.osc(440);
3529 // mth_octave_filterbank_test = sig : fi.mth_octave_filterbank(3, 2, 8000, 2);
3530 // ...
3531 //-----
3532 declare mth_octave_filterbank author "Julius O. Smith III";
3533 declare mth_octave_filterbank copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
      jos@ccrma.stanford.edu>";
3534 declare mth_octave_filterbank license "MIT-style STK-4.3 license";
3535
3536 mth_octave_filterbank(O,M,ftop,N) = an.mth_octave_analyzer(O,M,ftop,N) : delayeq(N)
3537 with {
3538   fc(n) = ftop * 2^(float(n-N+1)/float(M)); // -3dB crossover frequencies
3539   ap(n) = highpass_plus_lowpass(0,fc(n)); // delay-equalizing allpass
3540   delayeq(N) = par(i,N-2,apchain(i+1)), _, _;
3541   apchain(i) = seq(j,N-1-i,ap(j+1));
3542 };
3543
3544 // dc-inverted version. This reduces the delay-equalizer order for odd O.
3545 // Negating the input signal makes the dc band noninverting
3546 // and all higher bands sign-inverted (if preferred).
3547 declare mth_octave_filterbank_alt author "Julius O. Smith III";
3548 declare mth_octave_filterbank_alt copyright "Copyright (C) 2003-2019 by Julius O. Smith III
      <jos@ccrma.stanford.edu>";
3549 declare mth_octave_filterbank_alt license "MIT-style STK-4.3 license";
3550
3551 //-----`(fi.)mth_octave_filterbank_alt`-----
3552 //
3553 // #### Usage
3554 //
3555 // ...
3556 // _ : mth_octave_filterbank_alt(O,M,ftop,N) : par(i,N,_)
3557 // ...
3558 //
3559 // Where:
3560 //
3561 // * `O`: filter order used to split each band
3562 // * `M`: number of band-slices per octave
3563 // * `ftop`: highest band-split crossover frequency
3564 // * `N`: total number of bands
3565 //
3566 // #### Test
3567 // ...
3568 // fi = library("filters.lib");
3569 // os = library("oscillators.lib");
3570 // sig = os.osc(440);
3571 // mth_octave_filterbank_alt_test = sig : fi.mth_octave_filterbank_alt(3, 2, 8000, 2);
3572 // ...
3573 //-----
3574 mth_octave_filterbank_alt(O,M,ftop,N) = an.mth_octave_analyzer(O,M,ftop,N) : delayeqi(0,N)
3575 with {
3576   fc(n) = ftop * 2^(float(n-N+1)/float(M)); // -3dB crossover frequencies
3577   ap(n) = highpass_minus_lowpass(0,fc(n)); // half the order of 'plus' case
3578   delayeqi(N) = par(i,N-2,apchain(i+1)), _, *(-1.0);
3579   apchain(i) = seq(j,N-1-i,ap(j+1));
3580 };
3581
3582 // Note that even-order cases require complex coefficients.

```

```

3583 // See Vaidyanathan 1993 and papers cited there for more info.
3584 declare mth_octave_filterbank3 author "Julius O. Smith III";
3585 declare mth_octave_filterbank3 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
      jos@ccrma.stanford.edu>";
3586 declare mth_octave_filterbank3 license "MIT-style STK-4.3 license";
3587
3588 //-----`(fi.)mth_octave_filterbank3`-----
3589 //
3590 // #### Usage
3591 //
3592 // ``
3593 // _ : mth_octave_filterbank3(M,ftop,N) : par(i,N,_)
3594 // ``
3595 //
3596 // Where:
3597 //
3598 // * `M`: number of band-slices per octave
3599 // * `ftop`: highest band-split crossover frequency
3600 // * `N`: total number of bands
3601 //
3602 // #### Test
3603 // ``
3604 // fi = library("filters.lib");
3605 // os = library("oscillators.lib");
3606 // sig = os.osc(440);
3607 // mth_octave_filterbank3_test = sig : fi.mth_octave_filterbank3(2, 8000, 2);
3608 // ``
3609 //-----
3610 mth_octave_filterbank3(M,ftop,N) = mth_octave_filterbank_alt(3,M,ftop,N);
3611
3612 declare mth_octave_filterbank5 author "Julius O. Smith III";
3613 declare mth_octave_filterbank5 copyright "Copyright (C) 2003-2019 by Julius O. Smith III <
      jos@ccrma.stanford.edu>";
3614 declare mth_octave_filterbank5 license "MIT-style STK-4.3 license";
3615
3616 //-----`(fi.)mth_octave_filterbank5`-----
3617 //
3618 // #### Usage
3619 //
3620 // ``
3621 // _ : mth_octave_filterbank5(M,ftop,N) : par(i,N,_)
3622 // ``
3623 //
3624 // Where:
3625 //
3626 // * `M`: number of band-slices per octave
3627 // * `ftop`: highest band-split crossover frequency
3628 // * `N`: total number of bands
3629 //
3630 // #### Test
3631 // ``
3632 // fi = library("filters.lib");
3633 // os = library("oscillators.lib");
3634 // sig = os.osc(440);
3635 // mth_octave_filterbank5_test = sig : fi.mth_octave_filterbank5(2, 8000, 2);
3636 // ``
3637 //-----
3638 mth_octave_filterbank5(M,ftop,N) = mth_octave_filterbank(5,M,ftop,N);
3639
3640 declare mth_octave_filterbank_default author "Julius O. Smith III";
3641 declare mth_octave_filterbank_default copyright "Copyright (C) 2003-2019 by Julius O. Smith
      III <jos@ccrma.stanford.edu>";
3642 declare mth_octave_filterbank_default license "MIT-style STK-4.3 license";
3643
3644 //-----`(fi.)mth_octave_filterbank_default`-----
3645 //
3646 // #### Test
3647 // ``

```

```

3648 // fi = library("filters.lib");
3649 // os = library("oscillators.lib");
3650 // sig = os.osc(440);
3651 // mth_octave_filterbank_default_test = sig : fi.mth_octave_filterbank_default(2, 8000, 2);
3652 // ...
3653 //-----
3654 mth_octave_filterbank_default = mth_octave_filterbank5;
3655
3656
3657 //=====Arbitrary-Crossover Filter-Banks and Spectrum Analyzers=====
3658 // These are similar to the Mth-octave analyzers above, except that the
3659 // band-split frequencies are passed explicitly as arguments.
3660 //=====
3661
3662 // ACKNOWLEDGMENT
3663 // Technique for processing a variable number of signal arguments due
3664 // to Yann Orlarey (as is the entire Faust framework!)
3665
3666 //-----`(fi.)filterbank`-----
3667 // Filter bank.
3668 // `filterbank` is a standard Faust function.
3669 //
3670 // ### Usage
3671 //
3672 // ...
3673 // _ : filterbank (0,freqs) : par(i,N,_) // Butterworth band-splits
3674 // ...
3675 // Where:
3676 //
3677 // * `0`: band-split filter order (odd integer required for filterbank[i], a constant
3678 //   numerical expression)
3679 // * `freqs`: (fc1,fc2,...,fcNs) [in numerically ascending order], where
3680 //   Ns=N-1 is the number of octave band-splits
3681 //   (total number of bands N=Ns+1).
3682 //
3683 // If frequencies are listed explicitly as arguments, enclose them in parens:
3684 //
3685 // ...
3686 // _ : filterbank(3,(fc1,fc2)) : _,_,_
3687 // ...
3688 //
3689 // ### Test
3690 //
3691 // fi = library("filters.lib");
3692 // os = library("oscillators.lib");
3693 // src = os.osc(440);
3694 // filterbank_test = src : fi.filterbank(3, (500, 2000));
3695 // ...
3696 //-----
3697 declare filterbank author "Julius O. Smith III";
3698 declare filterbank copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
3699 stanford.edu>";
3700 declare filterbank license "MIT-style STK-4.3 license";
3701 filterbank(0,lfreqs) = an.analyzer(0,lfreqs) : delayeq(nb)
3702 with {
3703   nb = ba.count(lfreqs);
3704   fc(n) = ba.take(n, lfreqs);
3705   ap(n) = highpass_plus_lowpass(0,fc(n));
3706   delayeq(1) = _,_; // par(i,0,...) does not fly
3707   delayeq(nb) = par(i,nb-1,apchain(nb-1-i)),_,_;
3708   apchain(0) = _;
3709   apchain(i) = ap(i) : apchain(i-1);
3710 };
3711
3712 //-----`(fi.)filterbanki`-----
3713 // Inverted-dc filter bank.
3714 //
3715 // ### Usage

```

```

3714 //
3715 // ...
3716 // _ : filterbanki(0,freqs) : par(i,N,_) // Inverted-dc version
3717 // ...
3718 //
3719 // Where:
3720 //
3721 // * `0`: band-split filter order (odd integer required for `filterbank[i]`, a constant
    numerical expression)
3722 // * `freqs`: (fc1,fc2,...,fcNs) [in numerically ascending order], where
    //      Ns=N-1 is the number of octave band-splits
3723 //      (total number of bands N=Ns+1).
3724 //
3725 //
3726 // If frequencies are listed explicitly as arguments, enclose them in parens:
3727 //
3728 // ...
3729 // _ : filterbanki(3,(fc1,fc2)) : _,_,_
3730 // ...
3731 //
3732 // #### Test
3733 // ...
3734 // fi = library("filters.lib");
3735 // os = library("oscillators.lib");
3736 // src = os.osc(440);
3737 // filterbanki_test = src : fi.filterbanki(3, (500, 2000));
3738 // ...
3739 //-----
3740 declare filterbanki author "Julius O. Smith III";
3741 declare filterbanki copyright "Copyright (C) 2003-2019 by Julius O. Smith III <jos@ccrma.
    stanford.edu>";
3742 declare filterbanki license "MIT-style STK-4.3 license";
3743 filterbanki(0,lfreqs) = _ <: bsplit(nb)
3744 with {
3745     nb = ba.count(lfreqs);
3746     fc(n) = ba.take(n, lfreqs);
3747     lp(n) = lowpass(0,fc(n));
3748     hp(n) = highpass(0,fc(n));
3749     ap(n) = highpass_minus_lowpass(0,fc(n));
3750     bsplit(0) = *(-1.0);
3751     bsplit(i) = (hp(i) : delayeq(i-1)), (lp(i) <: bsplit(i-1));
3752     delayeq(0) = _; // moving the *(-1) here inverts all outputs BUT dc
3753     delayeq(i) = ap(i) : delayeq(i-1);
3754 };
3755
3756
3757 //=====State Variable Filters
    =====
3758 // #### References
3759 //
3760 // * Solving the continuous SVF equations using trapezoidal integration
3761 // * <https://cytomic.com/files/dsp/SvfLinearTrapOptimised2.pdf>
3762 //=====
3763
3764 //-----`(fi.)svf`-----
3765 // An environment with `lp`, `bp`, `hp`, `notch`, `peak`, `ap`, `bell`, `ls`, `hs` SVF based
    filters.
3766 // All filters have `freq` and `Q` parameters, the `bell`, `ls`, `hs` ones also have a `gain`
    third parameter.
3767 //
3768 // #### Usage
3769 //
3770 // ...
3771 // _ : svf.xx(freq, Q, [gain]) : _
3772 // ...
3773 //
3774 // Where:
3775 //
3776 // * `freq`: cut frequency

```

```

3777 // * `Q`: quality factor
3778 // * `[gain]`: gain in dB
3779 //
3780 // #### Test
3781 // ...
3782 // fi = library("filters.lib");
3783 // os = library("oscillators.lib");
3784 // sig = os.osc(440);
3785 // svf_lp_test = fi.svf.lp(1000, 0.707, sig);
3786 // ...
3787 //-----
3788 declare svf author "Oleg Nesterov";
3789 declare svf copyright "Copyright (C) 2020 Oleg Nesterov <oleg@redhat.com>";
3790 declare svf license "MIT-style STK-4.3 license";
3791
3792 svf = environment {
3793
3794     // Internal implementation
3795     svf(T,F,Q,G) = tick ~ (_,_) : !,!,si.dot(3, mix)
3796     with {
3797         tick(ic1eq, ic2eq, v0) =
3798             2*v1 - ic1eq,
3799             2*v2 - ic2eq,
3800             v0, v1, v2
3801         with {
3802             v1 = ic1eq + g *(v0-ic2eq) : /(1 + g*(g+k));
3803             v2 = ic2eq + g * v1;
3804         };
3805
3806         A = pow(10.0, G/40.0);
3807
3808         g = tan(F * ma.PI/ma.SR) : case {
3809             (7) => /(sqrt(A));
3810             (8) => *(sqrt(A));
3811             (t) => _;
3812         } (T);
3813
3814         k = case {
3815             (6) => 1/(Q*A);
3816             (t) => 1/Q;
3817         } (T);
3818
3819         mix = case {
3820             (0) => 0, 0, 1;
3821             (1) => 0, 1, 0;
3822             (2) => 1, -k, -1;
3823             (3) => 1, -k, 0;
3824             (4) => 1, -k, -2;
3825             (5) => 1, -2*k, 0;
3826             (6) => 1, k*(A*A-1), 0;
3827             (7) => 1, k*(A-1), A*A-1;
3828             (8) => A*A, k*(1-A)*A, 1-A*A;
3829         } (T);
3830     };
3831
3832     // External API
3833     lp(f,q) = svf(0, f, q, 0);
3834     bp(f,q) = svf(1, f, q, 0);
3835     hp(f,q) = svf(2, f, q, 0);
3836     notch(f,q) = svf(3, f, q, 0);
3837     peak(f,q) = svf(4, f, q, 0);
3838     ap(f,q) = svf(5, f, q, 0);
3839     bell(f,q,g) = svf(6, f, q, g);
3840     ls(f,q,g) = svf(7, f, q, g);
3841     hs(f,q,g) = svf(8, f, q, g);
3842 };
3843
3844

```

```

3845 //-----`(fi.)svf_morph`-----
3846 // An SVF-based filter that can smoothly morph between
3847 // being lowpass, bandpass, and highpass.
3848 //
3849 // #### Usage
3850 //
3851 // ...
3852 // _ : svf_morph(freq, Q, blend) : _
3853 // ...
3854 //
3855 // Where:
3856 //
3857 // * `freq`: cutoff frequency
3858 // * `Q`: quality factor
3859 // * `blend`: [0..2] continuous, where 0 is `lowpass`, 1 is `bandpass`, and 2 is `highpass`.
3860 // For performance, the value is not clamped to [0..2].
3861 //
3862 // #### Test
3863 // ...
3864 // fi = library("filters.lib");
3865 // os = library("oscillators.lib");
3866 // sig = os.osc(440);
3867 // svf_morph_test = fi.svf_morph(1000, 0.707, 1, sig);
3868 // ...
3869 //
3870 // #### Example test program
3871 // ...
3872 // process = no.noise : svf_morph(freq, q, blend)
3873 // with {
3874 //   blend = hslider("Blend", 0, 0, 2, .01) : si.smoo;
3875 //   q = hslider("Q", 1, 0.1, 10, .01) : si.smoo;
3876 //   freq = hslider("freq", 5000, 100, 18000, 1) : si.smoo;
3877 // };
3878 // ...
3879 //
3880 // #### References
3881 //
3882 // <https://github.com/mtytel/vital/blob/636ca0ef517a4db087a6a08a6a8a5e704e21f836/src/
3883 // synthesis/filters/digital_svf.cpp#L292-L295>
3884 //-----
3885 declare svf_morph author "David Braun and Clarence W. Rowley";
3886 declare svf_morph copyright "Copyright (C) 2024 David Braun <brown@ccrma.stanford.edu>";
3887 declare svf_morph license "MIT-style STK-4.3 license";
3888 svf_morph(f, q, _b, x) = svf.lp(f,q,x)*w_LP + svf.bp(f,q,x)*w_BP + svf.hp(f,q,x)*w_HP
3889 with {
3890   b = _b - 1; // b is now -1 to 1.
3891   w_LP = max(-b, 0);
3892   w_BP = sqrt(1 - b^2);
3893   w_HP = max(b, 0);
3894 };
3895
3896 //-----`(fi.)svf_notch_morph`-----
3897 // An SVF-based notch-filter that can smoothly morph between
3898 // being lowpass, notch, and highpass.
3899 //
3900 // #### Usage
3901 //
3902 // ...
3903 // _ : svf_notch_morph(freq, Q, blend) : _
3904 // ...
3905 //
3906 // Where:
3907 //
3908 // * `freq`: cutoff frequency
3909 // * `Q`: quality factor
3910 // * `blend`: [0..2] continuous, where 0 is `lowpass`, 1 is `notch`, and 2 is `highpass`. For

```

```

        performance, the value is not clamped to [0..2].
3911 //
3912 // ##### Test
3913 // ...
3914 // fi = library("filters.lib");
3915 // os = library("oscillators.lib");
3916 // sig = os.osc(440);
3917 // svf_notch_morph_test = fi.svf_notch_morph(1000, 0.707, 1, sig);
3918 // ...
3919 //
3920 // ##### Example test program
3921 //
3922 // ...
3923 // process = no.noise : svf_notch_morph(freq, q, blend)
3924 // with {
3925 //   blend = hslider("Blend", 0, 0, 2, .01) : si.smoo;
3926 //   q = hslider("Q", 1, 0.1, 10, .01) : si.smoo;
3927 //   freq = hslider("freq", 5000, 100, 18000, 1) : si.smoo;
3928 // };
3929 // ...
3930 //
3931 // ##### References
3932 //
3933 // <https://github.com/mtytel/vital/blob/636ca0ef517a4db087a6a08a6a8a5e704e21f836/src/
3934 // synthesis/filters/digital_svf.cpp#L256C36-L263>
3935 //-----
3936 declare svf_notch_morph author "David Braun and Clarence W. Rowley";
3937 declare svf_notch_morph copyright "Copyright (C) 2024 David Braun <braun@ccrma.stanford.edu>";
3938 declare svf_notch_morph license "MIT-style STK-4.3 license";
3939 svf_notch_morph(f, q, _b, x) = svf.lp(f,q,x)*w_LP + svf.hp(f,q,x)*w_HP
3940 with {
3941   b = _b - 1; // b is now -1 to 1.
3942   w_LP = min(1-b, 1);
3943   w_HP = min(1+b, 1);
3944 };
3945
3946 //-----`(fi.)SVFTPT`-----
3947 //
3948 // Topology-preserving transform implementation following Zavalishin's method.
3949 //
3950 // Outputs: lowpass, highpass, bandpass, normalised bandpass, notch, allpass,
3951 // peaking.
3952 //
3953 // Each individual output can be recalled with its name in the environment as in:
3954 //   `SVFTPT.LP2(1000.0, .707)`
3955 //
3956 // The 7 outputs can be recalled by using `SVF` name as in:
3957 //   `SVFTPT.SVF(1000.0, .707)`
3958 //
3959 // Even though the implementation is different, the characteristics of this
3960 // filter are comparable to those of the `svf` environment in this library.
3961 //
3962 // ##### Usage:
3963 //
3964 // ...
3965 // _ : SVFTPT.xxx(CF, Q) : _
3966 // ...
3967 //
3968 // Where:
3969 //
3970 // * `xxx` can be one of the following: `LP2`, `HP2`, `BP2`, `BP2Norm`, `Notch2`, `AP2`, `
3971 //   Peaking2`
3972 // * `CF`: cutoff in Hz
3973 // * `Q`: resonance
3974 //
3975 // ##### Test

```

```

3975 // ```
3976 // fi = library("filters.lib");
3977 // os = library("oscillators.lib");
3978 // sig = os.osc(440);
3979 // SVFTPT_SVF_test = fi.SVFTPT.SVF(1000, 0.707, sig);
3980 // ```
3981 //-----
3982 declare SVFTPT author "Dario Sanfilippo";
3983 declare SVFTPT copyright
3984 "Copyright (C) 2024 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
3985 declare SVFTPT license "MIT License";
3986 SVFTPT = environment {
3987   SVF(CF, Q, x) = f ~ si.bus(2) : (!, !, _, _, _, _, _, _, _)
3988   with {
3989     g = tan(CF * ma.PI * ma.T);
3990     R2 = 1.0 / Q;
3991     gPlusR2 = g + R2;
3992     f(s0, s1) = u0 , u1 , LP , HP , BP , BPNorm , Notch , AP , Peaking
3993     with {
3994       HP = (x - s0 * gPlusR2 - s1) / (1.0 + g * gPlusR2);
3995       v0 = HP * g;
3996       BP = s0 + v0;
3997       v1 = BP * g;
3998       LP = s1 + v1;
3999       BPNorm = BP * R2;
4000       Notch = x - BPNorm;
4001       AP = x - BP * (R2 + R2);
4002       Peaking = LP - HP;
4003       u0 = v0 + BP;
4004       u1 = v1 + LP;
4005     };
4006   };
4007   LP2(CF, Q, x) = SVF(CF, Q, x) : ba.selectn(7, 0);
4008   HP2(CF, Q, x) = SVF(CF, Q, x) : ba.selectn(7, 1);
4009   BP2(CF, Q, x) = SVF(CF, Q, x) : ba.selectn(7, 2);
4010   BP2Norm(CF, Q, x) = SVF(CF, Q, x) : ba.selectn(7, 3);
4011   Notch2(CF, Q, x) = SVF(CF, Q, x) : ba.selectn(7, 4);
4012   AP2(CF, Q, x) = SVF(CF, Q, x) : ba.selectn(7, 5);
4013   Peaking2(CF, Q, x) = SVF(CF, Q, x) : ba.selectn(7, 6);
4014 };
4015
4016 //-----` (fi.)dynamicSmoothing`-----
4017 //
4018 // Adaptive smoother based on Andy Simper's paper.
4019 //
4020 // This filter uses both the lowpass and bandpass outputs of a
4021 // state-variable filter. The lowpass is used to smooth out the input signal,
4022 // the bandpass, which is a smoothed out version of the highpass, provides
4023 // information on the rate of change of the input. Hence, the bandpass signal
4024 // can be used to adjust the cutoff of the filter to quickly follow the input's
4025 // fast and large variations while effectively filtering out local
4026 // perturbations.
4027 //
4028 // This implementation does not use an approximation for the CF computation,
4029 // and it deploys guards to prevent overshooting with extreme sensitivity
4030 // values.
4031 //
4032 // ### Usage:
4033 //
4034 // ```
4035 // _ : dynamicSmoothing(sensitivity, baseCF) : _
4036 // ```
4037 //
4038 // Where:
4039 //
4040 // * `sensitivity`: sensitivity to changes in the input signal.
4041 // The range is, theoretically, from 0 to INF, though anything between
4042 // 0.0 and 1.0 should be reasonable

```

```

4043 // * `baseCF`: cutoff frequency, in Hz, when there is no variation in the
4044 //   input signal
4045 //
4046 // #### Test
4047 // ```
4048 // fi = library("filters.lib");
4049 // os = library("oscillators.lib");
4050 // sig = os.osc(440);
4051 // dynamicSmoothing_test = fi.dynamicSmoothing(0.5, 500, sig);
4052 // ```
4053 //
4054 // #### References
4055 //
4056 // * <https://cytomic.com/files/dsp/DynamicSmoothing.pdf>
4057 //-----
4058 declare dynamicSmoothing author "Dario Sanfilippo";
4059 declare dynamicSmoothing copyright
4060   "Copyright (C) 2024 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
4061 declare dynamicSmoothing license "MIT License";
4062 dynamicSmoothing(sensitivity, baseCF, x) = f ~ _ : ! , _
4063   with {
4064     f(s) = SVFTPT.BP2(CF, .5 , x) , SVFTPT.LP2(CF, .5, x)
4065     with {
4066       CF = min(ma.SR * .125, baseCF + sensitivity * abs(s) * ma.SR * .5);
4067     };
4068   };
4069
4070
4071 //-----`(fi.)oneEuro`-----
4072 // The One Euro Filter €(1 Filter) is an adaptive lowpass filter.
4073 // This kind of filter is commonly used in object-tracking,
4074 // not necessarily audio processing.
4075 //
4076 // #### Usage
4077 //
4078 // ```
4079 // _ : oneEuro(derivativeCutoff, beta, minCutoff) : _
4080 // ```
4081 //
4082 // Where:
4083 //
4084 // * `derivativeCutoff`: Used to filter the first derivative of the input. 1 Hz is a good
4085 //   default.
4086 // * `beta`: "Speed" parameter where higher values reduce latency.
4087 // * `minCutoff`: Minimum cutoff frequency in Hz. Lower values remove more jitter.
4088 //
4089 // #### Test
4090 // ```
4091 // fi = library("filters.lib");
4092 // os = library("oscillators.lib");
4093 // sig = os.osc(440);
4094 // oneEuro_test = sig : fi.oneEuro(1, 0.5, 5);
4095 // ```
4096 //
4097 // #### References
4098 //
4099 // * <https://gery.casiez.net/1euro/>
4100 //-----
4101 declare oneEuro author "David Braun";
4102 declare oneEuro copyright "Copyright (C) 2024 by David Braun <braun@ccrma.stanford.edu>";
4103 declare oneEuro license "MIT";
4104
4105 oneEuro(derivativeCutoff, beta, minCutoff) = _oneEuro ~ _
4106   with {
4107     // exponential moving average that calculates its own alpha from a cutoff in Hz
4108     // _ : ema(cutoff) : _
4109     ema(cutoff) = tick ~ _
4110     with {

```

```

4110     alpha = 1.0 / (1.0 + ma.SR / (2 * ma.PI * cutoff));
4111     tick(prev, x) = prev*(1-alpha) + x*alpha;
4112 };
4113
4114 _oneEuro(prev, x) = x : ema(adaptiveCutoff)
4115 with {
4116     derivative = (x - prev)*ma.SR;
4117     derivativeFiltered = derivative : ema(derivativeCutoff);
4118     adaptiveCutoff = minCutoff + beta * abs(derivativeFiltered);
4119 };
4120 };
4121
4122
4123 //=====Linkwitz-Riley 4th-order 2-way, 3-way, and 4-way crossovers=====
4124 //
4125 // The Linkwitz-Riley (LR) crossovers are designed to produce a fully-flat
4126 // magnitude response when their outputs are combined. The 4th-order
4127 // LR filters (LR4) have a 24dB/octave slope and they are rather popular audio
4128 // crossovers used in multi-band processing.
4129 //
4130 // The LR4 can be constructed by cascading two second-order Butterworth
4131 // filters. For the second-order Butterworth filters, we will use the SVF
4132 // filter implemented above by setting the Q-factor to 1.0 / sqrt(2.0).
4133 // These will be cascaded in pairs to build the LR4 highpass and lowpass.
4134 // For the phase correction, we will use the 2nd-order Butterworth allpass.
4135 //
4136 // #### References
4137 //
4138 // Zavalishin, Vadim. "The art of VA filter design." Native Instruments, Berlin, Germany
4139 // (2012).
4140 //=====
4141
4142 //-----`(fi).lowpassLR4`-----
4143 // 4th-order Linkwitz-Riley lowpass.
4144 //
4145 // #### Usage
4146 //
4147 // ```
4148 // _ : lowpassLR4(cf) : _
4149 // ```
4150 //
4151 // Where:
4152 //
4153 // * `cf`: lowpass cutoff in Hz
4154 //
4155 // #### Test
4156 // ```
4157 // fi = library("filters.lib");
4158 // os = library("oscillators.lib");
4159 // src = os.osc(440);
4160 // lowpassLR4_test = src : fi.lowpassLR4(1000);
4161 // ```
4162 //-----
4163 declare lowpassLR4 author "Dario Sanfilippo";
4164 declare lowpassLR4 copyright
4165 "Copyright (C) 2022 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
4166 declare lowpassLR4 license "MIT-style STK-4.3 license";
4167 lowpassLR4(cf, x) = x : seq(i, 2, svf.lp(cf, 1.0 / sqrt(2.0)));
4168
4169
4170 //-----`(fi).highpassLR4`-----
4171 // 4th-order Linkwitz-Riley highpass.
4172 //
4173 // #### Usage
4174 //
4175 // ```
4176 // _ : highpassLR4(cf) : _

```

```

4177 // ``
4178 //
4179 // Where:
4180 //
4181 // * `cf`: highpass cutoff in Hz
4182 //
4183 // #### Test
4184 // ``
4185 // fi = library("filters.lib");
4186 // os = library("oscillators.lib");
4187 // src = os.osc(440);
4188 // highpassLR4_test = src : fi.highpassLR4(1000);
4189 // ``
4190 //-----
4191 declare highpassLR4 author "Dario Sanfilippo";
4192 declare highpassLR4 copyright
4193   "Copyright (C) 2022 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
4194 declare highpassLR4 license "MIT-style STK-4.3 license";
4195 highpassLR4(cf, x) = x : seq(i, 2, svf.hp(cf, 1.0 / sqrt(2.0)));
4196
4197 //-----`fi.crossover2LR4`-----
4198 // Two-way 4th-order Linkwitz-Riley crossover.
4199 //
4200 //
4201 // #### Usage
4202 //
4203 // ``
4204 // _ : crossover2LR4(cf) : si.bus(2)
4205 // ``
4206 //
4207 // Where:
4208 //
4209 // * `cf`: crossover split cutoff in Hz
4210 //
4211 // #### Test
4212 // ``
4213 // fi = library("filters.lib");
4214 // os = library("oscillators.lib");
4215 // src = os.osc(440);
4216 // crossover2LR4_test = src : fi.crossover2LR4(1000);
4217 // ``
4218 //-----
4219 declare crossover2LR4 author "Dario Sanfilippo";
4220 declare crossover2LR4 copyright
4221   "Copyright (C) 2022 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
4222 declare crossover2LR4 license "MIT-style STK-4.3 license";
4223 crossover2LR4(cf, x) = lowpassLR4(cf, x) , highpassLR4(cf, x);
4224
4225 //-----`fi.crossover3LR4`-----
4226 // Three-way 4th-order Linkwitz-Riley crossover.
4227 //
4228 //
4229 // #### Usage
4230 //
4231 // ``
4232 // _ : crossover3LR4(cf1, cf2) : si.bus(3)
4233 // ``
4234 //
4235 // Where:
4236 //
4237 // * `cf1`: crossover lower split cutoff in Hz
4238 // * `cf2`: crossover upper split cutoff in Hz
4239 //
4240 // #### Test
4241 // ``
4242 // fi = library("filters.lib");
4243 // os = library("oscillators.lib");
4244 // src = os.osc(440);

```

```

4245 // crossover3LR4_test = src : fi.crossover3LR4(500, 2000);
4246 // ```
4247 //-----
4248 declare crossover3LR4 author "Dario Sanfilippo";
4249 declare crossover3LR4 copyright
4250 "Copyright (C) 2022 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
4251 declare crossover3LR4 license "MIT-style STK-4.3 license";
4252 crossover3LR4(cf1, cf2, x) =
4253     crossover2LR4(cf1, x) : svf.ap(cf2, 1.0 / sqrt(2.0)) , crossover2LR4(cf2);
4254
4255
4256 //-----`(fi.)crossover4LR4`-----
4257 // Four-way 4th-order Linkwitz-Riley crossover.
4258 //
4259 // #### Usage
4260 //
4261 // ```
4262 // _ : crossover4LR4(cf1, cf2, cf3) : si.bus(4)
4263 // ```
4264 //
4265 // Where:
4266 //
4267 // * `cf1`: crossover lower split cutoff in Hz
4268 // * `cf2`: crossover mid split cutoff in Hz
4269 // * `cf3`: crossover upper split cutoff in Hz
4270 //
4271 // #### Test
4272 // ```
4273 // fi = library("filters.lib");
4274 // os = library("oscillators.lib");
4275 // src = os.osc(440);
4276 // crossover4LR4_test = src : fi.crossover4LR4(300, 1000, 3000);
4277 // ```
4278 //-----
4279 declare crossover4LR4 author "Dario Sanfilippo";
4280 declare crossover4LR4 copyright
4281 "Copyright (C) 2022 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
4282 declare crossover4LR4 license "MIT-style STK-4.3 license";
4283 crossover4LR4(cf1, cf2, cf3, x) =
4284     crossover2LR4(cf2, x) :
4285         svf.ap(cf3, 1.0 / sqrt(2.0)) ,
4286         svf.ap(cf1, 1.0 / sqrt(2.0)) :
4287             crossover2LR4(cf1) ,
4288             crossover2LR4(cf3);
4289
4290
4291 //-----`(fi.)crossover8LR4`-----
4292 // Eight-way 4th-order Linkwitz-Riley crossover.
4293 //
4294 // #### Usage
4295 //
4296 // ```
4297 // _ : crossover8LR4(cf1, cf2, cf3, cf4, cf5, cf6, cf7) : si.bus(8)
4298 // ```
4299 //
4300 // Where:
4301 //
4302 // * `cf1`: lowest crossover cutoff frequency in Hz
4303 // * `cf2`: second crossover cutoff frequency in Hz
4304 // * `cf3`: third crossover cutoff frequency in Hz
4305 // * `cf4`: fourth crossover cutoff frequency in Hz
4306 // * `cf5`: fifth crossover cutoff frequency in Hz
4307 // * `cf6`: sixth crossover cutoff frequency in Hz
4308 // * `cf7`: highest crossover cutoff frequency in Hz
4309 //
4310 // #### Test
4311 // ```
4312 // fi = library("filters.lib");

```

```

4313 // os = library("oscillators.lib");
4314 // src = os.osc(440);
4315 // crossover8LR4_test = src : fi.crossover8LR4(100, 200, 400, 800, 1600, 3200, 6400);
4316 // ```
4317 //-----
4318 declare crossover8LR4 author "Dario Sanfilippo";
4319 declare crossover8LR4 copyright
4320 "Copyright (C) 2022 Dario Sanfilippo <sanfilippo.dario@gmail.com>";
4321 declare crossover8LR4 license "MIT-style STK-4.3 license";
4322 crossover8LR4(cf1, cf2, cf3, cf4, cf5, cf6, cf7, x) =
4323   crossover2LR4(cf4, x) :
4324     (ap(cf6) : ap(cf5) : ap(cf7)) ,
4325     (ap(cf2) : ap(cf1) : ap(cf3)) :
4326     crossover2LR4(cf2) ,
4327     crossover2LR4(cf6) :
4328     ap(cf3) ,
4329     ap(cf1) ,
4330     ap(cf7) ,
4331     ap(cf5) :
4332     crossover2LR4(cf1) ,
4333     crossover2LR4(cf3) ,
4334     crossover2LR4(cf5) ,
4335     crossover2LR4(cf7)
4336   with {
4337     ap(cf) = svf.ap(cf, 1.0 / sqrt(2.0));
4338   };
4339
4340
4341 //===== Standardized Filters =====
4342 //=====
4343 //
4344 // This section provides filters that are defined by national or
4345 // international standards, e.g. for measurement applications.
4346
4347 //-----`fi.)itu_r_bs_1770_4_kfilter`-----
4348 // The prefilter from Recommendation ITU-R BS.1770-4 for loudness
4349 // measurement. Also known as "K-filter". The recommendation defines
4350 // biquad filter coefficients for a fixed sample rate of 48kHz (page
4351 // 4-5). Here, we construct biquads for arbitrary samplersates. The
4352 // resulting filter is normalized, such that the magnitude at 997Hz is
4353 // unity gain 1.0.
4354 //
4355 // Please note, the ITU-recommendation handles the normalization in
4356 // equation (2) by subtracting 0.691dB, which is not needed with
4357 // `itu_r_bs_1770_4_kfilter`.
4358 //
4359 // One option for future improvement might be, to round those filter
4360 // coefficients, that are almost equal to one. Second, the maximum
4361 // magnitude difference at 48kHz between the ITU-defined filter and
4362 // `itu_r_bs_1770_4_kfilter` is 0.001dB, which obviously could be
4363 // less.
4364 //
4365 // ### Usage
4366 //
4367 // ```
4368 // _ : itu_r_bs_1770_4_kfilter : _
4369 // ```
4370 //
4371 // ### Test
4372 //
4373 // fi = library("filters.lib");
4374 // os = library("oscillators.lib");
4375 // src = os.osc(440);
4376 // itu_r_bs_1770_4_kfilter_test = src : fi.itu_r_bs_1770_4_kfilter;
4377 // ```
4378 //
4379 // ### References
4380 //

```

```

4381 // * <https://www.itu.int/rec/R-REC-BS.1770>
4382 // * <https://gist.github.com/jkbd/07521a98f7873a2dc3dbe16417930791>
4383 //-----
4384 declare itu_r_bs_1770_4_kfilter author "Jakob Dübel";
4385 declare itu_r_bs_1770_4_kfilter copyright "Copyright (C) 2022 Jakob Dübel";
4386 declare itu_r_bs_1770_4_kfilter license "ISC license";
4387
4388 itu_r_bs_1770_4_kfilter = stage1 : stage2 : normalize997Hz
4389 with {
4390     freq2k(f_c) = tan((ma.PI * f_c)/ma.SR);
4391
4392     stage1 = tf22t(b0,b1,b2,a1,a2)
4393     with {
4394         f_c = 1681.7632251028442; // Hertz
4395         gain = 3.9997778685513232; // Decibel
4396         K = freq2k(f_c);
4397         V_0 = pow(10, (gain/20.0));
4398
4399         denominator = 1.0 + sqrt(2.0)*K + K^2;
4400         b0 = (V_0 + sqrt((2.0*V_0))*K + K^2) / denominator;
4401         b1 = 2.0*(K^2 - V_0) / denominator;
4402         b2 = (V_0 - sqrt(2.0*V_0)*K + K^2) / denominator;
4403
4404         a1 = 2*(K^2 - 1) / denominator;
4405         a2 = (1 - sqrt(2.0)*K + K^2) / denominator;
4406     };
4407
4408     stage2 = tf22t(b0,b1,b2,a1,a2)
4409     with {
4410         f_c = 38.135470876002174; // Hertz
4411         Q = 0.5003270373223665;
4412         K = freq2k(f_c);
4413
4414         denominator = (K^2) * Q + K + Q;
4415         b0 = Q / denominator;
4416         b1 = -2*Q / denominator;
4417         b2 = b0;
4418
4419         a1 = (2*Q * (K^2 - 1)) / denominator;
4420         a2 = ((K^2) * Q - K + Q) / denominator;
4421     };
4422
4423     normalize997Hz = *(0.9273671710547968);
4424 };
4425
4426
4427 //=====Averaging Functions=====
4428 //=====
4429 //
4430 // These are a set of samplerate independent averaging functions based on
4431 // moving-average and one-pole filters with specific response characteristics.
4432
4433 //-----`(fi.)avg_rect`-----
4434 // Moving average.
4435 //
4436 // ### Usage
4437 //
4438 // ```
4439 // _ : avg_rect(period) : _
4440 // ```
4441 //
4442 // Where:
4443 //
4444 // * `period`: averaging frame duration in seconds
4445 //-----
4446 declare avg_rect author "Dario Sanfilippo and Julius O. Smith III";
4447 declare avg_rect copyright "Copyright (C) 2020 Dario Sanfilippo
4448     <sanfilippo.dario@gmail.com> and

```

```

4449     2003-2020 by Julius O. Smith III <jos@ccrma.stanford.edu>;
4450 declare avg_rect license "MIT-style STK-4.3 license";
4451 avg_rect(period, x) = x : ba.slidingMean(rint(period * ma.SR));
4452
4453
4454 //-----`(fi.)avg_tau`-----
4455 // Averaging function based on a one-pole filter and the tau response time.
4456 // Tau represents the effective length of the one-pole impulse response,
4457 // that is, tau is the integral of the filter's impulse response. This
4458 // response is slower to reach the final value but has less ripples in
4459 // non-steady signals.
4460 //
4461 // #### Usage
4462 //
4463 // ...
4464 // _ : avg_tau(period) : _
4465 // ...
4466 //
4467 // Where:
4468 //
4469 // * `period`: time, in seconds, for the system to decay by 1/e,
4470 // or to reach 1-1/e of its final value.
4471 //
4472 // #### References
4473 //
4474 // * <https://ccrma.stanford.edu/~jos/mdft/Exponentials.html>
4475 //-----
4476 declare avg_tau author "Dario Sanfilippo and Julius O. Smith III";
4477 declare avg_tau copyright "Copyright (C) 2020 Dario Sanfilippo
4478 <sanfilippo.dario@gmail.com> and
4479 2003-2020 by Julius O. Smith III <jos@ccrma.stanford.edu>";
4480 declare avg_tau license "MIT-style STK-4.3 license";
4481 avg_tau(period, x) = fi.lptau(period, x);
4482
4483
4484 //-----`(fi.)avg_t60`-----
4485 // Averaging function based on a one-pole filter and the t60 response time.
4486 // This response is particularly useful when the system is required to
4487 // reach the final value after about `period` seconds.
4488 //
4489 // #### Usage
4490 //
4491 // ...
4492 // _ : avg_t60(period) : _
4493 // ...
4494 //
4495 // Where:
4496 //
4497 // * `period`: time, in seconds, for the system to decay by 1/1000,
4498 // or to reach 1-1/1000 of its final value.
4499 //
4500 // #### References
4501 //
4502 // * <https://ccrma.stanford.edu/~jos/mdft/Audio\_Decay\_Time\_T60.html>
4503 //-----
4504 declare avg_t60 author "Dario Sanfilippo and Julius O. Smith III";
4505 declare avg_t60 copyright "Copyright (C) 2020 Dario Sanfilippo
4506 <sanfilippo.dario@gmail.com> and
4507 2003-2020 by Julius O. Smith III <jos@ccrma.stanford.edu>";
4508 declare avg_t60 license "MIT-style STK-4.3 license";
4509 avg_t60(period, x) = fi.lpt60(period, x);
4510
4511
4512 //-----`(fi.)avg_t19`-----
4513 // Averaging function based on a one-pole filter and the t19 response time.
4514 // This response is close to the moving-average algorithm as it roughly reaches
4515 // the final value after `period` seconds and shows about the same
4516 // oscillations for non-steady signals.

```

```

4517 //
4518 // ##### Usage
4519 //
4520 // ...
4521 // _ : avg_t19(period) : _
4522 // ...
4523 //
4524 // Where:
4525 //
4526 // * `period`: time, in seconds, for the system to decay by 1/e^2.2,
4527 // or to reach 1-1/e^2.2 of its final value.
4528 //
4529 // ##### References
4530 //
4531 // Zölzer, U. (2008). Digital audio signal processing (Vol. 9). New York: Wiley.
4532 //-----
4533 declare avg_t19 author "Dario Sanfilippo and Julius O. Smith III";
4534 declare avg_t19 copyright "Copyright (C) 2020 Dario Sanfilippo
4535 <sanfilippo.dario@gmail.com> and
4536 2003-2020 by Julius O. Smith III <jos@ccrma.stanford.edu>";
4537 declare avg_t19 license "MIT-style STK-4.3 license";
4538 avg_t19(period, x) = fi.lpt19(period, x);
4539
4540
4541 //=====Kalman Filters=====
4542 //=====
4543 //
4544 // Functions related to the Kalman filter.
4545
4546 kalmanEnv = environment {
4547
4548     matMul = la.matMul;
4549
4550     // Predicts the next state.
4551     // * `N`: State size
4552     // * `M`: Measurement size
4553     // * `F`: State transition matrix (NxN)
4554     // * `B`: Control input matrix (NxM)
4555     // Implicit arguments:
4556     // * `x`: Current state (Nx1)
4557     // * `u`: Control input (Mx1)
4558     predictState(N, M, F, B) = si.vecOp((Fx, Bu), +)
4559     with {
4560         // Fx = F @ x, where F is NxN and x is Nx1
4561         Fx = matMul(N, N, N, 1, F, si.bus(N));
4562
4563         // Bu = B @ u, where B is NxM and u is Mx1
4564         Bu = matMul(N, M, M, 1, B, si.bus(M));
4565     };
4566
4567     // Predicts the covariance matrix for the next state.
4568     // * `N`: State size
4569     // * `F`: State transition matrix (NxN)
4570     // * `Q`: Process noise covariance matrix (NxN)
4571     // * `P`: Current state covariance matrix (NxN)
4572     // No implicit arguments.
4573     predictCovariance(N, F, Q, P) = si.vecOp((FPF_T, Q), +)
4574     with {
4575         // FP = F @ P
4576         FP = matMul(N, N, N, N, F, P);
4577
4578         // FPF_T = FP @ F^T, where FP is NxN and F^T is NxN
4579         FPF_T = matMul(N, N, N, N, FP, la.transpose2(N, N, F));
4580     };
4581
4582     // Calculates the Kalman gain (NxM).
4583     // * `N`: State size
4584     // * `M`: Measurement size

```

```

4585 // * `H`: Observation matrix (MxN)
4586 // * `R`: Measurement noise covariance matrix (MxM)
4587 // Implicit arguments:
4588 // * `P_pred`: Predicted covariance matrix (NxN)
4589 kalmanGain(N, M, H, R) = P_pred <:
4590   matMul(N, M, M, M, matMul(N, N, N, M, P_pred, H_T), S_inv)
4591 with {
4592   P_pred = si.bus(N*N);
4593   // H_T = Transpose of H
4594   H_T = la.transpose2(M, N, H);
4595
4596   // HP = H * P_pred, where H is MxN and P_pred is NxN
4597   HP = matMul(M, N, N, N, H, P_pred);
4598
4599   // HPH_T = HP @ H^T, where HP is MxN and H^T is NxM, resulting in MxM
4600   HPH_T = matMul(M, N, N, M, HP, H_T);
4601
4602   // S_inv = inverse(HPH_T + R)
4603   S_inv = si.vecOp((HPH_T, R), +) : la.inverse(M);
4604 };
4605
4606 // Updates the state estimate based on the Kalman gain and measurement.
4607 // * `N`: State size
4608 // * `M`: Measurement size
4609 // * `H`: Observation matrix (MxN)
4610 // Implicit arguments:
4611 // * `K`: Kalman gain matrix (NxM)
4612 // * `x_pred`: (Nx1)
4613 // * `z`: (Mx1)
4614 updateState(N, M, H) = si.bus(N*M), si.bus(N), si.bus(M) <:
4615   si.vecOp((get_x_pred, K_mul_residual(N, M, H)), +)
4616 with {
4617   get_x_pred = par(i, N*M, !), par(i, N, _), par(i, M, !);
4618 };
4619
4620 // todo: find a way to not use mySwap.
4621 // Its purpose is to route K, x, z into K, z, x
4622 mySwap(N, M) = si.bus(N*M), si.bus(N), si.bus(M) <:
4623   select_K, select_z, select_x_pred
4624 with {
4625   select_K = par(i, N*M, _), par(i, N, !), par(i, M, !);
4626   select_x_pred = par(i, N*M, !), par(i, N, _), par(i, M, !);
4627   select_z = par(i, N*M, !), par(i, N, !), par(i, M, _);
4628 };
4629
4630 K_mul_residual(N, M, H) = mySwap(N, M) : matMul(N, M, M, 1, si.bus(N*M), residual)
4631 with {
4632   // residual = z - H @ x_pred, where z is Mx1 and H @ x_pred is Mx1
4633   residual = si.vecOp((si.bus(M), matMul(M, N, N, 1, H, si.bus(N))), -);
4634 };
4635
4636 // Updates the covariance matrix based on the Kalman gain and observation matrix.
4637 // * `N`: State size
4638 // * `M`: Measurement size
4639 // * `H`: Observation matrix (MxN)
4640 // * `K`: Kalman gain matrix (NxM)
4641 // * `P_pred`: Predicted covariance matrix (NxN)
4642 // No implicit arguments.
4643 updateCovariance(N, M, H, K, P_pred) =
4644   matMul(N, N, N, N, I_minus_KH, P_pred)
4645 with {
4646   // KH = K @ H, where K is NxM and H is MxN
4647   KH = matMul(N, M, M, N, K, H);
4648
4649   // I_minus_KH = I - K @ H, where both are NxN
4650   I_minus_KH = si.vecOp((la.identity(N), KH), -);
4651 };
4652

```

```

4653 // implicit arguments:
4654 // * `u`: Control input (Mx1)
4655 // * `z`: Measurement signal (Mx1)
4656 kalmanFilter(N, M, B, R, H, Q, F, reset) = (p, x, u, z <: pNew, xNew) - (pBus, xBus) :
    pCut, xBus
4657 with {
4658     doReset = reset > 0;
4659
4660     pBus = si.bus(N*N);
4661     pCut = par(i, N*N, !);
4662     pInit = la.diag(N, par(i, N, 100)); // large value for initial uncertainty in P.
4663     p = pBus, pInit : ba.selectbus(N*N, 2, doReset);
4664
4665     xBus = si.bus(N);
4666     xCut = par(i, N, !);
4667     xInit = par(i, N, 0); // initialize with zeros.
4668     x = xBus, xInit : ba.selectbus(N, 2, doReset);
4669
4670     u = si.bus(M);
4671     z = si.bus(M);
4672
4673     uCut = par(i, M, !);
4674     zCut = par(i, M, !);
4675
4676     predCov = predictCovariance(N, F, Q, pBus);
4677     gain = kalmanGain(N, M, H, R);
4678
4679     xNew = pBus, xBus, u, z : updateState(N, M, H, gain(predCov), predictState(N, M, F, B,
        xBus, u), z);
4680     pNew = pBus, xCut, uCut, zCut : predCov <: updateCovariance(N, M, H, gain(pBus), pBus)
        ;
4681 };
4682 };
4683
4684 //-----`fi.kalman`-----
4685 // The Kalman filter. It returns the state (a bus of size `N`).
4686 // Note that the only compile-time constant arguments are `N` and `M`.
4687 // Other arguments are capitalized because they're matrices, and it makes
4688 // reading them much easier.
4689 //
4690 // #### Usage
4691 // ```
4692 // kalman(N, M, B, R, H, Q, F, reset, u, z) : si.bus(N)
4693 // ```
4694 //
4695 // Where:
4696 //
4697 // * `N`: State size (constant int)
4698 // * `M`: Measurement size (constant int)
4699 // * `B`: Control input matrix (NxM)
4700 // * `R`: Measurement noise covariance matrix (MxM)
4701 // * `H`: Observation matrix (MxN)
4702 // * `Q`: Process noise covariance matrix (NxN)
4703 // * `F`: State transition matrix (NxN)
4704 // * `reset`: Reset trigger. Whenever `reset>0`, the internal state `x` and covariance matrix
    `P` are reset.
4705 // * `u`: Control input (Mx1)
4706 // * `z`: Measurement signal (Mx1)
4707 //
4708 // #### Example test programs
4709 // Demo 1 `(N=1, M=1)` (don't listen, just use oscilloscope):
4710 // ```
4711 // process = fi.kalman(N, M, B, R, H, Q, F, reset, u, z) : it.interpolate_linear(filteredAmt,
    z)
4712 // with {
4713 //     B = 1.;
4714 //     R = 0.1;

```

```

4716 // H = 1;
4717 // Q = .01;
4718 // F = la.identity(N);
4719 // reset = button("reset");
4720 //
4721 // // Dimensions
4722 // N = 1; // State size
4723 // M = 1; // Measurement size
4724 //
4725 // freq = hslider("Freq", 1, 0.01, 10, .01);
4726 // u = 0.; // constant input
4727 // trueState = os.osc(freq)*.5 + u;
4728 // noiseGain = hslider("Noise Gain", .1, 0, 1, .01);
4729 //
4730 // filteredAmt = hslider("Filter Amount", 1, 0, 1, .01) : si.smoo;
4731 //
4732 // measurementNoise = no.noise*noiseGain;
4733 // z = trueState + measurementNoise; // Observed state
4734 // };
4735 // ...
4736 //
4737 // Demo 2 `(N=2, M=1)` (don't listen, just use oscilloscope)
4738 // ...
4739 //
4740 // process = fi.kalman(N, M, B, R, H, Q, F, reset, u, z)
4741 // with {
4742 //   B = par(i, N, 0);
4743 //   R = (0.1);
4744 //   H = (1, 0);
4745 //   Q = la.diag(2, par(i, N, .1));
4746 //   F = la.identity(N);
4747 //   reset = 0;
4748 //   u = si.bus(M);
4749 //   z = si.bus(M);
4750 // }
4751 // // Dimensions
4752 // N = 2; // State size
4753 // M = 1; // Measurement size
4754 // };
4755 // ...
4756 // #### References
4757 //
4758 // * <https://en.wikipedia.org/wiki/Kalman\_filter>
4759 // * <https://www.cs.unc.edu/~welch/kalman/index.html>
4760 //-----
4761 declare kalman author "David Braun";
4762 declare kalman license "MIT License";
4763 kalman = kalmanEnv.kalmanFilter;
4764
4765
4766 /*****
4767 # Licenses
4768
4769 ## STK 4.3 License
4770
4771 Permission is hereby granted, free of charge, to any person obtaining a copy of
4772 this software and associated documentation files (the "Software"), to deal in
4773 the Software without restriction, including without limitation the rights to
4774 use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies
4775 of the Software, and to permit persons to whom the Software is furnished to do
4776 so, subject to the following conditions:
4777
4778 The above copyright notice and this permission notice shall be included in all
4779 copies or substantial portions of the Software.
4780
4781 Any person wishing to distribute modifications to the Software is asked to send
4782 the modifications to the original developer so that they can be incorporated
4783 into the canonical version. For software copyrighted by Julius O. Smith III,

```

```

4784 email your modifications to <jos@ccrma.stanford.edu>. This is, however, not a
4785 binding provision of this license.
4786
4787 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
4788 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
4789 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
4790 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
4791 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
4792 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
4793 SOFTWARE.
4794
4795 -----
4796
4797 ## LGPL License
4798
4799 This program is free software; you can redistribute it and/or modify it under
4800 the terms of the GNU Lesser General Public License as published by the Free
4801 Software Foundation; either version 2.1 of the License, or (at your option) any
4802 later version.
4803
4804 This program is distributed in the hope that it will be useful, but WITHOUT ANY
4805 WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A
4806 PARTICULAR PURPOSE. See the GNU Lesser General Public License for more details.
4807
4808 You should have received a copy of the GNU Lesser General Public License along
4809 with the GNU C Library; if not, write to the Free Software Foundation, Inc.,
4810 59 Temple Place, Suite 330, Boston, MA 02111-1307 USA.
4811
4812 -----
4813
4814 ## ISC License
4815
4816 Permission to use, copy, modify, and/or distribute this software for any
4817 purpose with or without fee is hereby granted, provided that the above
4818 copyright notice and this permission notice appear in all copies.
4819
4820 THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARRANTIES WITH
4821 REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY
4822 AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY SPECIAL, DIRECT,
4823 INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM
4824 LOSS OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR
4825 OTHER TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR
4826 PERFORMANCE OF THIS SOFTWARE.
4827
4828 *****/

```

Listing 10: delays.lib

```

1 //##### delays.lib #####
2 // Delays library. Its official prefix is `de`.
3 //
4 // This library provides reusable building blocks for delay-based processing:
5 // single and multi-tap delays, fractional delays and utilities for echo and spatial effects.
6 //
7 // The Delays library is organized into 4 sections:
8 //
9 // * [Basic Delay Functions](#basic-delay-functions)
10 // * [Lagrange Interpolation](#lagrange-interpolation)
11 // * [Thiran Allpass Interpolation](#thiran-allpass-interpolation)
12 // * [Others](#others)
13 //
14 // #### References
15 //
16 // * <https://github.com/grame-cncm/faustlibraries/blob/master/delays.lib>

```

```

17 //#####
18
19 /*****
20 *****/
21 FAUST library file
22 Copyright (C) 2003-2016 GRAME, Centre National de Creation Musicale
23 -----
24 This program is free software; you can redistribute it and/or modify
25 it under the terms of the GNU Lesser General Public License as
26 published by the Free Software Foundation; either version 2.1 of the
27 License, or (at your option) any later version.
28
29 This program is distributed in the hope that it will be useful,
30 but WITHOUT ANY WARRANTY; without even the implied warranty of
31 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
32 GNU Lesser General Public License for more details.
33
34 You should have received a copy of the GNU Lesser General Public
35 License along with the GNU C Library; if not, write to the Free
36 Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
37 02111-1307 USA.
38
39 EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
40 larger FAUST program which directly or indirectly imports this library
41 file and still distribute the compiled code generated by the FAUST
42 compiler, or a modified version of this compiled code, under your own
43 copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
44 grants you the right to freely choose the license for the resulting
45 compiled code. In particular the resulting compiled code has no obligation
46 to be LGPL or GPL. For example you are free to choose a commercial or
47 closed source license or any other license if you decide so.
48 *****/
49 *****/
50
51 ma = library("maths.lib");
52 ba = library("basics.lib");
53 si = library("signals.lib");
54 fi = library("filters.lib");
55
56 declare name "Faust Delay Library";
57 declare version "1.2.0";
58
59 //=====Basic Delay Functions=====
60 //=====
61
62 //-----`(de.)delay`-----
63 // Simple `d` samples delay where `n` is the maximum delay length as a number of
64 // samples. Unlike the `@` delay operator, here the delay signal `d` is explicitly
65 // bounded to the interval [0..n]. The consequence is that delay will compile even
66 // if the interval of d can't be computed by the compiler.
67 // `delay` is a standard Faust function.
68 //
69 // ### Usage
70 //
71 // ```
72 // _ : delay(n,d) : _
73 // ```
74 //
75 // Where:
76 //
77 // * `n`: the max delay length in samples
78 // * `d`: the delay length in samples (integer)
79
80 // ### Test
81 // ```
82 // de = library("delays.lib");
83 // os = library("oscillators.lib");
84 // delay_test = os.osc(440) : de.delay(44100, 22050);

```

```

85 // ```
86 //
87 // TODO: add MBH np2
88 //-----
89 delay(n,d,x) = x @ min(n, max(0,d));
90
91
92 //-----`de.)fdelay`-----
93 // Simple `d` samples fractional delay based on 2 interpolated delay lines where `n` is
94 // the maximum delay length as a number of samples.
95 //
96 // `fdelay` is a standard Faust function.
97 //
98 // #### Usage
99 //
100 // ```
101 // _ : fdelay(n,d) : _
102 // ```
103 //
104 // Where:
105 //
106 // * `n`: the max delay length in samples
107 // * `d`: the delay length in samples (float)
108 //
109 // #### Test
110 // ```
111 // de = library("delays.lib");
112 // os = library("oscillators.lib");
113 // fdelay_test = os.osc(440) : de.fdelay(44100, 22050.5);
114 // ```
115 //-----
116 fdelay(n,d,x) = delay(n+1,int(d),x)*(1 - ma.frac(d)) + delay(n+1,int(d)+1,x)*ma.frac(d);
117
118
119 //-----`de.)sdelay`-----
120 // s(mooth)delay: a mono delay that doesn't click and doesn't
121 // transpose when the delay time is changed.
122 //
123 // #### Usage
124 //
125 // ```
126 // _ : sdelay(n,it,d) : _
127 // ```
128 //
129 // Where:
130 //
131 // * `n`: the max delay length in samples
132 // * `it`: interpolation time (in samples), for example 1024
133 // * `d`: the delay length in samples (float)
134 //
135 // #### Test
136 // ```
137 // de = library("delays.lib");
138 // os = library("oscillators.lib");
139 // sdelay_test = os.osc(440) : de.sdelay(44100, 1024, 22050.5);
140 // ```
141 //-----
142 sdelay(n, it, d) = ctrl(it,d),_ : ddi(n)
143 with {
144 // ddi(n,i,d0,d1)
145 // DDI (Double Delay with Interpolation) : the input signal is sent to two
146 // delay lines. The outputs of these delay lines are crossfaded with
147 // an interpolation stage. By acting on this interpolation value one
148 // can move smoothly from one delay to another. When <i> is 0 we can
149 // freely change the delay time <d1> of line 1, when it is 1 we can freely change
150 // the delay time <d0> of line 0.
151 //
152 // <n> = maximal delay in samples

```

```

153 // <i> = interpolation value between 0 and 1 used to crossfade the outputs of the
154 //      two delay lines (0.0: first delay line, 1.0: second delay line)
155 // <d0> = delay time of delay line 0 in samples between 0 and <N>-1
156 // <d1> = delay time of delay line 1 in samples between 0 and <N>-1
157 // <> = the input signal we want to delay
158 ddi(n, i, d0, d1) = _<: delay(n,d0), delay(n,d1) : si.interpolate(i);
159
160 // ctrl(it,d)
161 // Control logic for a Double Delay with Interpolation according to two
162 //
163 // USAGE : ctrl(it,d)
164 // where :
165 // <it> an interpolation time (in samples, for example 256)
166 // <d> a delay time (in samples)
167 //
168 // ctrl produces 3 outputs : an interpolation value <i> and two delay
169 // times <d0> and <d1>. These signals are used to control a ddi (Double Delay with
170 //      Interpolation).
171 // The principle is to detect changes in the input delay time d, then to
172 // change the delay time of the delay line currently unused and then by a
173 // smooth crossfade to remove the first delay line and activate the second one.
174 //
175 // The control logic has an internal state controlled by 4 elements
176 // <v> : the interpolation variation (0, 1/it, -1/it)
177 // <i> : the interpolation value (between 0 and 1)
178 // <d0>: the delay time of line 0
179 // <d1>: the delay time of line 1
180 //
181 // Please note that the last stage (!,_,_,_) cut <v> because it is only
182 // used internally.
183 ctrl(it, d) = \v,ip,d0,d1).(nv, nip, nd0, nd1)
184 with {
185 // interpolation variation
186 nv = ba.if (v!=0.0, // if variation we are interpolating
187 ba.if ((ip>0.0) & (ip<1.0), v, 0), // should we continue or not ?
188 ba.if ((ip==0.0) & (d!=d0), 1.0/it, // if true xfade from d0 to d1
189 ba.if ((ip==1.0) & (d!=d1), -1.0/it, // if true xfade from d1 to d0
190 0)); // nothing to change
191 // interpolation value
192 nip = ip+nv : min(1.0) : max(0.0);
193
194 // update delay time of line 0 if needed
195 nd0 = ba.if ((ip >= 1.0) & (d!=d), d, d0);
196
197 // update delay time of line 1 if needed
198 nd1 = ba.if ((ip <= 0.0) & (d!=d), d, d1);
199 } ~ (_,_,_,_) : (!,_,_,_);
200
201
202 //-----`de.)prime_power_delays`-----
203 // Prime Power Delay Line Lengths.
204 //
205 // #### Usage
206 //
207 // ...
208 // si.bus(N) : prime_power_delays(N,pathmin,pathmax) : si.bus(N);
209 //
210 //
211 // Where:
212 //
213 // * `N`: positive integer up to 16 (for higher powers of 2, extend 'primes' array below)
214 // * `pathmin`: minimum acoustic ray length in the reverberator (in meters)
215 // * `pathmax`: maximum acoustic ray length (meters) - think "room size"
216 //
217 // #### Test
218 //
219 // de = library("delays.lib");

```

```

220 // prime_power_delays_test = de.prime_power_delays(4, 1, 10);
221 // ...
222 //
223 // #### References
224 //
225 // * <https://ccrma.stanford.edu/~jos/pasp/Prime\_Power\_Delay\_Line.html>
226 //-----
227 declare prime_power_delays author "Julius O. Smith III";
228
229 prime_power_delays(N,pathmin,pathmax) = par(i,N,delayvals(i)) with {
230     Np = 16;
231     primes = 2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53;
232     prime(n) = primes : ba.selector(n,Np); // math.lib
233
234     // Prime Power Bounds [matlab: floor(log(maxdel)./log(primes(53)))]
235     maxdel = 8192; // more than 63 meters at 44100 samples/sec & 343 m/s
236     ppbs = 13,8,5,4, 3,3,3,3, 2,2,2,2, 2,2,2,2; // 8192 is enough for all
237     ppb(i) = ba.take(i+1,ppbs);
238
239     // Approximate desired delay-line lengths using powers of distinct primes:
240     c = 343; // soundspeed in m/s at 20 degrees C for dry air
241     dmin = ma.SR*pathmin/c;
242     dmax = ma.SR*pathmax/c;
243     dl(i) = dmin * (dmax/dmin)^(i/float(N-1)); // desired delay in samples
244     ppwr(i) = floor(0.5+log(dl(i))/log(prime(i))); // best prime power
245     delayvals(i) = prime(i)^ppwr(i); // each delay a power of a distinct prime
246 };
247
248
249 //=====Lagrange Interpolation=====
250 //=====
251
252 //-----`(de.)fdelaylti` and `(de.)fdelayltv`-----
253 // Fractional delay line using Lagrange interpolation.
254 //
255 // #### Usage
256 //
257 // ...
258 // _ : fdelaylt[i|v](N, n, d) : _
259 // ...
260 //
261 // Where:
262 //
263 // * `N=1,2,3,...` is the order of the Lagrange interpolation polynomial (constant numerical
264 //   expression)
265 // * `n`: the max delay length in samples
266 // * `d`: the delay length in samples
267 //
268 // `fdelaylti` is most efficient, but designed for constant/slowly-varying delay.
269 // `fdelayltv` is more expensive and more robust when the delay varies rapidly.
270 //
271 // Note: the requested delay should not be less than `(N-1)/2`.
272 //
273 // #### Test
274 // ...
275 // de = library("delays.lib");
276 // os = library("oscillators.lib");
277 // fdelaylti_test = os.osc(440) : de.fdelaylti(3, 44100, 22050.5);
278 // ...
279 // #### References
280 //
281 // * <https://ccrma.stanford.edu/~jos/pasp/Lagrange\_Interpolation.html>
282 // * [Fixed-delay case](<https://ccrma.stanford.edu/~jos/Interpolation/  
Efficient_Time_Invariant_Lagrange_Interpolation.html)>
283 // * [Variable-delay case](<https://ccrma.stanford.edu/~jos/Interpolation/  
Time_Varying_Lagrange_Interpolation.html)>
284 // * Timo I. Laakso et al., "Splitting the Unit Delay - Tools for Fractional Delay Filter

```

```

285 // * Philippe Depalle and Stephan Tassart, "Fractional Delay Lines using Lagrange
    Design", IEEE Signal Processing Magazine, vol. 13, no. 1, pp. 30-60, Jan 1996.
    Interpolators", ICMC Proceedings, pp. 341-343, 1996.
286 //-----
287 declare fdelaylti author "Julius O. Smith III";
288
289 fdelaylti(N,n,d,x) = delay(n,id,x) <: seq(i,N,section(i)) : !, _
290 with {
291     o = (N-1.00001)/2; // offset to -center FIR interpolator
292     dmo = d - o; // assumed nonnegative [d > (N-1)/2]
293     id = int(dmo);
294     fd = o + ma.frac(dmo);
295     section(i,x,y) = (x-x') * c(i) <: _,+(y);
296     c(i) = (i - fd)/(i+1);
297 };
298
299 declare fdelayltv author "Julius O. Smith III";
300
301 fdelayltv(N,n,d,x) = sum(i, N+1, delay(n,id+i,x) * h(N,fd,i))
302 with {
303     o = (N-1.00001)/2; // -center FIR interpolator
304     dmo = d - o; // assumed nonnegative [d > (N-1)/2]
305     id = int(dmo);
306     fd = o + ma.frac(dmo);
307     h(N,d,n) = facs1(N,d,n) * facs2(N,d,n);
308     facs1(N,d,n) = select2(n,1,prod(k,max(1,n),select2(k<n,1,fac(d,n,k))));
309     facs2(N,d,n) = select2(n<N,1,prod(1,max(1,N-n),fac(d,n,1+n+1)));
310     fac(d,n,k) = (d-k)/((n-k)+(n=k));
311     // Explicit formula for Lagrange interpolation coefficients:
312     // h_d(n) = \prod_{\stackrel{k=0}{k \neq n}}^N \frac{d-k}{n-k}, n=0:N
313 };
314
315 //-----` (de.)fdelay[N]`-----
316 // For convenience, `fdelay1`, `fdelay2`, `fdelay3`, `fdelay4`, `fdelay5`
317 // are also available where `N` is the order of the interpolation, built using `fdelayltv`.
318 //-----
319 declare fdelay1 author "Julius O. Smith III";
320 declare fdelay2 author "Julius O. Smith III";
321 declare fdelay3 author "Julius O. Smith III";
322 declare fdelay4 author "Julius O. Smith III";
323 declare fdelay5 author "Julius O. Smith III";
324
325 fdelay1 = fdelayltv(1);
326 fdelay2 = fdelayltv(2);
327 fdelay3 = fdelayltv(3);
328 fdelay4 = fdelayltv(4);
329 fdelay5 = fdelayltv(5);
330
331 //=====Thiran Allpass Interpolation=====
332 // Thiran Allpass Interpolation.
333 //
334 // #### References
335 //
336 // * <https://ccrma.stanford.edu/~jos/pasp/Thiran_Allpass_Interpolators.html>
337 //=====
338
339 //-----` (de.)fdelay[N]a`-----
340 // Delay lines interpolated using Thiran allpass interpolation.
341 //
342 // #### Usage
343 //
344 // ...
345 // _ : fdelay[N]a(n, d) : _
346 // ...
347 //
348 // (exactly like `fdelay`)
349 //
350 // Where:

```

```

351 //
352 // * `N=1,2,3, or 4` is the order of the Thiran interpolation filter (constant numerical
    expression),
353 // and the delay argument is at least `N-1/2`. First-order: `d` at least 0.5, second-order
    : `d` at least 1.5,
354 // third-order: `d` at least 2.5, fourth-order: `d` at least 3.5.
355 // * `n`: the max delay length in samples
356 // * `d`: the delay length in samples
357 //
358 // #### Test
359 // ```
360 // de = library("delays.lib");
361 // os = library("oscillators.lib");
362 // fdelay2a_test = os.osc(440) : de.fdelay2a(44100, 22050.5);
363 // ```
364 //
365 // #### Note
366 //
367 // The interpolated delay should not be less than `N-1/2`.
368 // (The allpass delay ranges from `N-1/2` to `N+1/2`).
369 // This constraint can be alleviated by altering the code,
370 // but be aware that allpass filters approach zero delay
371 // by means of pole-zero cancellations.
372 //
373 // Delay arguments too small will produce an UNSTABLE allpass!
374 //
375 // Because allpass interpolation is recursive, it is not as robust
376 // as Lagrange interpolation under time-varying conditions
377 // (you may hear clicks when changing the delay rapidly).
378 //
379 //-----
380 declare fdelay1a author "Julius O. Smith III";
381
382 fdelay1a(n,d,x) = delay(n,id,x) : fi.tf1(eta,1,eta)
383 with {
384   o = 0.49999; // offset to make life easy for allpass
385   dmo = d - o; // assumed nonnegative
386   id = int(dmo);
387   fd = o + ma.frac(dmo);
388   eta = (1-fd)/(1+fd); // allpass coefficient
389 };
390
391 declare fdelay2a author "Julius O. Smith III";
392 fdelay2a(n,d,x) = delay(n,id,x) : fi.tf2(a2,a1,1,a1,a2)
393 with {
394   o = 1.49999;
395   dmo = d - o; // delay range is [order-1/2, order+1/2]
396   id = int(dmo);
397   fd = o + ma.frac(dmo);
398   a1o2 = (2-fd)/(1+fd); // share some terms (the compiler does this anyway)
399   a1 = 2*a1o2;
400   a2 = a1o2*(1-fd)/(2+fd);
401 };
402
403 declare fdelay3a author "Julius O. Smith III";
404 fdelay3a(n,d,x) = delay(n,id,x) : fi.iir((a3,a2,a1,1),(a1,a2,a3))
405 with {
406   o = 2.49999;
407   dmo = d - o;
408   id = int(dmo);
409   fd = o + ma.frac(dmo);
410   a1o3 = (3-fd)/(1+fd);
411   a2o3 = a1o3*(2-fd)/(2+fd);
412   a1 = 3*a1o3;
413   a2 = 3*a2o3;
414   a3 = a2o3*(1-fd)/(3+fd);
415 };
416

```

```

417 declare fdelay4a author "Julius O. Smith III";
418 fdelay4a(n,d,x) = delay(n,id,x) : fi.iir((a4,a3,a2,a1,1),(a1,a2,a3,a4))
419 with {
420     o = 3.49999;
421     dmo = d - o;
422     id = int(dmo);
423     fd = o + ma.frac(dmo);
424     a1o4 = (4-fd)/(1+fd);
425     a2o6 = a1o4*(3-fd)/(2+fd);
426     a3o4 = a2o6*(2-fd)/(3+fd);
427     a1 = 4*a1o4;
428     a2 = 6*a2o6;
429     a3 = 4*a3o4;
430     a4 = a3o4*(1-fd)/(4+fd);
431 };
432
433 //=====Others=====
434 //=====
435
436 //-----`(de.)multiTapSincDelay`-----
437 //
438 // Variable delay line using multi-tap sinc interpolation.
439 //
440 // This function implements a continuously variable delay line by superposing (2K+2)
441 // auxiliary delayed signals
442 // whose positions and gains are determined by a sinc-based interpolation method. It extends
443 // the traditional
444 // crossfade delay technique to significantly reduce spectral coloration artifacts, which
445 // are problematic in
446 // applications like Wave Field Synthesis (WFS) and auralization.
447 //
448 // Operation:
449 //
450 // - If tau1 and tau2 are very close (|tau2 - tau1| 0), a simple fixed fractional delay is
451 // applied
452 // - Otherwise, a variable delay is synthesized by:
453 //
454 //   - Computing (2K+2) taps symmetrically distributed around tau1 and tau2
455 //   - Applying sinc-based weighting to each tap, based on its offset from the target
456 //   interpolated delay tau
457 //   - Summing all the weighted taps to produce the output
458 //
459 // Features:
460 //
461 // - Smooth delay variation without introducing Doppler pitch shifts
462 // - Significant reduction of comb-filter coloration compared to classical crossfading
463 // - Switching between fixed and variable delay modes to ensure stability
464 //
465 // #### Usage
466 //
467 // _ : multiTapSincDelay(K, MaxDelay, tau1, tau2, alpha) : _
468 //
469 // Where:
470 //
471 // * `K` (integer): number of auxiliary tap pairs (a constant numerical expression). Total
472 //   number of taps = 2*K + 2
473 // * `MaxDelay`: maximum allowable delay in samples (buffer size)
474 // * `tau1`: initial delay in samples (can be fractional)
475 // * `tau2`: target delay in samples (can be fractional)
476 // * `alpha`: interpolation factor between tau1 and tau2 (in [0,1] with 0 = tau1, 1 = tau2)
477 //
478 // #### Test
479 //
480 // de = library("delays.lib");
481 // os = library("oscillators.lib");
482 // multiTapSincDelay_test = os.osc(440) : de.multiTapSincDelay(2, 4096, 1024.0, 1536.0, 0.5)

```

```

479 // ---;
480 //
481 // ##### References
482 //
483 // T. Carpentier, "Implementation of a continuously variable delay line by crossfading
    between several tap delays", 2024: <https://hal.science/hal-04646939>
484 //-----
485 multiTapSincDelay(K, MaxDelay, tau1, tau2, alpha, input) = output with {
486
487     // Total number of taps used in the sum (2K + 2).
488     numTaps = 2 * K + 2;
489
490     // Normalized Sinc function: sinc(x) = sin(pi*x)/(pi*x)
491     // Handles the case where x 0 to avoid division by zero and return 1.
492     sinc(x) = ba.if(abs(x) < ma.EPSILON, 1, sin(ma.PI * x) / (ma.PI * x));
493
494     // Computes the difference between two delays: delta = tau2 - tau1
495     delta(t1, t2) = t2 - t1;
496
497     // Computes the target interpolated delay: tau = (1-alpha)*tau1 + alpha*tau2
498     // This is the center of the sinc function for the hk gains.
499     tau(t1, t2, a) = (1 - a) * t1 + a * t2;
500
501     // Computes the position (delay in samples) of the i-th tap (tk),
502     // according to equation (17) of the article.
503     tk(i, t1, t2) = ba.if(i <= K,
504         t1 - (K - i) * delta(t1, t2),
505         t2 + (i - K - 1) * delta(t1, t2));
506
507     // Computes the gain (hk) of the i-th tap, according to equation (19) of the paper.
508     // hk = sinc((tk - tau) / delta)
509     hk(i, t1, t2, a) = sinc(offset) with {
510         d = delta(t1, t2);
511         denom = ba.if(abs(d) < ma.EPSILON, 1, d);
512         offset = (tk(i, t1, t2) - tau(t1, t2, a)) / denom;
513     };
514
515     // Computes the sum of delayed signals weighted by their gains.
516     // This is the core of the variable multi-tap effect.
517     tapSum(sig, t1, t2, a) =
518         // Iterates from i = 0 to numTaps-1
519         sum(i, numTaps,
520             // For each tap 'i':
521             // 1. Delays the signal 'sig' by tk(i) samples (fractional)
522             // using an internal delay line (fdelay).
523             // 2. Multiplies the delayed signal by the gain hk(i).
524             sig : fdelay(MaxDelay, tk(i, t1, t2)) * hk(i, t1, t2, a)
525         );
526
527     // Simple function to apply a fixed delay (potentially fractional).
528     // Used when tau1 and tau2 are very close.
529     fixedDelay(x, t) = x : fdelay(MaxDelay, t);
530
531     // Determines if delays tau1 and tau2 are close enough to be
532     // considered fixed (to avoid unstable calculations when delta 0).
533     // isFixed equals 1 if delta 0, otherwise 0.
534     isFixed = abs(delta(tau1, tau2)) < ma.EPSILON;
535
536     // Computes the output corresponding to the fixed delay (used if isFixed=1).
537     fd = fixedDelay(input, tau1);
538
539     // Computes the output corresponding to the variable multi-tap delay (used if isFixed=0).
540     mt = tapSum(input, tau1, tau2, alpha);
541
542     // Selects the final output based on isFixed.
543     output = ba.if(isFixed, fd, mt);
544 };

```

```

545
546
547 ///////////////////////////////////////////////////Deprecated Functions//////////////////////////////////////
548 // This section implements functions that used to be in music.lib but that are now
549 // considered as "deprecated".
550 ///////////////////////////////////////////////////
551
552 delay1s(d) = delay(65536,d);
553 delay2s(d) = delay(131072,d);
554 delay5s(d) = delay(262144,d);
555 delay10s(d) = delay(524288,d);
556 delay21s(d) = delay(1048576,d);
557 delay43s(d) = delay(2097152,d);
558
559 fdelay1s(d) = fdelay(65536,d);
560 fdelay2s(d) = fdelay(131072,d);
561 fdelay5s(d) = fdelay(262144,d);
562 fdelay10s(d) = fdelay(524288,d);
563 fdelay21s(d) = fdelay(1048576,d);
564 fdelay43s(d) = fdelay(2097152,d);

```